Parte I. Introducción a la Ingeniería del Software.

Tema 1. Sistemas software complejos.

1.1.- Introducción.

1.2.- Evolución de la industria del software.

1.3.- Características del software.

1.4.- Aplicaciones del software.

1.5.- Problemas del software.

1.6.- Definición de Ingeniería del Software.

Tema 1. Sistemas software complejos.

1.1.- Introducción.

La situación actual en los sistemas informáticos se caracteriza por una rápida evolución de los componentes hardware, que incrementan continuamente sus prestaciones manteniendo o incluso disminuyendo sus precios, junto con una fuerte tendencia a la estandarización (ordenadores personales, estaciones de trabajo con sistema operativo UNIX, sistemas distribuidos funcionando sobre plataformas heterogéneas, etc.) y una gran diversidad de marcas y modelos con prestaciones y precios similares. En este escenario, la potencia de los grandes ordenadores de las décadas pasadas está hoy disponible en un miniordenador e incluso en un ordenador personal. El software es el mecanismo que nos permite utilizar y explotar este potencial.

Esto hace que, a la hora de plantearnos la adquisición de un sistema informático completo, ya sea para gestionar una empresa, para controlar un proceso industrial, o para uso doméstico, el software es lo que marca la diferencia. Entre varios productos de características hardware similares, nos decidiremos por una determinada compañía vendedora basándonos en las prestaciones, inteligencia, calidad y facilidad de uso de su software.

Por otra parte, el desarrollo de software no es una tarea fácil. La complejidad actual de los sistemas informáticos hace a veces necesario el desarrollo de proyectos software de decenas de miles de líneas de código. Esto no puede ser abordado directamente, empezando a programar sin más. Es necesario analizar qué es lo que tenemos que hacer, cómo lo vamos a hacer, cómo se van a coordinar todas las personas que van a intervenir en el proyecto y cómo vamos a controlar el desarrollo del mismo de forma que al final obtengamos los resultados esperados.

Como vemos, el software es actualmente, dentro de cualquier sistema basado en el uso de ordenadores, el componente cuyo desarrollo presenta mayores problemas: es el más difícil de planificar, el que tiene mayor probabilidad de fracaso, y el que tiene menos posibilidades de que se cumplan las estimaciones de costes iniciales. Por otra parte, la demanda de software (y también la complejidad del software que se demanda) aumentan continuamente, lo que aumenta la magnitud de estos problemas.

De todas formas, no hay que ser demasiado catastrofistas. El desarrollo de software es una actividad muy reciente (apenas tiene 50 años), si la comparamos con otras actividades de ingeniería (p.ej. la construcción de puentes o incluso la ingeniería eléctrica, de la que deriva la ingeniería de hardware), y la disciplina que se encarga de establecer un orden en el desarrollo de sistemas de software (esto es, la Ingeniería del Software) es aún más reciente. Existen buenos métodos de

desarrollo de software pero quizás el problema esté en que no están lo suficientemente difundidos o valorados. Sólo recientemente, estas técnicas están logrando una amplia aceptación.

1.2.- Evolución de la industria del software.

Hemos dicho que el software era el factor decisivo a la hora de elegir entre varias soluciones informáticas disponibles para un problema dado, pero esto no ha sido siempre así. En los primeros años de la informática, el hardware tenía una importancia mucho mayor que en la actualidad. Su coste era comparativamente mucho más alto y su capacidad de almacenamiento y procesamiento, junto con su fiabilidad, era lo que limitaba las prestaciones de un determinado producto. El software se consideraba como un simple añadido, a cuyo desarrollo se dedicaba poco esfuerzo y no se aplicaba ningún método sistemático. La programación era un arte de andar por casa, y el desarrollo de software se realizaba sin ninguna planificación. La mayoría del software se desarrollaba y era utilizado por la misma persona u organización. La misma persona lo escribía, lo ejecutaba y, si fallaba, lo depuraba. Debido a que la movilidad en el trabajo era baja, los ejecutivos estaban seguros de que esa persona estaría allí cuando se encontrara algún error. Debido a este entorno personalizado del software, el diseño era un proceso implícito, realizado en la mente de alguien y la documentación normalmente no existía.

En este contexto, las empresas de informática se dedicaron a mejorar las prestaciones del hardware, reduciendo los costes y el consumo de los equipos, y aumentando la velocidad de cálculo y la capacidad de almacenamiento. Para ello, tuvieron que dedicar grandes esfuerzos a investigación y aplicaron métodos de ingeniería industrial al análisis, diseño, fabricación y control de calidad de los componentes hardware. Como consecuencia de esto, el hardware se desarrolló rápidamente y la complejidad de los sistemas informáticos aumentó notablemente, necesitando de un software cada vez más complejo para su funcionamiento. Surgieron entonces las primeras casas de software, y el mercado se amplió considerablemente. Con ello aumentó la movilidad laboral, y con la marcha de un trabajador desaparecían las posibilidades de mantener o modificar los programas que éste había desarrollado.

Al no utilizarse metodología alguna en el desarrollo del software, los programas contenían numerosos errores e inconsistencias, lo que obligaba a una depuración continua, incluso mucho después de haber sido entregados al cliente. Estas continuas modificaciones no hacían sino aumentar la inconsistencia de los programas, que se alejaban cada vez más de la corrección y se hacían prácticamente ininteligibles. Los costes se disparaban y frecuentemente era más rápido (y por tanto más barato) empezar de nuevo desde cero, desechando todo el trabajo anterior, que intentar adaptar un programa preexistente a un cambio de los requisitos. Sin embargo, los nuevos programas no estaban exentos de errores ni de futuras modificaciones, con lo que la situación volvía a ser la misma. Había comenzado la denominada crisis del software.

Hoy en día, la distribución de costes en el desarrollo de sistemas informáticos ha cambiado drásticamente. El software, en lugar del hardware, es el elemento principal del coste. Esto ha hecho que las miradas de los ejecutivos de las empresas se centren en el software y a que se formulen las siguientes preguntas:

- ¿ Por qué lleva tanto tiempo terminar los programas ?
- ¿ Por qué es tan elevado el coste ?
- ¿ Por qué no es posible encontrar todos los errores antes de entregar el software al cliente ?
- ¿ Por qué resulta tan difícil constatar el progreso conforme se desarrolla el software ?

Estas y otras muchas preguntas son una manifestación del carácter del software y de la forma en que se desarrolla y han llevado a la aparición y la adopción paulatina de la ingeniería del software.

1.3.- Características del software.

Una definición de software podría ser la siguiente:

Software: (1) instrucciones de ordenador que cuando se ejecutan proporcionan la función y el comportamiento deseado, (2) estructuras de datos que facilitan a los programas manipular adecuadamente la información, y (3) documentos que describen la operación y el uso de los programas.

Por tanto, el software incluye no sólo los programas de ordenador, sino también las estructuras de datos que manejan esos programas y toda la documentación que debe acompañar al proceso de desarrollo, mantenimiento y uso de dichos programas.

Según esto, el software se diferencia de otros productos que los hombres puedan construir en que es, por su propia naturaleza lógico. En el desarrollo del hardware, el proceso creativo (análisis, diseño, construcción, prueba) se traduce finalmente en una forma material, en algo físico. Por el contrario, el software es inmaterial y por ello tiene unas características completamente distintas al hardware. Entre ellas podemos citar:

1.- El software se desarrolla, no se fabrica en sentido estricto.

Existen similitudes entre el proceso de desarrollo del software y el de otros productos industriales, como el hardware. En ambos casos existen fases de análisis, diseño y desarrollo o construcción, y la buena calidad del producto final se obtiene mediante un buen diseño. Sin embargo, en la fase de producción del software pueden producirse problemas que afecten a la calidad y que no existen, o son fácilmente evitables, en el caso del software

Por otro lado, en el caso de producción de hardware a gran escala, el coste del producto acaba dependiendo exclusivamente del coste de los materiales empleados y del propio proceso de producción, reduciéndose la repercusión en el coste de las fases de ingeniería previas. En el software, el desarrollo es una más de las labores de ingeniería, y la producción a gran o pequeña escala no influye en el impacto que tiene la ingeniería en el coste, al ser el producto inmaterial. Por otro lado, la replicación del producto (lo que sería equivalente a la producción del producto hardware) no presenta problemas técnicos, y no se requiere un control de calidad individualizado.

El diferente impacto que tiene la ingeniería en el desarrollo de productos hardware y software puede verse en el siguiente ejemplo:

	Ingeniería	Producción o Desarrollo	Coste unitario / 100 unidades	Coste unitario / 100.000 unidades
Hardware	1000	50 c.u.	60	50.01
Software	1000	2000	30	0.03

Tabla 1.1. Influencia de los costes de ingeniería en el coste total del producto

Los costes del software se encuentran en la ingeniería (incluyendo en ésta el desarrollo), y no en la producción, y es ahí donde hay que incidir para reducir el coste final del producto.

2.- El software no se estropea.

Podemos comparar las curvas de índices de fallos del hardware y el software en función del tiempo. En el caso del hardware (figura 1.1), se tiene la llamada *curva de bañera*, que indica que el hardware presenta relativamente muchos fallos al principio de su vida. Estos fallos son debidos fundamentalmente a defectos de diseño o a la baja calidad inicial de la fase de producción. Una vez corregidos estos defectos, la tasa de fallos cae hasta un nivel estacionario y se mantiene así durante un cierto periodo de tiempo. Posteriormente, la tasa de fallos vuelve a incrementarse debido al deterioro de los componentes, que van siendo afectados por la suciedad, vibraciones y la influencia de muchos otros factores externos. Llegados a este punto, podemos sustituir los componentes defectuosos o todo el sistema por otros nuevos, y la tasa de fallos vuelve a situarse en el nivel estacionario.

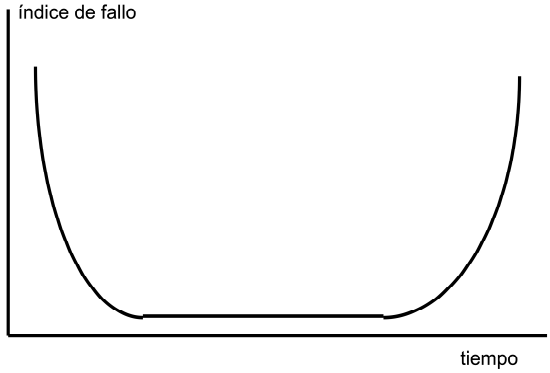


Figura. 1.1. Curva de fallos del hardware

El software no es susceptible a los factores externos que hacen que el hardware se estropee. Por tanto la curva debería seguir la forma de la figura 1.2. Inicialmente la tasa de fallos es alta, debido a la presencia de errores no detectados durante el desarrollo, los denominados errores latentes. Una vez corregidos estos errores, la tasa de fallos debería alcanzar el nivel estacionario y mantenerse ahí indefinidamente.

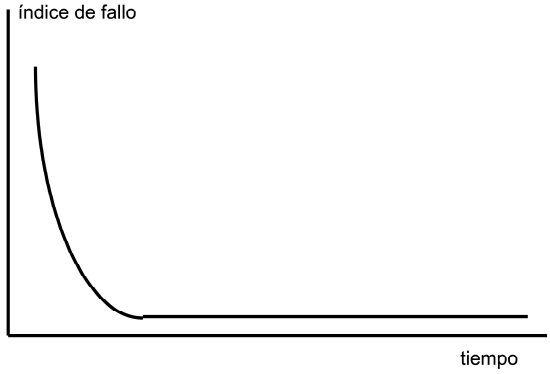


Figura 1.2. Curva ideal de fallos del software

Pero esto no es más que una simplificación del modelo real de fallos de un producto software. Durante su vida, el software sufre cambios, debidos al mantenimiento. El mantenimiento puede deberse a la corrección de errores latentes o a cambios en los requisitos iniciales del producto. Conforme se hacen cambios es bastante probable que se introduzcan nuevos errores, con lo que se producen picos en la curva de fallos.

Estos errores pueden corregirse, pero el efecto de los sucesivos cambios hace que el producto se aleje cada vez más de las especificaciones iniciales de acuerdo a las cuales fue desarrollado, conteniendo cada vez más errores latentes. Además, con mucha frecuencia se solicita - y se emprende - un nuevo cambio antes de haber conseguido corregir todos los errores producidos por el cambio anterior.

Por estas razones, el nivel estacionario que se consigue después de un cambio es algo superior al que el que había antes de efectuarlo (figura 1.3.), degradándose poco a poco el funcionamiento del sistema. Podemos decir entonces que el software no se estropea, pero se deteriora.

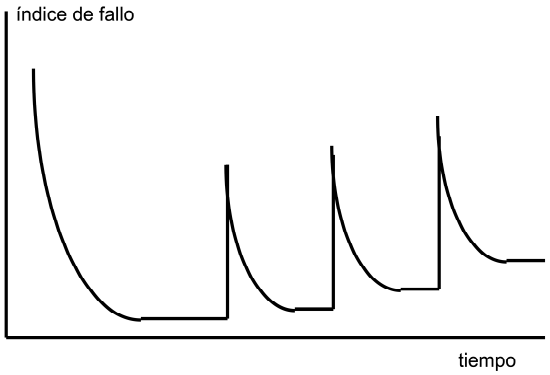


Figura 1.3. Curva real de fallos del software

Además, cuando un componente software se deteriora, no podemos sustituirlo por otro, como en el caso del hardware: no existen piezas de repuesto. Cada fallo del software indica un fallo en el diseño o en el proceso mediante el cual se transformó el diseño en código máquina ejecutable. La solución no es sustituir el componente defectuoso por otro (que sería idéntico y contendría los mismos errores) sino un nuevo diseño y desarrollo del producto. Por tanto, el mantenimiento del software tiene una complejidad mucho mayor que el mantenimiento del hardware.

3.- La mayoría del software se construye a medida.

El diseño de hardware se realiza, en gran medida, a base de componentes digitales existentes, cuyas características se comprueban en un catálogo y que han sido exhaustivamente probados por el fabricante y los usuarios anteriores. Estos componentes cumplen unas especificaciones claras y tienen unas interfaces definidas.

El caso del software es bien distinto. No existen catálogos de componentes, y aunque determinados productos como sistemas operativos, editores, entornos de ventanas y bases de datos se venden en grandes ediciones, la mayoría del software se fabrica a medida, siendo la reutilización muy baja. Se puede comprar software ya desarrollado, pero sólo como unidades completas, no como componentes que pueden ser reensamblados para construir nuevos programas. Esto hace que el impacto de los costes de ingeniería sobre el producto final sea muy elevado, al dividirse entre un número de unidades producidas muy pequeño.

Se ha escrito mucho sobre reutilización del software, y han sido muchos los intentos para conseguir aumentar el nivel de reutilización, normalmente con poco éxito. Uno de los ejemplos de reutilización más típicos, que ha venido usándose desde hace tiempo, son las bibliotecas. Durante

los años sesenta se empezaron a desarrollar bibliotecas de subrutinas científicas, reutilizables en una amplia gama de aplicaciones científicas y de ingeniería. La mayor parte de los lenguajes modernos incluyen bibliotecas de este tipo, así como otras para facilitar la entrada/salida y más recientemente los entornos de ventanas. Sin embargo esta aproximación no puede aplicarse fácilmente a otros problemas también de uso muy frecuente, como puede ser la búsqueda de un elemento en una estructura de datos, debido a la gran variación que existe en cuanto a la organización interna de estas estructuras y en la composición de los datos que contienen. Existen algoritmos para resolver estos problemas, pero no queda más remedio que programarlos una y otra vez, adaptándolos a cada situación particular.

Un nuevo intento de conseguir la reutilización se produjo con la utilización de técnicas de programación estructurada y modular. Sin embargo, se dedica por lo general poco esfuerzo al diseño de módulos lo suficientemente generales para ser reutilizables, y en todo caso, no se documentan ni se difunden todo lo que sería necesario para extender su uso, con lo que la tendencia habitual es diseñar y programar módulos muy semejantes una y otra vez. La programación estructurada permite diseñar programas con una estructura más clara, y que, por tanto, sean más fáciles de entender. Esta estructura interna más clara y estudiada, permite la reutilización de módulos dentro de los programas, o incluso dentro del proyecto que se está desarrollando, pero la reutilización de código en proyectos diferentes es muy baja.

La última tendencia para conseguir la reutilización es el uso de técnicas orientadas a objetos, que permiten la programación por especialización. Los objetos, que encapsulan tanto datos como los procedimientos que los manejan - los métodos -, haciendo los detalles de implementación invisibles e irrelevantes a quien los usa, disponen de interfaces claras, los errores cometidos en su desarrollo pueden ser depurados sin que esto afecte a la corrección de otras partes del código y pueden ser heredados y reescritos parcialmente, haciendo posible su reutilización aún en situaciones no contempladas en el diseño inicial.

1.4.- Aplicaciones del software.

El software puede aplicarse a numerosas situaciones del mundo real. En primer lugar, a todos aquellos problemas para los que se haya establecido un conjunto específico de acciones que lleven a su resolución (esto es, un algoritmo). En estos casos, utilizaremos lenguajes de programación procedimentales para implementar estos algoritmos. También puede aplicarse a situaciones en las que el problema puede describirse formalmente, por lo general en forma recursiva. En estos casos no necesitamos describir el método de resolución, es decir *cómo se resuelve el problema*, sino que bastará con describir en problema en sí, indicando cuál es la solución deseada, y utilizaremos lenguajes declarativos para ello. También puede aplicarse a problemas que los humanos resolvemos utilizando multitud de reglas heurísticas posiblemente

contradictorias, para lo cual utilizaremos un sistema experto e incluso para problemas de los cuales no tenemos una idea clara de cómo se resuelven, pero de los que conocemos cuál es la solución apropiada para algunos ejemplos de los datos de entrada. En este caso utilizaremos redes neuronales.

En cualquier caso, es difícil establecer categorías genéricas significativas para las aplicaciones del software. Conforme aumenta la complejidad del mismo se hace más complicado establecer compartimentos nítidamente separados. No obstante la siguiente clasificación ha venido aceptándose tradicionalmente:

Software de sistemas.

Está formado por todos aquellos programas cuya finalidad es servir al desarrollo o al funcionamiento de otros programas. Estos programas son muy variados: editores, compiladores, sistemas operativos, entornos gráficos, programas de telecomunicaciones, etc. pero se caracterizan por estar muy próximos al hardware, por ser utilizados concurrentemente por numerosos usuarios y por tratarse de programas de amplia difusión, no estando diseñados normalmente a medida. Esto permite un mayor esfuerzo en su diseño y optimización, pero también les obliga a ser muy fiables, cumpliendo estrictamente las especificaciones para las que fueron creados.

Software de tiempo real.

Esta formado por todos aquellos programas que miden, analizan y controlan los sucesos del mundo real a medida que ocurren, debiendo reaccionar de forma correcta a los estímulos de entrada en un tiempo máximo prefijado. Deben, por tanto, cumplir unos requisitos temporales muy estrictos y, dado que los procesos que controlan pueden ser potencialmente peligrosos, tienen que ser fiables y tolerantes a fallos. Por otro lado, no suelen ser muy complejos y precisan de poca interacción con el usuario.

Software de gestión.

El procesamiento de información de gestión constituye, casi desde los inicios de la informática la mayor de las áreas de aplicación de los ordenadores. Estos programas utilizan grandes cantidades de información almacenadas en bases de datos con objeto de facilitar las transacciones comerciales o la toma de decisiones. Además de las tareas convencionales de procesamiento de datos, en las que el tiempo de procesamiento no es crítico y los errores pueden ser corregidos a posteriori, incluyen programas interactivos que sirven de soporte a transacciones comerciales.

Software científico y de ingeniería.

Otro de los campos clásicos de aplicación de la informática. Se encarga de realizar complejos cálculos sobre datos numéricos de todo tipo. En este caso la corrección y exactitud de las operaciones que realizan es uno de los requisitos básicos que deben de cumplir.

El campo del software científico y de ingeniería se ha visto ampliado últimamente con el desarrollo de los sistemas de diseño, ingeniería y fabricación asistida por ordenador (CAD, CAE y CAM), los simuladores gráficos y otras aplicaciones interactivas que lo acercan más al software de tiempo real e incluso al software de sistemas.

Software de ordenadores personales.

El uso de ordenadores personales y de uso doméstico se ha generalizado a lo largo de la pasada década. Aplicaciones típicas son los procesadores de textos, las hojas de cálculo, bases de datos, aplicaciones gráficas, juegos, etc. Son productos de amplia difusión orientados a usuarios no profesionales, por lo que entre sus requisitos se encuentran la facilidad de uso y el bajo coste.

Software empotrado.

Software empotrado es aquel que va instalado en otros productos industriales, como por ejemplo la electrónica de consumo, dotando a estos productos de un grado de inteligencia cada vez mayor. Se aplica a todo tipo de productos, desde un vídeo doméstico hasta un misil con cabeza atómica, pasando por algunos sistemas de control de los automóviles, y realiza funciones muy diversas, que pueden ir desde complicados cálculos en tiempo real a sencillas interacciones con el usuario facilitando el manejo del aparato que los incorpora. Comparten características con el software de sistemas, el software de tiempo real, el software de ingeniería y científico y el software de ordenadores personales.

Software de inteligencia artificial.

El software basado en lenguajes procedimentales es útil para realizar de forma rápida y fiable operaciones que para el ser humano son tediosas e incluso inabordables. Sin embargo, es difícilmente aplicable a problemas que requieran la aplicación de funciones intelectuales más elevadas, por triviales que nos puedan parecer. El software de inteligencia artificial trata de dar respuesta a estas deficiencias, basándose en el uso de lenguajes declarativos, sistemas expertos y redes neuronales.

Como vemos, el software permite aplicaciones muy diversas, pero en todas ellas podemos encontrar algo en común: el objetivo es que el software desempeñe una determinada función, y

además, debe hacerlo cumpliendo una serie de requisitos. Esos pueden ser muy variados: corrección, fiabilidad, respuesta en un tiempo determinado, facilidad de uso, bajo coste, etc., pero siempre existen y no podemos olvidarnos de ellos a la hora de desarrollar el software.

1.5. Problemas del software.

Hemos hablado de una *crisis del software*. Sin embargo, por crisis entendemos normalmente un estado pasajero de inestabilidad, que tiene como resultado un cambio de estado del sistema o una vuelta al estado inicial, en caso de que se tomen las medidas para superarla. Teniendo en cuenta esto, el software, más que padecer una crisis podríamos decir que padece una enfermedad crónica. Los problemas que surgieron cuando se empezó a desarrollar software de una cierta complejidad siguen existiendo actualmente, sin que se haya avanzado mucho en los intentos de solucionarlos.

Estos problemas son causados por las propias características del software y por los errores cometidos por quienes intervienen en su producción. Entre ellos podemos citar:

La planificación y la estimación de costes son muy imprecisas.

A la hora de abordar un proyecto de una cierta complejidad, sea en el ámbito que sea, es frecuente que surjan imprevistos que no estaban recogidos en la planificación inicial, y como consecuencia de estos imprevistos se producirá una desviación en los costes del proyecto. Sin embargo, en el desarrollo de software lo más frecuente es que la planificación sea prácticamente inexistente, y que nunca se revise durante el desarrollo del proyecto. Sin una planificación detallada es totalmente imposible hacer una estimación de costes que tenga alguna posibilidad de cumplirse, y tampoco se pueden identificar las tareas conflictivas que pueden desviarnos de los costes previstos.

Entre las causas de este problema podemos citar:

- No se recogen datos sobre el desarrollo de proyectos anteriores, con lo que no se acumula experiencia que pueda ser utilizada en la planificación de nuevos proyectos.

- Los gestores de los proyectos no están especializados en la producción de software. Tradicionalmente, los responsables del desarrollo del software han sido ejecutivos de nivel medio y alto sin conocimientos de informática, siguiendo el principio de 'Un buen gestor puede gestionar cualquier proyecto'. Esto es cierto, pero no cabe duda de que también es necesario conocer las características específicas del software, aprender las técnicas que se aplican en su desarrollo y conocer una tecnología que evoluciona continuamente.

La productividad es baja.

Los proyectos software tienen, por lo general, una duración mucho mayor a la esperada. Como consecuencia de esto los costes se disparan y la productividad y los beneficios disminuyen. Uno de los factores que influyen en esto, es la falta de unos propósitos claros o realistas a la hora de comenzar el proyecto. La mayoría del software se desarrolla a partir de una especificaciones ambiguas o incorrectas, y no existe apenas comunicación con el cliente hasta la entrega del producto. Debido a esto son muy frecuentes las modificaciones de las especificaciones sobre la marcha o los cambios de última hora, después de la entrega al cliente. No se realiza un estudio detallado del impacto de estos cambios y la complejidad interna de las aplicaciones crece hasta que se hacen virtualmente imposibles de mantener y cada nueva modificación, por pequeña que sea, es más costosa, y puede provocar el fallo de todo el sistema.

Debido a la falta de documentación sobre cómo se ha desarrollado el producto o a que las sucesivas modificaciones - también indocumentadas - han desvirtuado totalmente el diseño inicial, el mantenimiento de software puede llegar a ser una tarea imposible de realizar, pudiendo llevar más tiempo el realizar una modificación sobre el programa ya escrito que analizarlo y desarrollarlo entero de nuevo.

La calidad es mala.

Como consecuencia de que las especificaciones son ambiguas o incluso incorrectas, y de que no se realizan pruebas exhaustivas, el software contiene numerosos errores cuando se entrega al cliente. Estos errores producen un fuerte incremento de costes durante el mantenimiento del producto, cuando ya se esperaba que el proyecto estuviese acabado. Sólo recientemente se ha empezado a tener en cuenta la importancia de la prueba sistemática y completa, y han empezado a surgir conceptos como la fiabilidad y la garantía de calidad.

El cliente queda insatisfecho.

Debido al poco tiempo e interés que se dedican al análisis de requisitos y a la especificación del proyecto, a la falta de comunicación durante el desarrollo y a la existencia de numerosos errores en el producto que se entrega, los clientes suelen quedar muy poco satisfechos de los resultados. Consecuencia de esto es que las aplicaciones tengan que ser diseñadas y desarrolladas de nuevo, que nunca lleguen a utilizarse o que se produzca con frecuencia un cambio de proveedor a la hora de abordar un nuevo proyecto.

1.6. Definición de Ingeniería del Software.

El desarrollo de sistemas de software complejos no es una actividad trivial, que pueda aobrdarse sin una preparación previa. El considerar que un proyecto de desarrollo de software podía abordarse como cualquier otro ha llevado a una serie de problemas que limitan nuestra capacidad de aprovechar los recursos que el hardware pone a nuestra disposición.

Los problemas tradicionales en el desarrollo de software no van a desaparecer de la noche a la mañana, pero identificarlos y conocer sus causas es el único método que nos puede llevar hacia su solución.

No existe una fórmula mágica para solucionar estos problemas, pero combinando métodos aplicables a cada una de las fases del desarrollo de software, construyendo herramientas para automatizar estos métodos, utilizando técnicas para garantizar la calidad de los productos desarrollados y coordinando todas las personas involucradas en el desarrollo de un proyecto, podremos avanzar mucho en la solución de estos problemas. De ello se encarga la disciplina llamada *Ingeniería del Software*.

Una de las primeras definiciones que se dio de la ingeniería del software es la siguiente:

El establecimiento y uso de principios de ingeniería robustos, orientados a obtener software económico, que sea fiable y funcione de manera eficiente sobre máquinas reales.

La ingeniería del software abarca un conjunto de tres elementos clave: métodos, herramientas y procedimientos, que faciliten al gestor el control el proceso de desarrollo y suministren a los implementadores bases para construir de forma productiva software de alta calidad.

Los **métodos** indican cómo construir técnicamente el software, y abarcan una amplia serie de tareas que incluyen la planificación y estimación de proyectos, el análisis de requisitos, el diseño de estructuras de datos, programas y procedimientos, la codificación, las pruebas y el mantenimiento. Los métodos introducen frecuentemente una notación específica para la tarea en cuestión y una serie de criterios de calidad.

Las **herramientas** proporcionan un soporte automático o semiautomático para utilizar los métodos. Existen herramientas automatizadas para cada una de las fases vistas anteriormente, y sistemas que integran las herramientas de cada fase de forma que sirven para todo el proceso de desarrollo. Estas herramientas se denominan CASE (Computer Assisted Software Engineering).

Los **procedimientos** definen la secuencia en que se aplican los métodos, los documentos que se requieren, los controles que permiten asegurar la calidad y las directrices que permiten a los gestores evaluar los progresos.

Software

(1) instrucciones de ordenador que cuando se ejecutan proporcionan la función y el comportamiento deseado, (2) estructuras de datos que facilitan a los programas manipular adecuadamente la información, y (3) documentos que describen la operación y el uso de los programas.

Características del software.

El software se desarrolla, no se fabrica en sentido estricto.

El software no se estropea.

La mayoría del software se construye a medida.

	Ingeniería	Producción o Desarrollo	Coste unitario / 100 unidades	Coste unitario / 100.000 unidades
Hardware	1000	50 c.u.	60	50.01
Software	1000	2000	30	0.03

Tabla 1.1. Influencia de los costes de ingeniería en el coste total del producto

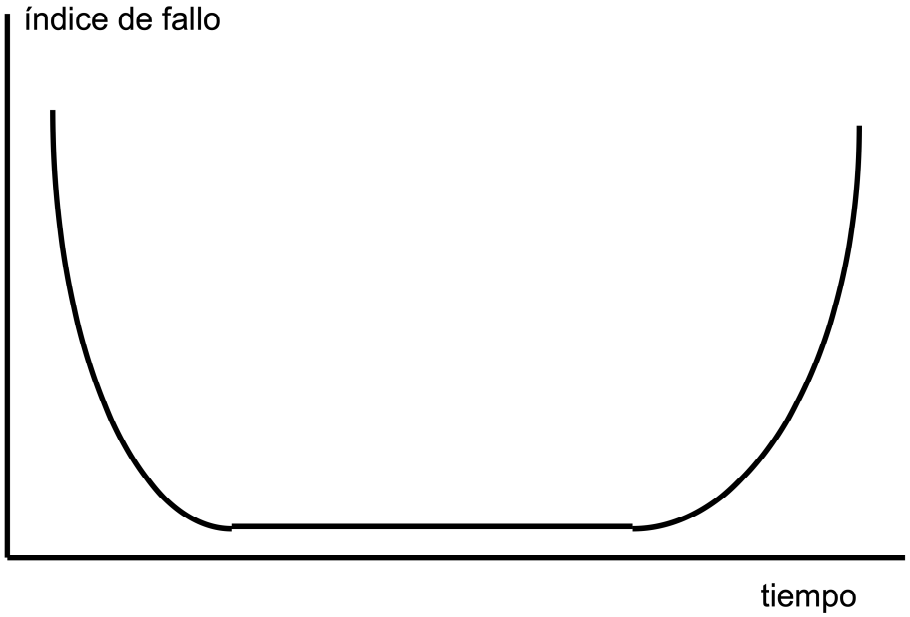


fig. 1.1. Curva de fallos del hardware

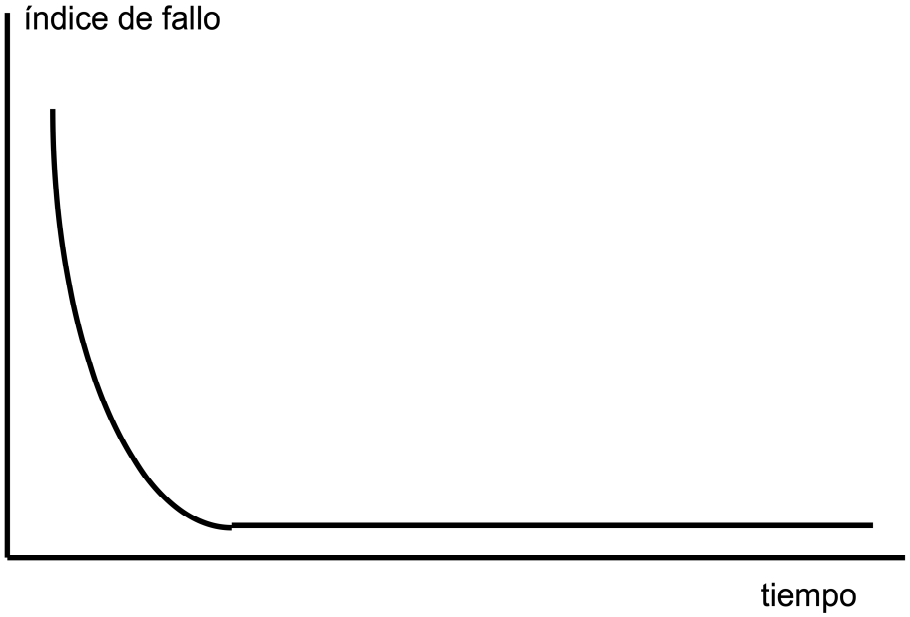


fig. 1.2. Curva ideal de fallos del software

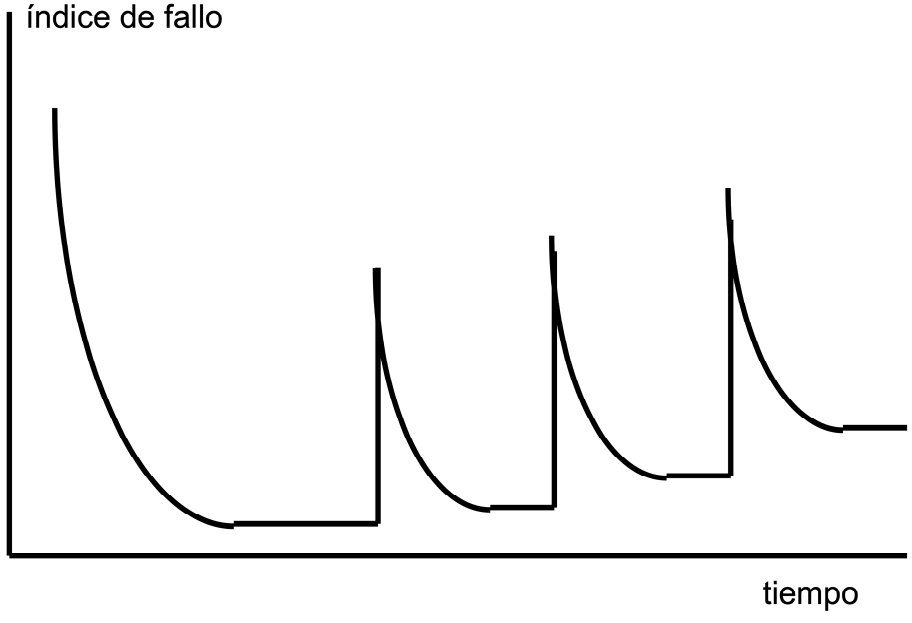


fig. 1.3. Curva real de fallos del software

Aplicaciones del software.

Software de sistemas.

Software de tiempo real.

Software de gestión.

Software científico y de ingeniería.

Software de ordenadores personales.

Software empotrado.

Software de inteligencia artificial.

Problemas del software.

La planificación y la estimación de costes son muy imprecisas.

La productividad es baja.

La calidad es mala.

El cliente queda insatisfecho.

Ingeniería del software.

El establecimiento y uso de principios de ingeniería robustos, orientados a obtener software económico, que sea fiable y funcione de manera eficiente sobre máquinas reales.

- Métodos
- Herramientas
- Procedimientos

Tema 2. El ciclo de vida.

2.1.- Concepto de ciclo de vida.

2.2.- Ciclo de vida en cascada.

2.3.- El modelo contractual.

2.4.- Uso de técnicas de cuarta generación.

2.5.- Construcción de prototipos.

2.6.- El modelo en espiral.

2.7.- Visión genérica de la ingeniería del software.

2.8.- Ingeniería inversa y reingeniería del software.

Tema 2. El ciclo de vida.

2.1.- Concepto de ciclo de vida.

Por ciclo de vida, se entiende la sucesión de etapas por las que pasa el software desde que un nuevo proyecto es concebido hasta que se deja de usar.

Cada una de estas etapas lleva asociada una serie de tareas que deben realizarse, y una serie de documentos (en sentido amplio: software) que serán la salida de cada una de estas fases y servirán de entrada en la fase siguiente.

Existen diversos modelos de ciclo de vida, es decir, diversas formas de ver el proceso de desarrollo de software, y cada uno de ellos va asociado a un paradigma de la ingeniería del software, es decir, a una serie de métodos, herramientas y procedimientos que debemos usar a lo largo de un proyecto. En este tema veremos algunos de los principales modelos de ciclo de vida.

La elección de un paradigma u otro se realiza de acuerdo con la naturaleza del proyecto y de la aplicación, los métodos a usar y los controles y entregas requeridos.

2.2.- El ciclo de vida en cascada (o ciclo de vida clásico).

Este paradigma es el más antiguo de los empleados en la IS y se desarrolló a partir del ciclo convencional de una ingeniería. No hay que olvidar que la IS surgió como copia de otras ingenierías, especialmente de las del hardware, para dar solución a los problemas más comunes que aparecían al desarrollar sistemas de software complejos.

Es un ciclo de vida en sentido amplio, que incluye no sólo las etapas de ingeniería sino toda la vida del producto: las pruebas, el uso (la vida útil del software) y el mantenimiento, hasta que llega el momento de sustituirlo (Figura 2.1)

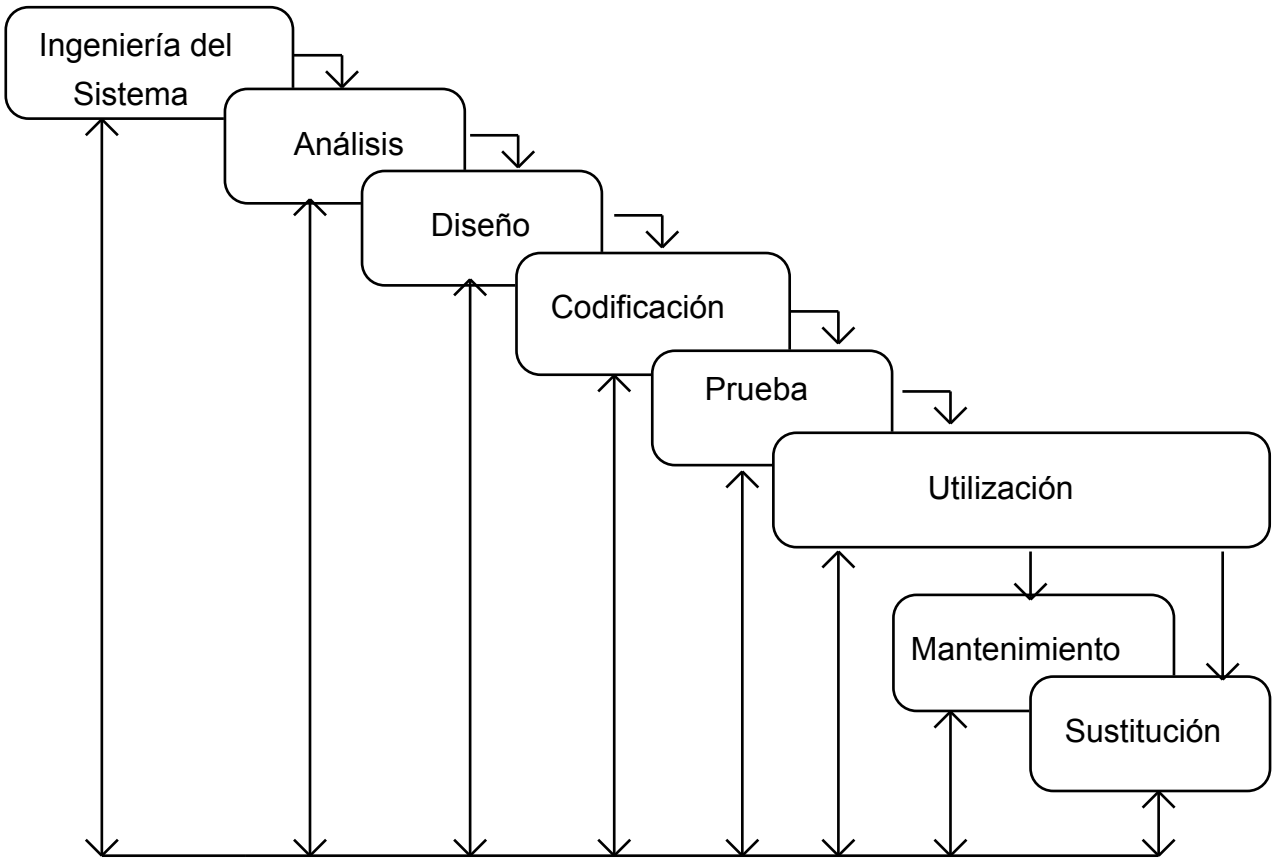


Figura 2.1. Ciclo de vida en cascada.

El ciclo de vida en cascada exige un enfoque sistemático y secuencial del desarrollo de software, que comienza en el nivel de la ingeniería de sistemas y avanza a través de fases secuenciales sucesivas. Estas fases son las siguientes:

Ingeniería y análisis del sistema.

El software es siempre parte de un sistema mayor, por lo que siempre va a interrelacionarse con otros elementos, ya sea hardware, máquinas o personas. Por esto, el primer paso del ciclo de vida de un proyecto consiste en un análisis de las características y el comportamiento del sistema del cual el software va a formar parte. En el caso de que queramos construir un sistema nuevo, por ejemplo un sistema de control, deberemos analizar cuáles son los requisitos y la función del sistema, y luego asignaremos un subconjunto de estos requisitos al software. En el caso de un sistema ya existente (supongamos, por ejemplo, que queremos informatizar una empresa) deberemos analizar el funcionamiento de la misma, - las operaciones que se llevan a cabo en ella -, y asignaremos al software aquellas funciones que vamos a automatizar.

La ingeniería del sistema comprende, por tanto, los requisitos globales a nivel del sistema, así como una cierta cantidad de análisis y de diseño a nivel superior, es decir sin entrar en mucho detalle.

Análisis de requisitos del software.

El análisis de requisitos debe ser más detallado para aquellos componentes del sistema que vamos a implementar mediante software. El ingeniero del software debe comprender cuáles son los datos que se van a manejar, cuál va a ser la función que tiene que cumplir el software, cuáles son los interfaces requeridos y cuál es el rendimiento que se espera lograr.

Los requisitos, tanto del sistema como del software deben documentarse y revisarse con el cliente.

Diseño.

El diseño se aplica a cuatro características distintas del software: la estructura de los datos, la arquitectura de las aplicaciones, la estructura interna de los programas y las interfaces.

El diseño es el proceso que traduce los requisitos en una representación del software de forma que pueda conocerse la arquitectura, funcionalidad e incluso la calidad del mismo antes de comenzar la codificación.

Al igual que el análisis, el diseño debe documentarse y forma parte de la configuración del software (el control de configuraciones es lo que nos permite realizar cambios en el software de forma controlada y no traumática para el cliente).

Codificación.

La codificación consiste en la traducción del diseño a un formato que sea legible para la máquina. Si el diseño es lo suficientemente detallado, la codificación es relativamente sencilla, y puede hacerse - al menos en parte - de forma automática, usando generadores de código.

Podemos observar que estas primeras fases del ciclo de vida consisten básicamente en una traducción: en el análisis del sistema, los requisitos, la función y la estructura de este se traducen a un documento: el análisis del sistema que está formado en parte por diagramas y en parte por descripciones en lenguaje natural. En el análisis de requisitos se profundiza en el estudio del componente software del sistema y esto se traduce a un documento, también formado por diagramas y descripciones en lenguaje natural. En el diseño, los requisitos del software se traducen a una serie de diagramas que representan la estructura del sistema software, de sus datos, de sus programas y de sus interfaces. Por último, en la codificación se traducen estos diagramas de diseño a un lenguaje fuente, que luego se traduce - se compila - para obtener un programa ejecutable.

Prueba.

Una vez que ya tenemos el programa ejecutable, comienza la fase de pruebas. El objetivo es comprobar que no se hayan producido errores en alguna de las fases de traducción anteriores, especialmente en la codificación. Para ello deben probarse todas las sentencias, no sólo los casos normales y todos los módulos que forman parte del sistema.

Utilización.

Una vez superada la fase de pruebas, el software se entrega al cliente y comienza la vida útil del mismo. La fase de utilización se solapa con las posteriores - el mantenimiento y la sustitución - y dura hasta que el software, ya reemplazado por otro, deje de utilizarse.

Mantenimiento.

El software sufrirá cambios a lo largo de su vida útil. Estos cambios pueden ser debidos a tres causas:

- Que, durante la utilización, el cliente detecte errores en el software: los errores latentes.
- Que se produzcan cambios en alguno de los componentes del sistema informático: por ejemplo cambios en la máquina, en el sistema operativo o en los periféricos.
- Que el cliente requiera modificaciones funcionales (normalmente ampliaciones) no contempladas en el proyecto.

En cualquier caso, el mantenimiento supone volver atrás en el ciclo de vida, a las etapas de codificación, diseño o análisis dependiendo de la magnitud del cambio.

El modelo en cascada, a pesar de ser lineal, contiene flujos que permiten la vuelta atrás. Así, desde el mantenimiento se vuelve al análisis, el diseño o la codificación, y también desde cualquier fase se puede volver a la anterior si se detectan fallos. Estas vueltas atrás no son controladas, ni quedan explícitas en el modelo, y este es uno de los problemas que presenta este paradigma

Sustitución.

La vida del software no es ilimitada y cualquier aplicación, por buena que sea, acaba por ser sustituida por otra más amplia, más rápida o más bonita y fácil de usar.

La sustitución de un software que está funcionando por otro que acaba de ser desarrollado es una tarea que hay que planificar cuidadosamente y que hay que llevar a cabo de forma organizada. Es conveniente realizarla por fases, si esto es posible, no sustituyendo todas las aplicaciones de golpe, puesto que la sustitución conlleva normalmente un aumento de trabajo para los usuarios, que tienen que acostumbrarse a las nuevas aplicaciones, y también para los implementadores, que tienen que corregir los errores que aparecen. Es necesario hacer un trasvase de la información que maneja el sistema viejo a la estructura y el formato requeridos por el nuevo. Además, es conveniente mantener los dos sistemas funcionando en paralelo durante algún tiempo para comprobar que el sistema nuevo funcione correctamente y para asegurarnos el funcionamiento normal de la empresa aún en el caso de que el sistema nuevo falle y tenga que volver a alguna de las fases de desarrollo.

La sustitución implica el desarrollo de programas para la interconexión de ambos sistemas, el viejo y el nuevo, y para trasvasar la información entre ambos, evitando la duplicación del trabajo de las personas encargadas del proceso de datos, durante el tiempo en que ambos sistemas funcionen en paralelo.

El ciclo de vida en cascada es el paradigma más antiguo, más conocido y más ampliamente usado en la IS. No obstante, ha sufrido diversas críticas, debido a los problemas que se plantean al intentar aplicarlo a determinadas situaciones. Entre estos problemas están:

- En realidad los proyectos no siguen un ciclo de vida estrictamente secuencial como propone el modelo. Siempre hay iteraciones. El ejemplo más típico es la fase de mantenimiento, que implica siempre volver a alguna de las fases anteriores, pero también es muy frecuente en que una fase, por ejemplo el diseño, se detecten errores que obliguen a volver a la fase anterior, el análisis.

- Es difícil que se puedan establecer inicialmente todos los requisitos del sistema. Normalmente los clientes no tienen conocimiento de la importancia de la fase de análisis o bien no han pensado en todo detalle que es lo que quieren que haga el software. Los requisitos se van aclarando y refinando a lo largo de todo el proyecto, según se plantean dudas concretas en el diseño o la codificación. Sin embargo, el ciclo de vida clásico requiere la definición inicial de todos los requisitos y no es fácil acomodar en él las incertidumbres que suelen existir al comienzo de todos los proyectos.

- Hasta que se llega a la fase final del desarrollo: la codificación, no se dispone de una versión operativa del programa. Como la mayor parte de los errores se detectan cuando el cliente puede probar el programa no se detectan hasta el final del proyecto, cuando son más costosos de corregir y más prisa (y más presiones) hay por que el programa se ponga definitivamente en marcha.

Todos estos problemas son reales, pero de todas formas es mucho mejor desarrollar software siguiendo el modelo de ciclo de vida en cascada que hacerlo sin ningún tipo de guías. Además, este modelo describe una serie de pasos genéricos que son aplicables a cualquier otro paradigma, refiriéndose la mayor parte de las críticas que recibe a su carácter secuencial.

2.3.- El modelo contractual.

El modelo de ciclo de vida en cascada, no nos indica nada sobre la relación entre las diversas partes involucradas en el desarrollo de software, ni tampoco sobre los documentos que sirven de entrada y salida de cada una de las fases del proceso. Por ello, se ha propuesto el modelo contractual, que no es más que una extensión más detallada del modelo clásico.

Fig. 2.2. El modelo contractual (fotocopia)

En este modelo, cada fase de desarrollo, ya sea el análisis, el diseño, la implementación, etc., es contemplada como el sujeto de un contrato entre dos partes, llamadas respectivamente el proveedor y el cliente. La finalización de cada fase se produce con la firma, por parte del cliente y del proveedor, de un documento contractual, (en sentido amplio: software) producido por el proveedor a partir de una solicitud de servicios que el cliente ha facilitado al inicio de la fase.

En cada fase existen, por tanto, un proveedor y un cliente. El cliente realiza una petición de servicios, expresando sus necesidades. A partir de esta petición, el proveedor redacta un contrato de servicio, cuyos detalles se discuten con el cliente. Finalmente, se firma el contrato y se pasa a la fase siguiente, en la que el proveedor se convierte en cliente (en una especie de subcontratación).

Así, en la fase de análisis, el cliente que ha encargado el proyecto actúa de cliente y el analista actúa de proveedor, y esta fase termina con un acuerdo de ambos respecto a que la especificación del sistema, redactada a partir de la definición informal de requisitos realizada por el cliente, es satisfactoria y puede servir de contrato para formalizar la transacción entre ambos.

En la fase de diseño, el analista actúa de cliente, y proporciona como petición de servicio la especificación. A su vez, esta fase acaba cuando el diseño realizado por el diseñador es aceptado como satisfactorio y conforme con los términos del contrato, por el analista. Y así sucesivamente.

El modelo contractual, tiene la ventaja de dar un mayor rigor a la transición entre cada una de las fases del ciclo de vida y permite determinar quién es el responsable en caso de que surjan problemas. Así, el cliente es responsable de los incrementos de coste producidos por un cambio en los requisitos, puesto que pretende que se realice un servicio que no estaba previsto en el contrato, y el proveedor es responsable de los gastos ocasionados (o de aceptar una disminución en el precio) si el producto finalmente entregado no cumple todas las condiciones del contrato, es decir, si contiene errores.

2.4.- Uso de técnicas de cuarta generación.

Por *técnicas de cuarta generación* se entiende un conjunto muy diverso de métodos y herramientas que tienen por objeto el facilitar el desarrollo de software. Pero todos ellos tienen algo en común: facilitan al que desarrolla el software el especificar algunas características del mismo a alto nivel. Luego, la herramienta genera automáticamente el código fuente a partir de esta especificación.

Los tipos más habituales de generadores de código cubren uno o varios de los siguientes aspectos:

- **Acceso a bases de datos utilizando lenguajes de consulta de alto nivel** (derivados normalmente de SQL). Con ello no es necesario conocer la estructura de los ficheros o tablas ni de sus índices.

- **Generación de código**. A partir de una especificación de los requisitos se genera automáticamente toda la aplicación.

- **Generación de pantallas**. Permiten diseñar la pantalla dibujándola directamente, incluyendo además el control del cursor y la gestión de errores de los datos de entrada.

- **Gestión de entornos gráficos.**

- **Generación de informes.** (de forma similar a las pantallas).

Esta generación automática permite reducir la duración de las fases del ciclo de vida clásico, especialmente la fase de codificación, quedando el ciclo de vida según se indica en la figura 2.3.

Al igual que en otros paradigmas, el proceso comienza con la recolección de requisitos, que pueden ser traducidos directamente a código fuente usando un generador de código. Sin embargo el problema es el mismo que se plantea en el ciclo de vida clásico: es muy difícil que se puedan establecer todos los requisitos desde el comienzo: el cliente puede no estar seguro de lo que necesita, o, aunque lo sepa, puede ser difícil expresarlo de la forma en que precisa la herramienta de cuarta generación para poder entenderla.

Si la especificación es pequeña, podemos pasar directamente del análisis de requisitos a la generación automática de código, sin realizar ningún tipo de diseño. Pero si la aplicación es grande, se producirán los mismos problemas que si no usamos técnicas de cuarta generación: mala calidad, dificultad de mantenimiento y poca aceptación por parte del cliente). Es necesario, por tanto,

realizar un cierto grado de diseño (al menos lo que hemos llamado una estrategia de diseño, puesto que el propio generador se encarga de parte de los detalles del diseño tradicional: descomposición modular, estructura lógica y organización de los ficheros, etc.).

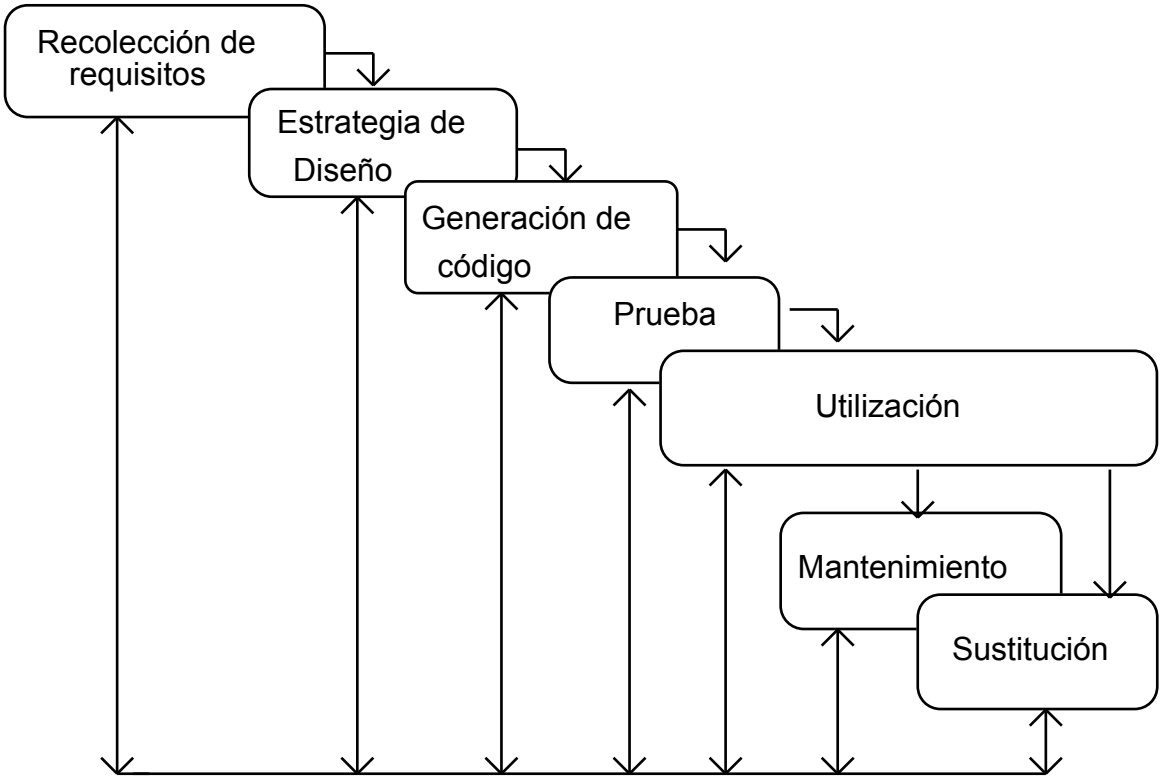


Figura 2.3. Ciclo de vida usando técnicas de cuarta generación.

Las herramientas de cuarta generación se encargan también de producir automáticamente la documentación del código generado, pero esta documentación es de ordinario muy parca y, por ello, difícil de seguir. Es necesario completarla hasta obtener una documentación con sentido.

Con respecto a las pruebas, podemos suponer (aunque nunca hay que fiarse) que el código generado es correcto y acorde con la especificación, y que no contiene los típicos errores de la codificación manual. Pero en cualquier caso es necesaria la fase de pruebas, en primer lugar para comprobar la eficiencia del código generado (la generación automática de los accesos a bases puede producir código muy eficiente cuando el volumen de información es grande (p.ej.: las distintas formas de relacionar tablas en SQL), también para detectar los errores en la especificación a partir de la cual se generó el código, y, por último, para que el cliente compruebe si el producto final satisface sus necesidades.

El resto de las fases del ciclo de vida usando estas técnicas es igual a las del paradigma del ciclo de vida en cascada, del que este no es más que una adaptación a las nuevas herramientas de producción de software.

Como conclusión, podemos decir que, mediante el uso de técnicas de cuarta generación no se han obtenido (afortunadamente) los resultados previstos cuando estas herramientas comenzaron a desarrollarse a principios de los ochenta (estos resultados incluían la desaparición de la codificación manual y con ello de los programadores, e incluso de los analistas, al poder encargarse el propio cliente 'con unos pequeños conocimientos técnicos' de manejar el generador), puesto que los avances en procesamiento de lenguaje natural (siempre ambiguo) no han sido (ni se espera que sean en un futuro próximo) demasiado grandes ni se han desarrollado lenguajes formales de especificación con la potencia expresiva necesaria.

Sin embargo, estas herramientas consiguen reducir el tiempo de desarrollo de software, eliminando las tareas más repetitivas y tediosas (ej. control de la entrada/salida por terminal) y aumentan la productividad de los programadores, por lo que son ampliamente utilizadas en la actualidad, especialmente si nos referimos a el acceso a bases de datos, la gestión de la entrada/salida por terminal y la generación de informes, y forman parte de muchos de los lenguajes de programación que se usan actualmente, sobre todo en el campo del software de gestión (ej.: Informix, Natural).

No obstante, entre las críticas más habituales están:

- **No son más fáciles de utilizar que los lenguajes de tercera generación**. En concreto, muchos de los lenguajes de especificación que utilizan pueden considerarse como lenguajes de programación, de un nivel algo más alto que los anteriores, pero que no logran prescindir de la codificación en sí, sino que simplemente la disfrazan de 'especificación'.

- **El código fuente que producen es ineficiente**.(el ejemplo de antes de SQL). Al estar generado automáticamente no pueden hacer uso de los *trucos* habituales para aumentar el rendimiento, que se basan en el buen conocimiento de cada caso particular. Esta crítica podría aplicarse a cualquier lenguaje de programación con respecto al ensamblador (los programas codificados en ensamblador siempre serán más rápidos y más pequeños que los generados por cualquier compilador), pero la reducción de los tiempos de desarrollo y el continuo aumento de la potencia de cálculo de los ordenadores compensan ampliamente esta menor eficiencia (salvo en excepciones).

- **Sólo son aplicables al software de gestión**. Esto es cierto, la mayoría de las herramientas de cuarta generación están orientadas a la generación de informes a partir de grandes bases de datos, pero últimamente están surgiendo herramientas que generan *esquemas de código* para aplicaciones de ingeniería y de tiempo real.

2.5.- Construcción de prototipos.

Dos de las críticas que se hacían al modelo de ciclo de vida en cascada eran que es difícil tener claros todos los requisitos del sistema al inicio del proyecto, y que no se dispone de una versión operativa del programa hasta las fases finales del desarrollo, lo que dificulta la detección de errores y deja también para el final el descubrimiento de los requisitos inadvertidos en las fases de análisis. Para paliar estas deficiencias se ha propuesto un modelo de ciclo de vida basado en la construcción de prototipos.

En primer lugar, hay que ver si el sistema que tenemos que desarrollar es un buen candidato a utilizar el paradigma de ciclo de vida de construcción de prototipos o al modelo en espiral. En general, cualquier aplicación que presente mucha interacción con el usuario, o que necesite algoritmos que puedan construirse de manera evolutiva, yendo de lo mas general a lo más específico es una buena candidata. No obstante, hay que tener en cuenta la complejidad: si la aplicación necesita que se desarrolle una gran cantidad de código para poder tener un prototipo que enseñar al usuario, las ventajas de la construcción de prototipos se verán superadas por el esfuerzo de desarrollar un prototipo que al final habrá que desechar o modificar mucho. También hay que tener en cuenta la disposición del cliente para probar un prototipo y sugerir modificaciones de los requisitos. Puede ser que el cliente 'no tenga tiempo para andar jugando' o 'no vea las ventajas de este método de desarrollo'.

También es conveniente construir prototipos para probar la eficiencia de los algoritmos que se van a implementar, o para comprobar el rendimiento de un determinado componente del sistema, por ejemplo, una base de datos o el soporte hardware, en condiciones similares a las que existirán durante la utilización del sistema. Es bastante frecuente que el producto de ingeniería desarrollado presente un buen rendimiento durante la fase de pruebas realizada por los ingenieros antes de entregarlo al cliente (pruebas que se realizarán normalmente con unos pocos registros en la base de datos o un único terminal conectado al sistema), pero que sea muy ineficiente, o incluso inviable, a la hora de almacenar o procesar el volumen real de información que debe manejar el cliente. En estos casos, la construcción de un prototipo de parte del sistema y la realización de pruebas de rendimiento, sirven para decidir, antes de empezar la fase de diseño, cuál es el modelo más adecuado de entre la gama disponible para el soporte hardware o cómo deben hacerse los accesos a la base de datos para obtener buenas respuestas en tiempo cuando la aplicación esté ya en funcionamiento.

En otros casos, el prototipo servirá para modelar y poder mostrar al cliente cómo va a realizarse la E/S de datos en la aplicación, de forma que éste pueda hacerse una idea de como va a ser el sistema final, pudiendo entonces detectar deficiencias o errores en la especificación aunque el modelo no sea más que una cáscara vacía.

Según esto un prototipo puede tener alguna de las tres formas siguientes:

- un prototipo, en papel o ejecutable en ordenador, que describa la interacción hombre-máquina y los listados del sistema.
- un prototipo que implemente algún(os) subconjunto(s) de la función requerida, y que sirva para evaluar el rendimiento de un algoritmo o las necesidades de capacidad de almacenamiento y velocidad de cálculo del sistema final.
- un programa que realice en todo o en parte la función deseada pero que tenga características (rendimiento, consideración de casos particulares, etc.) que deban ser mejoradas durante el desarrollo del proyecto.

La secuencia de tareas del paradigma de construcción de prototipos puede verse en la figura 2.4.

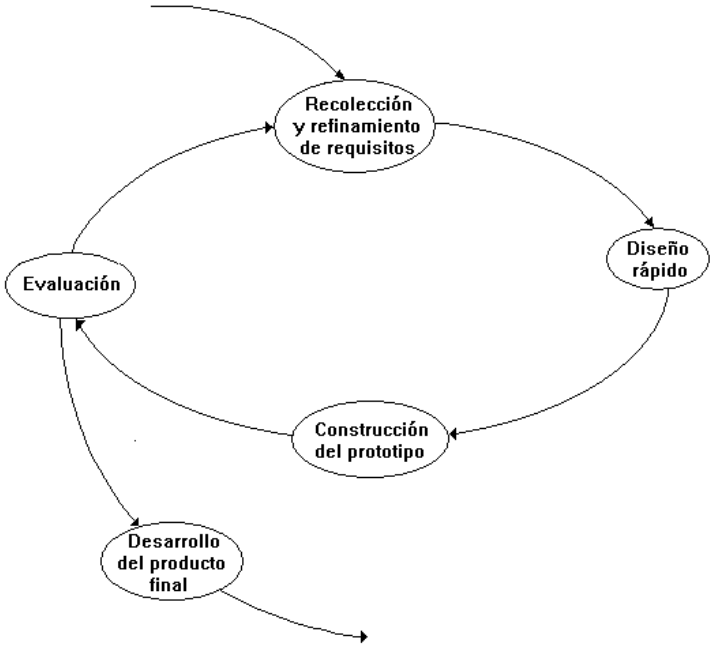


Fig. 2.4. Construcción de prototipos.

Si se ha decidido construir un prototipo, lo primero que hay que hacer es realizar un modelo del sistema, a partir de los requisitos que ya conozcamos. En este caso no es necesario realizar una definición completa de los requisitos, pero sí es conveniente determinar al menos las áreas donde será necesaria una definición posterior más detallada.

Luego se procede a diseñar el prototipo. Se tratará de un diseño rápido, centrado sobre todo en la arquitectura del sistema y la definición de la estructura de las interfaces más que en aspectos procedimentales de los programas: nos fijaremos más en la forma y en la apariencia que en el contenido.

A partir del diseño construiremos el prototipo. Existen herramientas especializadas en generar prototipos ejecutables a partir del diseño. Otra opción sería utilizar técnicas de cuarta generación. La posibilidad más reciente es consiste en el uso de especificaciones formales, que

últimamente tienden al desarrollo de entornos interactivos (ej. OBJ) que faciliten el desarrollo incremental de especificaciones y permitan la prueba de estas especificaciones.

En cualquier caso, el objetivo es siempre que la codificación sea rápida, aunque sea en detrimento de la calidad del software generado.

Una vez listo el prototipo, hay que presentarlo al cliente para que lo pruebe y sugiera modificaciones. En este punto el cliente puede ver una implementación de los requisitos que ha definido inicialmente y sugerir las modificaciones necesarias en las especificaciones para que satisfagan mejor sus necesidades.

A partir de estos comentarios del cliente y los cambios que se muestren necesarios en los requisitos, se procederá a construir un nuevo prototipo y así sucesivamente hasta que los requisitos queden totalmente formalizados, y se pueda entonces empezar con el desarrollo del producto final.

Por tanto, el prototipado es una técnica que sirve fundamentalmente para la fase de análisis de requisitos, pero lleva consigo la obtención de una serie de subproductos que son útiles a lo largo del desarrollo del proyecto:

- Gran parte del trabajo realizado durante la fase de diseño rápido (especialmente la definición de pantallas e informes) puede ser utilizada durante el diseño del producto final. Además, tras realizar varias vueltas en el ciclo de construcción de prototipos, el diseño del mismo se parece cada vez más al que tendrá el producto final.

- Durante la fase de construcción de prototipos será necesario codificar algunos componentes del software que también podrán ser reutilizados en la codificación del producto final, aunque deban de ser optimizados en cuanto a corrección o velocidad de procesamiento.

No obstante, hay que tener en cuenta que el prototipo no es el sistema final, puesto que normalmente apenas es utilizable. Será demasiado lento, demasiado grande, inadecuado para el volumen de datos necesario, contendrá errores (debido al diseño rápido), será demasiado general (sin considerar casos particulares, que debe tener en cuenta el sistema final) o estará codificado en un lenguaje o para una máquina inadecuadas, o a partir de componentes software previamente existentes. No hay que preocuparse de haber desperdiciado tiempo o esfuerzos construyendo prototipos que luego habrán de ser desechados, si con ello hemos conseguido tener más clara la especificación del proyecto, puesto que el tiempo perdido lo ahorraremos en las fases siguientes, que podrán realizarse con menos esfuerzo y en las que se cometerán menos errores que nos obliguen a volver atrás en el ciclo de vida.

Hay que tener en cuenta que un análisis de requisitos incorrecto o incompleto, cuyos errores y deficiencias se detecten a la hora de las pruebas o tras entregar el software al cliente, nos obligará

a repetir de nuevo las fases de análisis, diseño y codificación, que habíamos realizado cuidadosamente, pensando que estabamos desarrollando el producto final. Al tener que repetir estas fases, sí que estaremos desechando una gran cantidad de trabajo, normalmente muy superior al esfuerzo de construir un prototipo basándose en un diseño rápido, en la reutilización de trozos de software preexistentes y en herramientas de generación de código para informes y manejo de ventanas.

Uno de los problemas que suelen aparecer siguiendo el paradigma de construcción de prototipos, es que con demasiada frecuencia el prototipo pasa a ser parte del sistema final, bien sea por presiones del cliente, que quiere tener el sistema funcionando lo antes posible o bien porque los técnicos se han acostumbrado a la máquina, el sistema operativo o el lenguaje con el que se desarrolló el prototipo. Se olvida aquí que el prototipo ha sido construido de forma acelerada, sin tener en cuenta consideraciones de eficiencia, calidad del software o facilidad de mantenimiento, o que las elecciones de lenguaje, sistema operativo o máquina para desarrollarlo se han hecho basándose en criterios como el mejor conocimiento de esas herramientas por parte los técnicos que en que sean adecuadas para el producto final.

El utilizar el prototipo en el producto final conduce a que éste contenga numerosos errores latentes, sea ineficiente, poco fiable, incompleto o difícil de mantener. En definitiva a que tenga poca calidad, y eso es precisamente lo que queremos evitar aplicando la ingeniería del software.

2.6.- El modelo en espiral.

El modelo en espiral combina las principales ventajas del modelo de ciclo de vida en cascada y del modelo de construcción de prototipos. Proporciona un modelo *evolutivo* para el desarrollo de sistemas de software complejos, mucho más realista que el ciclo de vida clásico, y permite la utilización de prototipos en cualquier etapa de la evolución del proyecto. Este es un modelo relativamente nuevo (fue propuesto en 1988) y no ha sido tan usado como los dos anteriores, aunque es de esperar que se extienda cada vez más.

Otra característica de este modelo es que incorpora en el ciclo de vida el análisis de riesgos. (que ya veremos lo que es). Los prototipos se utilizan como mecanismo de reducción del riesgo, permitiendo finalizar el proyecto antes de haberse embarcado en el desarrollo del producto final, si el riesgo es demasiado grande.

Fig. 2.5. El modelo en espiral.

El modelo en espiral define cuatro tipos de actividades, y representa cada uno de ellos en un cuadrante:

Planificación.

Consiste en determinar los objetivos del proyecto, las posibles alternativas y las restricciones. Esta fase equivale a la de recolección de requisitos del ciclo de vida clásico e incluye además la planificación de las actividades a realizar en cada iteración.

Análisis de riesgo.

Una de las actividades de la planificación de proyectos (que se verá en IS: Proyectos) es el análisis de riesgos. El desarrollo de cualquier proyecto complejo lleva implícito una serie de riesgos: unos relativos al propio proyecto (los riesgos que pueden hacer que el proyecto fracase) y otros relativos a las decisiones que deben tomarse durante su desarrollo (los riesgos asociados a que una de estas decisiones sea errónea).

El análisis de riesgos consiste en cuatro actividades principales:

- **Identificar los riesgos**. Pueden ser: riesgos del proyecto (presupuestarios, de organización, del cliente. etc.), riesgos técnicos (problemas de diseño, codificación, mantenimiento), riesgos del negocio (riesgos de mercado: que se adelante la competencia o que el producto no se venda bien).

- **Estimación de riesgos**. Consiste en evaluar, para cada riesgo identificado, la probabilidad de que ocurra y las consecuencias, es decir, el coste que tendrá en caso de que ocurra.

- **Evaluación de riesgos**. Consiste en establecer unos niveles de referencia para el incremento de coste, de duración del proyecto y para la degradación de la calidad que si se superan harán que se interrumpa el proyecto. Luego hay que relacionar cuantitativamente cada uno de los riesgos con estos niveles de referencia, de forma que en cualquier momento del proyecto podamos calcular si hemos superado alguno de los niveles de referencia.

- **Gestión de riesgos**. Consiste en supervisar el desarrollo del proyecto, de forma que se detecten los riesgos tan pronto como aparezcan, se intenten minimizar sus daños y exista un apoyo previsto para las tareas críticas (aquéllas que más riesgo encierran).

Ingeniería.

Consiste en el desarrollo del sistema o de un prototipo del mismo.

Evaluación del cliente.

Consiste en la valoración, por parte del cliente, de los resultados de la ingeniería.

En la primera iteración se definen los requisitos del sistema y se realiza la planificación inicial del mismo. A continuación se analizan los riesgos del proyecto, basándonos en los requisitos iniciales y se procede a construir un prototipo del sistema. Entonces el cliente procede a evaluar el prototipo y con sus comentarios, se procede a refinar los requisitos y a reajustar la planificación inicial, volviendo a empezar el ciclo.

En cada una de las iteraciones se realiza el análisis de riesgos, teniendo en cuenta los requisitos y la reacción del cliente ante el último prototipo. Si los riesgos son demasiado grandes se terminará el proyecto, aunque lo normal es que se siga avanzando a lo largo de la espiral.

Con cada iteración, se construyen sucesivas versiones del software, cada vez más completas, y aumenta la duración de las operaciones del cuadrante de ingeniería, obteniéndose al final el sistema de ingeniería completo.

La diferencia principal con el modelo de construcción de prototipos, es que en éste los prototipos se usan para perfilar y definir los requisitos. Al final, el prototipo se desecha y comienza el desarrollo del software siguiendo el ciclo clásico. En el modelo en espiral, en cambio, los prototipos son sucesivas versiones del producto, cada vez más detalladas (el último es el producto en sí) y constituyen el esqueleto del producto de ingeniería. Por tanto deben construirse siguiendo estándares de calidad.

2.7.- Visión genérica de la ingeniería del software.

Independientemente del paradigma que se utilice, del área de aplicación, y del tamaño y la complejidad del proyecto, el proceso de desarrollo de software contiene siempre una serie de fases genéricas, existentes en todos los paradigmas. Estas fases son la definición, el desarrollo y el mantenimiento.

Definición.

La fase de definición se centra en el **qué**. Durante esta fase, se intenta identificar:

- qué información es la que tiene que ser procesada.
- qué función y rendimiento son los que se esperan.
- qué restricciones de diseño existen.
- qué interfaces deben utilizarse.
- qué lenguaje de programación, sistema operativo y soporte hardware van a ser utilizados.
- qué criterios de validación se necesitan para conseguir que el sistema final sea correcto.

Aunque los pasos concretos dependen del modelo de ciclo de vida utilizado, en general se realizarán tres tareas específicas:

Análisis del sistema.

Como ya hemos visto, el análisis del sistema define el papel de cada elemento de un sistema informático, estableciendo cuál es el papel del software dentro de ese sistema.

Análisis de requisitos del software.

El análisis del sistema proporciona el ámbito del software, su relación con el resto de componentes del sistema, pero antes de empezar a desarrollar es necesario hacer una definición más detallada de la función del software.

Existen dos formas de realizar el análisis y refinamiento de los requisitos del software. Por una parte, se puede hacer un análisis formal del ámbito de la información para establecer modelos del fujo y la estructura de la información. Luego se amplían unos modelos para convertirlos en una especificación del software. La otra alternativa consiste en construir un prototipo del software, que será evaluado por el cliente para intentar consolidar los requisitos. Los requisitos de rendimiento y las limitaciones de recursos se traducen en directivas para la fase de diseño.

El análisis y definición de los requisitos es una tarea que debe llevarse a cabo conjuntamente por el desarrollador de software y por el cliente. La especificación de requisitos del software es el documento que se produce como resultado de esta etapa.

Planificación del proyecto software.

Durante esta etapa se lleva a cabo el análisis de riesgos, se definen los recursos necesarios para desarrollar el software y se establecen las estimaciones de tiempo y costes. El propósito de

esta etapa de planificación es proporcionar una indicación preliminar de la viabilidad del proyecto de acuerdo con el coste y con la agenda que se hayan establecido. Posteriomente, la gestión del proyecto durante el desarrollo del mismo realiza y revisa el plan de proyecto de software.

Desarrollo.

La fase de definición se centra en el cómo.

- cómo ha de ser la arquitectura de la aplicación.
- cómo han de ser las estructuras de datos.
- cómo han de implementarse los detalles procedimentales de los módulos.
- cómo van a ser las interfaces.
- cómo ha de traducirse el diseño a un lenguaje de programación.

cómo van a realizarse las pruebas.

Aunque, al igual que antes, los pasos concretos dependen del modelo de ciclo de vida utilizado, en general se realizarán cuatro tareas específicas:

Diseño.

El diseño del software traduce los requisitos a un conjunto de representaciones (gráficas, en forma de tabla o basadas en algún lenguaje apropiado) que describen cómo van a estructurarse los datos, cuál va a ser la arquitectura de la aplicación, cuál va a ser la estructura de cada programa y cómo van a ser las interfaces. Es necesario seguir criterios de diseño que nos permitan asegurar la calidad del producto.

Una vez finalizado el diseño es necesario revisarlo para asegurar la completitud y el cumplimiento de los requisistos del software.

Codificación.

En esta fase, el diseño se traduce a un lenguaje de programación, dando como resultado un programa ejecutable. La buena calidad de los programas desarrollados depende en gran medida de la calidad del diseño.

Una vez codificados los programas debe revisarse su estilo y claridad, y se comprueba que haya una correspondencia con la estructura de los mismos definida en la fase de diseño.

El listado fuente de cada módulo (o el programa fuente en soporte magnético) pasa a formar parte de la configuración del sistema.

Pruebas.

Una vez que tenemos implementado el software es preciso probarlo, para detectar errores de codificación, de diseño o de especificación. Las pruebas son necesarias para encontrar el mayor número posible de errores antes de entregar el programa al cliente.

Es necesario probar cada uno de los componentes por separado (cada uno de los módulos o programas) para comprobar el rendimiento funcional de cada una de estas unidades.

A continuación se procede a integrar los componentes para probar toda la arquitectura del software, y probar su funcionamiento y las interfaces. En este punto hay que comprobar si se cumplen todos los requisitos de la especificación.

Se puede desarrollar un plan y procedimiento de pruebas y guardar información sobre los casos de pruebas y los resultados de las mismas.

Garantía de calidad.

Una vez terminada la fase de pruebas, el software está casi preparado para ser entregado al cliente.

Mantenimiento.

La fase de mantenimiento se centra en los cambios que va a sufrir el software a lo largo de su vida útil. Como ya hemos dicho, estos cambio pueden deberse a la corrección de errores, a cambios en el entorno inmediato del software o a cambios en los requisitos del cliente, dirigidos normalmente a ampliar el sistema.

La fase de mantenimiento vuelve a aplicar los pasos de las fases de definición y de desarrollo, pero en el contexto de un software ya existente y en funcionamiento.

Figura 2.6. Diagrama de etapas de la Ingeniería del Software.

2.8. Ingeniería inversa y reingeniería del software.

Los modelos descritos en las secciones anteriores dan una visión *directa* de la IS como un proceso evolutivo en el que el software pasa por etapas sucesivas de gestación, construcción, uso y sustitución. Sin embargo existen determinadas tareas dentro de la IS que obligan a realizar el camino inverso, lo que ha llevado a acuñar el término de ingeniería inversa.

Como ya habíamos visto en el tema anterior, en sus primeros tiempos, e incluso recientemente, el software se desarrollaba sin ninguna planificación, lo que ha llevado a la existencia de numerosas aplicaciones actualmente en funcionamiento que carecen por completo de documentación. Con el tiempo, y con la marcha de las personas que las han programado, desaparece por completo el conocimiento que se tiene sobre estas aplicaciones, y después de años de procesamiento automático de la información, incluso las personas que las manejan desconocen cuáles son exactamente las transformaciones que realizan sobre los datos que manejan.

Esto causa graves problemas a la hora de intentar realizar cualquier cambio en estas aplicaciones, al intentar optimizarlas o adaptarlas a para que funcionen sobre nuevos ordenadores, sistemas operativos o lenguajes o a la hora de reescribirlas según los nuevos estilos de trabajo con ordenador (primero fue la entrada de datos interactiva usando pantallas de texto, hoy son los entornos gráficos de ventanas).

La ingeniería inversa consiste en analizar un programa en un esfuerzo de representarlo en un mayor nivel de abstracción que el código fuente, de forma que se extraiga información del diseño de datos, de la arquitectura y del detalle procedimental del mismo, para poder entenderlo.

La reingeniería del software no sólo recupera información sobre el diseño de un programa existente sino que utiliza esta información para reestructurar o reconstruir el programa existente, con vistas a adaptarlo a un cambio, a ampliarlo o a mejorar su calidad general, con el objetivo de conseguir una mayor facilidad de mantenimiento en el futuro (esto es lo que se denomina mantenimiento preventivo).

Existen herramientas CASE especializadas en ingeniería inversa y reingeniería, pero así como las herramientas de ingeniería 'normal' producen buenos resultados, éstas son aún muy rudimentarias, estando muy limitados tanto su campo de aplicación como los resultados que producen.

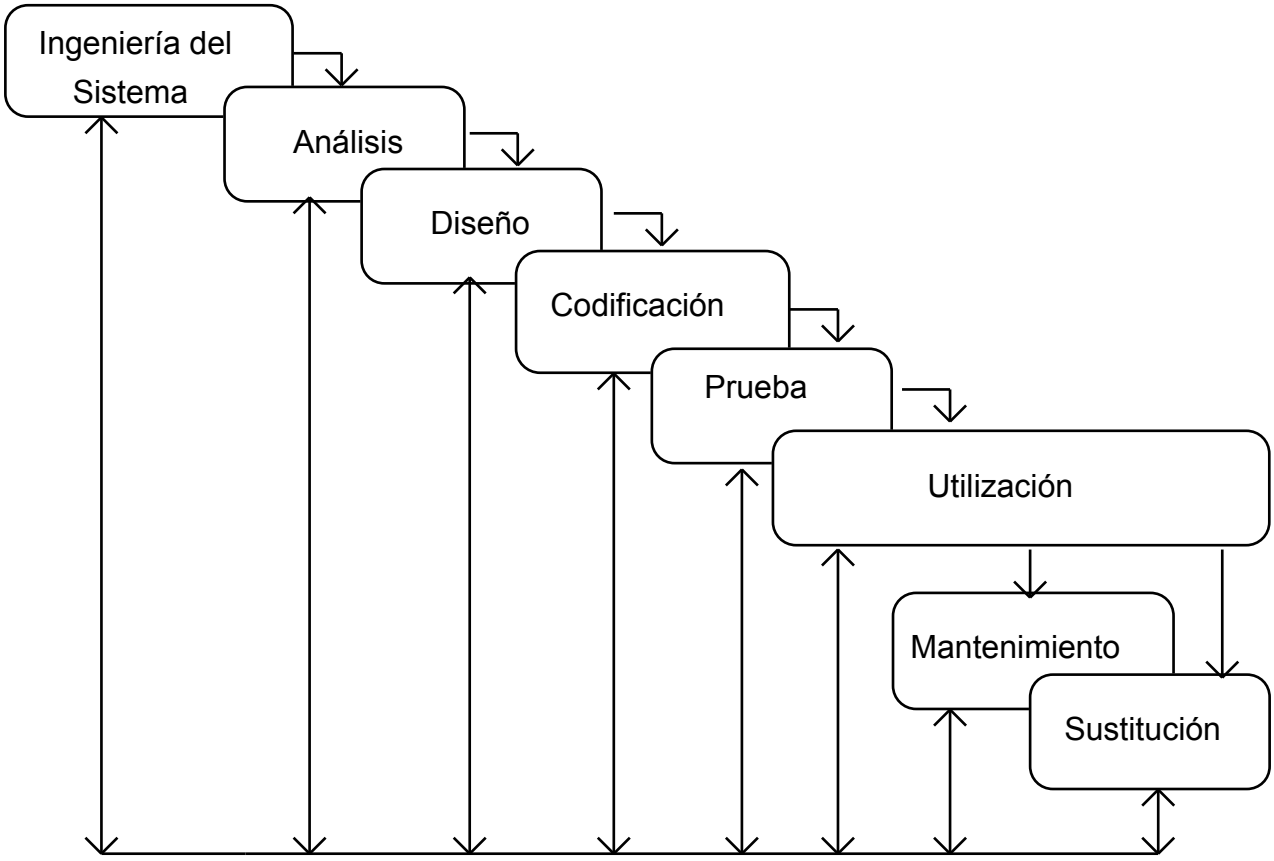


Fig. 2.1. Ciclo de vida clásico o en cascada

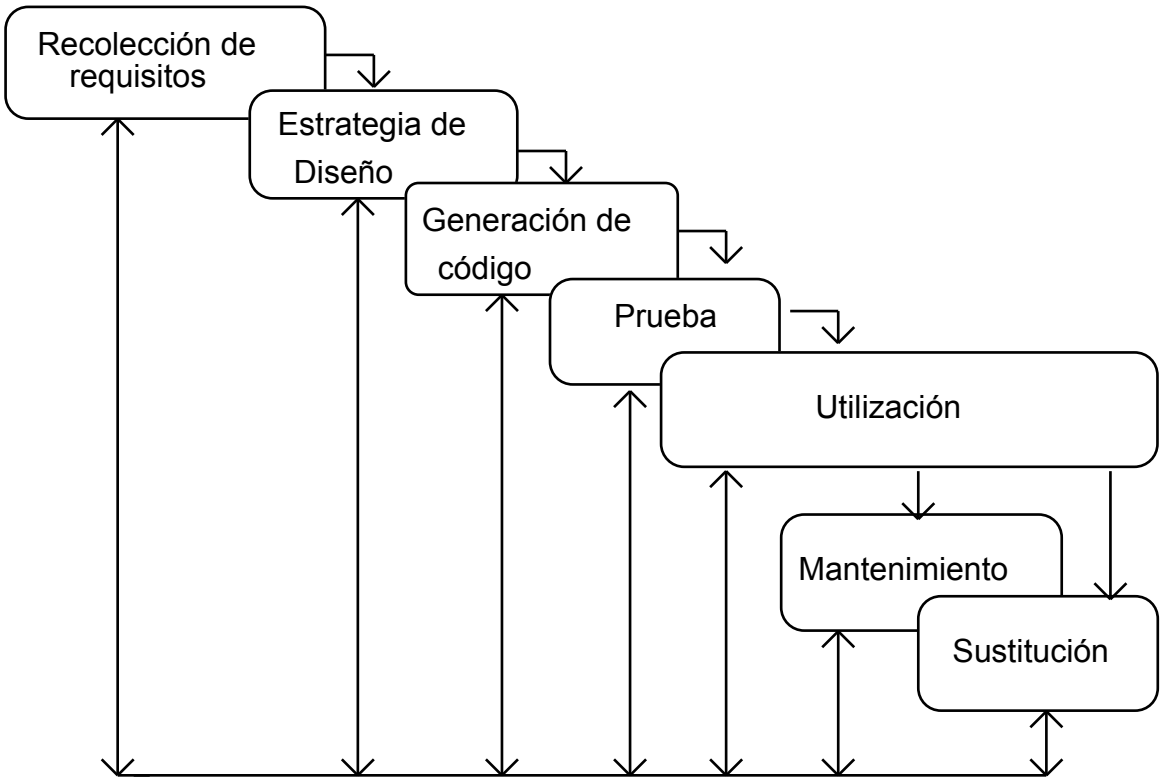


Fig. 2.3. Ciclo de vida usando técnicas de cuarta generación.

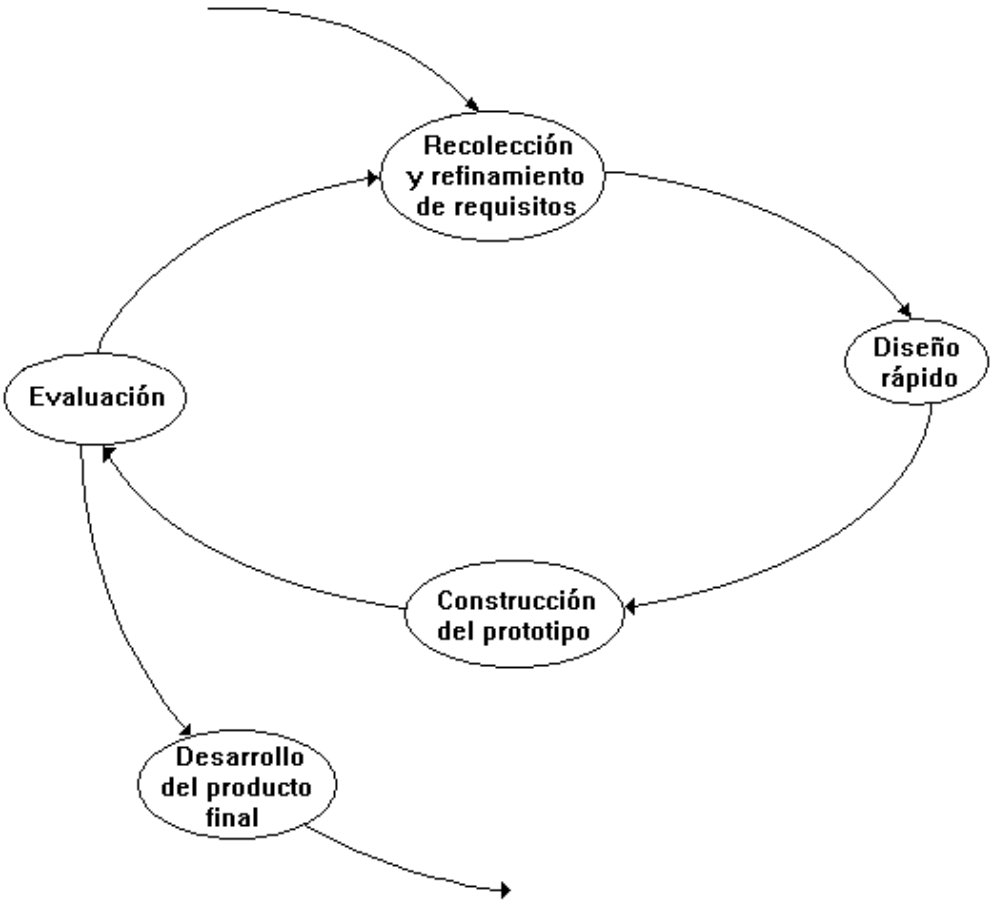


Fig. 2.4. Construcción de prototipos.

Visión genérica de la Ingeniería del Software.

Definición. ¿Qué?

Análisis del sistema.

Establecer el ámbito del software.

Análisis de requisitos del sistema software.

Definición detallada de la función del software.

Planificación.

Análisis de riesgos.
Asignación de recursos.
Definición de tareas.
Estimación de costes.

Desarrollo. ¿Cómo?

Diseño.

Arquitectura de la aplicación.
Estructura de los datos.
Estructura interna de los programas.
Diseño de las interfaces.

Codificación.

Pruebas.

Mantenimiento. El cambio.

Corrección de errores.
Cambios en el entorno.
Cambios en los requisitos.

Parte II. Análisis de requisitos.

Tema 3. Objetivos y conceptos básicos.

3.1. Introducción.

Análisis de requisitos del sistema

3.2. Objetivos del análisis de requisitos del sistema.

3.3. Fases del análisis de sistemas.

3.4. Representación de la arquitectura del sistema.

Análisis de requisitos del software

3.5. Objetivos del análisis de requisitos el software.

3.6. El papel del analista.

3.7 Recogida inicial de requisitos

3.8. Especificación preliminar.

3.9. Principios del análisis de requisitos.

Objetivos de la especificación.

3.10. Principios de especificación.

3.11. Especificación del sistema.

3.12. Especificación del software.

3.13. Especificación formal y especificación informal.

Tema 3. Análisis de requisitos. Objetivos y conceptos básicos.

3.1. Introducción.

Según vimos en el tema anterior, existen diversos modelos de ciclo de vida para el desarrollo de software, pero en todos ellos se puede observar la existencia de una fase denominada Análisis de requisitos.

Entre las tareas que hay que realizar en esta fase están el estudio de las características y la función del sistema, la definición de los requisitos del software y del sistema del que el software forma parte, así como la planificación inicial del proyecto y, posiblemente, algunas tareas relacionadas con el análisis de riesgos.

Todas estas tareas deben realizarse al comienzo del proyecto, pero el principal problema que se nos presenta es que, en estos momentos iniciales, es difícil tener una idea clara - o, al menos, es difícil llegar a expresarla - de cuáles son los requisitos del sistema y del software, y llegar a comprender en su totalidad la función que el software debe realizar. Por esto, algunos de los modelos de ciclo de vida estudiados proponen enfoques cíclicos de refinamiento de los requisitos o incluso de todo el proceso de desarrollo de software.

El análisis de requisitos es el primer paso técnico del proceso de ingeniería del software. Es aquí donde se refina la declaración general del ámbito del software en una *especificación* concreta que se convierte en la base de todas las actividades de ingeniería del software que siguen. Algunos modelos de ciclo de vida distinguen una fase de análisis de requisitos y otra de especificación del sistema. En cualquier caso, el análisis del sistema concluye con la especificación de los requisitos del mismo.

El análisis se centra en los ámbitos de información, funcional y de comportamiento del problema en cuestión. Para comprender mejor lo que se requiere se divide el problema en partes y se desarrollan representaciones o modelos que muestran la esencia de los requisitos (modelo esencial) y, posteriormente, los detalles de implementación (modelo de implementación). Como resultado del

análisis se desarrolla la *especificación de requisitos*, un documento que describe el problema analizado y muestra la estructura de la solución propuesta.

La forma de especificar un sistema tiene una gran influencia en la calidad de la solución implementada finalmente. Tradicionalmente los ingenieros de software han venido trabajando con especificaciones incompletas, inconsistentes o erróneas, lo que invariablemente lleva a la confusión y a la frustración en todas las etapas del ciclo de vida. Como consecuencia de esto, la calidad, la corrección y la completitud del software disminuyen

A lo largo del curso, veremos que los requisitos del software se pueden analizar de varias formas diferentes, no necesariamente excluyentes, cada una de ellas asociadas a un modelo del ciclo de vida. Las técnicas de análisis que veremos conducen a una especificación en papel o basada en ordenador (si utilizamos herramientas de análisis o CASE), que contiene descripciones gráficas y textuales (normalmente en lenguaje natural o alguna forma de pseudocódigo) de los requisitos del software. Por otro lado, la construcción de prototipos lleva a una especificación ejecutable, es decir, el prototipo sirve como representación de los requisitos. Por último, los lenguajes de especificación formal conducen a representaciones de los requisitos que pueden ser analizadas o verificadas formalmente. Sin embargo, sea cual sea el método elegido, existen una serie de características comunes:

Métodos de análisis. Características comunes.

- *Realizan una abstracción de las características del sistema, es decir, consisten en desarrollar un modelo del mismo.*
- *Representan el sistema de forma jerárquica, basándose en mecanismos de partición del problema y estableciendo varios niveles de detalle.*
- *Definen cuidadosamente las interfaces del sistema, tanto las interfaces externas, que relacionan el sistema con su entorno, como de las internas, las que se establecen entre los distintos módulos definidos.*
- *Sirven de base para las etapas posteriores de diseño y de implementación.*
- *Distinguen entre requisitos esenciales y de implementación.*
- *No prestan demasiada atención a la representación de las restricciones o de criterios de validación (exceptuando los métodos formales).*

Es por esta última deficiencia por la que surge la necesidad de los métodos de especificación formal, especialmente en aquellos sistemas en los que la corrección, completitud o fiabilidad del

software juegan un papel fundamental. Ejemplos de este tipo de sistemas pueden ser los protocolos de comunicación, el software de control de una central nuclear o el de gestión del tráfico aéreo.

Análisis de requisitos del sistema.

El análisis de requisitos del sistema constituye el primer intento de comprender cuál va a ser la función y ámbito de información de un nuevo proyecto. El objetivo es conseguir representar un sistema en su totalidad, incluyendo hardware, software y personas, mostrando la relación entre los diversos componentes y sin entrar en la estructura interna de los mismos. En algunos casos se nos plantearán diferentes posibilidades y tendremos que realizar un estudio de cada una de ellas.

Indudablemente, los esfuerzos realizados en esta fase producen beneficios en las fases posteriores. Sin embargo se nos pueden plantear las siguientes dudas:

- **¿Cuánto tiempo debe dedicarse al análisis del sistema?** Dependerá de la naturaleza, el tamaño y la complejidad de la aplicación. Una directriz que se podría aplicar es dedicar entre un 10% y un 20% del esfuerzo total estimado al análisis del sistema y otro 10% o 20% al análisis de requisitos del software. El esfuerzo que se le dedica normalmente es mucho menor:. En la mayoría de los proyectos la mayor parte del esfuerzo se va en la codificación, precisamente por la dificultad de realizar la codificación cuando no se ha hecho un buen análisis previo.

- **¿Quién debe hacerlo?** La respuesta es fácil: el analista de sistemas. Este debe ser una persona con buena formación técnica y con experiencia. No obstante, el analista no trabaja de forma aislada sino en estrecho contacto con el personal de dirección, técnico y administrativo tanto del cliente como del que desarrolla el software.

- **¿Por qué es una tarea tan difícil?** Básicamente porque consiste en la traducción de unas ideas vagas de necesidades de software en un conjunto concreto de funciones y restricciones. Además el analista debe extraer información dialogando con muchas personas y cada una de ellas se expresará de una forma distinta, tendrá conocimientos informáticos y técnicos distintos, y tendrá unas necesidades y una idea del proyecto muy particulares.

3.2. Objetivos del análisis de requisitos del sistema.

El papel del analista de sistemas es el de definir los elementos de un sistema informático dentro del contexto del sistema en que va a ser usado. Hay que identificar las funciones del sistema y asignarlas a alguno de sus componentes. Para ello, el analista de sistemas parte de los objetivos y restricciones definidos por el usuario y realiza una representación de la función del sistema, de la información que maneja, de sus interfaces y del rendimiento y las restricciones del mismo.

En la mayoría de los casos, el proyecto empieza con un concepto más bien vago y ambiguo de cuál es la función deseada. Entonces el analista debe delimitar el sistema, indicando el ámbito de funcionamiento y el rendimiento deseados. Por ejemplo, en el caso de un sistema de control, no basta con decir algo así como *'el sistema debe reaccionar rápidamente en caso de que la señal de entrada supere los límites de seguridad'* sino que hay que definir cuáles son los límites de seguridad de la señal de entrada, en cuánto tiempo debe producirse la reacción y cómo ha de ser esta reacción.

Una vez que se ha logrado delimitar la función, el ámbito de información, las restricciones, el rendimiento y las interfaces del sistema, el analista debe proceder a la asignación de funciones. Las funciones del sistema deben de ser asignadas a alguno de sus componentes (ya sean éstos software, hardware o personas). El analista debe estudiar varias opciones de asignación (considerando, por ejemplo, la posibilidad de automatizar o no alguna de estas funciones), teniendo en cuenta las ventajas e inconvenientes de cada una de ellas (en cuanto a viabilidad, costes de desarrollo y funcionamiento y fiabilidad) y decidirse por una de ellas, o bien presentar un estudio razonado de las opciones a quienes tengan que tomar la decisión. Para explicar todo lo anterior podemos poner el siguiente ejemplo:

Especificación informal del sistema de clasificación de paquetes.

El sistema de clasificación de paquetes debe realizarse de forma que los paquetes que se mueven a lo largo de una cinta transportadora sean identificados (para lo cual van provistos de un código numérico) y clasificados en alguno de los seis contenedores situados al final de la cinta. Los paquetes de cada tipo aparecen en la cinta siguiendo una distribución aleatoria y están espaciados de manera uniforme. La velocidad de la cinta debe ser tan alta como sea posible; como mínimo el sistema debe ser capaz de clasificar 10 paquetes por minuto. La carga de paquetes al principio de la cinta puede realizarse a una velocidad máxima de 20 paquetes por minuto. El sistema debe funcionar las 24 horas del día, siete días a la semana.

Función del sistema.

Realizar la clasificación de paquetes que llegan por una cinta transportadora en seis compartimentos distintos, dependiendo del tipo de cada paquete.

Información que se maneja.

Los paquetes disponen de un código numérico de identificación.

Debe existir una tabla o algoritmo que asigne a cada número de paquete el contenedor donde debe ser clasificado.

Interfaces del sistema.

El sistema de clasificación se relaciona con otros dos sistemas. Por un lado tenemos la cinta transportadora. Parece conveniente que el sistema de clasificación pueda parar el funcionamiento de la cinta o del sistema de carga de paquetes en la cinta, en caso de que no pueda realizar la clasificación (por ejemplo si se produce una avería). Por otro lado, el sistema deposita paquetes en los contenedores, pero no se establece ningún mecanismo de vaciado o sustitución de los contenedores si se llenan. El sistema debería ordenar la sustitución o vaciado del contenedor o esperar mientras un contenedor esté lleno.

Como la descripción realizada por el cliente no establece los mecanismos para solventar estas dos situaciones estos detalles deben ser discutidos con el cliente.

Rendimiento.

El sistema debe ser capaz de clasificar al menos 10 paquetes por minuto. No es necesario que el sistema sea capaz de clasificar más de 20 paquetes por minuto.

Restricciones.

El sistema debe tener un funcionamiento continuo. Por tanto, debemos evitar la parada del sistema incluso en el caso de que para alguno de los componentes del mismo se averíen.

El documento no indica restricciones sobre la eficacia del sistema, es decir, sobre cuál es el porcentaje máximo que se puede tolerar de paquetes que pueden ser clasificados de forma errónea. Estos detalles también deben ser aclarados con el cliente.

Asignación de funciones.

Podemos considerar tres asignaciones posibles:

Asignación 1.

*Esta asignación propone una **solución manual** para implementar el sistema. Los recursos que utiliza son básicamente personas, y se requiere además algo de documentación, definiendo las características del puesto de trabajo y del sistema de turnos y una tabla que sirva al operador para relacionar los códigos de identificación de los paquetes con el contenedor donde deben ser depositados.*

La inversión necesaria para poner en marcha este sistema es mínima, pero requiere una gran cantidad de mano de obra (varios turnos de trabajo y operadores de guardia) con lo que los costes de funcionamiento serán elevados. Además hay que tener en cuenta que lo rutinario del trabajo provocará una falta de motivación en los operarios, lo que a la larga se acabará traduciendo en un mayor absentismo laboral. Todos estos factores deben de tenerse en cuenta a la hora de elegir esta u otra opción.

Asignación 2.

*En este caso, la solución es automatizada. Los recursos que se utilizan son: hardware (el lector de códigos de barras, el controlador, el **mecanismo de distribución**), software (para el lector de códigos de barras y el controlador, y una base de datos que permita asignar a cada código su contenedor) y personas (si en caso de avería la distribución se va a hacer manualmente). Cualquiera de los elementos hardware y software tendrán la correspondiente documentación sobre cómo han sido construidos y un manual de usuario.*

Aquí si hay que realizar una cierta inversión, para comprar o desarrollar los componentes del sistema, pero los costes de funcionamiento serán sin duda menores (sólo el consumo de energía eléctrica). Hay que tener en cuenta que el uso de dispositivos mecánicos (el mecanismo de distribución) va a introducir unos costes de mantenimiento y paradas por avería o mantenimiento con una cierta frecuencia.

Asignación 3.

*Los recursos que utilizamos aquí son: hardware (el lector, el **brazo robot**), software (el del lector, el del robot, incluyendo la tabla o algoritmo de clasificación) y la documentación y manuales correspondientes a estos elementos.*

En este caso la inversión inicial es, sin duda, la más elevada. Los costes de funcionamiento son bajos pero hay que considerar también el coste de mantenimiento del robot, que posiblemente tenga que ser realizado por personal especializado. Los únicos motivos que nos harían decidir por esta opción en vez de la anterior vendrían dados por una mayor velocidad, un menor número de errores o unas menores necesidades de mantenimiento o frecuencia de averías.

Por otra parte esta solución puede tener problemas de viabilidad (si no encontramos un brazo robot que sea capaz de atrapar los paquetes según pasan por la cinta).

Además de las tres opciones propuestas, el ingeniero de sistemas debe considerar también la adopción de *soluciones estándar* al problema. Hay que estudiar si existe ya un producto comercial que realice la función requerida para el sistema o si alguna de las partes del mismo pueden ser adquiridas a un tercero. Aparte de considerar el precio de estos productos habrá que tener también en cuenta los

costes del mantenimiento y el riesgo que se asocia al depender de una tecnología que no es propia (¿es la empresa proveedora estable?, ¿cuál es la calidad de sus productos?) valorando todo esto frente a los riesgos asociados a realizar el desarrollo nosotros mismos.

La labor del ingeniero o analista de sistemas consiste, en definitiva, en asignar a cada elemento del sistema un ámbito de funcionamiento y de rendimiento. Después, el ingeniero del software se encargará de refinar este ámbito para el componente software del sistema y de producir un elemento funcional, que sea capaz de ser integrado con el resto de los elementos del sistema.

3.3. Fases del análisis de sistemas.

El análisis del sistema comprende las siguientes fases o tareas:

- *Identificación de las necesidades del cliente.*
- *Evaluación de la viabilidad del sistema.*
- *Análisis económico y técnico del proyecto.*
- *Asignación de funciones a cada uno de los elementos del sistema.*
- *Obtención de una definición del sistema que sea la base del trabajo posterior.*

3.3.1. Identificación de las necesidades.

Para identificar las necesidades, el analista del sistema debe reunirse con el cliente o con su representante. Juntos, proceden a definir los objetivos del sistemas, la información que se va a suministrar, la información que se va a obtener, las funciones y el rendimiento requerido.

El analista debe ser capaz de distinguir entre lo que *necesita* el cliente (lo que es para él imprescindible), y lo que *quiere* el cliente (aquello que le sería útil pero no imprescindible).

3.3.2. Estudio de viabilidad.

Cualquier proyecto sería viable si dispusiésemos de recursos humanos, temporales y económicos ilimitados. Pero los recursos son siempre limitados: existen restricciones sobre el número de personas que se pueden dedicar (especialmente si se trata de personal del cliente), sobre cuanto dinero nos podemos gastar en el proyecto (si la inversión necesaria para el desarrollo es demasiado alta el sistema no compensará los ahorros que se produzcan con su uso) y sobre los tiempos de entrega

(nadie compra software a cinco años vista, además, según pasa el tiempo aumentan las posibilidades de que cambien los requisitos).

Por esto, es conveniente estudiar la viabilidad del proyecto lo antes posible, puesto que así se pueden ahorrar meses de esfuerzos y millones de pesetas en el desarrollo de un proyecto que al final se muestre como inviable. El estudio de viabilidad está muy relacionado con el análisis de riesgos: si el riesgo de un proyecto es grande, se reducen las posibilidades de producir software de calidad, es decir, disminuye la viabilidad.

Nos centraremos fundamentalmente en tres aspectos:

- **Viabilidad económica.** Consiste en comparar los beneficios futuros de la utilización del sistema con los costes de su desarrollo. La justificación económica es normalmente la principal consideración a la hora de decidir realizar o no cualquier proyecto. Por esto, aparte de decidir la viabilidad o no viabilidad se necesita también un análisis económico completo (no sólo si va a producir beneficios sino qué beneficios va a producir, a qué plazo, etc.).

- **Viabilidad técnica.** Consiste en determinar si es posible desarrollar o no el producto, teniendo en cuenta sus restricciones, basándonos en los recursos humanos y técnicos a nuestro alcance. Como los objetivos, funciones y rendimiento son confusos al inicio del proyecto, cualquier cosa puede parecer inicialmente viable. Debido a esto, es conveniente revisar el estudio de viabilidad técnica una vez que las especificaciones estén más claras. Además, el análisis de viabilidad técnica debe ser completado con un estudio técnico del sistema en proyecto, que permita determinar las características técnicas del nuevo sistema y qué mejoras introduce sobre el proceso actual.

- **Viabilidad legal.** Consiste en determinar si el proyecto infringe alguna disposición legal sobre el derecho a la intimidad de las personas, las normas de seguridad en el trabajo, de calidad de los productos, las leyes de Copyright si se van a utilizar componentes comprados a terceros para integrarlos en el sistema o basarnos en ellos para el desarrollo del producto software, y otras.

3.3.3. Análisis económico.

El análisis económico del proyecto consistirá en señalar los costes de desarrollo del proyecto y compararlos con los beneficios que producirá una vez desarrollado.

Los costes de desarrollo pueden ser cuantificados fácilmente (inversiones en equipos, horas-hombre necesarias en las fases de análisis, diseño y codificación). Sin embargo, suele ser más difícil determinar los beneficios futuros que producirá el proyecto una vez listo para ser usado. Algunos de estos beneficios pueden ser tangibles (disminución del tiempo necesario para realizar determinadas tareas) pero otros muchos son intangibles (mayor satisfacción o cualificación profesional de los usuarios del sistema, incremento de la capacidad de producción, etc.), por lo que puede ser difícil realizar comparaciones coste-beneficio directas.

Hay que tener en cuenta que la mayoría de los sistemas de procesamiento de la información se realizan teniendo como principal objetivo una mayor cantidad, calidad y rapidez de acceso a la información, de forma que se pueda realizar una mejor gestión de la empresa. Todos estos beneficios citados son intangibles y, aunque se traducirán sin lugar a dudas en beneficios tangibles (ahorro en los costes de producción o en las tareas administrativas, incremento de ventas como resultado de una mayor rapidez de respuesta a las necesidades del mercado, etc.) es muy difícil formular numéricamente esta relación. Por este motivo, los beneficios intangibles se incluirán en el análisis económico para reforzar la justificación del proyecto, pero esta justificación debe basarse en la medida de lo posible en los beneficios, medibles y demostrables que producirá el nuevo sistema.

Los costes y beneficios pueden ser directos o indirectos dependiendo de si es posible establecer una relación de los mismos con la implantación del software. Por ejemplo un aumento de la capacidad de gestión de un almacén es un beneficio directo de la implantación de un sistema informático para el control del mismo. El acondicionamiento del almacén para adaptarlo al nuevo sistema (redistribución de espacios, etc.) puede considerarse un coste indirecto.

La única forma de demostrar estos beneficios será comparando los modos de trabajo con el nuevo sistema con los modos de trabajo actuales. El nuevo sistema, cambiará los modos de trabajo de la empresa u organización en la que se instale de forma que se producirá:

- una disminución del tiempo necesario para realizar determinadas tareas.
- una menor necesidad de mano de obra para realizar el trabajo actual.
- un aumento de productividad, que permitirá aumentar la producción con la mano de obra actual.

Cualquiera de los tres puntos anteriores puede ser evaluado cuantitativamente, de forma que se puede determinar el ahorro que se producirá con la puesta en marcha del nuevo sistema.

Pongamos como ejemplo el sistema de clasificación de paquetes visto antes. Supongamos que el sistema actual es el que corresponde a la *Asignación 1* (es decir, es un sistema manual) y que el nuevo sistema es el indicado en la *Asignación 3* (utilización de un lápiz óptico y de un brazo robot).

En primer lugar, calculemos los costes del sistema actual. Supongamos que para el funcionamiento en turno ininterrumpido, incluyendo operadores de guardia, necesitamos 7 operadores y que el coste salarial (incluyendo cargas sociales) de cada operador es de 2.5 M/año. Podemos calcular el coste/hora del sistema actual, que resulta ser de 2000 ptas/hora, aproximadamente.

Supongamos también que las inversiones necesarias para poner en marcha el sistema nuevo son de 40 M, que los costes de mantenimiento del brazo robot son de 1 M/año y los costes de funcionamiento (gastos de electricidad) son de 2 M/año. Además, debido a las paradas por mantenimiento y averías, el sistema nuevo solo estará útil el 95% del tiempo, debiendo efectuarse la clasificación manual el 5% del tiempo restante. Esto resulta en unos gastos de puesta en marcha de 40M y unos gastos de funcionamiento de 3.875 M/año. Podemos representar los beneficios y costes acumulados del nuevo sistema en una gráfica (fig. 3.1) y calcular el tiempo de amortización (el punto donde se cortan ambas curvas: indica cuánto tiempo ha de pasar para que los beneficios del sistema nuevo compensen los costes iniciales de puesta en marcha). Este tiempo de amortización resulta ser de 2.9 años, aproximadamente.

Sistema de clasificación de paquetes. Análisis coste-beneficio	
Costes del sistema actual:	*17.5 Mptas/año*
Costes del nuevo sistema:	
Inversión inicial:	*40 Mptas*
Funcionamiento:	*2 Mptas/año*
Mantenimiento:	*1 Mptas/año*
Operatividad:	*0.95*
Total coste anual:	*3.875Mptas/año*

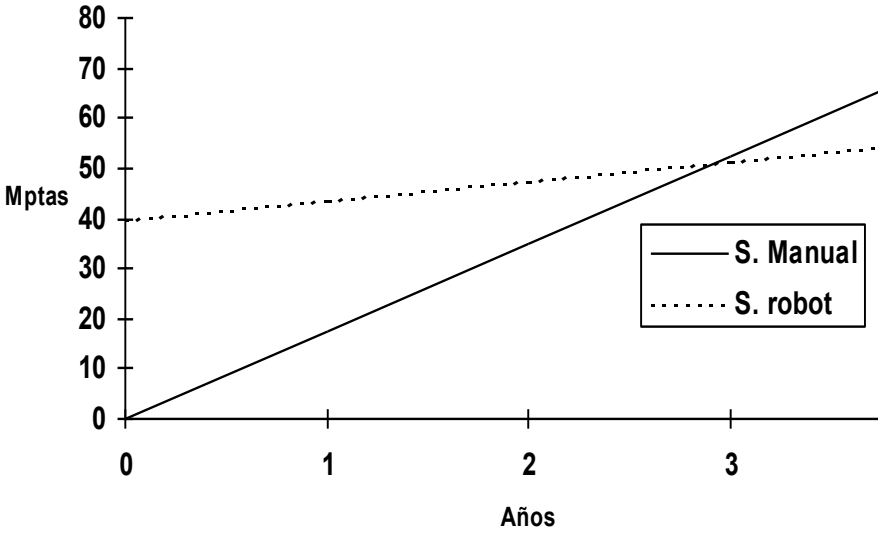


Fig. 3.1. Análisis coste beneficio

3.3.4. Análisis técnico.

Con el análisis técnico se pretende estudiar las características técnicas del nuevo sistema: capacidad, rendimiento, fiabilidad, seguridad, etc., de forma que se complemente el análisis de coste-beneficio con las mejoras técnicas que pueda proporcionar el sistema a los modos de trabajo de la empresa u organización donde se implante.

Para hacer un buen análisis técnico tendremos que modelar el sistema de alguna forma, y realizar un estudio analítico de las características del modelo propuesto o bien realizar algún tipo de simulación con el modelo. (Cosas todas ellas fuera del ámbito de esta asignatura).

Los resultados del análisis técnico son otro de los criterios que permitirán decidir si seguir o no con el proyecto. Si el riesgo técnico es alto, o si el modelo o las simulaciones muestran que el sistema en proyecto no va a conseguir substanciales mejoras sobre el sistema actual, podemos cancelar el proyecto.

3.3.5. Asignación de funciones.

Como ya habíamos visto, para cada elemento del sistema puede haber una o varias asignaciones posibles. Es necesario estudiar cada una de las alternativas desde el punto de vista económico y técnico y elegir cuál es la más adecuada.

Para elegir entre las distintas alternativas debemos fijar una serie de parámetros de evaluación. Normalmente los parámetros de evaluación serán de orden económico (coste de las inversiones, ahorro que se producirá con el sistema nuevo), pero también pueden tenerse en cuenta parámetros relacionados con la fiabilidad del sistema, su capacidad, su rendimiento o la mejora de calidad de los productos, si se trata de un sistema de producción. Hay que ordenar cada uno de estos parámetros de acuerdo a su importancia, lo que dependerá de los objetivos generales de cada proyecto, y ver cuál de las distintas alternativas obtiene una mejor evaluación de acuerdo con los parámetros establecidos. Cuando dos o más alternativas satisfagan los parámetros de evaluación principales, optaremos entre ellas observando los parámetros de segundo orden y así sucesivamente.

Por ejemplo, en el caso del sistema de clasificación de paquetes, la alternativa elegida dependerá de qué parámetros consideremos más importantes. Si queremos que la inversión inicial sea pequeña la mejor alternativa será la primera. Si pretendemos en cambio unos costes de funcionamiento moderados, valorando que el plazo de amortización sea corto, la mejor opción será la segunda. Si damos más importancia a la fiabilidad y al bajo mantenimiento del sistema, la mejor opción puede ser la tercera.

Sistema de clasificación de paquetes. Criterios de evaluación.

	Alternativa 1	Alternativa 2	Alternativa 3
Inversión inicial	Nula	Moderada	Grande
Coste funcionamiento	Grande	Moderado	Moderado
Tiempo amortización	Nulo	Moderado	Grande
Fiabilidad	Moderada	Pequeña	Grande
Mantenimiento	Nulo	Grande	Moderado

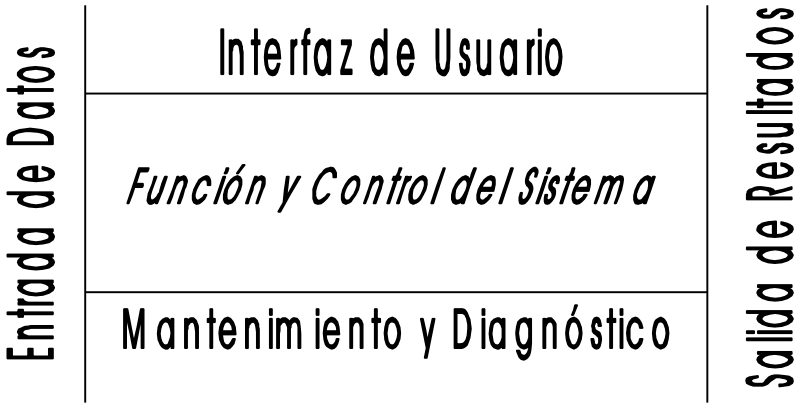
3.4. Representación de la arquitectura del sistema.

Una vez que hemos realizado la asignación de funciones podemos hacer un diagrama del sistema que represente las relaciones entre sus elementos y que sirva de base para el trabajo posterior. Para ello utilizaremos los diagramas de arquitectura.

Si la complejidad del sistema así lo aconseja podemos utilizar estos diagramas formando una jerarquía de niveles, en el nivel superior representaremos el sistema mediante un diagrama de contexto, e iremos detallando más la arquitectura en sucesivos diagramas de flujo.

3.4.1. Plantilla de diagramas de arquitectura.

Parta que los diagramas tengan un formato estándar, los dividiremos en cinco regiones:



3.4.2. Diagrama de contexto.

El diagrama de contexto representa el sistema en relación con su entorno. Sirve para definir los límites del sistema y muestra todos los productores y consumidores de información del sistema.

El centro del diagrama de contexto estará ocupado por el sistema, representado por una caja de esquinas redondeadas.  A su alrededor se situarán una serie de entidades o agentes externos (el entorno de sistema) representados mediante cajas de esquinas cuadradas.  Cada agente externo representa un productor o consumidor de información del sistema. Cada agente externo se sitúa en la región del diagrama que le corresponda, según su papel sea el de productor, consumidor, usuario o supervisor.

El sistema se relaciona con los agentes externos a través de flujos de información, representados mediante arcos orientados.

3.4.3. Diagramas de flujo.

Podemos describir la arquitectura del sistema en mayor grado de detalle a base de expandir el diagrama de contexto en una jerarquía de diagramas de flujo.

Cada subsistema del diagrama de contexto puede dar lugar a un diagrama de flujo, donde se describirá el sistema en mayor detalle, descomponiéndolo en nuevos subsistemas relacionados mediante flujos de información. Cada subsistema ocupa una región del diagrama dependiendo de cuál sea su función (adquisición de datos, salida de datos, interfaz, etc.).

Un detalle importante es que debe mantenerse la consistencia entre los diagramas de distinto nivel. Al expandir un determinado subsistema en un diagrama de flujo, los flujos de información que conectan dicho subsistema con otros o con agentes externos, deben figurar también (coincidiendo en número, sentido y nombre) en el diagrama de flujo resultado de dicha expansión. Estos flujos de información, tendrán un extremo libre, el que los conectaba con subsistemas que quedan fuera del ámbito del nuevo diagrama de flujo.

Análisis de requisitos del software.

Como resultado de la fase de análisis de requisitos del sistema, se ha asignado una función y un rendimiento al componente software del mismo. Para conseguir esta función y rendimiento, el ingeniero del software debe construir - o adquirir - una serie de componentes software. Desgraciadamente, los componentes software están muy poco estandarizados, por lo que las dos únicas opciones serán el adquirir un sistema software comercial que cumpla con los requisitos - si este sistema existe y logramos encontrarlo - o desarrollar (o encargar el desarrollo) de un sistema software a la medida.

3.5. Objetivos del análisis de requisitos del software.

El análisis de requisitos del software tiene como objeto desarrollar una representación del software que pueda ser revisada y aprobada por el cliente y es una tarea que sirve de enlace entre la asignación de funciones al software que se ha hecho en el análisis del sistema y el diseño del software.

Desde el punto de vista del analista de sistemas, el análisis de requisitos del software define con mayor precisión las funciones y rendimiento del software, las interfaces que ha de tener con otros componentes del sistema y las restricciones que debe cumplir.

Desde el punto de vista del diseñador, el análisis de requisitos proporciona una representación de la información y la función del sistema que él se encargará de traducir en un diseño de datos y programas.

Por último, el análisis de requisitos, incluido en la especificación del proyecto, permite a todos (incluido aquí el cliente) valorar la calidad del software una vez que haya sido construido.

La labor del analista debe centrarse en el **qué**, no en el **cómo**. (¿qué datos debe manejar el sistema software?, ¿qué función debe realizar?, ¿qué interfaces debe tener?, y ¿qué restricciones tiene que cumplir?)

El análisis de requisitos puede entenderse como una tarea de análisis síntesis: el ingeniero del software debe analizar el problema y los objetivos del cliente y, a partir de esto, sintetizar soluciones.

Inicialmente, el, ingeniero del software o analista, estudia la especificación resultado del análisis del sistema, con el objetivo de comprender cuál es el papel del software en el contexto del sistema. También debe revisar el plan del proyecto software (la planificación de proyectos se ve el año que viene) para comprobar las estimaciones que se han hecho a la hora de planificar el proyecto.

A continuación, debe ponerse en contacto con el equipo técnico y de gestión del cliente, para poder conocer los objetivos básicos del software tal como los entiende el cliente. Con la información obtenida de la especificación del sistema y de las entrevistas con el cliente/usuario el analista debe ir definiendo y refinando los flujos y la estructura de la información, la función del programa y su papel en el contexto del sistema, las características de las interfaces y las restricciones de diseño.

Con este conocimiento que se va adquiriendo del sistema software, el analista debe sintetizar una o varias soluciones al problema, comprobando que se ajustan al plan del proyecto y a las necesidades del cliente. Este proceso de análisis del problemas y síntesis de soluciones debe proseguir hasta que el analista y el cliente acuerden una solución y se pueda especificar el software de forma adecuada para que se puedan efectuar las siguientes fases de desarrollo.

Como ayuda en este proceso de análisis-síntesis, el analista utiliza modelos para entender mejor los flujos de información y la función de cada elemento del sistema software. Estos modelos servirán de base para el diseño de software.

A partir de estos modelos pueden desarrollarse prototipos del sistema software. Esto estará especialmente indicado cuando el cliente no esté seguro de lo que quiere realmente o cuando el analista necesite comprobar si una solución determinada resuelve efectivamente el problema. Los prototipos pueden ser utilizados por el cliente para refinar los requisitos o comprobar la validez de la solución propuesta.

3.6. El papel del analista.

El analista es el principal puente que se establece entre el cliente y los desarrolladores. No sólo desarrollará su trabajo en la fase de análisis de requisitos sino que lo normal es que esté presente en todas las revisiones que se hagan a lo largo del desarrollo del proyecto. En su trabajo, el analista debe

entrevistarse con múltiples personas, que tendrán una visión distinta del problema y de su solución incluso tendrán intereses contrapuestos.

Un buen analista, por tanto, debe facilidad de comunicación, debe ser capaz de extraer información relevante a partir de fuentes confusas o contradictorias, reorganizar esta información para sintetizar soluciones y debe servir de mediador entre las partes. Además de todo esto, debe tener los conocimientos técnicos y la experiencia necesarios para poder aplicar la informática a los problemas del cliente, debe conocer o ser capaz de asimilar conocimientos sobre el campo de actividad del cliente y debe tener claros los principios básicos de la ingeniería del software para poder incorporar estos principios a los requisitos del sistema de forma que se desarrolle software de calidad.

3.7. Recogida inicial de requisitos

Uno de los mayores problemas que surgen en la fase de recogida inicial de requisitos es cómo organizar toda la información que adquiere el analista en sus entrevistas con las personas involucradas en un proyecto y cómo poner de acuerdo a todas estas personas sobre cuál es la solución más adecuada. Mientras que los métodos clásicos se basan en entrevistas bilaterales (el analista y cada una de las partes) con lo que dejan para el analista toda la labor de organización y obtención de un consenso, últimamente se tiende a prácticas más relacionadas con el *brainstorming* en el que cada parte expone sus ideas y propuestas y se produce un debate de forma que las posiciones vayan acercándose sucesivamente hasta que se llegue a un consenso.

El método de trabajo sería el siguiente:

Inicialmente se llevan a cabo unas reuniones preliminares con el cliente (el que ha encargado el proyecto) con vistas a aclarar el ámbito del problema y los objetivos generales de una solución del mismo. Como resultado de estas reuniones, el analista y el cliente redactan una descripción breve del problema y los objetivos del proyecto y el esquema general de la solución.

A continuación se convoca una reunión en la que deben estar representados el analista, el cliente y el equipo de desarrollo. Los invitados a la reunión deben conocer previamente la descripción del problema y su solución redactada anteriormente y se les pide que elaboren una relación preliminar de los objetos o entidades que pueden identificar en el sistema o su entorno, las operaciones que se

realizan o sirven para interrelacionar estos objetos, las restricciones que debe cumplir el sistema y los rendimientos que se esperan del mismo. Para conducir la reunión se nombra a un moderador neutral.

La reunión comienza discutiendo la necesidad y justificación del proyecto (es decir, debatiendo el documento preliminar). Una vez que todo el mundo esté de acuerdo en la necesidad de desarrollar el producto, se puede pasar a presentar cada una de las listas de objetos, operaciones, restricciones y rendimientos realizadas individualmente. A partir de estas listas individuales se crean listas combinadas, en las que se agrupa lo redundante pero no se elimina nada.

Puede entonces comenzar la discusión sobre la lista combinada, en la que se pueden añadir nuevos elementos, eliminar otros o reescribirla de forma que refleje correctamente el sistema a desarrollar. Hay que realizar una *miniespecificación* de cada uno de los elementos de las listas, donde se defina brevemente cada uno de dichos elementos. El desarrollo de estas miniespecificaciones puede descubrir nuevos elementos o demostrar que alguno de ellos está incluido en otro. Se pueden dejar en el aire ciertos aspectos siempre y cuando se registre que deben ser tratados posteriormente.

Una vez concluida la reunión el analista debe elaborar un borrador de especificación del proyecto, combinando las miniespecificaciones de cada elemento, que servirá de base para todo su trabajo posterior.

Este enfoque en equipo de la recogida inicial de requisitos proporciona numerosas ventajas, entre las que se pueden citar la comunicación multilateral de todos los involucrados en el proyecto, el refinamiento instantáneo y en equipo de los requisitos a partir de las ideas previas individuales y la obtención de un documento que sirva de base para el proceso de análisis.

Recogida inicial de requisitos. Método de trabajo.

- *Redacción de un documento inicial:*
  - *Descripción del problema.*
  - *Objetivos.*
- *Propuesta de solución (si es posible).*
- *Creación de listas individuales de:*
  - *Objetos: del sistema y del entorno*
  - *Operaciones: relación entre los objetos*
  - *Restricciones*
  - *Rendimiento*
- *Creación de una lista conjunta*
- *Discusión de la lista conjunta*
- *Redacción de miniespecificaciones*
- *Redacción de un borrador de la especificación*

3.8. Especificación preliminar.

La especificación preliminar debe indicar qué es lo que hay que hacer, pero no cómo hay que hacerlo. Debe ser una descripción de las necesidades y no una propuesta de solución. El cliente debe indicar qué características son imprescindibles y cuáles opcionales, y debe evitar describir la estructura interna del sistema, para no reducir la flexibilidad en la implementación. Características como, rendimiento, protocolos de comunicación, standards de la IS (construcción modular, previsión de expansiones futuras, etc.), y, en algunos casos, la elección del soporte hardware y el lenguaje de implementación, pueden figurar en esta especificación. Otras decisiones de diseño, como el uso de un determinado algoritmo, no tienen nada que ver con el análisis y no son propiamente requisitos del sistema.

La especificación preliminar no es un documento inmutable. Normalmente será ambiguo, incompleto, incorrecto e inconsistente. Incluso aunque estén expresados de forma precisa, algunos de los requisitos reflejados pueden causar efectos secundarios no deseados (gran cantidad de información almacenada, tiempos de carga o de ejecución muy grandes) o llevar a costes de implementación demasiado grandes, frente a unos beneficios pequeños o innecesarios. Es necesario tomar la decisión de cumplir o no estos requisitos.

3.9. Principios del análisis de requisitos.

A lo largo de la historia de la Ingeniería del Software se han ido desarrollando diversos métodos de análisis del software, y diversas herramientas para facilitar el uso de estos métodos. Cada uno de los métodos tiene sus peculiaridades, utiliza una notación gráfica o textual distinta y tiene un campo de aplicación distinto. Sin embargo todos se basan en un conjunto de principios fundamentales.

Principios de análisis de requisitos del software

- *Identificar y representar el ámbito de información del problema.*
- *Modelar la información, la función y el comportamiento del sistema.*
- *Descomponer el problema de forma que se reduzca la complejidad.*
- *Avanzar desde lo más general a lo más detallado*

Los dos primeros puntos se refieren, por tanto, a conseguir representar el sistema y la información que maneja mediante un diagrama o una representación textual que permita comprender fácilmente qué información se maneja y qué funciones se realizan con esta información.

Los dos últimos se refieren a un método de trabajo *top-down*, que permita la división del problema en subproblemas, y vaya avanzando de los más general a lo más específico, de forma que la síntesis de la solución siga una estructura jerárquica.

3.9.1. El ámbito de información.

La tarea que realiza el software va a consistir siempre en procesar información: cualquier aplicación que consideremos responde a una estructura de entrada-procesamiento-salida, donde el software se encarga de procesar unos determinados datos de entrada para producir unos datos de salida. Estos datos pueden ser lógicos, numéricos, cadenas de caracteres, imágenes...

Pero en un sistema software, la información no está representada sólo por datos sino también por sucesos o eventos. En el sistema de clasificación de paquetes se procesa el número de identificación de cada paquete, obteniéndose el contenedor donde debe ser almacenado. Pero también se procesan sucesos: la clasificación de paquetes se para si sucede que los contenedores se llenan; o se genera un informe de los paquetes procesados si el operador lo solicita. Los eventos son normalmente

datos de tipo lógico (la activación o desactivación de la señal de un sensor, por ejemplo) y su procesamiento está ligado con el control del sistema.

En un sentido general, podemos decir que la información que maneja un sistema software se divide en datos y eventos. Los datos se refieren normalmente a la información que procesa el sistema y los eventos a cuándo debe procesarse esa información. El ámbito de información del sistema admite, por tanto, dos puntos de vista: por un lado, el flujo de datos (cómo se mueven los datos por el sistema y cómo se van transformando estos datos), y por otro, el flujo de control (cómo se controla el sistema y cuándo hay que realizar cada procesamiento).

3.9.2. Modelado del sistema software.

Los modelos se utilizan en el campo de la ingeniería para entender mejor lo que se quiere construir. En la Ingeniería del Software se utilizan modelos para la información que transforma el software (modelos de datos), para las funciones que transforman esa información (modelos de procesos) y también para definir el comportamiento del sistema (modelos de control). Todos los métodos de análisis que veremos posteriormente son en realidad métodos de modelado de sistemas.

Los modelos que se realizan en la fase de análisis sirven para tres cosas:

Utilidad de los modelos

- *Ayudan al analista a entender la información, la función y el comportamiento del sistema, con lo que el análisis puede hacerse de forma más fácil y sistemática.*
- *Sirven de base para el trabajo del diseñador. La arquitectura de las aplicaciones y los datos debe corresponderse con los modelos del análisis.*
- *Sirven también para realizar la validación del producto una vez desarrollado. Por una parte, el sistema final debe comportarse como indica el modelo. Por otra, el sistema final permite comprobar la consistencia y la eficacia de la especificación.*

Para realizar un modelo podemos utilizar una notación gráfica (y el modelo estará representado mediante una serie de diagramas) o bien una notación textual (y tendremos modelos en lenguaje natural o en un lenguaje de especificación más formal).

3.9.3. Descomposición del sistema.

La descomposición se utiliza para abordar problemas que son demasiado grandes o demasiado complejos para resolverlos directamente. Mediante descomposición, el problema se divide en subproblemas más sencillos, que realizan una función más clara y que pueden entenderse más fácilmente. La descomposición se puede aplicar a los ámbitos de la información, de la función o del comportamiento.

3.9.4. Análisis de lo general a lo específico.

Descomponiendo un problema en subproblemas, y definiendo modelos de cada uno de estos problemas y subproblemas, llegaremos a establecer una jerarquía de modelos, que irán desde el más general (el de arriba del todo) a los más específicos (en los niveles bajos de la jerarquía). Cada modelo de la jerarquía produce, por descomposición de las funciones que realiza, una serie de modelos, cada uno de los cuales realiza una de estas funciones y que tendrán un mayor nivel de detalle.

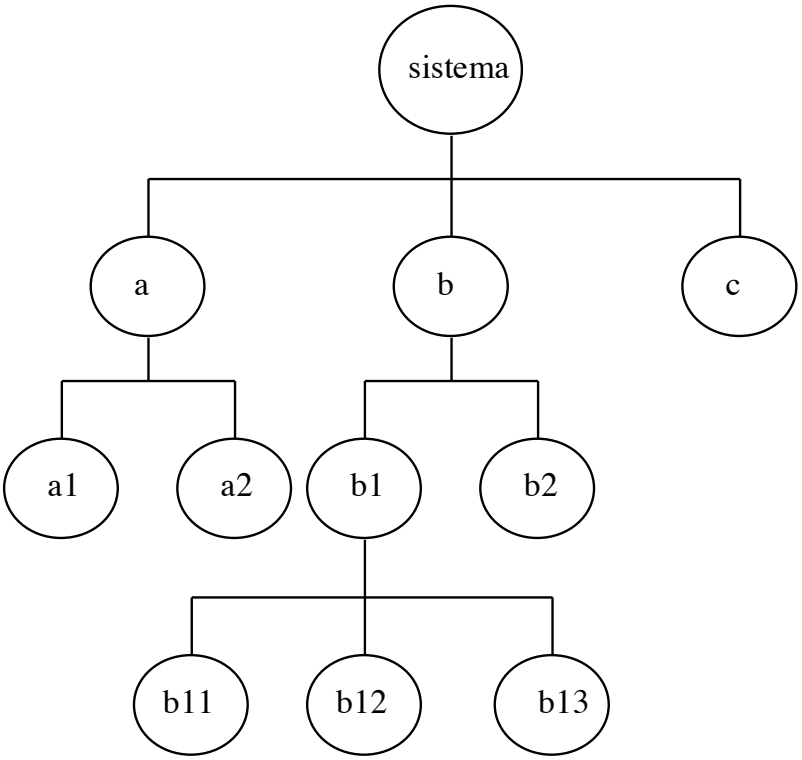


Fig. 3.3. Jerarquía de modelos

3.10. Principios de especificación.

La especificación, independientemente del modo en que se realice, puede ser vista como un proceso de representación. Los requisitos se representan de forma que conduzcan finalmente a una correcta implementación del sistema.. Podemos establecer una serie de principios que deben seguirse para obtener una buena especificación. Estos principios son los siguientes:

Principios de la especificación.

- *Es necesario separar funcionalidad e implementación.*
- *El lenguaje de especificación debe estar orientado al proceso.*
- *La especificación debe abarcar todo el sistema del que el software es parte.*
- *La especificación debe abarcar también el entorno del sistema.*
- *La especificación debe modelar el dominio del problema.*
- *La especificación debe ser operativa.*
- *La especificación debe ser ampliable y tolerante a la incompletitud.*
- *La especificación debe estar localizada y débilmente acoplada.*

• Es necesario separar funcionalidad e implementación.

Una especificación es, por definición, una descripción de lo que se quiere realizar, no de cómo se va a realizar o implementar. Como ejemplo de esto podemos tomar una especificación formal expresada mediante algún lenguaje declarativo: dado un conjunto de valores de entrada se produce otro de salida. Estas especificaciones se centran exclusivamente en el *qué* y no en el *cómo*. Esto es debido a que el resultado es una función matemática de la entrada. En estos casos de lo que se trata es de buscar alguna o todas las soluciones, tales que se cumpla P(entrada), donde P es un predicado que representa al sistema.

• El lenguaje de especificación debe estar orientado al proceso.

El ejemplo anterior muestra cómo se modelan sistemas que no están afectados por el entorno. Sin embargo, un caso más general debe considerar un sistema que interactúe con un entorno dinámico, modificando su comportamiento según los estímulos que recibe. Este podría ser el caso de un sistema empotrado. En este caso no podemos representar el sistema mediante una función matemática que relacione la entrada y la salida, sino que debemos emplear una descripción orientada al proceso, en la que la especificación del *qué* se consigue estableciendo un modelo del comportamiento deseado en términos de respuestas funcionales a distintos estímulos del entorno.

- **La especificación debe abarcar todo el sistema del que el software es parte.**

Un sistema está compuesto de partes que interactúan. El comportamiento de un componente específico, como es el software, sólo puede ser definido en el contexto del sistema completo. Por tanto, la especificación debe describir todos los componentes del sistema, estableciendo las interfaces entre estos componentes, y no sólo el componente software

- **La especificación debe abarcar también el entorno del sistema.**

Por el mismo motivo, hay que especificar también el entorno en el que opera el sistema. La especificación del entorno permite describir la interfaz del sistema de la misma forma que las interfaces de los componentes del mismo. De esta forma podemos representar sistemas dinámicos cuyo comportamiento varía dependiendo de los estímulos que reciban del entorno.

Esta especificación del entorno no se hace con vistas a implementarlo, puesto que ya nos viene dado y no podemos modificarlo, sino que sirve para definir los límites del sistema y su interfaz, permitiendo además probarlo.

- **La especificación debe modelar el dominio del problema.**

La especificación del sistema ha de ser un modelo del dominio del problema, en vez de un modelo de diseño o implementación. Debe describir el sistema tal como es percibido por los expertos en el dominio de aplicación. Los elementos del sistema deben corresponderse con objetos reales de dicho dominio, ya sean individuos, máquinas u organizaciones, y las acciones que se realizan deben corresponderse con lo que realmente ocurre en el dominio del problema. Además, deben poder describirse en la especificación las reglas o leyes que gobiernan los objetos del dominio de aplicación. Algunas de estas leyes son restricciones sobre los estados del sistema, del tipo de ‘dos objetos no pueden estar en el mismo lugar al mismo tiempo’ y otras describen cómo responden los objetos cuando se actúa sobre ellos (por ejemplo las leyes del movimiento de Newton). En este caso, son parte inherente de las especificaciones del sistema.

Para que esto sea posible, el formato de la especificación y su contenido deben ser adecuados al problema.

- **La especificación debe ser operativa.**

La especificación ha de servir para determinar si una implementación concreta la satisface. Esto puede hacerse bien mediante la verificación de la corrección del sistema implementado, cosa que normalmente no puede demostrarse, o bien mediante una serie de casos de prueba elegidos arbitrariamente.

A partir de los resultados de una implementación sobre un conjunto arbitrario de datos de entrada, debe ser posible usar la implementación para validar estos resultados, a pesar de que la especificación no describa el *cómo* sino solamente el *qué*.

Este principio no establece que la especificación tenga que ser ejecutable, sino más bien que pueda ser utilizada para demostrar teoremas.

- **La especificación debe ser ampliable y tolerante a la incompletitud.**

Una especificación es un modelo o abstracción de un sistema real, por tanto nunca será completa, y puede desarrollarse a distintos niveles de detalle. Normalmente el desarrollo de especificaciones se hace de forma incremental, estando en cada momento elementos del sistema parcialmente especificados. Aunque esto debilita el análisis, las herramientas de prueba de las especificaciones y de comprobación de que las implementaciones son correctas deben ser capaces de manejar especificaciones incompletas.

- **La especificación debe estar localizada y débilmente acoplada.**

Durante su desarrollo, una especificación sufre continuas modificaciones. Por este motivo, la estructura de la especificación debe permitir que estas modificaciones se realicen lo más fácilmente posible. El principio de localidad establece que si es necesario modificar un elemento de la especificación, esta modificación se realice sólo en un punto de la misma. El principio del acoplamiento débil establece que se puedan añadir y quitar partes de la especificación fácilmente.

Par cumplir estos dos principios la especificación ha de ser no redundante y modular, con interfaces breves y bien definidas.

3.11. Especificación del sistema.

La especificación del sistema es un documento que sirve de base para el análisis de cada uno de los componentes que intervienen en él, sean hardware, software o personas. Describe la función, rendimiento y restricciones que debe cumplir el sistema y limita cada uno de sus componentes, indicando el papel de cada uno de ellos y su interfaz.

La especificación del sistema es el documento que permite comprobar si el análisis realizado por el ingeniero de sistemas satisface las necesidades del cliente. Por este motivo, el cliente y el analista de sistemas deben revisar conjuntamente esta especificación para determinar si:

Evaluación inicial de la especificación

- *se ha delimitado correctamente el ámbito del proyecto.*
- *se ha definido correctamente la funcionalidad, las interfaces y el rendimiento.*
- *las necesidades del usuario y el análisis de riesgos justifican el desarrollo del proyecto.*
- *cliente y analista tienen la misma percepción de los objetivos del proyecto.*

Después de esta revisión con el usuario, es necesario realizar una evaluación técnica, para determinar si:

Evaluación técnica de la especificación

- *Las estimaciones de riesgos, coste y agenda se corresponden con la complejidad del proyecto.*
- *Todos los detalles técnicos (asignación de funciones, interfaces, rendimientos) están bien definidos.*
- *La especificación del sistema sirve de base para las fases siguientes (en concreto para la ingeniería del software).*

Un posible formato de especificación del sistema podría ser el siguiente:

Especificación de requisitos del sistema.

I. Introducción.
- A. Ámbito y propósito del documento.
- B. Descripción general.
  1. Objetivos.
  2. Restricciones.

II. Descripción funcional y de datos.
- A. Diagrama de contexto de arquitectura.
- B. Descripción del DCA.

III. Descripción de los subsistemas.
- A. Especificación del diagrama de arquitectura para el subsistema *i*.
  1. Diagrama de flujo de arquitectura.
  2. Narrativa del módulo del sistema.
  3. Rendimiento.
  4. Restricciones de diseño.
  5. Asignación de componentes.
- B. Diccionario de la arquitectura.

IV. Resultados de la simulación del sistema.
- A. Modelo usado para la simulación.
- B. Resultados de la simulación.
- C. Aspectos especiales de rendimiento.

V. Aspectos del proyecto.
- A. Costes del proyecto.
- B. Agenda.

VI. Apéndices.

3.12. Especificación del software.

La especificación del software es el documento que culmina las labores del análisis de requisitos. Debe contener una descripción detallada del ámbito de información, las funciones y el comportamiento asignados al software durante el análisis del sistema. Además debe contener información sobre los requisitos de rendimiento y restricciones de diseño y sobre las pruebas que se han de realizar para probar el software una vez que haya sido construido.

Existen numerosos esquemas para la especificación del software, que varían dependiendo de el método de análisis elegido. A titulo general, podemos proponer el siguiente:

Especificación de requisitos del software.

I. Introducción.
- A. Referencia del sistema.
- B. Descripción general.
  1. Objetivos.
  2. Restricciones.

II. Descripción de la información.
- A. Representación del flujo de la información.
  1. Diagramas de flujo de datos.
  2. Diagramas de flujo de control.
- B. Representación del contenido de la información.

III. Descripción funcional.
- A. Narrativa de procesamiento.
- B. Restricciones.
- C. Requisitos de rendimiento.
- D. Restricciones de diseño.

IV. Descripción del comportamiento.
- A. Especificaciones de control.
- B. Estados del sistema.
- C. Eventos y acciones.
- D. Restricciones de diseño.

V. Criterios de validación.
- A. Descripción de las pruebas.
- B. Respuesta esperada del software.
- C. Consideraciones especiales.

VI. Apéndices.

Este esquema se corresponde con un análisis realizado siguiendo métodos estructurados, pero podríamos adaptarlo fácilmente al AOO.

La introducción describe el software en el contexto del sistema del que va a formar parte, indicando cuáles son los objetivos y restricciones del sistema software.

La sección de descripción de la información da una descripción detallada del problema que tiene que resolver el software, indicando los flujos de información que se mueven entre las partes del mismo y el contenido de estos flujos de información (mediante un diccionario de datos). Si usamos análisis estructurado, aquí irían los DFDs y DFCs. Si usamos AOO, incluiríamos aquí los diagramas de objetos.

En la sección de descripción funcional figurarían las definiciones de cada una de las funciones que componen el sistema, mediante la especificación de las primitivas de proceso. Además, hay que definir y justificar todas las restricciones, incluidas las de rendimiento y de diseño que deban satisfacer estas funciones. En el caso del AOO, figurarían aquí también los DFDs.

La sección de descripción del comportamiento debe incluir las especificaciones de control del sistema, normalmente en forma de Diagramas de Estados, definiendo también, si es necesario, las acciones y actividades a realizar y los eventos o sucesos que disparan las transiciones.

Por último, la sección de criterios de validación debe definir las pruebas que permitan determinar si una implementación del sistema satisface la función, restricciones y rendimiento establecidos en la especificación. Esto no es una tarea trivial, pues precisa un buen conocimiento de los requisitos del software y no debemos dejarla para cuando el software ya esté acabado, puesto que entonces se tiende a pasarla por alto o a realizar las pruebas precisamente en función del software construido y no en función del proyectado.

Al igual que la especificación del sistema, la especificación de requisitos del software debe sufrir una revisión realizada conjuntamente por el cliente y el ingeniero del software. Esta revisión puede hacerse a dos niveles. En primer lugar, para determinar si el sistema cumple con la función, restricciones y rendimiento requeridos por el cliente, y para ver si ha de cambiarse algunas de las estimaciones de coste, riesgo o agenda definidas en el análisis del sistema.

El segundo nivel de revisión es mucho más detallado, y tiene por objeto detectar imprecisiones o ambigüedades en la especificación. Hay que revisar los términos vagos, como ‘a veces’, ‘a menudo’ o ‘alguno’ y las listas no exhaustivas, que acaban en ‘etc.’ o ‘y otros’ e intentar precisarlos. También hay que tener cuidado con las expresiones de certeza, como ‘siempre’, ‘todos’ o ‘nunca’ y comprobar si efectivamente son correctos (ej. el detalle de una factura siempre cabe en una página). En muchos casos, expresiones como ‘obviamente’ o ‘por tanto’ no son tan obvias e intentan ocultar una laguna en la especificación (ej. hoy es miércoles, por tanto, llueve).

3.13. Especificación formal y especificación informal.

Como vemos, el principal objetivo de la especificación es definir de forma clara y no ambigua la función y restricciones del software, de forma que se eviten problemas en las etapas de diseño y codificación. A partir de una especificación confusa o ambigua surgen continuamente dudas a la hora de implementar el software. Cada una de estas dudas plantea una serie de posibilidades y obliga a elegir entre varias alternativas durante el diseño y la implementación. Estas decisiones no se toman con un conocimiento completo del problema (las deben tomar diseñadores y programadores que nunca han hablado con el cliente), por lo que conducen a implementaciones que, con seguridad, no cumplirán las necesidades del cliente. Por otro lado, es imposible **demostrar** si una implementación concreta satisface una especificación.

Los métodos de análisis que se han usado tradicionalmente conducen a especificaciones de este tipo, redactadas en lenguaje natural o, en el mejor de los casos, en pseudocódigo. Para modelar y describir el software se utiliza una combinación de diagramas, textos, tablas y notaciones sencillas.

La otra alternativa consiste en describir la el sistema utilizando un lenguaje de especificación formal, basado en la lógica o las matemáticas, con una sintaxis y una semántica formales. De esta forma se elimina la ambigüedad y sería posible demostrar la corrección y completitud de la especificación e incluso de una implementación de esta especificación. Además, el rigor de la descripción matemática fuerza al ingeniero a pensar con más cuidado sobre el problema en cuestión. Las continuas revisiones que recomiendan los métodos clásicos se sustituyen aquí por análisis y demostraciones matemáticas, con la ventaja de que, si conseguimos demostrar que un programa es correcto, nunca presentará un error, cosa que no es posible conseguir utilizando métodos informales.

Los inconvenientes de las especificaciones formales son que son más difíciles de usar y de entender, por lo que, si bien son más precisas, es discutible que sean más claras. Además se pierde la posibilidad de revisarlas con el cliente, y las demostraciones de corrección y completitud son tan complejas que, hasta ahora, sólo han podido aplicarse a casos muy sencillos. Por estos motivos, el uso de especificaciones formales se ha venido reservando a aplicaciones muy concretas del software, en las que la seguridad y fiabilidad sean un criterio fundamental.

Sistema de clasificación de paquetes.

El sistema de clasificación de paquetes debe realizarse de forma que los paquetes que se mueven a lo largo de una cinta transportadora sean identificados (para lo cual van provistos de un código numérico) y clasificados en alguno de los seis contenedores situados al final de la cinta. Los paquetes de cada tipo aparecen en la cinta siguiendo una distribución aleatoria y están espaciados de manera uniforme. La velocidad de la cinta debe ser tan alta como sea posible; como mínimo el sistema debe ser capaz de clasificar 10 paquetes por minuto. La carga de paquetes al principio de la cinta puede realizarse a una velocidad máxima de 20 paquetes por minuto. El sistema debe funcionar las 24 horas del día, siete días a la semana.

Asignación 1.

Se enseña a un operador a clasificar y se le sitúa al final de la cinta transportadora. El operador lee el código de identificación del paquete y lo coloca en el contenedor adecuado. Para que el sistema funcione continuamente es preciso establecer turnos y para evitar la parada 'en caso de avería' del operador hay que establecer también turnos de guardia.

Asignación 2.

Se etiquetan los paquetes con el código de barras correspondiente a su código numérico. Se coloca un lector de códigos de barras, un controlador y un mecanismo de distribución (algún tipo de dispositivo mecánico, basado, por ejemplo, en una cinta transportadora y un émbolo que empuje el paquete cuando pasa por delante del contenedor adecuado) al final de la cinta. La salida del código de barras pasa al controlador, que controla el funcionamiento del mecanismo de distribución. Este sistema puede estar funcionando continuamente, pero para evitar la parada en caso de averías o mantenimiento será necesario duplicarlo, manteniendo uno de reserva, o bien prever la solución manual para estos casos.

Asignación 3.

Se coloca un lector de códigos de barras conectado a un brazo robot, al final de la cinta transportadora. La salida del código de barras pasa al controlador del robot, que coge el paquete y lo coloca en el contenedor apropiado. Para evitar la parada por avería o mantenimiento hay que duplicar el sistema o bien considerar la clasificación manual en estas situaciones.

Fases del análisis de sistemas.

- Identificación de las necesidades del cliente.
- Evaluación de la viabilidad del sistema.
- Análisis económico y técnico del proyecto.
- Asignación de funciones a cada uno de los elementos del sistema.
- Obtención de una definición del sistema que sea la base del trabajo posterior.

Sistema de clasificación de paquetes. Análisis coste/beneficio

Costes del sistema actual: 17.5 Mill. ptas./año

Costes del nuevo sistema:

Inversión inicial: 40 Mill. ptas.

Funcionamiento: 2 Mill. ptas./año

Mantenimiento: 1 Mill. ptas./año

Operatividad: 0.95

Total coste anual: 3.875Mptas/año

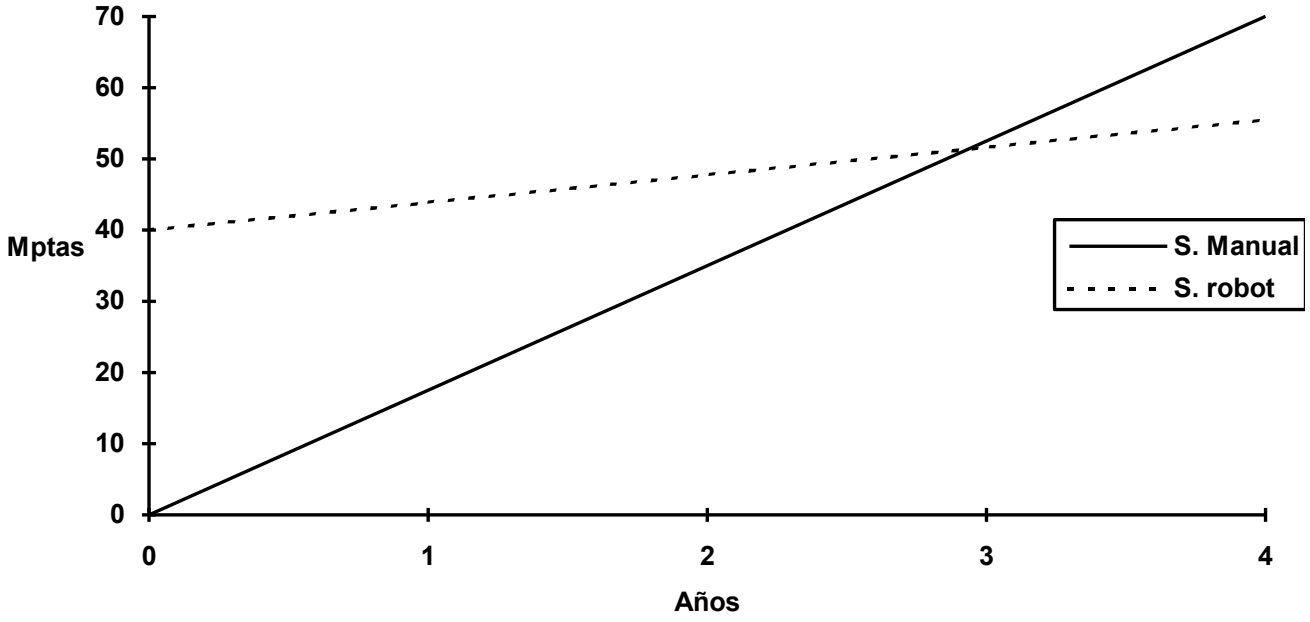


Fig. 3.1. Análisis coste/beneficio

Sistema de clasificación de paquetes. Criterios de evaluación.

	Alternativa 1	Alternativa 2	Alternativa 3
Inversión inicial	Nula	Moderada	Grande
Coste funcionamiento	Grande	Moderado	Moderado
Tiempo amortización	Nulo	Moderado	Grande
Fiabilidad	Moderada	Pequeña	Grande
Mantenimiento	Nulo	Grande	Moderado

Fig. 3.2. Criterios de evaluación de las diferentes alternativas.

Principios de análisis de requisitos del software

- Identificar y representar el ámbito de información del problema.
- Modelar la información, la función y el comportamiento del sistema.
- Descomponer el problema de forma que se reduzca la complejidad.
- Avanzar desde lo más general a lo más detallado

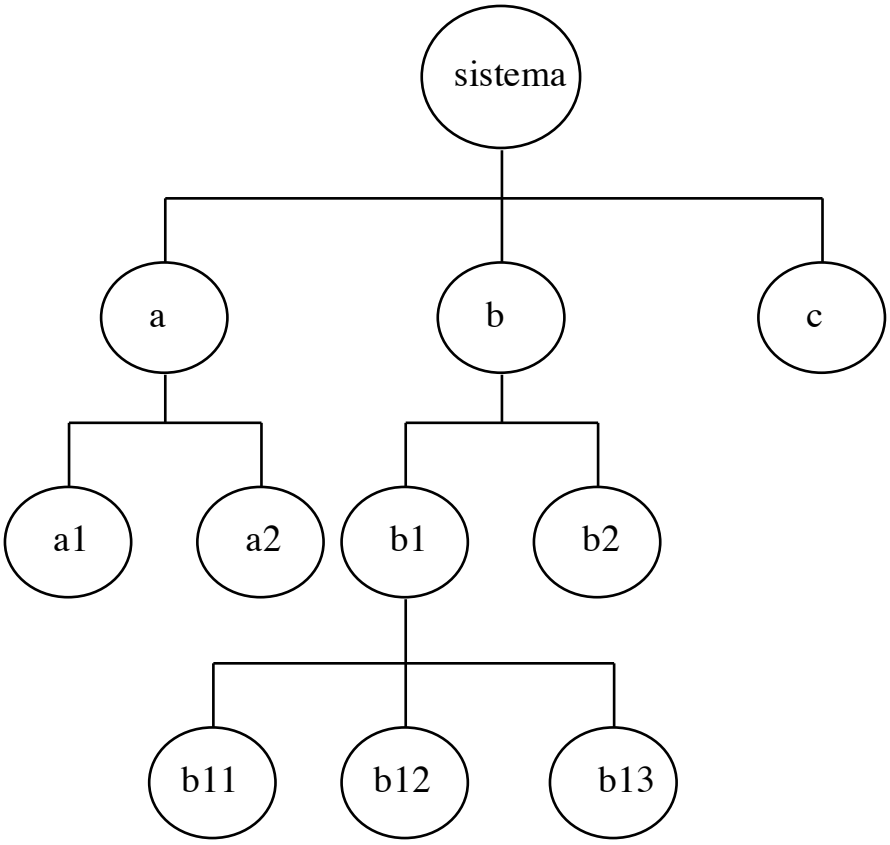


Fig. 3.3. Jerarquía de modelos.

Tema 4. Análisis estructurado.

4.1. Introducción.

4.2. Notaciones.

4.2.1. Diagramas de flujo de datos.

4.2.2. Especificaciones de proceso.

4.2.3. Diagramas de flujo de control.

4.2.4. Especificaciones de control.

4.2.5. Diagramas de estados.

4.2.6. Diagramas Entidad/Relación.

4.3. Diccionario de datos.

4.4. Metodología del análisis estructurado.

4.4.1. Fases.

4.4.2. Consistencia entre los modelos.

4.5. Modelos del sistema: esencial y de implementación.

4.6. Ejemplos.

Bibliografía

Ingeniería del Software: un enfoque práctico
R.S. Pressmann. Mc Graw Hill. Madrid, 1993. 3ª Edición.

Modern Structured Analysis
E. Yourdon. Prentice-Hall. New Jersey, 1989.

Strategies for Real-Time System Specification
D.J. Hatley & I.A. Pirbhai. Dorset House. New York, 1988.

Object-Oriented Modeling and Design
Rumbaugh, J. et al. Prentice Hall, 1991.

Tema 4. Análisis estructurado.

4.1. Introducción.

Todos los métodos de análisis de requisitos se basan en la construcción de modelos del sistema que se pretende desarrollar. Utilizando alguna notación, propia de cada método, creamos modelos que reflejen el sistema, y aplicamos técnicas de descomposición y razonamiento *top-down*, de tal forma que al final establecemos la esencia del sistema que pretendemos construir.

El desarrollo de modelos presenta algunas ventajas claras:

- **Permite centrarse en determinadas características del sistema, dejando de lado otras menos significativas.** Esto nos permite centrar las discusiones con el usuario en los aspectos más importantes del sistema, sin distraernos en características del sistema que sean irrelevantes.

- **Permite realizar cambios y correcciones en los requisitos a bajo coste y sin correr ningún riesgo.** Si nos damos cuenta que no habíamos entendido las necesidades del usuario o si el usuario ha cambiado de idea acerca de los requisitos del sistema, podemos cambiar el modelo o incluso desecharlo y empezar de nuevo. Si no hiciésemos modelos, los cambios en los requisitos sólo se efectuarían después de construir el producto software, y el coste sería muchísimo mayor.

- **Permite verificar que el ingeniero del software ha entendido correctamente las necesidades del usuario** y que las ha documentado de tal forma que los diseñadores y programadores pueden construir el software.

Sin embargo, no todas las técnicas de análisis logran estos tres objetivos: una descripción del sistema de 500 páginas (que sería, en sentido estricto, un modelo) oculta las características del sistema, tanto las relevantes como las irrelevantes, su desarrollo puede costar más que el del propio sistema, es difícil de modificar y no permite verificar si se han establecido los requisitos.

El análisis de requisitos clásico, usado hasta finales de los 70, consistía en redactar especificaciones funcionales, en forma de documentos textuales, de este tipo:

- **Eran monolíticas**. Para entender el sistema había que leerse la especificación de cabo a rabo. No había posibilidad de que ni el analista ni el usuario pudiesen centrarse en una determinada parte de la especificación, sin tener que leerse el resto.

- **Eran redundantes**. La misma información se repartía en diferentes partes del documento. Debido a esto cualquier cambio que se hiciese en la especificación debía reflejarse en varios puntos del documento. Esta situación produce con frecuencia inconsistencia: si no se cambiaba en todos estos lugares la misma información (p.ej. el mismo código) tenía definiciones distintas.

- **Eran ambiguas**. Al estar escritas en lenguaje natural, podían ser interpretadas de forma distinta por analistas, usuarios, diseñadores o programadores. Hay estudios que muestran que el 50% de los errores encontrados en el sistema final y el 70% del coste de la corrección de los errores surgía de este tipo de malentendidos.

- **Eran imposibles de mantener o modificar**. Por todas las razones anteriores, la especificación del sistema estaba totalmente obsoleta cuando finalizaba el desarrollo. En muchos casos, estaba incluso obsoleta cuando finalizaba la fase de análisis de requisitos. Debido a esto, la mayor parte del software de la época carece de una documentación fiable, incluso aunque se hubiese realizado análisis y redactado una especificación.

Por estos motivos fueron surgiendo nuevos métodos de análisis, cuyo objetivo era obtener especificaciones:

- **Gráficas**. Formadas por una colección de diagramas, acompañados de información textual detallada, que sirve de material de referencia, más que de cuerpo principal de la especificación.

- **Particionadas**. De forma que fuese posible leerse o trabajar sobre partes individuales de la especificación sin tener que leérsela toda.

- **Mínimamente redundantes**. De forma que los cambios en los requisitos necesitasen reflejarse en un sólo punto de la especificación.

- **Transparentes**. De forma que fuesen tan fáciles de leer y de entender que el que las utilizase no se diese cuenta de que está mirando una representación del sistema, en lugar del sistema en sí. Los sistemas son los que son complejos, las especificaciones tienen que ser claras y sencillas.

En este tema nos centraremos en los *Métodos de análisis estructurado*, que son los más utilizados en la actualidad. En los temas siguientes veremos métodos de análisis orientado a objetos y utilizando lenguajes de especificación formal.

Bajo el nombre genérico de *Análisis estructurado*, se engloban una serie de aportaciones de diversos autores entre los que cabe citar, por orden cronológico, a De Marco, Yourdon, Gane &

Sarson, Ward & Mellor y Hatley & Pirbhai. Cada uno de ellos ha desarrollado su propio método de análisis, mejorando, ampliando o adaptando los anteriores a algún campo de aplicación específico.

Siguiendo las técnicas del análisis estructurado podemos describir los sistemas desde tres puntos de vista:

MODELO	HERRAMIENTAS		
Datos	Diagramas de Entidad/Relación		Diccionario
Procesos	Diagramas de Flujo de Datos Especificaciones de Proceso	⇔	De
Control o Comportamiento	Diagramas de Flujo de Control Especificaciones de Control Diagramas de Transición de Estados		Datos

Fig. 4.1. Diferentes modelos de un sistema software.

Punto de vista del proceso.

Describiremos el sistema como un conjunto de operaciones de proceso de información. Estas operaciones reciben unos flujos de datos de entrada y los transforman en flujos de datos de salida. Para describir el sistema desde este punto de vista utilizaremos **Diagramas de Flujo de Datos** y **Especificaciones de procesos**.

Punto de vista de los datos.

Describiremos el sistema mediante un modelo de los datos que utiliza, haciendo explícitas las relaciones que se establecen entre esos datos. Para ello utilizaremos **Diagramas Entidad/Relación**.

El desarrollo del modelo de datos de un sistema evita el almacenamiento de información redundante e incoherente, garantiza la integridad y seguridad de los datos y simplifica el mantenimiento.

Punto de vista del comportamiento.

Describiremos el sistema como una sucesión de estados o modos de funcionamiento. Indicaremos también cuáles son las condiciones o eventos que hacen que el sistema pase de un modo a otro. Utilizaremos **Diagramas de Flujo de Control**,. **Especificaciones de Control** y **Diagramas de Estados**.

Es importante indicar que cualquiera de los tres tipos de diagramas citados van a describir el mismo sistema, que tendrá una implementación física determinada, variando simplemente el punto

de vista desde el que se contempla. La relación entre los distintos diagramas se establece a través del **Diccionario de Datos**.

Por último, hay que tener en cuenta la influencia que han tenido las herramientas de análisis en el uso y difusión de los métodos de análisis estructurado. Estas herramientas permiten dibujar los diagramas, hacen mucho más sencilla su modificación y comprueban su completitud y consistencia. Además muchas de estas herramientas sirven de soporte no sólo a la fase de análisis sino a todas las etapas del ciclo de vida. Estas son las herramientas CASE.

4.2. Notaciones.

En este punto vamos a estudiar las diferentes técnicas y notaciones de modelado de sistemas que puede utilizar un ingeniero del software. Combinando todas ellas se puede establecer un modelo completo del sistema, pero lo importante es usar aquéllos diagramas que nos sirvan en una situación determinada. Habrá situaciones en que lo importante sea modelar el flujo de datos, mientras que el control sea obvio. En otras situaciones los datos serán lo suficientemente complejos como para justificar un modelo de datos. En otras, p.ej. en sistemas de tiempo real, será necesario especificar el comportamiento del sistema.

En general, la mayoría de los sistemas requerirán utilizar varios modelos para representarlos. Cada modelo se centra en un número limitado de aspectos del sistema, dejando de lado otros (esta era una de las características de los modelos). Combinando los diversos modelos podemos tener una visión detallada de todas las características del sistema. Esto es especialmente cierto hoy en día, cuando los sistemas software manejan estructuras de datos complejas, realizan funciones complejas y tienen pautas de comportamiento complejas.

4.2.1. Diagramas de flujo de datos.

Como su propio nombre indica, un sistema de procesamiento de datos incluye tanto datos como procesos, y cualquier análisis de un sistema así debe incluir ambos aspectos. Necesitamos una técnica para modelar sistemas que describa:

- *Qué funciones son las que realiza el sistema.*
- *Qué interacción se produce entre estas funciones.*
- *Qué transformaciones de datos realiza el sistema.*
- *Qué datos de entrada se transforman en qué datos de salida.*

A medida que la información se mueve a través del software, va siendo modificada mediante una serie de transformaciones. El DFD es una técnica gráfica que representa el flujo de

información y las transformaciones que se aplican a los datos al moverse desde la entrada a la salida.

Elementos de un DFD.

Para representar el sistema mediante DFDs utilizaremos la notación de Yourdon (1975), posiblemente la más extendida. Esta notación es la indicada en la transparencia 4.3. Los elementos que aparecen en el DFD pueden ser:

- Procesos. Representan elementos software que transforman información. Son, por tanto, los componentes software que realizan cada una de las funciones del sistema, transformando datos de entrada en datos de salida.

- Entidades externas. Representan elementos del sistema informático o de otros sistemas adyacentes (en cualquier caso se trata de algo que está fuera de los límites del sistema software) que producen información que va a ser transformada por el software o que consumen información transformada por el software. Los flujos de datos que comuniquen el sistema con las entidades externas representan las interfaces del sistema. Las entidades externas sólo aparecen en el diagrama de contexto.

- Almacenes de datos. Representan información almacenada que puede ser utilizada por el software. Los almacenes de datos permiten guardar temporalmente información que luego puede ser procesada por el mismo proceso que la creó o por otro distinto. En la mayoría de los casos, utilizaremos almacenes de datos cuando dos procesos intercambian información pero no ocurren o se ejecutan simultáneamente. En otros casos, utilizaremos los almacenes como copia de seguridad de los datos, para evitar pérdidas de información en caso de que el sistema falle. Los almacenes de datos pueden ir desde registros temporales para almacenar un dato hasta ficheros o bases de datos.

- Flujos de datos. Representan datos o colecciones de datos que fluyen a través del sistema. La flecha indica el sentido de flujo. Posiblemente en los diagramas de nivel mayor existan flujos de datos bidireccionales, que luego son refinados en sucesivos diagramas, o incluso varios flujos de datos agrupados en uno sólo. Los flujos de datos conectan los procesos con otros procesos, con entidades externas o con almacenes de datos, y pueden converger o divergir si conectan un elemento del DFD con varios otros. Mientras que los almacenes de datos representan información estática o en reposo, los flujos de datos representan información en movimiento. Puede tratarse de un elemento de datos simple o compuesto (un registro) o incluso de una colección de datos de estructura compleja, p.ej. un árbol. En algunos casos, el flujo de datos puede representar elementos que no son datos,

sino p.ej. materiales, si estamos haciendo un diagrama de flujo de una cadena de producción, por ejemplo

Cualquiera de los elementos que aparecen en un DFD tiene que estar etiquetado con un nombre, corto y significativo. Los procesos van etiquetados con la función que realizan, (o bien con el nombre de la máquina, persona o grupo de personas que realizan esa función, en el caso de que no se trate de software). Lo mismo sucede con las entidades externas. Los flujos de datos van etiquetados con un nombre identificativo de la información que transportan, y posiblemente con el estado de dicha información (p. ej. *número de teléfono, número de teléfono correcto, número de teléfono erróneo*). Los almacenes de datos van etiquetados con un nombre significativo de la información que contienen, generalmente en plural.

Hay que tener en cuenta que un DFD no representa información sobre el comportamiento del sistema o sobre el control del mismo. Representa qué funciones o que transformaciones se realizan sobre los datos pero no cuándo se realizan o en qué secuencia.

Diagrama de contexto.

Se pueden utilizar DFDs para representar el sistema a cualquier nivel de abstracción. El DFD de nivel 0 se llama **diagrama de contexto** y en él el sistema está representado por un sólo *proceso*, que identifica cuál es la función principal del sistema, mostrando además los flujos de información que lo relacionan con otros sistemas: las *entidades externas*. El diagrama de contexto tiene una gran importancia puesto que resume el requisito principal del sistema de recibir ciertas entradas, procesarlas de acuerdo con determinada función y generar ciertas salidas. A partir del diagrama de contexto podemos ir construyendo nuevos diagramas que vayan definiendo con mayor nivel de detalle los flujos de datos y procesos de transformación que ocurren en el sistema, de forma que al final obtenemos una jerarquía de diagramas.

En general, cualquier proceso que aparezca en un DFD puede ser descrito más detalladamente en un nuevo. DFD. A esto lo llamaremos **explosión** de un proceso. En este DFD el proceso que estamos describiendo aparece descompuesto en una serie de subprocesos o subsistemas, cada uno encargado de realizar un aspecto determinado del proceso original. Los flujos de datos que entraban y salían del proceso que estamos describiendo deben entrar y salir del DFD que lo desarrolla. Además de estos flujos, el DFD contendrá por lo general nuevos flujos que comunican los procesos que figuran en él y posiblemente almacenes de datos. Las entidades externas sólo aparecen en el DFD de contexto.

Dado que en el DFD de contexto nuestro sistema estará representado por un sólo proceso. Este DFD de nivel 0 sólo dará lugar a un DFD de nivel 1. A su vez, este puede dar lugar a tantos DFDs de nivel 2 como procesos contenga, y así sucesivamente hasta que hayamos alcanzado un

nivel en el que los procesos sean lo suficientemente simples como para no necesitar su descripción más detallada en un DFD. De este forma, el modelo de procesos del sistema va a consistir en una jerarquía de DFDs.

Una ventaja de los DFDs es que no sólo se utilizan para representar modelos del software sino también en muchos otros campos, como la investigación operativa o la organización empresarial. Eso hace que estén ampliamente difundidos y que sea relativamente fácil mostrarlos al usuario y que éste los entienda.

DFDs. Reglas de construcción.

- *Un DFD debe contener menos de 10 elementos.*
- *Cada elemento de un DFD debe tener un nombre corto e identificativo.*
- *Es necesario numerar los procesos.*
- *Para modelar sistemas complejos se utiliza la explosión, que da como resultado DFDs a distintos niveles de detalle.*
- *No es conveniente utilizar más de 7 u 8 niveles.*
- *Los DFDs de niveles inferiores desarrollan de forma más concreta los procesos de niveles superiores.*
- *La explosión se realiza hasta alcanzar un nivel de especificación mínimo y sencillo.*
- *Debe mantenerse la consistencia de nombres en los distintos DFDs.*
- *Debe mantenerse la consistencia entre los distintos niveles, utilizando la regla de equilibrio.*
- *En cada DFD hijo deben representarse los mismos flujos de datos que en el proceso padre.*
- *No existen conexiones entre entidades externas*
- *No existen conexiones entre entidades externas y almacenes*
- *No existen conexiones entre almacenes*

Regla de equilibrio.

Normalmente, el sistema que estamos modelando será lo suficientemente complejo como para necesitar varios DFDs. Construiremos entonces una jerarquía de DFDs.

Cada DFD (hijo) de un nivel *n* será resultado de la explosión de un proceso (padre) de un DFD de nivel *n-1*. Es necesario que el título del DFD sea el nombre del proceso que desarrolla, que la numeración de los procesos en el DFD hijo se derive del número del DFD padre (p.ej. 3, 3.1, 3.2) y además hay que mantener la consistencia de los flujos de datos en ambos.

La regla de equilibrio dice que los flujos de datos que entran o salen del proceso padre deben aparecer en el DFD hijo, manteniendo el nombre y el sentido.

En el diagrama hijo estos flujos de datos tendrán un extremo libre, puesto que conectaban el proceso padre con algún elemento que ya no va a estar representado en el diagrama hijo. Una excepción son los flujos que conectan los procesos con almacenes de datos. Se recomienda representar los almacenes de datos en todos los DFDs en los que intervienen.

La regla de equilibrio también puede tener excepciones. Antes habíamos hablado de flujos de datos compuestos, e incluso bidireccionales (que normalmente transmiten una pregunta o petición y su respuesta). Estos flujos de datos suelen utilizarse en los DFDs de los niveles más bajos (es decir, los más altos de la jerarquía) para modelar el sistema con cierto grado de abstracción y para evitar representar muchos flujos de datos, lo que haría más confuso el diagrama. En los DFDs de niveles superiores, es necesario descomponer estos flujos en varios (p.ej. petición y respuesta). Para realizar esta descomposición podemos hacer divergir el flujo en el DFD hijo o al menos dejar reflejada esta descomposición en el diccionario de datos (que ya se verá qué es).

4.2.2. Especificaciones de proceso.

Los DFDs permiten identificar las principales funciones o transformaciones que realiza un sistema, y las relaciones entre estas funciones pero no indican nada acerca de los detalles de cómo se realizan estas transformaciones. Para definir los detalles de qué información de entrada se transforma en qué información de salida y cómo se realiza esta información se necesita una descripción textual de los procesos. Para esto utilizaremos las especificaciones de proceso.

En los DFDs de menor nivel (los más altos en la jerarquía), los procesos se describen mediante un nuevo DFD, que define más detalladamente las funciones que realiza y los flujos que maneja. Este proceso de descomposición debe continuar hasta que se alcance un nivel en el que un proceso puede ser descrito textualmente de forma sencilla y no ambigua. Estos procesos de los nodos hoja de la jerarquía se suelen llamar **primitivas de proceso**.

Una forma de especificar una primitiva de proceso es dando un algoritmo. Esto no quiere decir que el algoritmo propuesto en el análisis se corresponda con la implementación final del proceso, sino que es simplemente una forma de describir la operación. La elección del algoritmo definitivo se hace en la fase de diseño, considerando además de la especificación, criterios de eficiencia y las características del lenguaje de implementación.

Alternativamente, una primitiva de proceso puede ser descrita mediante una definición, sin especificar un algoritmo (p.ej. el proceso *Invertir Matriz* calcula la inversa de la matriz que recibe como entrada), matemáticamente o mediante un lenguaje de especificación formal, en lenguaje natural o mediante una tabla que indica los valores de los flujos de salida a partir de los valores de los flujos de entrada.

En cualquier caso, las especificaciones de proceso son textos breves (unas pocas líneas, en cualquier caso menos de una página), que definen la función que realiza la primitiva de proceso para transformar los flujos de entrada en flujos de salida.

4.2.3. Diagramas de flujo de control.

Los DFDs son muy útiles a la hora de modelar sistemas de proceso de datos, describen las funciones o transformaciones que realiza el sistema y cómo fluyen los datos a través del mismo.

Sin embargo, en los DFDs no se representa explícitamente el control o flujo de sucesos del sistema, por lo que estos diagramas no aportan ninguna información sobre cómo se comporta el sistema, sobre cuándo se realizan los procesos o en qué orden. En determinados sistemas de software de gestión, este funcionamiento puede ser intuitivo, y no necesitaremos reflejarlo en el modelo, pero en otras aplicaciones (p. ej. relacionadas con sistemas de tiempo real o con automatismos) sí que es importante que el modelo incluya los aspectos de comportamiento y control del sistema.

Podemos considerar tres situaciones posibles, referidas a cómo se comporta el sistema:

- Los procesos que figuran en el DFD están activos siempre. En este caso no necesitamos especificar el control del sistema.

- Los procesos se activan cuando llegan datos a través de sus flujos de entrada, transforman estos datos y emiten los resultados a través de los flujos de salida, permaneciendo entonces inactivos hasta la llegada de nuevos datos. Este comportamiento está implícito en la notación usada para los DFDs por lo que tampoco será necesario especificar el control. Muchas de las aplicaciones de gestión pueden ser descritas de esta forma.

- Cada proceso pasa por periodos de actividad e inactividad que están gobernados por mecanismos más complejos que los descritos anteriormente. Por lo general, un proceso se activará cuando se produzca determinada situación o suceso en el sistema y permanecerá activo hasta que se produzca otra situación. Comportamientos de este tipo no pueden ser reflejados de forma adecuada en los DFDs por lo que en estos casos necesitaremos describir el modelo de control o comportamiento del sistema, al menos para estos procesos que siguen patrones de comportamiento complejos. En aquéllas aplicaciones en las que el software controle el funcionamiento de otros dispostivos (p. ej. en un sistema empotrado) nos encontraremos con situaciones de este tipo.

Nota: Hay que tener en cuenta que el modelo de control del sistema, establece el comportamiento de éste a alto nivel: qué procesos se encuentran activos en cada momento o en que secuencia se realizan las transformaciones de los datos. Existe un control de bajo nivel que se refiere a los saltos condicionales y a los bucles de los programas. Este control de bajo nivel estaría reflejado en las PSPECs de los procesos.

Estas falta de expresividad de los DFDs llevaron a proponer varias extensiones de los mismos para modelar el control del sistema. Las más importantes son las de Ward & Mellor (1985) y Hatley & Pirbhai (1987). Nosotros vamos a seguir la notación de estos últimos.

Para modelar el control del sistema Hatley y Pirbhai proponen el uso de Diagramas de Flujo de Control (DFCs), similares a los DFDs ya vistos. Su método se basa en eliminar de los DFDs todo lo relativo a información de control : sucesos, señales, condiciones de datos, etc. y construir una jerarquía de DFCs paralela a la de DFDs. En estos DFCs se especifica todo el flujo de sucesos, señales y condiciones de datos del sistema, es decir, se indican los elementos de información que intervienen en el control de los procesos.

Nota: Otros autores (W & M) proponen incluir toda la especificación del control en los DFDs, por lo que no utilizan DFCs. Hatley y Pirbhai prefieren separar ambos modelos, para dejar más claro lo que es procesamiento y lo que es control, y para evitar recargar los DFDs con más información. No obstante, en sistemas sencillos con mecanismos de control sencillos sería admisible utilizar un único diagrama combinado, siempre y cuando quede reflejado lo que son datos y lo que son eventos.

Según lo anterior, construiremos una jerarquía de DFCs , empezando por un diagrama de contexto, de forma que a cada DFD (o al menos para aquéllos que sea necesario representar su control) le corresponda un DFC. En cada par DFD/DFC representaremos los mismos procesos y las mismas entidades externas, puesto que ambos representan modelos de la misma parte del sistema con el mismo nivel de detalle, aunque con puntos de vista distintos.

Las reglas sobre denominación numeración, relaciones padre - hijo y equilibrio que se aplican a los DFCs son las mismas que se establecen para los DFDs.

Elementos de un DFC.

Los elementos que aparecen en un DFC son prácticamente los mismos que en los DFDs.

- **Procesos, entidades externas y almacenes de datos**. Serán los mismos y tendrán el mismo significado que en el DFD al que corresponden. Se incluyen en el DFC básicamente como referencia, para mostrar a qué procesos afectan los mecanismos de control que estamos describiendo.

- **Flujos de control**. Se representan mediante trazos discontinuos y modelan el flujo de información de control en el sistema. Habrá procesos o entidades externas que generen información de control y otras que la consuman. La única diferencia es que los flujos de control transportan señales discretas (normalmente lógicas o de tipo enumerado) y son impulsos o eventos, es decir tienen una duración instantánea, mientras que los flujos de datos pueden transportar cualquier tipo de datos (posiblemente estructuras de datos complejas) y pueden transformar datos de manera continua.

- **Almacenes de control**. Se representan igual que los almacenes de datos pero con trazos discontinuos. Permiten almacenar información de control, para ser utilizada posteriormente.

Ventanas a especificaciones de control. Se representan mediante barras. Estas ventanas reciben y emiten flujos de control y representan la transformación de flujos de control en el sistema.

Los procesos de un DFC no representan procesamiento ni transformación de los flujos de control (lo que se hace en las CSPECs) ni tampoco representan los estados del sistema (que se representan en los DEs). Simplemente representan a los mismos procesos de los DFDs, y lo que indica el DFC es simplemente qué flujos de control reciben o generan estos procesos.

Otra detalle importante es que un flujo de control que entra en un proceso no indica que ese proceso se active mediante ese flujo. La activación y desactivación de procesos se indica en la CSPEC. Un flujo de control que entra en un proceso puede indicar dos cosas: bien que va a ser utilizado como una dato más para que el proceso lleve a cabo su función de transformación, o bien que va a ser utilizado para controlar alguno de los procesos en los que se descompone éste. Del mismo modo, un flujo de control que sale de un proceso indica simplemente que ha sido generado por éste e intervendrá en el control del comportamiento de algún otro.

Las especificaciones de control figuran normalmente en un nivel alto de la jerarquía de diagramas. Se encargan de controlar el funcionamiento del sistema, activando o desactivando sus procesos. A bajo nivel, mucho procesamiento de control puede representarse en las primitivas de proceso sin que esto influya en los resultados del análisis.

Cómo separar datos y control.

De entre todos los flujos de información que maneja un sistema software, ¿cómo podemos clasificar unos como datos y otros como control? y ¿qué procesamiento corresponde a procesos de datos y qué procesamiento corresponde al control del sistema? (los procesos de control no existen como tales en la notación H & P, sino que están incluidos en las CSPECs).

No hay unas normas estrictas para distinguir entre unos y otros, y en algunos casos se puede hacer de forma arbitraria. Como criterio de decisión utilizaremos el siguiente:

Modelaremos como señales de control únicamente aquéllas que intervengan en la activación y desactivación de algunos de los procesos del sistema. El resto las modelaremos como datos.

En determinadas situaciones, un elemento de información determinado se utiliza como dato en un proceso y como control en otro. En este caso, podemos modelarlo como dato (trazo continuo) en el DFD y como control (trazo discontinuo) en el DFC, pero asignaremos a ambos flujos el mismo nombre para mostrar que son el mismo.

Cuándo usar especificaciones de control.

Lo más conveniente es utilizar DFDs y flujos de datos siempre que sea posible, puesto que son más sencillos de realizar y de entender (la notación de los DFCs es más oscura y el uso de ambos obliga a mirar los dos a la vez). Sólo para aquellos aspectos del sistema que no podamos modelar con DFDs utilizaremos DFCs. Esto significa reducir el control en la medida de lo posible.

Pese a esto, a la hora de hacer el análisis de requisitos del sistema existe la tendencia de profundizar demasiado en el modelo de control del sistema. Con frecuencia se especifican detalles de implementación como flags, contadores, llamadas a procedimientos e interrupciones, que no tienen nada que ver con los requisitos del sistema. En el análisis de requisitos nos debemos centrarnos en el **modelo abstracto o lógico** del sistema más que en su implementación. Según esto el modelo de control debe ser usado para describir la lógica que controla los procesos principales del sistema y no para describir de forma detallada interacciones entre primitivas de proceso. Esto último puede ser descrito perfectamente utilizando el modelo implícito en los DFDs, que consiste en que cuando un proceso recibe un dato lo procesa de forma inmediata e instantánea. Como este no es el funcionamiento que tendrá el sistema real, existe la tendencia de enviar, junto con el dato, una señal de control que no tiene más objetivo que activar el proceso receptor cuando llegue el dato. Esto no es necesario y no debería hacerse.

Un ejemplo muy común de especificación de control excesiva es el caso en que un proceso genera flujos de salida alternativos, de los cuales sólo se usa uno de cada vez. Podemos entonces generar señales de control para activar/desactivar los procesos receptores pero esto no es un requisito del sistema. Desde el punto de vista de los requisitos podemos dejar todos los procesos funcionando en paralelo; sólo procesarán información cuando reciban entradas. Haciendo esto, podemos expresar los requisitos de forma sencilla y dejar las manos libres a los diseñadores para decidir la implementación más adecuada, siempre y cuando se mantengan las transformaciones de entradas en salidas.

¿Para qué sirven los DFCs?

Considerado aisladamente, un solo muestran la información de control que existe en el sistema, junto con los procesos y entidades externas que generan y consumen dicha información de control. Para representar el comportamiento del sistema, hay que utilizarlos en combinación con las especificaciones de control.

4.2.4. Especificaciones de control.

Las especificaciones de control (CSPECs) son similares a las PSPECs. En ambos casos, la función de estas especificaciones es definir los detalles procedimentales de cómo se realiza el procesamiento de los flujos de entrada y salida.

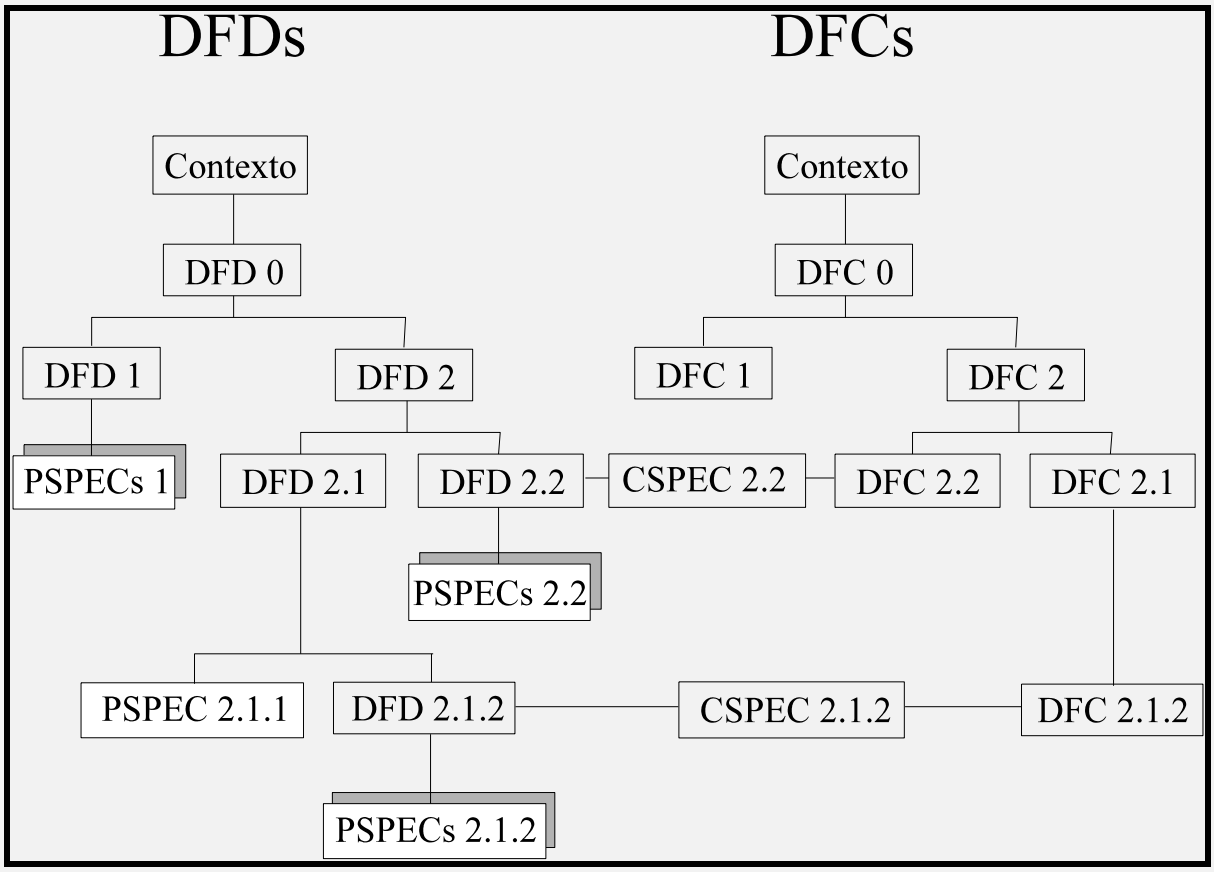
Sin embargo, hay una diferencia importante entre PSPECs y CSPECs. Las PSPECs se utilizan para describir las primitivas de proceso: aquéllos procesos del sistema que son lo suficientemente simples como para no necesitar crear un nuevo DFD. Las CSPECs sirven para modelar el comportamiento de un DFC, describiendo cómo se procesan los flujos de control. Habrá entonces como máximo una CSPEC por cada DFC de la jerarquía. La interfaz entre el DFC y la CSPEC se hace a través de las ventanas de control que aparecen en los DFCs.

Las CSPECs muestran cómo, a partir de las señales de control que entran en la ventana, se determina la activación o desactivación de procesos que figuran en el DFC correspondiente. Es decir, muestran cómo se calculan unos flujos de control de salida (los activadores de los procesos) a partir de otros de entrada (las señales de control que entran en la ventana).

Las CSPECs se caracterizan mediante sistemas combinacionales o, más frecuentemente, mediante sistemas secuenciales. Si son combinacionales su representación se hace mediante Tablas de Decisión (tablas de verdad que indican cómo se calculan los señales de salida en función de las de entrada) o mediante Tablas de Activación de Procesos (iguales que las anteriores, en las que se indica para cada proceso del DFC si esta activo o inactivo ante cada combinación de las señales de entrada); si son secuenciales se representan mediante Diagramas de Estados, que no son más que autómatas finitos.

Al hablar de los DFCs habíamos dicho que los flujos de control pueden figurar como salida de alguno de los procesos del sistema. En este caso se denominan condiciones de datos. Como consecuencia del procesamiento de los flujos de datos de entrada del proceso (procesamiento que se describe en la PSPEC asociada al proceso o a alguno de sus descendientes) se genera un flujo de control, que va a intervenir en el comportamiento de otro proceso del sistema. Estos flujos son el único enlace que se establece del modelo de procesos al modelo de control.

Relación entre los modelos de procesos y de control.
(DFDs, DFCs, PSPECs y CSPECs)



- *Los DFDs de menor nivel acaban en una PSPEC del mismo nombre.*
- *Las jerarquías de DFDs y DFCs son gemelas. Se establecen pares DFD - DFC.*
- *La jerarquía de DFCs suele acabar antes que la de DFDs.*
- *Algunos pares DFD - DFC tienen asociada una especificación de control CSPEC.*

Condiciones de datos.

Ej. El proceso *Comprobar Saldo* procesa *Número de Cuenta* e *Importe* y genera dos flujos de control: *Aceptar Operación* y *Rechazar Operación*.

Resumiendo, las condiciones de datos son flujos de control generados por un proceso del sistema. Figuran en el DFC y no en el DFD, pero es la PSPEC la que define cómo se calcula su valor a partir de los flujos de entrada del proceso.

Ventanas de control.

Estas condiciones de datos, junto con otros flujos de control provenientes del exterior del sistema (de las entidades externas) son los que deciden el comportamiento del sistema. Este comportamiento se define en las CSPECs que indican cómo se generan unos flujos de control a partir de otros o, más concretamente, cómo se activan o desactivan los procesos.

Para hacer referencia a que un flujo de control se calcula a partir de otros y a que esta transformación está definida en la CSPEC se utilizan las ventanas de control en los DFCs. Estas ventanas se representan mediante barras, donde entran y salen flujos de control. Las ventanas de control no están etiquetadas, puesto que todas las que aparecen en un determinado DFC hacen referencia a una única CSPEC que define cómo se realizan estas transformaciones.

Ej. El cajero automático utiliza dos flujos de control (*Saldo suficiente* e *Importe disponible en cajero*) para calcular los flujos de control *Activar Aceptar Operación* y *Activar Rechazar Operación*.

En un DFC podemos utilizar tantas ventanas de control como sea necesario con objeto de que el diagrama sea claro y todas ellas hacen referencia a la misma CSPEC.

Activadores de procesos.

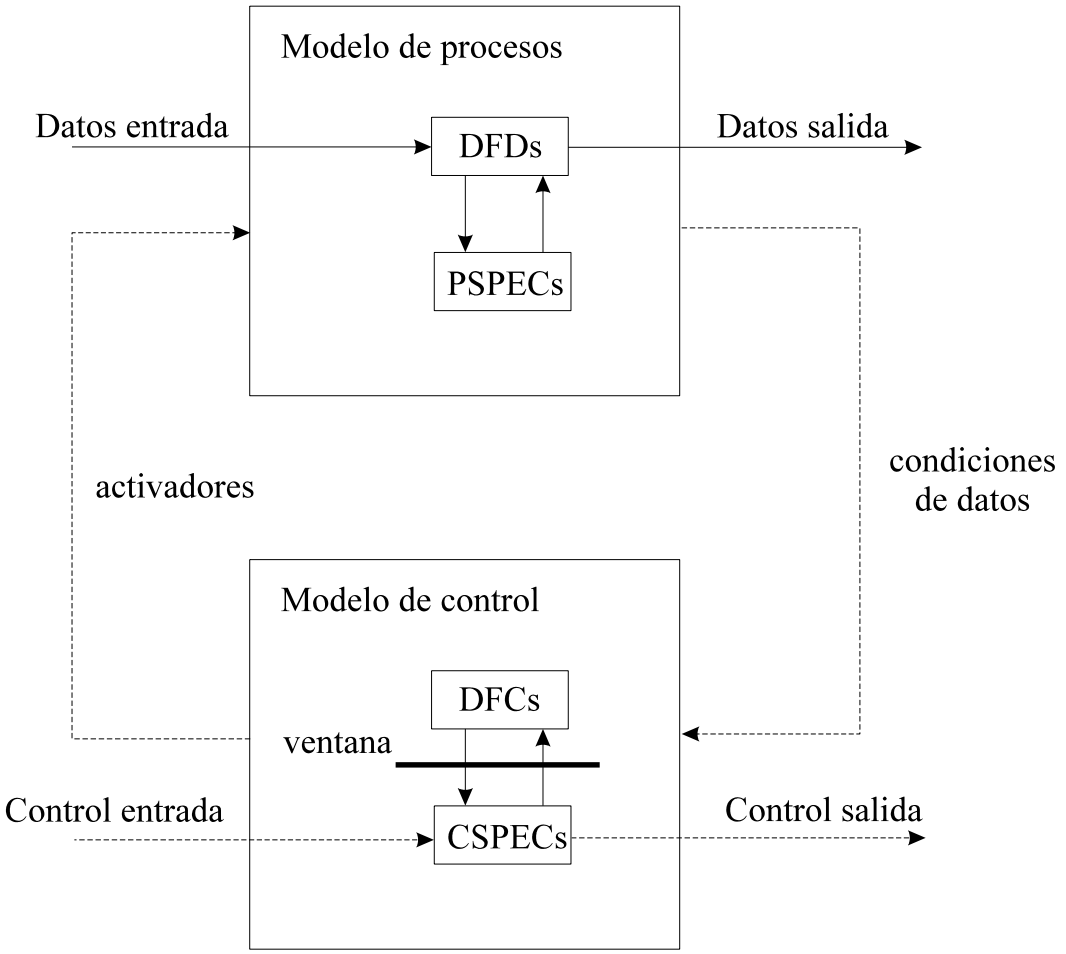
El objetivo del modelo de control es definir el comportamiento del sistema, es decir, cuándo se activan o desactivan los procesos. Estas activaciones y desactivaciones se modelan mediante unos flujos de control especiales, denominados Activadores de procesos. Estas señales de control toman sólo dos valores: On y Off. Los flujos activadores de procesos tienen el mismo nombre que los procesos que controlan, por lo que no suelen representarse en el DFC, salvo con fines de claridad (identificar qué procesos tienen activador).

En general, los flujos de control que entran en un proceso no sirven para activarlo. Se utilizan para transformar los flujos de entrada en flujos de salida según se describe en la PSPEC correspondiente o para intervenir en el control de algún subproceso de éste.

Dado que los modelos del sistema tienen estructura jerárquica, consideraremos que un proceso esta activo **sólo si todos sus antecesores están activos**. Esto es lógico si tenemos en cuenta que los descendientes de un proceso son subprocesos que están incluidos en él. Cuando se desactiva un proceso, el sistema se comporta cómo si este proceso **y todos sus descendientes** no existiesen, y se considera que sus salidas toman valor nulo.

Un proceso que no tiene activador se considera activo siempre que sus antecesores lo estén, y responde a los datos de entrada según le van llegando (este es el modelo de control implícito).

Relaciones detalladas entre DFDs, DFCs, PSPECs y CSPECs



En resumen, por cada DFD dibujaremos un DFC gemelo si alguno de los procesos del DFD genera o consume información de control. Además, si alguno de los procesos del DFD es activado o desactivado de forma no implícita, éste DFC llevará una ventana de control y haremos una CSPEC para indicar cuándo se realiza la activación y desactivación.

Una vez que hemos establecido la necesidad de utilizar CSPECs, debemos estudiar si podemos representarlas mediante un sistema combinacional (lo que podremos hacer si la función de control depende únicamente de los valore actuales de las señales de control disponibles) o si necesitamos un sistema secuencial (si la función de control depende de valores anteriores de las señales de control, que ya no están disponibles o si el comportamiento del sistema depende de su evolución anterior. Sólo en este caso utilizaremos CSPECs en forma de DEs.

4.2.5. Diagramas de estados.

Los sistemas complejos suelen presentan la propiedad de que los eventos pasados influyen en su comportamiento. Estos cambios no se limitan a cambios en los valores de los flujos de salida, sino que las computaciones pueden cambiar totalmente o incluso desaparecer, y el sistema se

comporta de una forma completamente diferente a lo largo del tiempo. Este tipo de comportamiento es difícil de representar utilizando un DFD, pero podemos utilizar técnicas basadas autómatas secuenciales. La forma más habitual de representar autómatas secuenciales es utilizando DEs.

Hay dos modelos muy conocidos de autómatas secuenciales: las máquinas de Moore y las máquinas de Mealy. En las primeras, las salidas se asocian con los estados, mientras que en las segundas se asocian con las transiciones. En los DEs utilizaremos indistintamente uno u otro, incluso un modelo mixto donde sea necesario.

El DE representa el comportamiento de un sistema, mostrando los estados en los que puede estar y los sucesos que hacen que el sistema cambie de estado. Además, el DE indica qué acciones se realizan cuando un sistema cambia de estado y qué actividades se realizan mientras el sistema está en un estado.

Elementos de un DE

Los elementos que aparecen en el DE son dos:

- **Estados**. Representados mediante rectángulos de esquinas redondeadas. Los estados muestran los distintos modos de comportamiento, escenarios o situaciones en que puede encontrarse el sistema. Cada estado representa un periodo de tiempo en el que el sistema muestra un cierto comportamiento. Generalmente los estados se asocian a la realización de un proceso o a un grupo de procesos del DFD. En otros casos, los estados representan al sistema *esperando* la ocurrencia de un determinado suceso (una petición de servicio o una entrada de datos, p.ej.).

  Los estados tienen un nombre que los identifica y pueden ir etiquetados con una **actividad**: aquellos procesos que están activos mientras el sistema esté en dicho estado. Las actividades se corresponden con las señales de salida de una máquina de Moore.

  Uno de los estados será el **estado inicial**, aquél en el que se sitúe el sistema cuando comience su funcionamiento (cuando se arranque el programa o se encienda el interruptor general). Un DE puede contener también **estados finales**, de los cuales no se salga mediante ninguna transición. Los estados iniciales y finales están marcados en el DE.

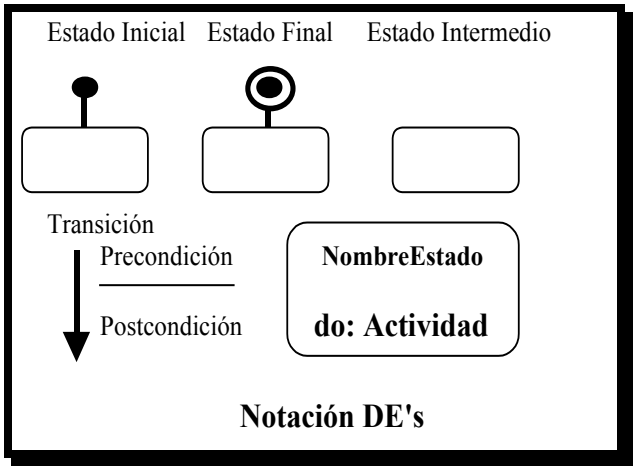
- **Transiciones**. Muestran las evoluciones posibles entre los estados de un sistema y se representan mediante flechas. Indican cuándo *evoluciona* el sistema de un estado a otro. Las transiciones van etiquetadas con dos elementos: la **condición** que hace que se dispare la transición y la **acción** que se produce como consecuencia de la transición. Las acciones corresponden con las señales de salida de una máquina de Mealy.

Las condiciones son generalmente condiciones de datos (esto es, flujos de control) y en este caso se denominan **eventos**, aunque para evitar tener que introducir señales ficticias en el modelo del sistema podemos establecer condiciones asociadas a flujos de datos (la ocurrencia de una determinada entrada de datos, o el que una señal tome determinado valor). En este caso se denominan **guardas** y se representan entre corchetes. Según esto la condición de disparo de una transición está formada por guardas y/o eventos. Para que una transición se dispare deben cumplirse sus guardas y producirse los eventos con los que esté etiquetada.

Las **acciones** consisten en generar un suceso o flujo de control o en activar un proceso del DFD (asignar valor *On* a un activador de procesos).

Tanto los eventos como las acciones son flujos de control, y deben figurar en el DFC (exceptuando los activadores de procesos, que normalmente no aparecen en los DFCs).

En el modelo de comportamiento del sistema consideraremos que las transiciones se realizan de forma instantánea, es decir, que al sistema no le lleva tiempo cambiar de estado. Esta no será la situación real del sistema una vez que lo implementemos, pero ahora estamos intentando establecer un modelo lógico o abstracto del sistema.



Como vemos, en el DE podemos representar las computaciones de dos formas distintas: como **acciones** (de duración idelmente instantánea con respecto a la escala de tiempo que utiliza el modelo) o como **actividades** (de duración prolongada en el tiempo, durante todo el intervalo de tiempo en el que el sistema permanezca en el estado correspondiente).

Con respecto a la activación de procesos, si realizamos ésta en un estado, consideraremos que dicho proceso permanece activo mientras el sistema permanece en dicho estado, desactivándose cuando lo abandone. Si, por el contrario, realizamos la activación en la transición supondremos que el proceso sigue activo hasta la siguiente activación, es decir, mientras el sistema

permanezca en el estado de salida de la transición. Como se puede apreciar, ambos casos son equivalentes, y no es necesaria la desactivación explícita de procesos.

El DE muestra, por tanto, el comportamiento del sistema. Estudiando el DE podemos comprobar si hay lagunas en el comportamiento especificado: si hay estados de los que no se sale mediante ninguna transición o hemos considerado todas las transiciones posibles (esto es si en un estado hemos previsto todos los sucesos que pueden ocurrir y causar una evolución del sistema).

Esto último es muy importante: supongamos un sistema con tres estados. Del estado inicial A, pasamos al estado B pulsando la tecla F1, y del B pasamos al C pulsando F2. El DE de la figura modela este comportamiento, pero no podemos quedarnos en modelar el comportamiento del sistema basándonos en un *comportamiento correcto del usuario*. ¿Qué sucede si el usuario pulsa dos veces la misma tecla? ¿O si pulsa cualquier otra? Si no especificamos el comportamiento del sistema ante estos eventos, posiblemente los programadores no crearán código para estas situaciones y el sistema final podrá presentar un comportamiento incontrolado, ante un usuario poco experto o malicioso. No podemos hacer suposiciones sobre el buen uso del sistema y tendremos que intentar dejarlo siempre todo bien atado.

Nosotros utilizaremos los DEs para modelar el comportamiento de todo el sistema o una parte de él, incluyendo estos diagramas en las CSPECs. Por tanto, los DEs van a ir asociados a un par DFD/DFC, indicando cómo se procesan los flujos de control que entran en las ventanas de control de los DFCs y cómo se activan los procesos que figuran en los DFDs. Los flujos de control que entran en la ventana van a ser condiciones de las transiciones del DE. Los flujos que salen de la ventana serán establecidos en las acciones y actividades de dicho DE.

Composición de DEs.

En algunos sistemas, los procesos de un DFD se activan y desactivan de forma más o menos independiente. Este es el caso de un sistema con dos o más procesos concurrentes que no interactúan entre sí o lo hacen de forma limitada. En estos casos realizaremos un DE compuesto, es decir formado por un DE para cada proceso o grupo de procesos que se comportan de forma independiente.

Ej. En la máquina expendedora (ejercicio 1), el control de la aceptación de monedas y el control de la selección de productos son prácticamente independientes. La única interacción entre ellos se produce a través del almacen IMPORTE. El DE se compone entonces de dos: uno para cada subsistema.

Anidamiento de DEs.

Igual que los DFDs y DFCs, los DEs también pueden anidarse, en caso de que sea necesario. Esto será útil cuando el DE sea complejo, y sea conveniente mostrarlo en varios niveles de abstracción o cuando un grupo de estados tienen un comportamiento común (reacciónan de una forma determinada ante un mismo evento (p. ej. un evento de error o de cancelación). En estos casos utilizaremos DEs anidados.

En cualquier caso, en DE anidado va a contener estados que se descomponen en un DE completo. Cualquier transición de entrada en dicho estado nos lleva al estado inicial del DE anidado. Los estados finales del DE anidado nos llevan a la transición de salida correspondiente del estado anidado.

Acciones dentro de los estados.

La inclusión de acciones de duración instantánea dentro de los estados amplía el modelo de automátas, dándole mayor expresividad y evitando duplicaciones innecesarias:

- ***Acciones de entrada****. Si una determinada acción ha de realizarse en todas las transiciones de entrada de un estado determinado, podemos ponerla como acción de entrada de dicho estado, evitando repetirla en cada transición.*
- ***Acciones de salida****. Igualmente, si una determinada acción ha de realizarse en todas las transiciones de salida de un estado, la pondremos como acción de salida del mismo, evitando repetirla en cada transición.*
- ***Eventos internos****. Si, mientras el sistema está en un estado determinado, puede producirse un evento que lleve asociada una determinada acción, pero que no provoque un cambio de estado, podemos representar esto como un evento interno del estado. En este caso no se realizarán las acciones de entrada y de salida del estado, puesto que se considera que el sistema no cambia de estado al atender dicho evento.*

4.2.6. Diagramas Entidad/Relación.

El tercer punto de vista que podemos considerar para hacer un modelo del sistema software, es el **punto de vista de los datos**. La notación básica de los DFDs, que representan los datos mediante almacenes de datos, es suficiente cuando la información que fluyen entre los procesos es relativamente simple. Sin embargo, las aplicaciones de software de gestión y cada vez más las aplicaciones de ingeniería y de tiempo real, utilizan colecciones de datos complejas. En estos casos no basta con saber en detalle qué información contiene cada almacén de datos, sino qué relaciones existen entre estos almacenes. Para ello es necesario desarrollar un modelo de datos del sistema, lo que haremos mediante Diagramas Entidad/Relación (DER), propuestos, entre otros, por Martin (1982) y Chen (1986).

Supongamos, por ejemplo, que un elemento de un almacén de datos sea un puntero a otro almacén de datos. Este tipo de relaciones no se reflejan en los por lo que necesitamos un diagrama específico para representarlas.

El modelado de datos se centra únicamente en los datos que maneja el proceso y en qué relaciones se establecen entre estos datos, dejando de lado las funciones que realiza el sistema y el control del mismo (Esta abstracción era una de las características de los modelos).

La notación de los DER y las reglas de construcción son ya suficientemente conocidas. En este apartado vamos a dar un repaso breve y luego nos centraremos en la relación entre los DERs y el resto de modelos del sistema.

Componentes de un DER.

Los componentes que pueden aparecer en un DER son cuatro:

- **Objetos (o tipos de objetos).** *Representados mediante un rectángulo etiquetado. Modelan una colección o conjunto de objetos o cosas del mundo real, cuyos miembros individuales (instancias) juegan un papel necesario en el sistema (sin ellos el sistema no podría operar), pueden ser identificados individualmente (mediante alguna clave o identificador) y pueden ser descritos mediante una serie de datos elementales (atributos).*
- **Relaciones.** *Los objetos de un DER se conectan unos a otros mediante relaciones, representadas mediante un rombo etiquetado. Cada instancia de una relación representa una asociación entre cero o más instancias de cada uno de los objetos conectados a ella. En algunos casos la relación asocia diferentes instancias del mismo objeto. (p.ej. relación casado entre instancias del objeto persona).*

Podemos definir la cardinalidad de la relación etiquetando los conectores con números o rangos (0:N). De esta forma indicamos cuántas instancias de cada objeto intervienen en la relación.

Hay que tener en cuenta que las relaciones representan algo que tiene que ser recordado por el sistema, es decir, que no puede ser calculado o derivado automáticamente, sino cada instancia de una relación, que asocia instancias de los diferentes objetos tiene que estar registrada o almacenada de alguna forma en el sistema (p.ej. relación pide entre compradores y artículos). Si una relación entre varios objetos puede ser calculada de alguna forma, no la representaremos en el diagrama.

Otro detalle es que las relaciones pueden asociar instancias de múltiples objetos, no sólo dos. Normalmente, la relación puede ser descrita desde el punto de vista de cada uno de los objetos asociados a ella (p.ej. relación negociar precio entre comprador, vendedor y agente). En otros casos, habrá un objeto que determine el punto de vista desde el que se debe leer la relación, en estas situaciones podemos asignar un sentido a los conectores.

Por último, en un DER no indicaremos todas las relaciones que existen entre los objetos, sino sólo aquéllas que sean relevantes para el sistema. Entre los objetos Vendedor y Cliente pueden

existir muchas relaciones distintas, aparte de la relación vende; puede suceder que algún cliente esté casado con un vendedor, que sea su padre o que sean amigos, pero posiblemente estas relaciones no sean relevantes para nuestro sistema. Sólo representaremos las relaciones que tengan algún significado en el contexto del sistema que estamos modelando.

- **Atributos.** *Los atributos son datos elementales asociados a un objeto o a una relación. Podemos representarlos mediante elipses que cuelguen del objeto o relación o no representarlos en el modelo, detallándolos entonces en la entrada del diccionario de datos dedicada al objeto o relación en donde están incluidos.*
- **Indicadores de supertipo y subtipo.** *Podemos establecer taxonomías entre los diferentes objetos relevantes para el sistema. Así, nuestro sistema puede trabajar con objetos Vehículo, Coche y Camión, siendo estos dos últimos un subtipo del primero. Conectaremos estos objetos de forma jerárquica. (Yourdon, fig. 12.8, p 242). Los objetos de los subtipos contendrán atributos comunes (que colgaremos del supertipo) y otros específicos, que colgaremos de cada subtipo particular.*

4.3. Diccionario de datos.

El diccionario de datos o diccionario de requisitos es el lugar donde van a estar contenidas las definiciones de todos los elementos que aparecen en los distintos diagramas del sistema, y sirve por tanto para establecer la relación entre los distintos modelos del mismo (procesos, comportamiento y datos). El diccionario de datos permite establecer que el analista y el usuario entienden de la misma forma cada uno de los elementos de los diagramas.

El análisis del sistema software no va a estar completo porque hagamos una serie de diagramas DFD, DFC, DER y DE. Necesitamos una cierta cantidad de información textual, que defina brevemente cada uno de los elementos de los modelos (con las etiquetas no basta), o que describa el *contenido* de dichos elementos (el contenido de los flujos y almacenes, de entidades y relaciones, el pseudocódigo de los elementos de proceso).

La información que suele contener el diccionario de datos es la siguiente:

- **Nombre**. *El nombre principal del elemento en el sistema.*
- **Alias.** *Otros nombres usados para el mismo elemento en diferentes partes del sistema. Aunque lo ideal sería que un mismo elemento se llamase igual en todos los modelos, hay que tener en cuenta que puede ser que la labor de análisis no sea hecha por una sola persona sino por un equipo de analistas, y en estos casos la disparidad de nombres suele ser inevitable.*
- **Uso.** *Un listado de dónde (qué procesos o diagramas) se usa el elemento y cómo se usa (entrada, salida, actualización). Esta información se puede generar automáticamente con una herramienta de análisis o una herramienta CASE y es muy útil a la hora de estimar el impacto de un cambio (si queremos modificar el tipo de un dato elemental, cuántos programas es necesario modificar).*
- **Descripción.** *Una descripción breve del significado del elemento, que complete la información del nombre.*
- **Contenido.** *Una relación de los datos elementales que contiene, en el caso de ser un flujo, un almacén, un objeto o una relación. Para la descripción del contenido de los elementos existe una notación normalizada.*

Descripción del contenido.

Constructor	Notación	Significado
	=	se compone de
Secuencia	+	y
Selección	[|]	o
Repetición	$\{\}^n$	n repeticiones de
Opción	( )	datos opcionales
	@	identificador
	" "	literal

- **Información adicional.** *Otras informaciones relativas al elemento (quién y cuándo lo definieron) o información específica del tipo de elemento de que se trate.*

En concreto, un diccionario de requisitos completo debe incluir entradas para los siguientes elementos:

- **Entidades externas:** *Descripción.*
- **Procesos:** *Descripción de la función que realizan.*
- **Flujos de datos:** *Descripción, dónde se usan y su contenido.*
- **Almacenes de datos:** *Descripción, dónde se usan y contenido.*
- **Datos elementales:** *Descripción, tipo y restricciones (p.ej. de rango o de valores)*
- **Primitivas de proceso:** *Descripción y pseudocódigo.*
- **Flujos de control:** *Descripción, dónde se usan y su contenido.*
- **Almacenes de control:** *Descripción, dónde se usan y contenido.*
- **Estados:** *Descripción.*
- **Transiciones:** *Condiciones de activación y acciones a realizar (si es necesario).*
- **Entidades:** *Descripción y atributos que contienen.*
- **Relaciones:** *Descripción y atributos que contienen.*

Actualmente, el diccionario de datos suele estar mecanizado, bien sea por separado o bien como parte de una herramienta CASE. Esto hace que las modificaciones sean más fáciles, que las comprobaciones de que cada modelo o el conjunto de los modelos son correctos se puedan hacer de forma automática o que sirva de ayuda para evaluar el impacto de un cambio en el sistema.

4.4. Metodología del análisis estructurado.

En el apartado anterior hemos visto diversas notaciones para el análisis estructurado de sistemas. Estas notaciones nos permiten modelar el software desde diversos puntos de vista. Vamos a ver ahora como podemos coordinar estos modelos y cuál es el método de trabajo apropiado.

4.4.1. Fases.

Creación del modelo de procesos.

El punto de partida será el modelo de procesos del sistema. Mediante el uso de DFDs y PSPECs podemos modelar el ámbito de información y ámbito funcional del sistema.

La tarea de análisis consiste básicamente en eso: en analizar o pensar, no en ponerse a dibujar diagramas a lo loco. Por esto, lo primero que tenemos que hacer es estudiar toda la documentación inicial que tengamos del sistema, bien sea una especificación preliminar, o el resultado de las reuniones o entrevistas preliminares con el usuario.

Podemos hacer un análisis gramatical de la información de entrada. Identificando los nombres y los verbos de la especificación preliminar podremos identificar muchos de los

componentes del sistema. Podemos clasificar estos elementos en entidades externas (nombres), flujos y almacenes de datos (nombres) y procesos o funciones (verbos). La especificación relaciona estos nombres y verbos entre sí. Esto nos permitirá establecer la relación entre procesos y el resto de los componentes. Podemos hacer listas separadas de cada uno de estos componentes. No serán correctas ni completas, pero es un buen punto de partida.

A continuación, comenzaremos por hacer el DFD de contexto, mostrando el sistema software en relación con las entidades externas con las que interactúa. En este nivel representamos el sistema mediante un sólo símbolo de proceso, no se suelen mostrar almacenes de datos, y se identifican las entradas y salidas principales del sistema (no se detallan todos los flujos de datos, sino que se los agrupa).

A continuación, vamos refinando el DFD de contexto en mayores niveles de detalle. Este es un proceso de descomposición funcional, en el que tenemos que ir descomponiendo el sistema en las distintas funciones que realiza. Hay que aplicar los principios de **acoplamiento mínimo** (máxima independencia) entre procesos distintos y **máxima cohesión cohesión** (fuerte conexión funcional entre todo lo que está dentro de un proceso). en cada proceso. Si todos los flujos de datos que aparecen en un DFD son utilizados en todos los procesos del mismo, posiblemente algo irá mal. Según esto, un proceso debe realizar una función clara y coherente.

El número de procesos que aparecen en un DFD debe estar (idealmente) entre 5 y 10. Poner menos llevará a tener que hacer demasiados DFDs y a que alguna de las burbujas no sea más que un saco de procesos (es decir, habrá poca cohesión). Poner más dificultará el conseguir una visión general del diagrama.

Según vamos descomponiendo, tenemos que tener cuidado en mantener la consistencia: consistencia de nombres, de numeración y de flujos de datos, aplicando la regla de equilibrio.

No hay que caer en detalles de implementación, el DFD pretende dar una visión de cómo se mueve idealmente la información entre los diferentes procesos, y no en ser un lenguaje de programación gráfico.

Según vamos incluyendo elementos en los DFDs hay que ir definiéndolos en el diccionario de datos del proyecto.

Seguiremos el proceso de descomposición hasta encontrar las primitivas de proceso: estas serán procesos sencillos, con máxima cohesión (realizan una única función). Los flujos de datos que entran y salen de las primitivas constituyen la interfaz de estos procesos. Describiremos estas primitivas mediante PSPECs, a ser posible en pseudocódigo, manteniendo la consistencia de flujos de entrada y salida con respecto a los DFDs.

Las PSPECs no contienen definiciones de datos, por tanto, no podemos ocultar en ellas almacenes de datos. Estos tienen que aparecer en los DFDs.

Creación del modelo de control.

No todos los sistemas necesitan de un modelo de control. En muchos casos, el comportamiento implícito en los DFDs servirá para mostrar cómo se comporta el sistema. Debido a esto, lo primero es determinar si es necesario o no desarrollar un modelo de control.

Aquí hay que tener especial cuidado en no caer en detalles de implementación, y empezar a pensar en qué programas llaman a qué programas o como se sincronizan los procesos. Esta es una tarea más propia del diseño.

Si hemos decidido desarrollar un modelo de comportamiento del sistema, debemos empezar por establecer una jerarquía de DFCs simétrica a la de DFDs. Cada par DFD/DFC contiene los mismos procesos, almacenes de datos y entidades externas, y en la misma posición, para facilitar la identificación.

A continuación eliminaremos del DFD todos los flujos que transporten información de control: aquélla que sirve para determinar el comportamiento del sistema, activando o desactivando procesos. Estos flujos de control los representaremos en los DFCs.

La jerarquía de DFCs será normalmente más corta que la de DFDs. Continuaremos desarrollando DFCs mientras los procesos que aparecen en ellos generen y sean controlados por los flujos de control que vayamos especificando. En aquellos casos en los que el comportamiento del sistema quede reflejado implícitamente en los DFDs no haremos DFCs.

A cada par DFD/DFC le corresponde una CSPEC. Las condiciones de datos generadas por los procesos y las señales de control provenientes del exterior del DFC entrarán en las ventanas de control. De estas ventanas saldrán los activadores de procesos (se pueden representar o no, opcionalmente) y las señales de control que salgan del DFC.

Hay que mantener la consistencia de flujos de control entre las ventanas de control y las CSPECs.

La CSPEC pude ser combinacional, secuencial o compuesta. Elegiremos siempre el modelo más sencillo posible. Las CSPECs combinacionales pueden representarse mediante tablas de decisión y tablas de activación de procesos. Las secuenciales mediante DEs.

Respecto a los DEs, hay que tener cuidado con las posibles omisiones de transiciones. ¿Existe alguna otra forma de llegar a un estado o salir de él?, ¿Están previstas todas las contingencias que pueden presentarse en cada estado?

Creación del modelo de datos.

Si los datos que maneja el sistema tienen una estructura compleja, no bastará con definir en el diccionario el contenido de cada uno de los almacenes de datos. Desarrollaremos un modelo de datos, utilizando DERs, donde se muestren las relaciones que existen entre los datos que maneja el sistema. Como punto de partida de este modelo utilizaremos los almacenes de datos y entidades externas que aparecen en el modelo de procesos.

4.4.2. Consistencia entre los modelos.

En los apartados anteriores hemos visto cómo desarrollar modelos procesos, de comportamiento y de datos de un sistema, cada uno de estos modelos se centra en un determinado aspecto del sistema. También hemos visto cómo definir todos los elementos de estos modelos en un diccionario de datos y una serie de reglas para determinar la consistencia de los diferentes diagramas de se utilizan en cada modelo.

El conjunto de modelos tiene como misión dar una visión global del sistema, desde diversos puntos de vista. Los modelos pueden parecer muy distintos pero todos ellos representan el mismo sistema. En este apartado vamos a ver como podemos asegurar la consistencia entre los distintos modelos

Los errores de consistencia más comunes se deben a elementos no definidos: podemos tener almacenes de datos no definidos en el DD o que no se corresponden con ningún objeto del DER. Otros errores se deben a inconsistencia entre los modelos: un mismo elemento 'real' del sistema tiene descripciones distintas y contradictorias en los diferentes modelos.

Vamos a ver una serie de reglas de consistencia entre los diferentes modelos:

Consistencia entre el modelo de procesos y el diccionario de datos.

Las reglas son las siguientes:

- **Cada elemento de los DFDs debe estar definido en el DD**. Las entidades externas, procesos, almacenes y flujos de datos tienen que estar definidos en el diccionario. Para entidades externas y procesos basta con una descripción breve de su significado, para almacenes y flujos de datos hay que describir también los datos elementales que contienen.

- **Cada dato elemental del DD tiene que estar incluido en algún flujo de datos**. Si no fuese así el diccionario contendrá datos elementales que no se usan en el sistema.

Además, si el sistema debe memorizar alguno de estos datos, debe figurar también en un almacén de datos de los DFDs.

Una excepción a esta regla serían las variables locales de las primitivas de proceso, que sólo aparecen en las PSPECs, pero definir estas variables locales en el DD no es lo habitual.

Consistencia entre el modelo de control y el diccionario de datos.

Las reglas son exactamente las mismas: cada flujo y almacén de control de los DFC y cada señal utilizada en las condiciones o acciones del DE, debe estar definido en el diccionario. Por otro lado, cada señal de control definida en el diccionario tiene que estar incluida en un flujo de control de los DFCs o tiene que ser una variable local de la CSPEC.

Consistencia entre el modelo de procesos y el modelo de control.

Las reglas ya son conocidas, simplemente las recordamos:

- **Cada DFC y cada CSPEC están asociados a un DFD**. (El recíproco no es necesariamente cierto). Los procesos, entidades externas y almacenes del DFD deben aparecer en el DFC correspondiente.

- **Cada estado del DER se asocia a un proceso o grupo de procesos del DFD, activos durante ese estado, o al sistema esperando que se produzca algún evento**. De acuerdo a esto, los nombres de estados y procesos deben estar relacionados: para los procesos utilizaremos infinitivos (p.ej. Comprobar Saldo Cliente) y para los estados, gerundios (p.ej. Comprobando Saldo Cliente).

Consistencia entre el modelo de datos y el diccionario de datos.

- Los objetos y almacenes de datos del DER tiene que estar definidos en el DD. Para cada uno de ellos hay que describir los datos elementales que contienen.

Consistencia entre el modelo de procesos y el modelo de datos.

El DER muestra el sistema desde un punto de vista muy distinto al de los DFDs, sin embargo hay una serie de reglas que deben cumplirse para que ambos sean consistentes:

- **Cada almacén de los DFD debe corresponderse con un objeto o relación del DER**. Ambos elementos representan necesidades de almacenamiento de información del sistema.

- **Los nombres de los elementos del DER y de los almacenes de los DFDs tienen que corresponderse**. El convenio que seguiremos es usar nombres en plural para los almacenes de datos (p.ej. Clientes) y en singular para los objetos (p.ej. Cliente).

- **Las entradas del diccionario deben aplicarse tanto a los almacenes de datos como a los objetos y relaciones correspondientes**. Hemos dicho que el diccionario debe describir el contenido de los almacenes y de los objetos y relaciones, pero no es necesario definir por separado estos contenidos, sino que, en el ejemplo anterior las definiciones del diccionario serían las siguientes:

  Clientes = { Cliente }

  Cliente = @NIF + Nombre + Dirección + ...

Como vemos, todas estas reglas son bastante mecánicas y muy tediosas de comprobar manualmente. La mayor parte de las herramientas de análisis realizan de forma automática este tipo de chequeos (exceptuando las convenciones de nombres). Sin embargo, es necesario conocerlas para entender cuál es la mecánica de análisis estructurado y poder desarrollar modelos que sean lo más consistentes posible antes de realizar estos chequeos automáticos y tener que cambiar todos los diagramas.

4.5. Modelos del sistema: esencial y de implementación.

El modelo esencial del sistema (algunas veces llamado modelo lógico del sistema) es un modelo de lo que el sistema debe hacer con objeto de satisfacer los requisitos del usuario. En el modelo esencial no figura (al menos idealmente) nada acerca de cómo se va a implementar el sistema. Es por tanto, un modelo abstracto del sistema, que supone que disponemos de una tecnología perfecta sin coste alguno.

Este modelo esencial es el resultado de la fase de análisis de requisitos del sistema, y de este tipo son todos los modelos que hemos estado viendo hasta ahora.

El modelo esencial tiene que estar completamente libre de detalles de implementación. Alguno de los errores típicos en los que se cae al hacerlo son:

- **Secuenciar de forma arbitraria las funciones del DFD**. Los procesos de los DFDs deben ser lo más concurrentes posible, no hay que pensar en que el sistema realiza una tarea, y luego otra, y luego otra. El único secuenciamiento de funciones que debe aparecer en los DFDs es el impuesto por la dependencia de datos. Si un proceso P2 recibe como entrada un flujo de datos generado por un proceso P1, no podrá realizarse hasta que P2

acabe. Cualquier otra secuencia es puramente arbitraria, y dependerá más de necesidades de implementación (p.ej. capacidad del ordenador) que de requisitos del sistema.

- **Utilizar ficheros temporales o de backup**. Los ficheros temporales se usan para conectar procesos que no pueden ejecutarse simultáneamente por problemas de capacidad o multiproceso del ordenador, no porque los procesos deban ejecutarse de forma secuencial. Los ficheros de backup se utilizan para evitar perder información en caso de que falle el sistema (sea el fallo hardware, software o humano). Supuesta una tecnología perfecta, no necesitamos de ninguno de ellos. No son requisitos esenciales del sistema sino de implementación.

- **Utilizar información redundante o derivada**. El incorporar en las bases de datos o en los procesos información redundante tiene más que ver con la eficiencia de la implementación que con los requisitos del modelo. (P.ej. los totales de factura se pueden calcular a partir de las líneas, sin embargo es normal almacenarlos para evitar tener que calcularlos cada vez que se accede a la factura). En el modelo esencial no usaremos nunca información redundante o derivada.

A partir del modelo esencial, los diseñadores podrían decidir cómo implementar el sistema, usando la tecnología disponible. Podrían decidir cuál va a ser el hardware, la base de datos, el lenguaje de programación, etc. y cómo implementar con estas herramientas los elementos del modelo esencial.

Sin embargo, lo normal es que el cliente proporcione más información que los simples requisitos del sistema, y que decida también sobre detalles de implementación: qué funciones van a ser manuales y cuáles automáticas, el formato de los informes y de las pantallas, requisitos operacionales de velocidad de cálculo o capacidad, etc. Toda esta información que el cliente proporciona al analista, que no se refiere a los requisitos esenciales del sistema, sino a los requisitos de implementación, formarán parte del modelo de implementación del sistema.

El desarrollo de un modelo de implementación es una tarea que está a caballo entre el análisis y el diseño. No puede ser desarrollado sólo por el analista y el cliente pues se necesita el consejo de diseñadores e implementadores sobre las elecciones de hardware y software y sobre la posibilidad de cumplir las restricciones de tiempos de respuesta y capacidades del sistema usando la tecnología elegida. Por otra parte, no puede ser realizado únicamente por diseñadores e implementadores porque el usuario debe definir una gran cantidad de requisitos de implementación, que irán surgiendo en la fase de análisis, y es al analista al que se los describe. (El analista es el principal vínculo entre el cliente y el equipo de desarrollo. El cliente tiene muy poco trato con diseñadores e implementadores).

Este modelo de implementación será una versión revisada y anotada del modelo esencial, donde se especifiquen todos estos detalles adicionales proporcionados por el usuario. Entre estas anotaciones podemos citar:

- **La elección de dispositivos de entrada y salida**. Desde el punto de vista del modelo esencial, el sistema recibe y emite flujos de datos a las entidades externas. Un modelo de implementación debe indicar qué dispositivos se utilizan para estas entradas o salidas: interruptores y señales luminosas o estaciones de trabajo, p.ej.

- **La elección de los dispositivos de almacenamiento**. Discos duros, discos ópticos, cintas magnéticas, etc.

- **El formato de las entradas y salidas**. Incluyendo aquí el tamaño de los datos (p.ej. longitud de los nombres, formato de fechas y números de teléfono, número de líneas máximo en un pedido, etc.). Todas estas restricciones deben ser incorporadas al diccionario de datos.

- **La secuencia de operaciones de entrada y salida.** Incluyendo la definición de cómo se van a hacer los diálogos del sistema con el usuario (orden de las pantallas de captura de datos y disposición de los datos en la pantalla, valores por defecto de los datos, mensajes de error y de ayuda, cómo se pasa de una pantalla a otra, etc.) Muchos de estos aspectos pueden definirse mediante DEs.

- **Volumen de datos**. El usuario debe indicar el volumen de datos que manejará el sistema de forma que se puedan prever las necesidades de almacenamiento y procesamiento.

- **Tiempo de respuesta**. El tiempo de respuesta de las aplicaciones interactivas y el tiempo máximo de procesamiento que puede consumir in proceso batch, etc.

- **Copias de seguridad y descarga de datos del sistema**. El usuario puede indicar cuándo y de qué datos es necesario hacer copia de seguridad o qué datos necesita mantener on-line (información del último año p.ej.) y qué datos se pueden descargar del sistema a cinta magnética. Habrá que prever mecanismos de recuperación o recarga de esa información cuando sea necesaria.

- **Seguridad**. Mecanismos de seguridad para evitar accesos no autorizados a todos o parte de los datos, a determinados procesos, etc.

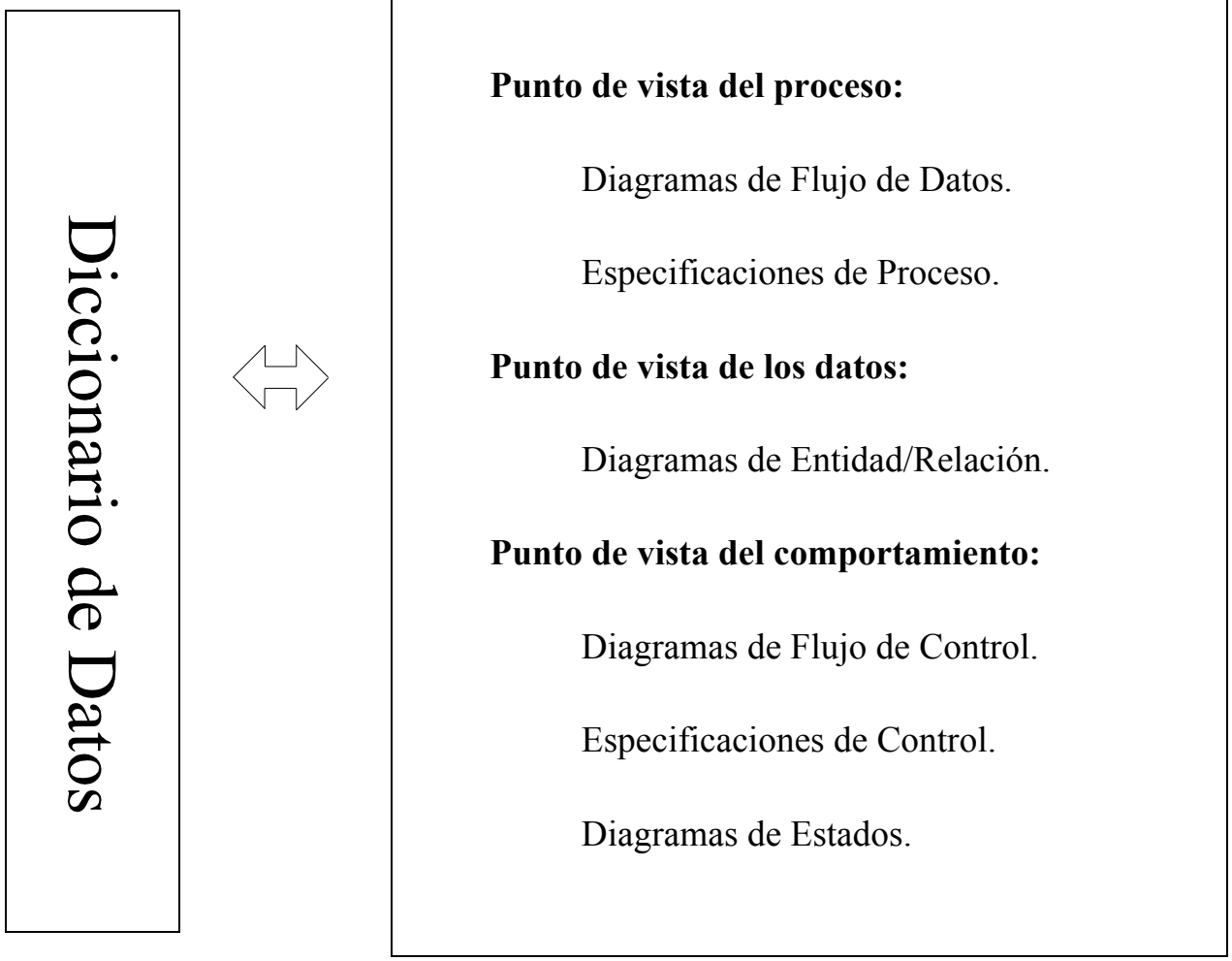
Especificaciones utilizando análisis clásico.

- Monolíticas.
- Redundantes.
- Ambiguas.
- Imposibles de mantener o modificar.

Especificaciones utilizando análisis estructurado.

- Gráficas.
- Particionadas.
- Mínimamente redundantes.
- Transparentes.

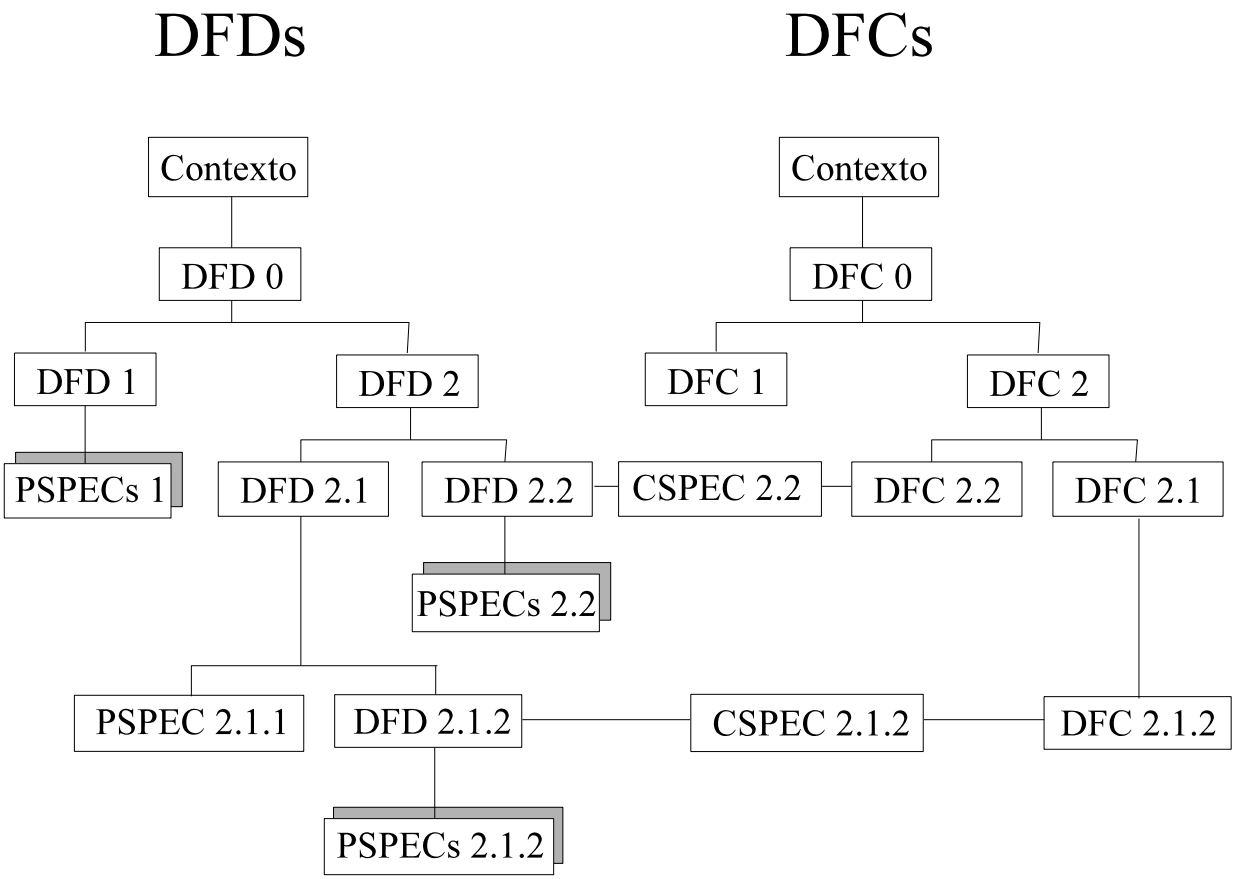
Diferentes tipos de modelos.



DFDs. Reglas de construcción.

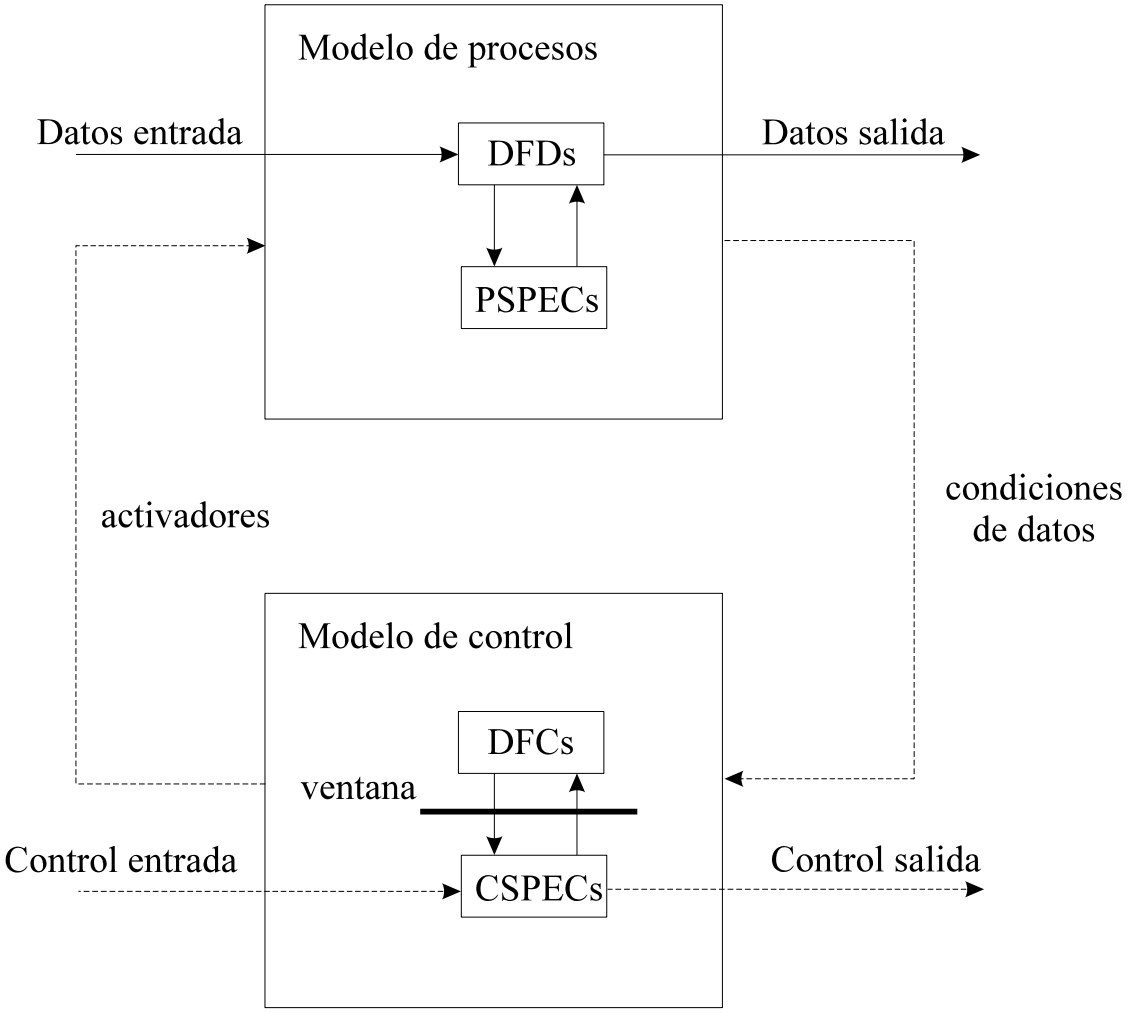
- Un DFD debe contener menos de 10 elementos.
- Cada elemento de un DFD debe tener un nombre corto e identificativo.
- Es necesario numerar los procesos.
- Para modelar sistemas complejos se utiliza la explosión, que da como resultado DFDs a distintos niveles de detalle.
- No es conveniente utilizar más de 7 u 8 niveles.
- Los DFDs de niveles inferiores desarrollan de forma más concreta los procesos de niveles superiores.
- La explosión se realiza hasta alcanzar un nivel de especificación mínimo y sencillo.
- Debe mantenerse la consistencia de nombres en los distintos DFDs.
- Debe mantenerse la consistencia entre los distintos niveles, utilizando la **regla de equilibrio**.
- En cada DFD hijo deben representarse los mismos flujos de datos que en el proceso padre.
- No existen conexiones entre entidades externas
- No existen conexiones entre entidades externas y almacenes
- No existen conexiones entre almacenes

Relación entre los modelos de procesos y de control.



- Los DFDs de menor nivel acaban en una PSPEC del mismo nombre.
- Las jerarquías de DFDs y DFCs son gemelas. Se establecen pares DFD - DFC.
- La jerarquía de DFCs suele acabar antes que la de DFDs.
- Algunos pares DFD - DFC tienen asociada una especificación de control CSPEC.

Relación entre los modelos de procesos y de control (2).



Descripción de cada elemento en el diccionario de datos.

- **Entidades externas**: Descripción.
- **Procesos**: Descripción de la función que realizan.
- **Flujos de datos**: Descripción, dónde se usan y su contenido.
- **Almacenes de datos**: Descripción, dónde se usan y contenido.
- **Datos elementales**: Descripción, tipo y restricciones (p.ej. de rango o de valores)
- **Primitivas de proceso**: Descripción y pseudocódigo.
- **Flujos de control**: Descripción, dónde se usan y su contenido.
- **Almacenes de control**: Descripción, dónde se usan y contenido.
- **Estados**: Descripción. Acciones y actividades a realizar.
- **Transiciones**: Condiciones de activación y acciones a realizar.
- **Entidades**: Descripción y atributos que contienen.
- **Relaciones**: Descripción y atributos que contienen.

Descripción del contenido.

Constructor	Notación	Significado
	=	se compone de
Secuencia	+	y
Selección	[|]	o
Repetición	$\{ \}^n$	n repeticiones de
Opción	( )	datos opcionales
	@	identificador
	“ “	literal

Consistencia entre los modelos y el diccionario de datos.

Entre el modelo de procesos y el diccionario de datos.

- Cada elemento de los DFDs debe estar definido en el DD.
- Cada dato elemental del DD tiene que estar incluido en algún flujo de datos.

Entre el modelo de control y el diccionario de datos.

- Cada elemento de los DFC debe estar definido en el diccionario.
- Cada señal utilizada en las condiciones o acciones del DE,. debe estar definido en el diccionario.
- Cada dato de control definido en el diccionario tiene que estar incluida en un flujo de control de los DFCs.

Entre el modelo de datos y el diccionario de datos.

- Los objetos y relaciones del DER tiene que estar definidos en el DD.

Consistencia entre los diferentes modelos.

Entre el modelo de procesos y el modelo de control.

- Cada DFC y cada CSPEC están asociados a un DFD.
- Los elementos del DFD deben aparecer en el DFC correspondiente.
- Cada estado del DE se asocia a un proceso o grupo de procesos del DFD, activos durante ese estado, o representa al sistema esperando que se produzca algún evento.

Entre el modelo de procesos y el modelo de datos.

- Cada almacén de los DFD debe corresponderse con un objeto o relación del DER.
- Los nombres de los elementos del DER y de los almacenes de los DFDs tienen que corresponderse.
- Las entradas del diccionario deben aplicarse tanto a los almacenes de datos como a los objetos y relaciones correspondientes.

Modelo esencial. En el modelo esencial no debemos:

- Secuenciar de forma arbitraria las funciones del DFD.
- Utilizar ficheros temporales o de backup.
- Utilizar información redundante o derivada.

Modelo de implementación. El modelo de implementación puede incluir:

- La elección de dispositivos de entrada y salida.
- La elección de los dispositivos de almacenamiento.
- El formato de las entradas y salidas.
- La secuencia de operaciones de entrada y salida.
- Volumen de datos.
- Tiempos de respuesta.
- Copias de seguridad y descarga de datos del sistema.
- Requisitos de seguridad.

Ejercicio 1

Desarrollar el diagrama de flujo de datos de una máquina expendedora (de dulces, café, etc.) como las que existen en el bar de la facultad. La máquina debe ir aceptando monedas, comprobar si son válidas y calcular su valor a partir de su tamaño y peso. Además, acumula el importe total pagado. El cliente elige uno de los productos disponibles pulsando un botón. Antes de dispensar el producto debe comprobar si hay existencias del producto elegido y si el pago realizado es suficiente. Para comprobar si el pago es suficiente la máquina dispone de una tabla de precios.

La máquina es capaz de dar cambio y puede devolver el importe a petición del cliente.

La máquina dispone de dos letreros luminosos, que indican si el importe pagado es insuficiente y si el producto está agotado.

Ejercicio 2

Desarrollar el diagrama de flujo de datos de un cajero automático.

Sólo consideraremos las operaciones de reintegro.

La máquina acepta la tarjeta y comprueba el número de identificación personal tecleado con el que está encriptado en la banda magnética de la tarjeta. Una vez hecho esto, el cliente teclea el importe que quiere retirar y la máquina consulta telefónicamente con la central del banco si el saldo de la cuenta es suficiente. Si no fuese posible establecer la comunicación sólo permite retirar cantidades de hasta 10.000 pesetas.

Si la operación es aceptada y el cajero tiene dinero suficiente para atender la petición, la operación queda registrada en el cajero y la máquina expulsa los billetes junto con un recibo.

Mientras sea posible, el cliente puede cancelar la operación pulsando la tecla CANCELAR.

Ejercicio 3

Realizar los Diagramas de Arquitectura, de Flujo de Datos, de Flujo de Control y las Especificaciones de Proceso y Control de una máquina fotocopiadora.

La fotocopiadora que vamos a modelar es muy similar a las que existen en el Servicio de Reprografía de la Universidad. Dispone de los siguientes elementos:

- Mecanismo de alimentación automática de originales, gobernado por un motor, que se encarga de llevar los originales al área de copia y expulsarlos, una vez copiados, a una bandeja de salida.

- Un único depósito de papel DIN A4.

- Un sistema de óptica, que se encarga de realizar las ampliaciones/reducciones según un porcentaje ajustado por el usuario.

- Un sistema de copia, consistente en una barra luminosa movida por un motor, que se desplaza por el original. La intensidad de la luz puede ser regulada por el usuario de forma que las copias sean más claras o más oscuras que el original.

- Un teclado de órdenes consistente en:

    - tecla de copia y tecla de parada.

    - teclado numérico para seleccionar el número de copias a realizar.

    - dos teclas (< , >) para ampliación/reducción.

    - dos teclas (< , >) para copia más clara/más oscura.

Una pantalla de visualización de mensajes de diagnóstico.

Tema 5. Análisis orientado a objetos.

5.1. Introducción y conceptos básicos.

5.2. Técnica de modelado de objetos (OMT).

5.2.1. Fases.

5.2.2. Modelos.

5.2.3. Diferencias con la metodología estructurada.

5.3. Casos de uso.

5.4. Modelo de objetos.

5.5. Modelo dinámico.

5.6. Modelo funcional.

5.7. Relación entre los modelos.

5.8. Método de análisis.

Bibliografía

Object-Oriented Analysis and Design with Applications

G. Booch. Benjamin Cummings, 1993. 2ª Edición.

Object-Oriented Analysis

P. Coad, E. Yourdon. Prentice-Hall, 1991. 2ª Edición.

Applying OMT

K. W. Derr. SIGS Books, 1995.

Object-Oriented Software Engineering: A Use Case Driven Approach

Ivar Jacobson et al. Addison-Wesley, 1992

Aplicaciones Informáticas de Gestión

M. G. Piattini et al. Ra-Ma, 1996.

Ingeniería del Software: un enfoque práctico

R.S. Pressmann. Mc Graw Hill, 1993. 3ª Edición.

UML. Unified Modeling Language

Rational Software Corporation, 1997, ver. 1.0. http://www.rational.com

Object-Oriented Modeling and Design

J. Rumbaugh et al. Prentice Hall, 1991.

The OMT Process

J. Rumbaugh. Rational Software Corporation, 1994, http://www.rational.com

[OMT] The Functional Model

J. Rumbaugh. Rational Software Corporation, 1994, http://www.rational.com

Tema 5. Análisis orientado a objetos.

5.1. Introducción y conceptos básicos.

Desde comienzos de la década de los 80, el paradigma "orientado a objetos" ha ido madurando como un enfoque de desarrollo de software alternativo a la programación estructurada o modular. Se empezó a crear diseños de aplicaciones de todo tipo usando una forma de pensar orientada a los objetos, y a implementar estos diseños utilizando lenguajes orientados a objetos. Sin embargo, el análisis de requisitos se quedó atrás. No se desarrollaron técnicas de análisis específicamente orientadas a objetos.

Esta situación ha ido cambiando poco a poco, a medida que se desarrollaban técnicas de análisis específicas para desarrollar software orientado a objetos, e incluso como complemento de otros métodos de análisis. Ejemplos de estas nuevas técnicas son los métodos de Coad/Yourdon, Jacobson, Booch y Rumbaugh (OMT) [1]. En este tema seguiremos principalmente esta última metodología.

El Análisis Orientado a Objetos (AOO) se basa en conceptos sencillos, conocidos desde la infancia y que aplicamos continuamente: objetos y atributos, el todo y las partes, clases y miembros. Puede parecer llamativo que se haya tardado tanto tiempo en aplicar estos conceptos al desarrollo de software. Posiblemente, una de las razones es el éxito de los métodos de análisis estructurados, basados en el concepto de flujo de información, que monopolizaron el análisis de sistemas software durante los últimos veinte años.

En cualquier caso, el paradigma orientado a objetos ha sufrido una evolución similar al paradigma de programación estructurada: primero se empezaron a utilizar los lenguajes de programación estructurados, que permiten la descomposición modular de los programas; esto condujo a la adopción de técnicas de diseño estructuradas y de ahí se paso al análisis estructurado. El paradigma orientado a objetos ha seguido el mismo camino: el uso de la Programación

---

[1] Recent polls among the readers of the Object Magazine and the C++ Report about the preferred OO methodology resulted in:

- Booch 55%
- Rumbaugh (OMT) 34%
- Coad/Yourdon 16%
- Jacobson 14%

Three of the leading methodologists (the 'amigos' Booch, Rumbaugh, Jacobson) are now working together on the UML (Unified Modeling Language), in order to define the standard notation for OO analysis and design. The version 1.0 of the UML should be released in early '97.

Orientada a Objetos (POO) ha modificado las técnicas de diseño para adaptarlas a los nuevos lenguajes y ahora se están empezando a utilizar técnicas de análisis basadas en este nueva forma de desarrollar software.

El AOO ofrece un enfoque nuevo para el análisis de requisitos de sistemas software. En lugar de considerar el software desde una perspectiva clásica de entrada/proceso/salida, como los métodos estructurados clásicos, se basa en modelar el sistema mediante los objetos que forman parte de él y las relaciones estáticas (herencia y composición) o dinámicas (uso) entre estos objetos. Este enfoque pretende conseguir modelos que se ajusten mejor al problema real, a partir del conocimiento del llamado *dominio del problema*[2], evitando que influyan en el análisis consideraciones de que estamos analizando un sistema para implementarlo en un ordenador. Desde este punto de vista, el AOO consigue una abstracción mayor que el análisis estructurado, que modela los sistemas desde un punto de vista más próximo a su implementación en un ordenador (entrada/proceso/salida).

Este intento de conocer el dominio del problema ha sido siempre importante; no tiene sentido empezar a escribir los requisitos funcionales de un sistema de control de tráfico aéreo, y menos aún diseñarlo o programarlo sin estudiar primero qué es el tráfico aéreo o qué se espera de un sistema de control de este tipo. La ventaja del AOO es que se basa en la utilización de objetos como abstracciones del mundo real. Esto nos permite centrarnos en los aspectos significativos del dominio del problema (en las características de los objetos y las relaciones que se establecen entre ellos) y este conocimiento se convierte en la parte fundamental del análisis del sistema software, que será luego utilizado en el diseño y la implementación.

Este enfoque no es totalmente nuevo, sino que puede considerarse como una extensión del modelado de datos (DER) que se utiliza en los métodos estructurados. Sin embargo, el modelado de datos mediante DER está más orientado al diseño de bases de datos y se centra exclusivamente en la identificación de los datos que maneja un sistema y en las relaciones *estáticas* que se establecen entre esos datos. En el AOO, los objetos encapsulan tanto atributos como procedimientos (operaciones que se realizan sobre los objetos), e incorpora además conceptos como el polimorfismo o la herencia que facilitan la reutilización de código.

El uso de AOO puede facilitar mucho la creación de prototipos, y las técnicas de desarrollo evolutivo de software. Los objetos son inherentemente reutilizables, y se puede crear un catálogo

[2] El término *dominio del problema* o *dominio de aplicación* es uno de los más usados en el paradigma orientado a objetos. Se refiere al campo de aplicación del sistema, es decir, a qué es el sistema, entendido desde su propio campo de aplicación, más que a su descripción en términos de una implementación en ordenador.

de objetos que podemos usar en sucesivas aplicaciones. De esta forma, podemos obtener rápidamente un prototipo del sistema, que pueda ser evaluado por el cliente, a partir de objetos analizados, diseñados e implementados en aplicaciones anteriores. Y lo que es más importante, dada la facilidad de reutilización de estos objetos, el prototipo puede ir evolucionando hacia convertirse en el sistema final, según vamos refinando los objetos de acuerdo a un proceso de especificación incremental.

5.1.1. Deficiencias del análisis estructurado.

- **Descomposición funcional**. El análisis estructurado se basa fundamentalmente en la descomposición funcional del sistema que queremos construir. Esta descomposición funcional requiere traducir el dominio del problema en una serie de funciones y subfunciones. El analista debe comprender primero el dominio del problema y a continuación documentar las funciones y subfunciones que debe proporcionar el sistema. El problema es que no existe un mecanismo para comprobar si la especificación del sistema expresa con exactitud los requisitos del sistema.

- **Flujo de datos**. El análisis estructurado muestra cómo fluye la información a través del sistema. Aunque este enfoque se adapta bien al uso de sistemas informáticos para implementar el sistema, no es nuestra forma habitual de pensar. Sin embargo, la abstracción y la clasificación sí son conceptos que manejamos habitualmente, aunque sea de forma inconsciente.

- **Modelo de datos**. El análisis clásico daba muy poca importancia al almacenamiento de datos. El análisis estructurado moderno incorpora modelos de datos, además de modelos de procesos y de comportamiento. Sin embargo la relación entre los modelos es muy débil, y hay muy poca influencia de un modelo en otro. En la práctica, los modelos de procesos y de datos de un mismo sistema se parecen muy poco. En muchos casos son visiones irreconciliables, no del mismo sistema, sino de dos puntos de vista totalmente diferentes de organizar la solución. Lo ideal sería que ambos modelos se complementasen, no por oposición sino de forma que el desarrollo de uno facilitase el desarrollo del otro.

5.1.2. Ventajas del AOO.

- **Dominio del problema**. El paradigma OO es más que una forma de programar. Es una forma de pensar acerca de un problema en términos del mundo real en vez de en términos de un ordenador. El AOO permite analizar mejor el dominio del problema, sin

pensar en términos de implementar el sistema en un ordenador. El AOO permite pasar directamente el dominio del problema al modelo del sistema.

- **Comunicación**. El concepto OO es más simple y está menos relacionado con la informática que el concepto de flujo de datos. Esto permite una mejor comunicación entre el analista y el experto en el dominio del problema (es decir, el cliente).

- **Consistencia**. Los objetos encapsulan tanto atributos como operaciones. Debido a esto, el AOO reduce la distancia entre el punto de vista de los datos y el punto de vista del proceso, dejando menos lugar a inconsistencias o disparidades entre ambos modelos.

- **Expresión de características comunes**. El paradigma OO utiliza la herencia para expresar explícitamente las características comunes de una serie de objetos. Estas características comunes quedan escondidas en otros enfoques y llevan a duplicar entidades en el análisis y código en los programas. Sin embargo, el paradigma OO pone especial énfasis en la reutilización, y proporciona mecanismos efectivos que permiten reutilizar aquello que es común, sin impedir por ello describir las diferencias.

- **Resistencia al cambio**. Los cambios en los requisitos afectan notablemente a la funcionalidad de un sistema, por lo que afectan mucho al software desarrollado con métodos estructurados. Sin embargo, los cambios afectan en mucha menor medida a los objetos que componen o maneja el sistema, que son mucho más estables. Las modificaciones necesarias para adaptar una aplicación basada en objetos a un cambio de requisitos suelen estar mucho más localizadas.

- **Reutilización**. Aparte de la reutilización interna, basada en la expresión explícita de características comunes, el paradigma OO desarrolla modelos mucho más próximos al mundo real, con lo que aumentan las posibilidades de reutilización. Es probable que en futuras aplicaciones nos encontremos con objetos iguales o similares a los de la actual.

5.1.3. Conceptos básicos.

Las técnicas orientadas a objetos se basan en organizar el software como una colección de objetos discretos que incorporan tanto estructuras de datos como comportamiento. Esto contrasta con la programación convencional, en la que las estructuras de datos y el comportamiento estaban escasamente relacionadas.

Las características principales del enfoque orientado a objetos son, en primer lugar:

Identidad.

Los datos se organizan en entidades discretas y distinguibles llamadas objetos. Estos objetos pueden ser concretos o abstractos, pero cada objeto tiene su propia identidad. Dicho de otra forma: dos objetos son distintos incluso aún en el caso de que los valores de todos sus atributos (p. ej. nombre y tamaño) coincidan. Dos manzanas pueden ser totalmente idénticas pero no por eso pierden su identidad: nos podemos comer una u otra.

Clasificación.

Los objetos que tengan los mismos atributos y comportamiento se agrupan en *clases*. Todas las manzanas tienen una serie de atributos comunes: tamaño, peso, grado de maduración, y un comportamiento común: podemos coger una manzana, moverla o comerla. Los valores de los atributos podrán ser distintos para cada una de ellas, pero todas comparten los mismos atributos y comportamiento (las operaciones que se pueden realizar sobre ellas). Una clase es una abstracción que describe propiedades (atributos y comportamiento) relevantes para una aplicación determinada, ignorando el resto. La elección de clases es arbitraria, y depende del dominio del problema.

Según esto, una clase es una abstracción de un conjunto posiblemente infinito de objetos individuales. Cada uno de estos objetos se dice que es una *instancia o ejemplar*[9] de dicha clase. Cada instancia de una clase tiene sus propios valores para sus atributos, pero comparte el nombre de estos atributos y las operaciones con el resto de instancias de su clase.

Polimorfismo.

El polimorfismo permite que una misma operación pueda llevarse a cabo de forma diferente en clases diferentes. Por ejemplo, la operación *mover*, es distinta para una pieza de ajedrez que para una ficha de parchís, pero ambos objetos pueden ser movidos. Una operación es una acción o transformación que realiza o padece un objeto. La implementación específica de una operación determinada en una clase determinada se denomina *método*.

Según lo dicho, una operación es una abstracción de un comportamiento similar (pero no idéntico) en diferentes clases de objetos. La semántica de la operación debe ser la misma para todas las clases. Sin embargo, cada método concreto seguirá unos pasos procedimentales específicos.

---

[9] *instancia*, a pesar de ser el término utilizado habitualmente en la bibliografía en castellano, es una mala traducción del término inglés *instance*, siendo *ejemplar* la traducción correcta. Algunos autores solventan el problema hablando de clases y objetos, pero esto último resulta a veces ambiguo.

Herencia.

El concepto de herencia se refiere a la compartición de atributos y operaciones basada en una relación jerárquica entre varias clases. Una clase puede definirse de forma general y luego refinarse en sucesivas subclases. Cada clase hereda todas las propiedades (atributos y operaciones) de su superclase y añade sus propiedades particulares.

La posibilidad de agrupar las propiedades comunes de una serie de clases en una superclase y heredar estas propiedades en cada una de las subclases es lo que permite reducir la repetición de código en el paradigma OO y es una de sus principales ventajas.

5.2. Técnica de modelado de objetos (OMT).

La esencia del desarrollo de software OO es la identificación y organización de conceptos del dominio del problema, más que en su implementación final usando un determinado lenguaje.

La Técnica de Modelado de Objetos (OMT, Rumbaugh, 1991) es un procedimiento que se basa en aplicar el enfoque orientado a objetos a todo el proceso de desarrollo de un sistema software, desde el análisis hasta la implementación. Los métodos de análisis y diseño que propone son independientes del lenguaje de programación que se emplee para la implementación. Incluso esta implementación no tiene que basarse necesariamente en un lenguaje OO.

OMT es una metodología OO de desarrollo de software basada en una notación gráfica para representar conceptos OO. La metodología consiste en construir un modelo del dominio de aplicación y ir añadiendo detalles a este modelo durante la fase de diseño. OMT consta de las siguientes fases o etapas.

5.2.1. Fases.

- **Conceptualización**. Consiste en la primera aproximación al problema que se debe resolver. Se realiza una lista inicial de requisitos y se describen los casos de uso.

- **Análisis**. El analista construye un modelo del dominio del problema, mostrando sus propiedades más importantes. Los elementos del modelo deben ser conceptos del dominio de aplicación y no conceptos informáticos tales como estructuras de datos. Un buen modelo debe poder ser entendido y criticado por expertos en el dominio del problema que no tengan conocimientos informáticos.

- **Diseño del sistema**. El diseñador del sistema toma decisiones de alto nivel sobre la arquitectura del mismo. Durante esta fase el sistema se organiza en subsistemas basándose tanto en la estructura del análisis como en la arquitectura propuesta.

- **Diseño de objetos**. El diseñador de objetos construye un modelo de diseño basándose en el modelo de análisis, pero incorporando detalles de implementación. El diseño de objetos se centra en las estructuras de datos y algoritmos que son necesarios para implementar cada clase. OMT describe la forma en que el diseño puede ser implementado en distintos lenguajes (orientados y no orientados a objetos, bases de datos, etc.).

- **Implementación**. Las clases de objetos y relaciones desarrolladas durante el análisis de objetos se traducen finalmente a una implementación concreta. Durante la fase de implementación es importante tener en cuenta los principios de la ingeniería del software de forma que la correspondencia con el diseño sea directa y el sistema implementado sea flexible y extensible. No tiene sentido que utilicemos AOO y DOO de forma que potenciemos la reutilización de código y la correspondencia entre el dominio del problema y el sistema informático, si luego perdemos todas estas ventajas con una implementación de mala calidad.

Algunas clases que aparecen en el sistema final no son parte del análisis sino que se introducen durante el diseño o la implementación. Este es el caso de estructuras como árboles, listas enlazadas o tablas *hash*, que no suelen estar presentes en el dominio de aplicación. Estas clases se añaden para permitir utilizar determinados algoritmos.

Los conceptos del paradigma OO pueden aplicarse durante todo el ciclo de desarrollo del software, desde el análisis a la implementación sin cambios de notación, sólo añadiendo progresivamente detalles al modelo inicial.

5.2.2. Modelos.

Al igual que los métodos estructurados, OMT utiliza tres tipos de modelos para describir un sistema:

- **Modelo de objetos**. Describe la estructura estática de los objetos de un sistema y sus relaciones. El modelo de objetos contiene diagramas de objetos. Un diagrama de objetos es un grafo cuyos nodos son clases y cuyos arcos son relaciones entre clases.

- **Modelo dinámico**. El modelo dinámico describe las características de un sistema que cambia a lo largo del tiempo. Se utiliza para especificar los aspectos de control de un sistema. Para representarlo utilizaremos DEs.

- **Modelo funcional**. Describe las transformaciones de datos del sistema. El modelo funcional contiene DFDs y especificaciones de proceso.

Los tres modelos son vistas ortogonales (independientes) del mismo sistema, aunque existen relaciones entre ellos. Cada modelo contiene referencias a elementos de los otros dos. Por ejemplo, las operaciones que se asocian a los objetos del modelo de objetos figuran también, de forma más detallada en el modelo funcional. El más importante de los tres es el modelo de objetos, porque es necesario describir qué cambia antes que decir cuándo o cómo cambia.

5.2.3. Diferencias con los métodos estructurados.

OMT invierte el método estructurado, en el que se da más importancia a la descomposición funcional del sistema y, por tanto, a los diagramas de proceso. Este enfoque de descomposición funcional puede parecer que lleva de forma más directa a una implementación del sistema, pero con frecuencia este sistema suele ser más frágil. Si cambian los requisitos, un sistema basado en descomposición funcional puede requerir una reestructuración masiva. Por el contrario, el enfoque OO se centra en primer lugar en identificar los objetos del dominio de aplicación y después en establecer procedimientos que los manejen. Aunque esto puede parecer más indirecto, el software OO se mantiene mejor ante los cambios de requisitos, porque se basa en la estructura subyacente del dominio de aplicación, en vez de en los requisitos funcionales de un determinado problema.

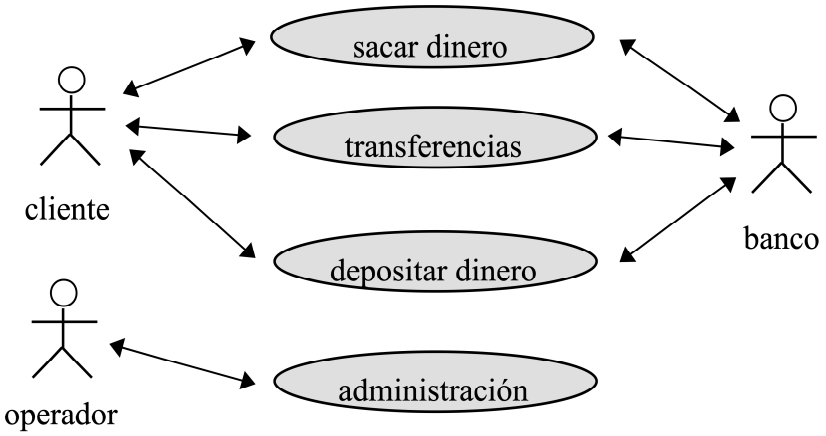
De todas formas, los modelos son los mismos que en el análisis estructurado. El modelo de objetos, el único que parece nuevo, es bastante similar a los DERs. Las diferencias principales consisten en la mayor importancia que se da el modelo de datos, por encima de los otros dos, y en el enfoque orientado a objetos de este modelo (encapsulando tanto datos como operaciones) en vez de estar orientado a bases de datos como los DERs.

En este sentido, puede compararse a los paquetes de lenguajes como ADA, con los que se implementan TADs. La diferencia reside en que en la POO, la herencia y el polimorfismo favorecen la reutilización de código y la programación por especialización, desde una perspectiva distinta al uso de paquetes genéricos en ADA.

5.3. Casos de uso.

Una forma de describir los requisitos iniciales del usuario, durante la fase de conceptualización, es construir casos de uso del sistema, descritos inicialmente por Jacobson en 1987 y actualmente incorporados a la mayor parte de las metodologías de AOO.

Un caso de uso está formado por una serie de interacciones entre el sistema y un *actor* (una entidad externa, ejerciendo un rol determinado), que muestran una determinada forma de utilizar el sistema. Cada interacción comienza con un evento inicial que el actor envía al sistema y continua con una serie de eventos entre el actor, el sistema y posiblemente otros actores involucrados.



Un caso de uso puede ser descrito en lenguaje natural, mediante trazas de eventos o mediante diagramas de interacción de objetos.

5.4. Modelo de objetos.

El modelo de objetos describe la estructura de los objetos de un sistema: su identidad, sus relaciones con otros objetos, sus atributos y sus operaciones. Los cambios y las transformaciones no tienen sentido a menos que haya algo que cambiar o transformar. OMT considera este modelo el más importante de los tres.

El modelo de objetos se representa gráficamente con diagramas de objetos y diagramas de instancias, que contienen clases de objetos e instancias, respectivamente. Las clases se disponen en jerarquías que comparten una estructura de datos y un comportamiento comunes, y se relacionan con otras clases. Cada clase define los atributos que contiene cada uno de los objetos o instancias y las operaciones que realizan o sufren estos objetos.

5.4.1. Elementos del modelo de objetos.

El modelo de objetos puede contener los siguientes elementos:

Objetos o instancias.

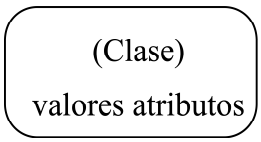
Un objeto es un concepto, una abstracción o una cosa con unos límites definidos y que es relevante para el problema en cuestión. Los modelos de objetos sirven tanto para obtener un conocimiento mejor del dominio de aplicación como de base para la implementación del sistema en un ordenador.

Una característica de los objetos es que tienen identidad y son distinguibles. Aunque dos objetos tengan los mismos valores para todos sus atributos son diferentes.

El término objeto está sobrecargado. Mediante él podemos referirnos tanto a clases de objetos (p. ej. el concepto abstracto mesa) como a las instancias de estas clases (una mesa determinada). Es mejor utilizar los términos clase e instancia para evitar confusiones.

La mayoría de las instancias de una clase derivan su individualidad de tener valores diferentes en alguno/s de sus atributos o de tener relaciones con instancias diferentes. No obstante pueden existir instancias con los mismos valores de los atributos e idénticas relaciones.

El símbolo gráfico para representar instancias es un rectángulo de esquinas redondeadas. Dentro del rectángulo figura la clase a la que pertenece la instancia (entre paréntesis) y los valores de sus atributos.



Las instancias figuran en diagramas de instancias, que se utilizan normalmente para describir ejemplos que aclaren un diagrama de objetos complejos o para describir escenarios determinados (p. ej. situaciones típicas o anómalas, escenarios de prueba, etc.).

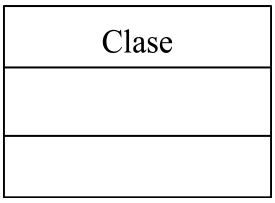
Clases.

Una clase o clase de objetos es una abstracción que describe un grupo de instancias con propiedades (atributos) comunes, comportamiento (operaciones) común, relaciones comunes con otros objetos y (lo que es más importante) **una semántica común**. Así un *caballo* y un *establo* tienen los dos un *coste* y una *edad*, pero posiblemente pertenezcan a clases diferentes (aunque esto

depende del dominio de aplicación: en una aplicación financiera ambos pertenecerían posiblemente a la misma clase: *Inversiones*).

La diferencia entre instancia y clase está en el grado de abstracción. Un objeto es una abstracción de un objeto del mundo real, pero una clase es una abstracción de un grupo de objetos del mundo real. La abstracción permite la generalización y evita la redefinición de las características (atributos, comportamiento o relaciones) comunes, de forma que se produce una reutilización de estas definiciones comunes por parte de cada uno de los objetos. Por ejemplo todas las elipses (instancias) comparten las mismos operaciones para dibujarlas o calcular su área.

El símbolo gráfico para representar clases es un rectángulo, en el que figura el nombre de la clase. Las clases se representan en los diagramas de clases, que son plantillas que describen un conjunto de posibles diagramas de instancias. Describen, por tanto el caso general.

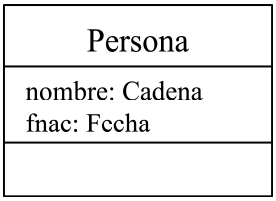


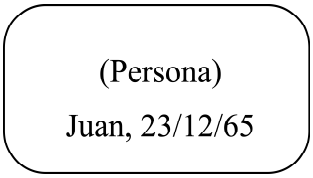
Atributos.

Un atributo es un dato contenido en todas las instancias de una clase. Cada atributo tiene un valor para cada una de las instancias. Varias clases pueden tener atributos comunes (p. ej. *nombre*, en las clases *Persona* y *Calle*) pero cada atributo debe ser único dentro de una clase.

Los atributos tienen que ser datos, no objetos. La diferencia entre unos y otros reside en la identidad: los objetos tienen identidad, pero los atributos no. Por ejemplo, todas las ocurrencias del valor '3' de un atributo son indistinguibles.

Los atributos se representan en el segundo área de los símbolos de clase e instancia. En las clases, figurará el nombre del atributo, el tipo y el valor por defecto. En las instancias, el valor del atributo para ese objeto determinado.

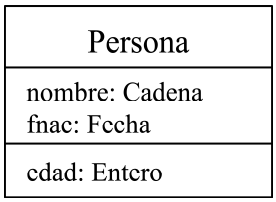


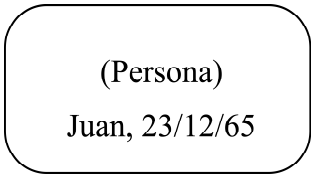


Operaciones.

Una operación o método es una función o transformación. Cada operación lleva implícito un objeto destino, sobre el que se va a realizar la operación. El comportamiento de la operación depende de la clase del objeto destino. Todos los objetos de una clase comparten las mismas operaciones o métodos. Cada objeto conoce la clase a que pertenece y, por tanto, la implementación correcta de la operación. Una misma operación puede aplicarse a objetos de clases distintas. En este caso diremos que la operación es polimórfica, y a la implementación de la operación en cada una de las clases la llamaremos método.

Una operación puede tener una serie de argumentos explícitos, además del objeto destino, que actúa siempre como argumento implícito.

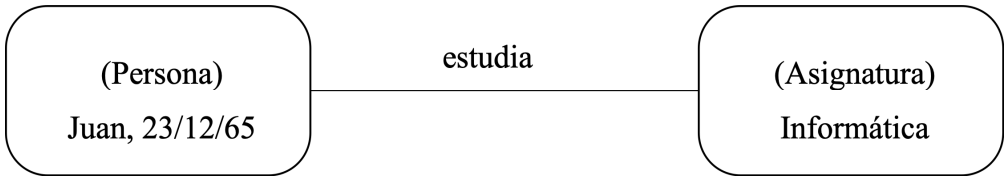




Las operaciones figuran en la tercer área del símbolo de las clases. Opcionalmente figuran también la lista de argumentos y el tipo de resultado de la operación (si es que la operación devuelve algún resultado). En los símbolos de instancia no figuran las operaciones.

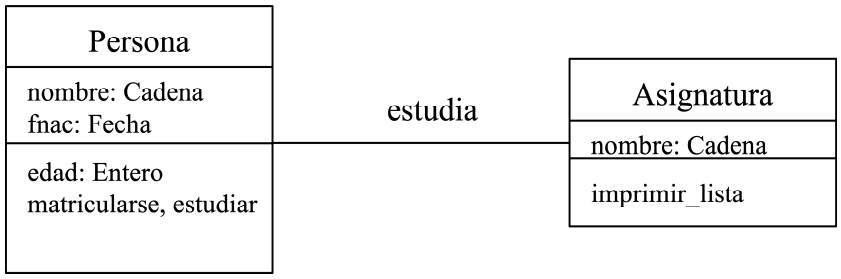
Enlaces.

Un **enlace** es una conexión entre dos o más instancias (objetos). Los enlaces pueden ser considerados como las instancias de las asociaciones.



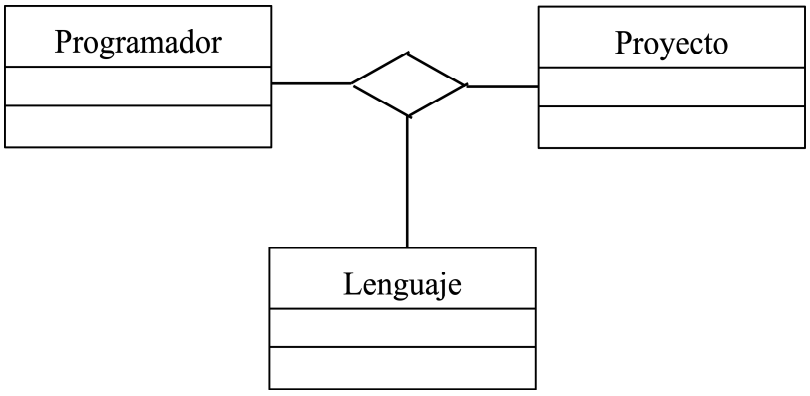
Asociaciones.

Una **asociación** es una abstracción de un grupo de enlaces con una estructura común (todos ellos conectan instancias de objetos de las mismas clases) y una semántica común (todos los enlaces tienen el mismo significado. Una asociación describe un conjunto de enlaces potenciales de la misma forma que una clase describe un conjunto de instancias potenciales.



Tanto los enlaces como las asociaciones se representan mediante arcos, generalmente etiquetados con el nombre de la asociación. Aunque la etiqueta induce a leer una asociación en un determinado sentido, las asociaciones pueden recorrerse en cualquier dirección.

Los enlaces y las asociaciones pueden conectar más de dos objetos. En este caso se representan mediante un rombo con conexiones a cada uno de los objetos:



La mayoría de las asociaciones conectan solamente dos clases de objetos y, siempre que sea posible usaremos asociaciones binarias porque son más sencillas de nombrar, entender y de implementar Sin embargo habrá casos en los que no podamos desglosar una asociación n-aria en varias asociaciones binarias. En el ejemplo anterior si desglosamos la asociación en tres (*Programador - conoce - Lenguaje, Proyecto - se implementa en - Lenguaje, y Programador - participa en - Proyecto*), no podemos representar que un *Programador* participa en un *Proyecto* programando en un *Lenguaje* determinado de lo que conoce.

Multiplicidad.

Cada asociación puede modelar la conexión un número indeterminado de objetos de las clases que conecta. Para representar el número de instancias de cada clase que pueden participar en una asociación utilizaremos la siguiente notación en cada extremo de la asociación:

- ○ Opcional. La asociación puede relacionar 0 ó 1 instancias de la clase
- ● Muchos. Significa de 0 a N.
- 3 Exactamente 3.

2,4 Dos o cuatro.

2-4 De dos a cuatro.

4+ Más de cuatro.

Exactamente 1.

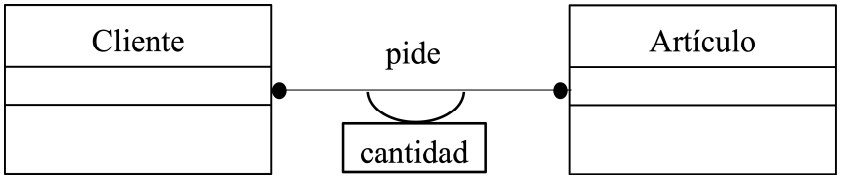
Una multiplicidad mayor que uno (p. ej. la existente entre las clases *Clase1* y *Clase2* abajo) indica que para cada instancia del *Clase1* pueden existir muchas instancias de la asociación (muchos enlaces) que lo conecten a instancias de la *Clase2*.



Con frecuencia la multiplicidad no viene expresada en la especificación preliminar, por lo que tendremos que decidirla en base al uso que queremos hacer del sistema. No es conveniente abusar de relaciones del tipo muchos a muchos pues, aunque son las más generales, van a sobrecargar el sistema a la hora de la implementación. De todas formas, no hay que preocuparse mucho de la multiplicidad en las etapas iniciales del análisis, cuando tengamos un mayor conocimiento del dominio del problema será más fácil decidir sobre ella.

Atributos de una asociación.

Al igual que los objetos contienen atributos, las asociaciones pueden contenerlos también. (p. ej. asociación *pide* entre *Clientes* y *Artículos*, que tiene un atributo *cantidad*). Estos atributos no pertenecen a ninguna de las clases relacionadas por lo que no hay más remedio que representarlos en la asociación. Estos atributos tendrán un valor propio para cada una de las instancias de la asociación (enlaces).



No obstante, si la asociación tiene multiplicidad 1/1 o 1/M podemos incluir estos atributos en la clase que tiene multiplicidad 1. Por ejemplo, en la relación *Persona - compra - Casa*, el atributo *fecha de compra*. puede incluirse en la clase *Casa*. Sin embargo, si la asociación es M/M, no podemos hacer esto: por ejemplo en un sistema de multipropiedad una casa puede pertenecer a varias personas, que la compran en fechas distintas. En cualquier caso no es conveniente incluir los

atributos de las asociaciones en alguna de las clases relacionadas, puesto que reducimos la flexibilidad de nuestro modelo, que tendrá que ser reestructurado en caso de que necesitemos cambiar la multiplicidad.

Una asociación con atributos se parece bastante a una clase. En determinados casos (si la asociación contiene también operaciones o si mantiene asociaciones con otras clases podemos modelarla como una clase.

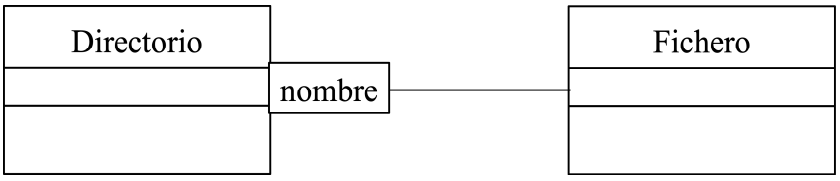
Calificación.

Un tipo especial de asociaciones son las asociaciones calificadas. Si queremos expresar mediante el modelo que *'un directorio contiene varios ficheros, cada uno de ellos identificados mediante un nombre'* podemos hacer una asociación 1/M normal, con *nombre fichero* como atributo de *Fichero* (1)



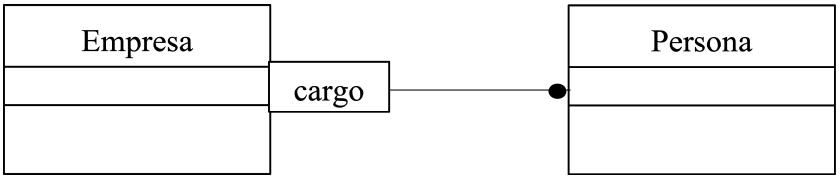
(1)

De esta forma, partiendo del directorio podemos acceder a todos sus ficheros mediante la asociación múltiple, pero así no expresamos que el nombre **identifica** a los ficheros dentro del directorio. La solución es utilizar *nombre fichero* como calificador en el extremo opuesto al M de la asociación (2), en este caso desaparece la multiplicidad en la asociación, puesto que utilizando el nombre la asociación múltiple se convierte en unitaria y, además, el modelo contiene una información mucho más exacta.

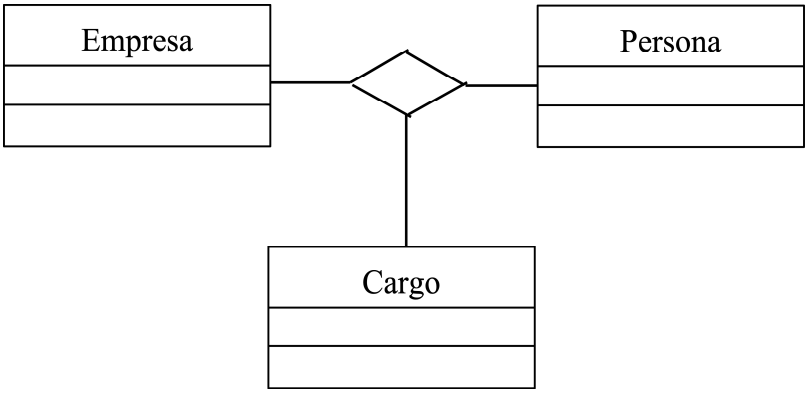


(2)

La calificación reduce la multiplicidad pero en algunos casos no se logra conseguir una multiplicidad unitaria. Tomemos como ejemplo el calificador *cargo* en la plantilla de una empresa. Suponemos que varias personas pueden ocupar el mismo cargo (3)

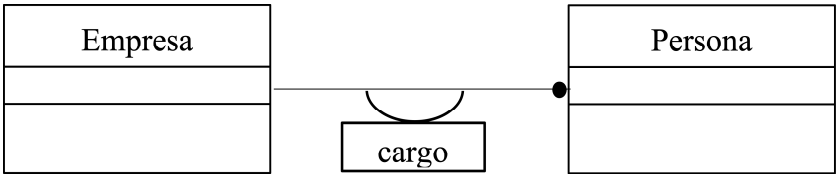


(3)



(4)

En algunos casos podemos considerar al calificador como un atributo especial del objeto situado al otro extremo de la asociación (1), o también podemos considerar la asociación calificada como una forma de asociación ternaria (4) o asociación con atributos (5). Por último si consideramos que una misma persona puede ocupar varios cargos distintos en la empresa, no podemos considerar *cargo* como un atributo de persona (6 *), pues esto significaría que una persona ocupa varias veces el mismo cargo en varias empresas.



(5)

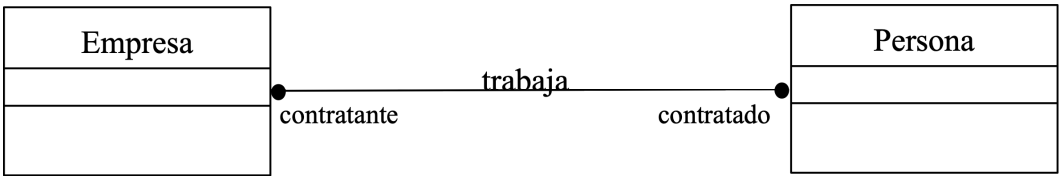
En resumen, la calificación nos permite reducir la multiplicidad y tiene una mayor potencia expresiva que las otras dos opciones (considerar el calificador como atributo de una de las clases o incluirlo como atributo de la asociación) por lo que optaremos por la asociación calificada cuando nos encontremos ante situaciones similares a las descritas.



(6 *)

Roles o papeles.

Para mayor claridad podemos etiquetar los extremos de las asociaciones con el rol o papel que representa la clase en la asociación. Así, en la relación *trabaja,* las personas juegan el papel de empleados mientras que las empresas juegan el papel de contratantes.

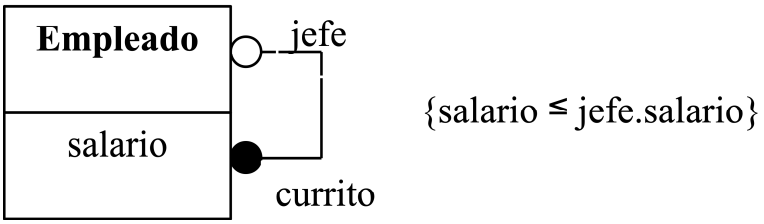


Restricciones

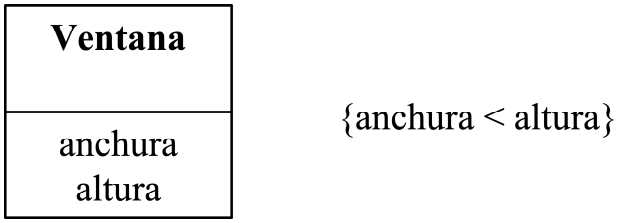
Las restricciones son relaciones funcionales entre entidades de un modelo de objetos. El término entidad incluye objetos, clases, atributos, enlaces y asociaciones. Una restricción restringe los valores que una entidad puede tomar.

Ejemplos:

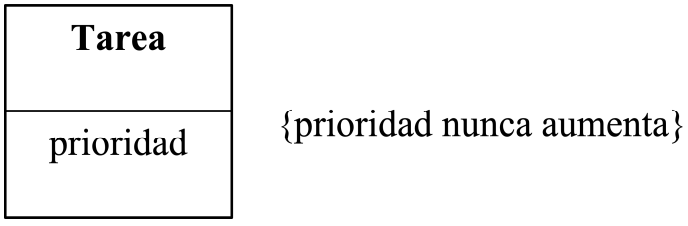
- Restricción entre objetos: El salario de un empleado no puede superar el salario del jefe.



- Restricciones entre atributos de un objeto: Una ventana tiene que ser más alta que ancha.



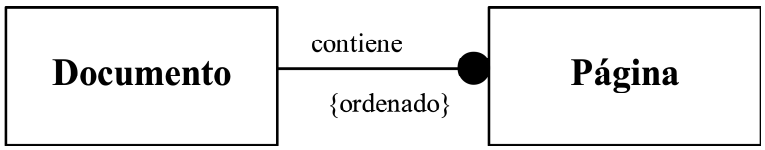
- Restricciones sobre un objeto a lo largo del tiempo: la prioridad de un trabajo no puede aumentar.



• Restricciones sobre enlaces:

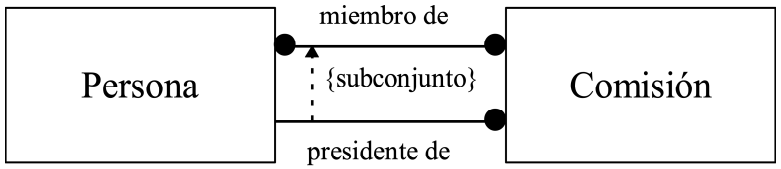
  ° La multiplicidad restringe el números de objetos relacionados con un objeto determinado.

  ° Por regla general, las instancias conectadas al extremo M de una asociación no tienen ningún orden, pero en algunos casos sí que existe este orden. Un ejemplo de relación ordenada es la que se establece entre un documento y sus páginas. Para expresar esta característica etiquetaremos el extremo M con **{ordenado}**.



Las restricciones simples pueden situarse en el modelo de objetos; restricciones complejas aparecerán en el modelo funcional. Las restricciones no tienen porqué aparecer inicialmente en el modelo de objetos, estas irán añadiéndose conforme se vaya concretando en la definición del modelo.

Las restricciones se escriben entre llaves y se colocan junto a la entidad restringida. Las restricciones se expresan en lenguaje natural o con ecuaciones. En caso de que existan varias entidades involucradas en una restricción, dibujaremos una línea de puntos entre las entidades. (Ejemplo: una asociación puede ser subconjunto de otra: el presidente de una comisión debe ser un miembro de la comisión -la asociación *presidente_de* es un subconjunto de la asociación *miembro_de*).

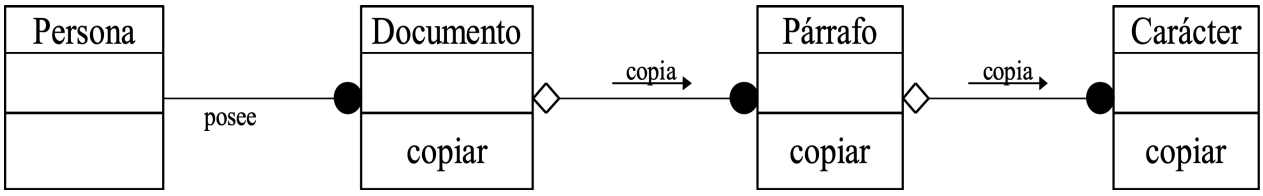


5.4.2. Relaciones de composición o agregación.

La composición nos permite representar las relaciones de *se compone de* o *es una parte de* que existen entre los objetos del modelo. Podemos utilizar asociaciones normales para representar la composición (y así lo hemos hecho en alguno de los ejemplos anteriores) pero la relación de composición tiene unas características especiales, por lo que se ha incluido un tipo de asociación especial para representarla.

Estas características son que la composición es transitiva, mientras que las relaciones normales no lo son. Por otra parte, la composición es antisimétrica y, por último, algunas propiedades del conjunto se propagan o influyen a las partes (p. ej. Un robot móvil con un brazo articulado: la posición del manipulador en el extremo del brazo depende no sólo de la posición (ángulos de rotación del brazo) sino también de la posición de todo el robot móvil.

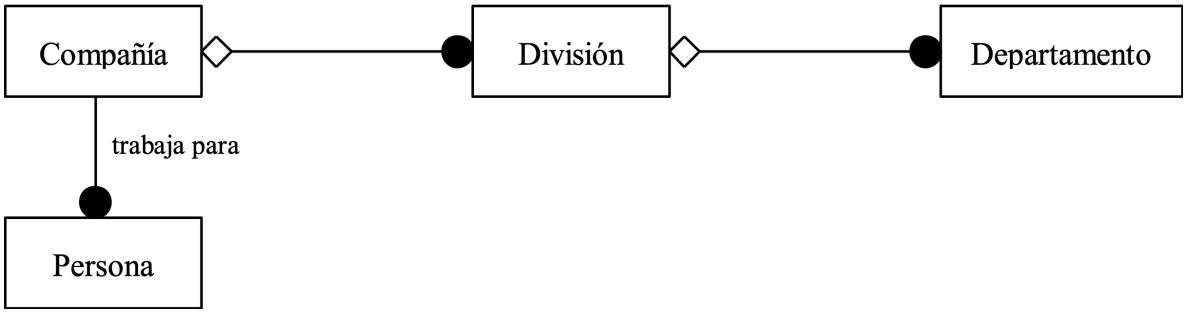
La relación de composición se expresa gráficamente con un rombo al extremo de la asociación (p. ej. los *Documentos* se componen de *Párrafos*, que a su vez se componen de *Caracteres*). Para modelar un objeto que se compone de objetos de varias clases distintas (p. ej. una cebra que se compone de una cabeza, un cuerpo, hasta cuatro patas y una cola opcional) podemos agrupar las relaciones de composición en una única asociación ramificada, indicando la multiplicidad en cada extremo.



A veces, la aplicación de una operación a un objeto supone la aplicación automática de esta operación a un conjunto de objetos. A esto se le llama propagación.. La operación de *copiar* se propaga del documento a sus párrafos y de estos a sus caracteres. La operación sin embargo no se propaga en la dirección inversa, el hecho de *copiar* un párrafo no implica la copia del documento.

Agregación y Asociación

Según lo visto, la agregación no es más que un tipo (frecuente) de asociación, que tiene unas características determinadas y que se representa en OMT mediante una notación gráfica especial. La diferencia entre asociación y agregación es fundamentalmente semántica, y hay que tener esto presente a la hora de decidir modelar una relación entre clases como asociación o como agregación. Por ejemplo, una *Empresa* es una agregación de *Divisiones*, que a su vez son agregaciones de *Departamentos*. Sin embargo, no es una agregación de *Personas*, puesto que *Empresa* y *Persona* son objetos independientes.

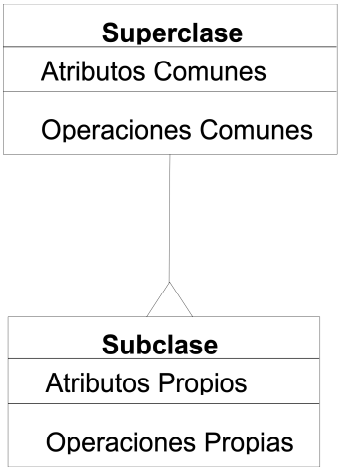


5.4.3. Generalización.

El último tipo de relación puede presentarse entre las clases es la relación de generalización o especialización (depende desde dónde se mire).

La generalización es una forma de abstracción que permite modelar que varias clases comparten una serie de características comunes, a la vez que tienen características propias. En el paradigma OO la generalización se relaciona con el mecanismo de **herencia**, mediante el que los objetos heredan las características comunes (atributos y operaciones) y es la fuente principal de la reutilización de código.

Generalización o **especialización** es la relación que se establece entre una clase y una o más versiones refinadas de esta clase. La clase general se denomina superclase y las clases especializadas, subclases.

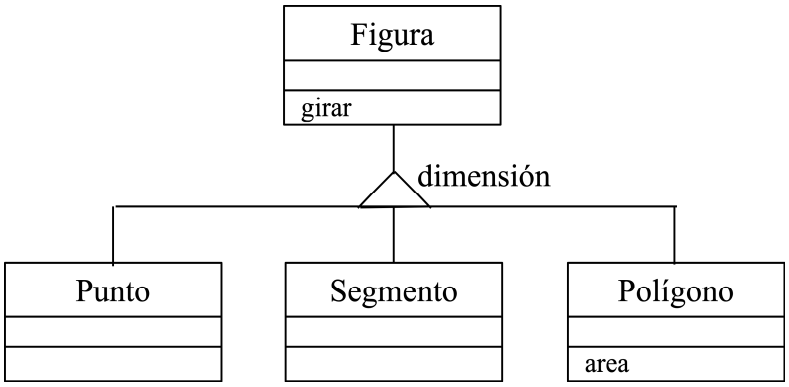


En la superclase definiremos los atributos y operaciones comunes a todas las subclases. En cada subclase definiremos los atributos y métodos específicos para esa subclase. El mecanismo de herencia garantiza que todas las subclases heredan los atributos y operaciones definidos en la superclase.

La relación de generalización puede modelarse en varios niveles y es transitiva: las instancias de una subclase heredan los atributos y operaciones de todas las clases antecesoras. Además, cada instancia de una subclase puede considerarse también una instancia de cada una de las clases antecesoras: una lista ordenada doblemente encadenada es también una lista doblemente encadenada y una lista.

Si hay varias clases que particularizan una dada, podemos modelar esto mediante una sola relación de generalización ramificada. En ocasiones, existe un atributo de la superclase cuyos

valores determinan la adscripción de un objeto a una u otra de las subclases. Un atributo de este tipo se denomina **discriminante.**



Redefinición.

Las subclases no sólo heredan los atributos y las operaciones de la superclase sino que también pueden redefinirlos. Esto se hace cuando un método general definido en una superclase no puede aplicarse tal como está a algunas de las subclases o puede realizarse una implementación más eficiente del método (p. ej. en la jerarquía anterior podemos redefinir el método *girar*. De la misma forma, las clases que hereden de *Polígono* redefinirán el método *área*.

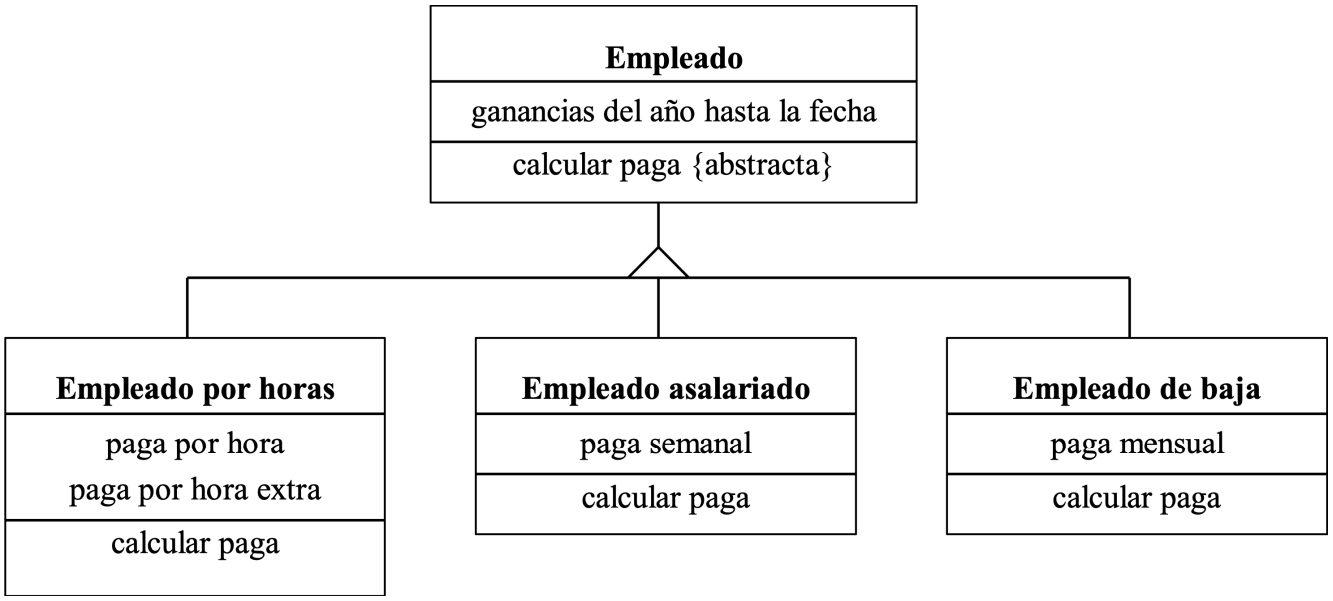
La redefinición no puede cambiar la semántica de la característica redefinida (la operación tiene que ser la misma, aunque el método cambie) ni puede cambiar la **signatura** (es decir la forma: el número y tipo de los parámetros) de la característica redefinida. Algunos lenguajes de POO permiten que la redefinición restrinja el tipo de los parámetros (que sea por ejemplo un subrango).

Clases Abstractas

Una clase abstracta es una clase que no tiene instancias directas pero cuyas clases descendientes (clases concretas) sí pueden tener instancias directas. Una clase concreta puede tener subclases abstractas, pero éstas han de tener subclases concretas (las hojas de una jerarquía de clases han de ser clases concretas).

Las clases abstractas organizan características comunes a varias clases. A veces es útil crear una superclase abstracta que encapsule aquellas características (atributos, operaciones y asociaciones) comunes a un conjunto de clases. Puede ser que esta clase abstracta aparezca en el dominio del problema o bien que se introduzca artificialmente para facilitar la reutilización de código.

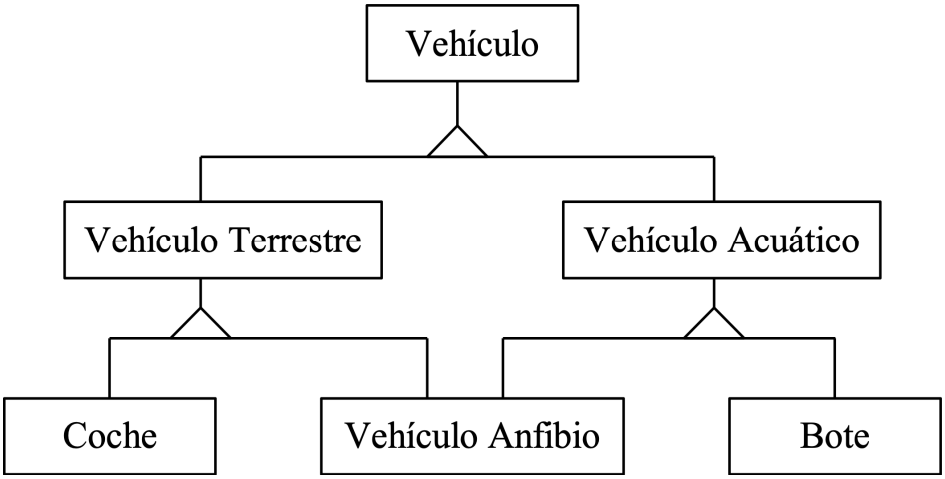
Normalmente las clases abstractas se usan para definir métodos que son heredados por sus subclases. Sin embargo, una clase abstracta puede definir el protocolo de una determinada operación sin proporcionar el correspondiente método. Esto de denomina operación abstracta, que define la forma de una operación para la que cada subclase concreta debe suministrar su propia implementación. Una clase concreta no puede tener operaciones abstractas porque los objetos de esta clase tendrían operaciones indefinidas. Para indicar que una operación es abstracta se coloca una restricción {abstracta} junto a ella.



Herencia múltiple

La herencia múltiple permite a una clase tener más de una superclase, y así heredar las características de todos sus padres. Esto complica las jerarquías de herencia, que dejan de ser árboles para convertirse en grafos. La gran ventaja de la herencia múltiple es el incremento en las posibilidades de reutilización. El inconveniente es la pérdida de simplicidad conceptual y de implementación.

La herencia múltiple presenta dos problemas: conflictos y herencia repetida. Cuando una clase hereda simultáneamente sus propiedades de varias clases, pueden existir en estas propiedades representadas por el mismo identificador con significados diferentes; a esto se le denomina *conflicto*. Los conflictos que provoca la herencia múltiple se agravan en presencia de situaciones en las que una clase es heredada por otra por medio de caminos distintos; es lo que se denomina *herencia repetida*.



Existen muchas propuestas para resolver este tipo de problemas, casi tantas como lenguajes orientados a objetos. Podemos encontrar desde sistemas que dejan al usuario su resolución (algunas versiones de Smalltalk), hasta los que prohiben situaciones conflictivas (C++, Eiffel), pasando por sistemas que usan algún heurístico linealizando el árbol de herencia (CommonLoops), o los que simplemente evitan el problema impidiendo la herencia múltiple (Java). Las ventajas e inconvenientes de las posibles alternativas de la herencia múltiple se han estudiado con cierta profundidad, llegando a la conclusión de que cada alternativa tiene razones a favor y en contra, lo que impide tomar decisiones de diseño que satisfagan todo tipo de exigencias. En la mayoría de los casos, la interpretación que se da a la herencia múltiple depende más de su adecuación a la implementación que a otros motivos.

5.4.4. Consejos prácticos.

Por último, una serie de consejos prácticos para realizar modelos de objetos. Muchos de estos consejos son muy similares a los de los métodos estructurados: lo importante de una notación gráfica es que represente de forma correcta y clara el dominio de aplicación.

- **No lanzarse a dibujar clases y asociaciones sin sentido.** Primero hay que entender el problema. Los métodos y herramientas no hacen análisis, el análisis lo hacemos nosotros ayudándonos con estos métodos y herramientas.

- **Intentar que el modelo sea simple, evitando complicaciones innecesarias.** De lo que se trata es de que el modelo sea claro, que alguien más pueda entenderlo, aceptarlo o criticarlo.

- **Los nombres de objetos, asociaciones, atributos y operaciones deben ser significativos**. Antes de poner un nombre hay que pensárselo un poco: el nombre representa al elemento. Las clases deben nombrarse con sustantivos y las asociaciones

con verbos. Por las asociaciones se envían mensajes, pero no objetos. Un nombre largo puede ser más significativo pero no será más claro.

- **No incluir en los objetos punteros a otros objetos**. Esto sólo sirve para que las relaciones entre objetos queden ocultas. Hay que modelar estas relaciones mediante asociaciones. Los punteros es como implementan algunos lenguajes las asociaciones, pero nuestro modelo debe ser independiente de la implementación.

- **Utilizar, si es posible, asociaciones binarias**. Las asociaciones de orden superior son más difíciles de nombrar, entender e implementar. De todas formas hay asociaciones que no podemos descomponer en binarias sin perder información.

- **Dejar la definición de la multiplicidad para cuando se tenga un mejor conocimiento del problema**. Pero no hay que olvidarse de ella. No hay que abusar de la multiplicidad 1/M o M/M pues, aunque sea más general la implementación es menos eficiente. Por otra parte, hay que tener cuidado con la multiplicidad 1/1. En muchos casos se trata de multiplicidad opcional (es decir, 1/0-1). La multiplicidad 1/1 implica que los objetos conectados no pueden existir de forma independiente, sino sólo por pares. Normalmente esta consideración es excesivamente estricta y puede llevarnos a perder información).

- **No incluir los atributos de las asociaciones en las clases**. Aunque las asociaciones sin atributos son más sencillas, el hacer esto limita la flexibilidad del modelo.

- **Utilizar preferentemente asociaciones calificadas en vez de ternarias o con atributos**. Proporcionan una información más exacta del dominio de aplicación.

- **Evitar las jerarquías de composición o generalización de muchos niveles**. Dificultan la legibilidad del modelo.

- **Revisar el modelo hasta que sea satisfactorio**. No podemos pretender que la primera versión sea correcta. Es conveniente mostrar el modelo a otras personas (no en el examen).

- **Documentar el modelo**. Una imagen vale más que mil palabras, pero una imagen con texto es siempre mucho más explicativa que otra sin él.

- **Utilizar sólo los elementos necesarios**. No se trata de lucirse y mostrar que se dominan todos los conceptos del modelo, sino de que el modelo sea correcto y claro.

5.5. Modelo dinámico.

5.5.1. Diagramas de estados.

El modelo dinámico del método OMT se corresponde con el modelo de control o modelo de comportamiento de las técnicas de análisis estructurado. En ambos casos, el modelo se representa mediante DEs, pero en el caso de OMT estos DEs se utilizan para modelar el comportamiento de cada clase de objetos, es decir, para modelar el comportamiento común a todas las instancias de una clase.

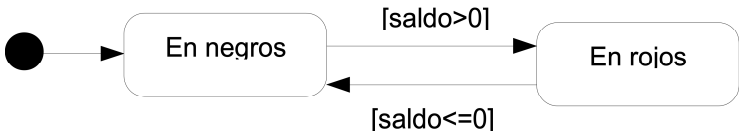
Al igual que todas las instancias de una clase comparten atributos, pero tienen valores particulares para esos atributos, todas las instancias de una clase se comportan de igual forma, pero pueden encontrarse en estados distintos.

Los elementos que figuran en un DE ya son conocidos, aquí sólo vamos a referirnos a las diferencias con respecto al análisis estructurado.

Estados y eventos.

Por *estado* de un objeto entendemos los valores de los atributos y los enlaces que mantiene un objeto en un momento determinado. Los objetos interactúan unos con otros y como consecuencia de esas interacciones cambian de estado (es decir, cambian el valor de sus atributos o sus enlaces con otros objetos).

Los estados que figuran en los DEs son abstracciones de los valores de los atributos y enlaces de un objeto, es decir, engloban conjuntos de valores de los atributos, de forma que muestran situaciones en las que el objeto presenta un determinado comportamiento. Por ejemplo, en el objeto *Cuenta Corriente*, el estado *En rojos* engloba todos los estados del objeto en los que el saldo es negativo.



Al definir los estados, debemos ignorar los atributos que no afectan al comportamiento del objeto y agrupar todas las combinaciones de valores de atributos y enlaces en las que el objeto presente el mismo comportamiento.

La interacción entre objetos se realiza mediante *eventos*. Como consecuencia de (la recepción de) un evento, un objeto puede realizar una serie de acciones y/o enviar eventos a otros

objetos y/o cambiar de estado. Los eventos son el método de intercambiar información entre los objetos. Esta información consistirá en la mayoría de los casos en una señal lógica, pero en otros puede tratarse de información más compleja, y el evento tendrá una serie de atributos:

cancelar	evento lógico
retirar(numero de cuenta, 5000)	evento con atributos

La respuesta de un objeto en un estado determinado ante un evento determinado puede depender de los valores exactos de los atributos, pero sólo cuantitativamente. Cualitativamente, la respuesta será la misma para todos los objetos que se encuentren en ese estado.

La relación o secuencia de estados, eventos, transiciones y acciones se representa en los DEs. El modelo dinámico es un conjunto de DEs, uno para cada clase de objetos que tenga un comportamiento dinámico no trivial (no es necesario hacer un DE para cada clase sino sólo para aquéllas que lo precisen). La relación entre los distintos DEs se establece mediante los eventos que emiten unos objetos y consumen otros (se emiten en un determinado DE y se consumen en otro).

Guardas.

La encapsulación propia del paradigma OO impide con carácter general el acceso directo a valores de los atributos de otros objetos. Por este motivo una guarda es una condición que se define sobre los **atributos propios** de un objeto [4]. Al recibir un evento, la transición etiquetada con dicho evento se dispara si y sólo si se satisface la guarda.

Actividades y acciones.

Las actividades y las acciones son las respuestas que tiene que dar un objeto ante los eventos que se producen en el exterior o las guardas (condiciones de datos) que se satisfacen internamente. Típicamente, las acciones y las actividades van a ser operaciones definidas en la clase a la que pertenece el objeto, aunque en ocasiones las acciones pueden consistir en enviar un mensaje a otro objeto.

Como ya sabemos, una **actividad** es una operación que necesita un tiempo para completarse, y se corresponde siempre con la ejecución de un determinado método o con el objeto inactivo, esperando a que se cumplan determinadas condiciones. Cuando un objeto entra en un

[4] En ocasiones podemos utilizar en una guardas atributos de objetos distintos a aquél cuyo DE estamos representando. Pero esto lo haremos sólo con caracter excepcional para evitar alargar innecesariamente el DE y siempre que no se puedan producir colisiones a la hora de acceder a estas variables externas.

estado, ejecuta la actividad correspondiente hasta que ésta finalice o bien hasta que el disparo de una transición nos haga abandonar dicho estado.

Por el contrario, una **acción** es una operación instantánea, que idealmente no necesita tiempo para realizarse y que se asocia, por tanto, al disparo de una determinada transición. Las acciones generalmente consisten en asignar valores a los atributos del objeto o en generar eventos para que sean tratados por otro proceso (con la notación *enviar: evento*). Por este motivo, las acciones no suelen corresponderse con operaciones del objeto, sino posiblemente con parte del código de ciertas operaciones.

Trazas de eventos.

Los eventos son el medio que utilizan los objetos para comunicarse entre sí. Podemos representar la secuencia de eventos que se ha producido entre los objetos de un determinado sistema mediante **una traza de eventos**. Esta traza es un diagrama que muestra la secuencia de envíos y recepciones de eventos entre varios objetos.

Las trazas de eventos son útiles para mostrar cómo un sistema ha llegado a un escenario determinado, y se mostrarían entonces junto con el diagrama de instancias que represente ese escenario, pero también son útiles para construir el modelo dinámico del sistema, pues muestran una secuencia lógica de eventos que podemos utilizar para realizar los DE de cada clase. A partir de varias trazas de eventos podemos construir el DE de una clase.

Cambiando el punto de vista, un diagrama de estados nos permite determinar la secuencia de estados por los que pasa un objeto a partir de una traza de eventos. Si el evento recibido figura en alguna de las transiciones que salen del estado, entonces la transición **se dispara** y el objeto pasa al estado de llegada de la transición.

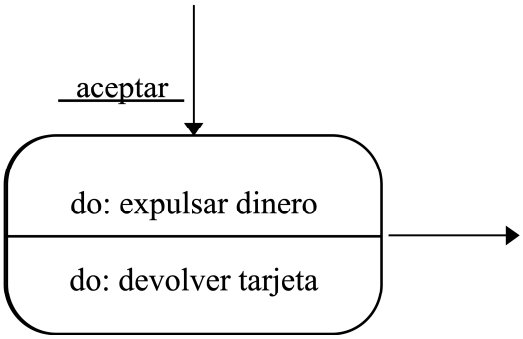
5.5.2. Composición en los DEs.

Una de las características del paradigma OO es la definición de objetos componiendo otros objetos. El DE de un objeto compuesto estará formado por la unión de los DEs de cada uno de los objetos que lo componen. Cada uno de los objetos estará en un estado determinado, por lo que el estado del objeto compuesto estará representado mediante una tupla que contiene cada uno de los estados de los componentes. Es decir, los objetos compuestos tienen estados compuestos.

Esta posibilidad nos permite describir la concurrencia, no sólo en los objetos compuestos, sino en cualquier objeto. Podemos descomponer un objeto simple en varias partes, cada una que contiene un subconjunto de sus atributos y enlaces, y realizar un DE para cada uno de esos subconjuntos. Al igual que antes, el estado del objeto vendrá representado por la composición de

estos DEs. Cada uno de los componentes de un objeto modelado mediante un DE compuesto evoluciona de forma concurrente al resto.

En algunos casos, podemos necesitar mostrar la concurrencia entre las actividades que se realizan dentro de un estado. Podemos encontrarnos con un estado en el que la actividad a realizar pueda ser descompuesta en una serie de pasos concurrentes. En este caso dividiremos el estado en dos (o más) subestados concurrentes.



Ejemplo: Las operaciones *Expulsar Dinero* y *Devolver Tarjeta* del cajero automático.

La sincronización de las transiciones de salida de cada uno de los subestados se hace juntando estas transiciones para formar la transición de salida del objeto compuesto.

5.5.3. Generalización en los DEs.

Generalización de estados.

Puede ocurrir que el objeto tenga un comportamiento tan complejo que el DE que lo modele sea demasiado grande y difícil de leer. En estos casos, podemos hacer DEs anidados, en los que un estado del DE padre se corresponda con un DE hijo. En este DE hijo se detalla el estado padre en una secuencia de estados, transiciones, acciones y actividades. Esta forma de anidamiento es similar al anidamiento que se producía en los DEs del análisis estructurado.

Podemos utilizar esta posibilidad en el caso de que la actividad que se asocie a un estado pueda ser descompuesta en una serie de operaciones secuenciales. En el DE hijo representaremos cada una de estas operaciones en un nuevo estado. Las transiciones que entran y salen del estado padre deben figurar en los estados iniciales y finales del DE hijo.

Alternativamente, podemos utilizar el anidamiento de estados para representar las transiciones comunes a una serie de estados, al igual que hacíamos en el análisis estructurado. Cada estado anidado o **subestado** comparte las transiciones de salida del estado padre o **superestado**, por otra parte las transiciones que entran en el padre nos llevan al estado inicial del DE anidado.

Cuando un objeto esté en un superestado, habrá de estar necesariamente en uno y sólo en uno de los subestados correspondientes.

Generalización de eventos.

Al igual que los estados también podemos generalizar los eventos. Podemos agrupar los eventos en clases hasta conseguir una jerarquía de eventos, lo que nos permite utilizar diferentes grados de abstracción en diferentes partes del modelo.

5.5.5. Herencia de DEs.

Según hemos dicho a cada clase del modelo de objetos se asocia un DE que modela su comportamiento, pero ¿qué sucede cuando dos clases están relacionadas mediante generalización? Entre las propiedades que se heredan de la clase padre está también el comportamiento, por lo que las subclases **heredan el DE de la superclase.**

Es posible que la subclase añada atributos u operaciones con respecto a la superclase. Entonces podemos considerar el DE de la subclase como la composición del DE de la superclase con el/los DEs que modelen el comportamiento del objeto con respecto a los nuevos atributos, al igual que hacíamos con los objetos compuestos.

En otros casos, la especialización del DE consistirá en el refinamiento de alguno de los estados del DE de la superclase, descomponiendo este estado en el DE de la subclase. Esto se corresponde con una división más detallada del espacio de estados heredado (el conjunto de valores de los atributos heredados).

En cualquier caso existe la posibilidad de conflicto: si la subclase redefine el comportamiento de los objetos respecto a alguno de los atributos heredados. No se pueden incluir transiciones o estados nuevos en el DE heredado puesto que entonces no habría una correspondencia entre ambos DEs. Estas situaciones, en las que las clases herederas modifican tienen un comportamiento distinto al de la clase padre y no es posible establecer una correspondencia entre ambos, son bastante frecuentes en la práctica. En estos casos no podemos hablar de herencia del DE sino de una redefinición completa del mismo. Esto es una limitación del modelo que no está resuelta y que contrasta con las posibilidades de redefinición de operaciones y atributos al especializar una clase. Esta limitación puede presentarse también cuando el comportamiento relacionado con los atributos nuevos definidos en la clase hija no puede modelarse de forma independiente a los heredados, sino que requiera realizarse de forma conjunta.

5.5.6. Modelo dinámico: consejos prácticos.

- **No todas las clases necesitan un DE**. Sólo haremos DEs para aquellas clases que lo necesiten. Habrá clases cuyo comportamiento sea trivial, por ejemplo aquéllas clases que cuyo comportamiento sea siempre igual, es decir, que posean un único estado.

- **Utilizar trazas de eventos como ayuda para construir el DE**. Las trazas de eventos describen una secuencia lógica de eventos en el sistema. Podemos utilizarlas para construir el esqueleto del DE, empezando por la secuencia de operaciones más habitual en el sistema. Luego debemos ir ampliando para que contemple otras secuencias cada vez menos frecuentes, no olvidándonos de las secuencias de error.

- **Para definir estados, sólo hay que tener en cuenta los atributos que influyen en el comportamiento**. Los estados se definen a partir de subconjuntos de valores de los atributos. Sólo consideraremos los atributos que influyen en el comportamiento (los nombres no) y agruparemos los valores de estos atributos en conjuntos de forma que los objetos que estén en un estado presenten un comportamiento común.

- **Decidir la granularidad de estados y eventos según las necesidades del modelo**. Los estados se asocian a actividades, que normalmente pueden ser descompuestas en fases, con varios niveles de granularidad. Según esto podemos hacer también los DEs con distinto tamaño de grano. Escogeremos el más adecuado según las necesidades de la aplicación y el grado de detalle que le queramos dar al modelo.

- **Distinguir entre eventos y guardas**. Los eventos son mensajes que utilizan los objetos para comunicarse. En los DEs los eventos que recibe un objeto aparecen en las transiciones e indican que transición se dispara cuando se recibe un evento. Las guardas son condiciones que se establecen sobre valores de los atributos del propio objeto del que estamos haciendo el DE.

- **Distinguir entre actividades y acciones**. Las actividades son operaciones que tienen una duración, por lo que se representan en los estados. Las acciones son operaciones que (idealmente) se realizan instantáneamente, y se representan en las transiciones.

- **Utilizar diagramas anidados cuando una transición se aplique a varios estados**. Esto nos permite *reutilizar* las transiciones aplicando los principios de generalización y especialización y la regla de herencia a los DEs.

En los diagramas anidados hay que tener en cuenta la consistencia, al menos de las transiciones de salida. Las transiciones de salida del estado padre deben figurar en alguno de los estados del DE hijo. Las de entrada no hace falta pues suponemos que el DE hijo comienza siempre por el estado inicial.

- **Realizar un DE compuesto si:**

  - **Existen relaciones de composición entre los objetos**. El DE del objeto compuesto estará formado por el conjunto de los DEs de los componentes. El estado de un objeto compuesto es una tupla, cuyos elementos son los estados de los objetos componentes.

  - **Existe concurrencia interna en el objeto**. Si el objeto es concurrente (si puede o debe realizar varias acciones al mismo tiempo) podemos dividir los estados en subestados concurrentes, en cada uno de ellos se realiza una actividad.

  - **Podemos dividir el comportamiento en facetas independientes**. Si el comportamiento presenta facetas que dependen de conjuntos disjuntos de atributos, podemos modelar este comportamiento mediante varios DEs, uno para cada conjunto. Estos DEs serán mucho más sencillos puesto que sólo van a manejar un subconjunto de los eventos, sin que tengamos que preocuparnos de definir transiciones para el resto.

  - **Existen relaciones de herencia entre los objetos**. Si el objeto hijo define nuevos atributos, haremos un DE para definir el comportamiento según los valores de estos atributos nuevos. Si el objeto hijo descompone uno o más estados del padre, tendremos un DE anidado.

- **En caso de herencia de entre clases, el DE de la subclase no puede añadir estados o transiciones al DE padre**. Como un objeto puede ser visto tanto como una instancia de la clase padre como una instancia de la clase hijo, debe existir una correspondencia entre los estados y transiciones de los DEs de ambas. Si añadimos un estado en el DE del hijo ya no existe esa correspondencia, y un objeto en ese nuevo estado, considerado como una instancia de la clase padre, estaría un estado indeterminado.

- **Comprobar la consistencia de los eventos que se generan en un DE y se utilizan en otros**. Los eventos que se utilizan para disparar las transiciones de un DE deben ser generados por algún objeto. Los eventos que genera un objeto deben utilizarse en algún otro DE.

5.6. Modelo funcional.

El modelo funcional describe las computaciones que se realizan en un sistema, mostrando cómo se derivan los valores de salida a partir de los de entrada. El modelo funcional muestra cómo se realizan las operaciones del modelo de datos y las actividades acciones de los DEs.

La existencia de un modelo funcional basado en el uso de DFDs distingue OMT de otras metodologías de desarrollo orientado a objetos. Sin embargo, no podemos considerar esto como algo positivo de OMT puesto que no es fácil la adecuación del modelo de procesos del análisis estructurado al tipo de código existente en un sistema orientado a objetos. Por este motivo apenas se usa y, de hecho, escritos recientes de Rumbaugh descartan el uso de DFDs para representar el modelo funcional del sistema, optando, en su lugar, por incluir casos de uso, diagramas de interacción de objetos y especificaciones de operaciones similares a las PSPECs del análisis estructurado.

La especificación de una operación puede realizarse mediante precondiciones y postcondiciones. Una precondición indica suposiciones sobre el estado del sistema al comienzo de la operación. Una postcondición describe el estado del sistema una vez realizada la operación. Las precondiciones y postcondiciones pueden expresarse mediante algún lenguaje formal, aunque en la mayoría de los casos basta con utilizar lenguaje natural, siempre que el significado quede claro.

Clase: Array[T]

Operación: ordenar

Función: reorganiza los elementos del array, de forma que queden en orden creciente.

Entradas: ninguna.

Salidas: ninguna.

Objetos modificados: el propio objeto.

Precondiciones

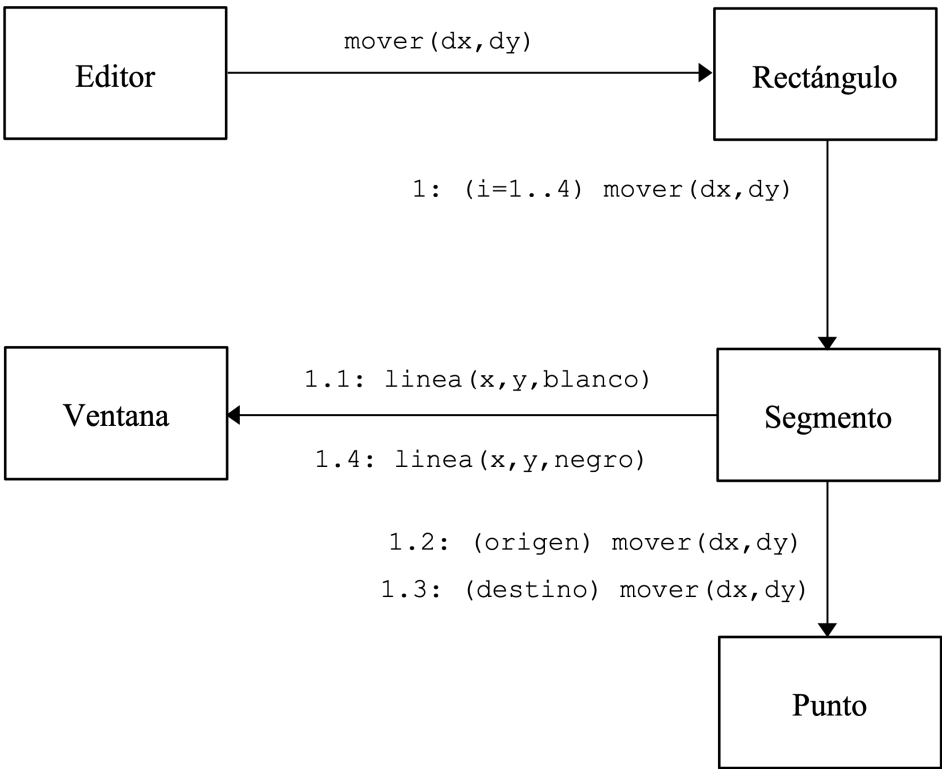
- Todos los elementos han de ser de tipo T o de un subtipo de T.
- El tipo T tiene una operacion *comparar*, que compara dos elementos y devuelve los valores LT, EQ, GT si el primero es menor, igual o mayor, respectivamente que el segundo. La operación *comparar* define una ordenación total de los valores de T.
- Se permite que el array tenga valores duplicados.

Postcondiciones

- Los elementos del array quedan ordenados de acuerdo a la función de comparación.
- El array contiene exactamente el mismo número de ocurrencias de cada valor de T que antes de ejecutar la operación.

En general, la implementación de las operaciones puede derivarse del modelo de comportamiento durante el diseño del sistema. Sin embargo, durante el análisis, será necesario describir, al menos, las operaciones de aquéllas clases para las que no se ha realizado diagrama de estados.

La implementación de una operación puede invocar otras operaciones, pero esto es un aspecto de diseño y no de especificación. Para mostrar la colaboración entre distintos objetos a la hora de realizar una operación determinada podemos realizar un *diagrama de interacción de objetos*.



5.7. Relación entre los modelos.

El modelo de objetos describe la estructura de datos sobre la que operan los modelos funcional y dinámico. Las operaciones en el modelo de objetos se corresponden con las acciones y actividades del modelo dinámico y con los procesos del modelo funcional.

El modelo dinámico describe el control de los objetos. El DE describe el comportamiento de los objetos de una clase, mostrando secuencias válidas de cambios de comportamiento en las clases del modelo de objetos. Los estados son clases de equivalencia de los valores de atributos y enlaces del objeto, que normalmente se asocian a la ejecución de una determinada operación en el objeto. Los eventos aparecen también como operaciones en el modelo de objetos.

Un subestado refina los valores de atributos y enlaces que puede tener un objeto. Una jerarquía de estados de un objeto es equivalente a una jerarquía de restricciones sobre el objeto. Como los LOO no suelen permitir restricciones de valores sobre el tipo de los atributos heredados o los parámetros de las operaciones (no se puede cambiar la signatura, ni siquiera restringiéndola), el modelo dinámico es el sitio adecuado para representar estas restricciones.

En la mayoría de los LOO, las instancias no pueden cambiar la clase a la que pertenecen a lo largo de su vida, pero sí que pueden cambiar de estado. Por tanto, para modelar objetos que van a estar sujetos a restricciones cambiantes, modelaremos estas restricciones como estados en lugar de como clases.

Las jerarquías de objetos y de eventos son totalmente independientes. Los eventos sirven para intercambiar información entre objetos de clases distintas. Las transiciones se suelen representar mediante operaciones, cuyo nombre es el evento que dispara la transición.

El modelo funcional describe funciones invocadas por operaciones en el modelo de objetos o acciones y actividades en el modelo dinámico. Las funciones operan sobre los valores de datos especificados en el modelo de objetos. El modelo de objetos muestra las entidades que realizan o padecen las funciones. El modelo dinámico muestra la secuencia en la que se realizan las funciones.

En resumen:

- **Relaciones con el modelo de objetos.**

  El modelo funcional muestra las operaciones que se realizan en cada clase y los argumentos de estas operaciones. El modelo dinámico muestra los estados de cada objeto y las operaciones que éstos realizan al recibir eventos y cambiar de estado.

- **Relaciones con el modelo dinámico.**

  El modelo funcional muestra las definiciones de las acciones y actividades del modelo dinámico. El modelo de objetos muestra los objetos que sufren o realizan las acciones y actividades del modelo dinámico.

- **Relaciones con el modelo funcional.**

  El modelo de objetos muestra las entidades que realizan o padecen las funciones del modelo funcional. El modelo dinámico muestra la secuencia en que se realizan las funciones del modelo funcional.

5.8. Método de análisis.

El objetivo del AOO es modelar el mundo real, de forma que pueda ser entendido. Es necesario abstraer las características importantes del problema en primer lugar, dejando los detalles para más adelante. Un buen modelo de análisis debe indicar lo que hay que hacer, sin restringir cómo hay que hacerlo, evitando tomar anticipadamente decisiones de implementación.

El modelo de análisis se compone del modelo de objetos, que representa la estructura estática de la información, el modelo dinámico, que indica la secuencia de eventos, y el modelo funcional, que muestra las transformaciones de datos. No todos ellos tienen la misma importancia, y la necesidad de desarrollar cada uno de ellos depende del dominio del problema. Habrá casos en que no sea necesario realizar el modelo dinámico o el funcional. El modelo de objetos siempre es necesario si vamos a hacer AOO.

El análisis no es una actividad secuencial, sino que los modelos se construyen de forma iterativa, añadiendo nuevas características o modificando el modelo según se refinan los requisitos y también a partir del conocimiento del dominio de aplicación obtenido al construir cada uno de los modelos.

Estrictamente, podemos dividir la etapa de análisis dentro de la metodología OMT en tres fases: modelo de objetos, modelo dinámico y modelo funcional. Sin embargo, si consideramos la fase inicial de conceptualización, debemos incluir también los casos de uso.

5.8.1. Casos de uso.

Los casos de uso describen las distintas formas de utilizar el sistema, visto desde su exterior, es decir, desde el punto de vista del usuario. Para realizar los diagramas de casos de uso se puede proceder como se describe a continuación:

- **Establecer los límites del sistema**. Hay que determinar qué objetos forman parte del sistema, qué objetos interactúan con él y cuáles no. Los casos de uso consideran el sistema como una caja negra, es decir, no entran a describir su estructura interna.

- **Determinar los actores que interactúan con el sistema**. Un actor es un rol o papel que juega un objeto externo en su relación con el sistema. Para determinar los actores debemos observar los objetos físicos que interactúan con el sistema, que en muchos casos juegan múltiples roles. Por ejemplo, una misma persona puede ser Usuario, Operador y Administrador de un cierto sistema. Cada rol es un actor diferente.

- Para cada actor, **determinar las diferentes formas en las que usa el** sistema. Por cada una de esas formas tendremos un caso de uso.

  No debería haber demasiados casos de uso en un sistema. Entre 5 y 10 casos bastan normalmente para mostrar los usos principales del sistema. Si no es así, es necesario utilizar un menor nivel de detalle. En cada caso de uso se debe escoger un nivel de detalle similar.

- **Identificar el evento inicial** que comienza el caso de uso.

- **Determinar la condición de terminación** que finaliza el caso de uso. A menudo un caso de uso puede ser descrito con diferentes niveles de detalle.

- **Describir la situación típica** en la que ocurre el caso de uso. Si hay variaciones o excepciones, es necesario describirlas también. Basta con utilizar lenguaje natural, un caso de uso no necesita ser demasiado formal.

5.8.2. Modelo de objetos.

Empezaremos a modelar el sistema realizando su modelo de objetos. Este modelo muestra la estructura estática de los datos del mundo real y las relaciones entre estos datos. El modelo de objetos precede normalmente al dinámico y al funcional porque normalmente está mejor definido en la especificación preliminar, es menos dependiente de detalles de la aplicación, es más estable respecto a la evolución de la solución y es más fácil de entender que el resto.

Los pasos a seguir son los siguientes:

Identificar objetos y clases.

Durante el proceso de desarrollo aparecen tres categorías de objetos: objetos del dominio, objetos de aplicación y objetos internos.

Los **objetos del dominio** son significativos desde el punto de vista del dominio del problema. Existen de forma independiente a la aplicación y tienen sentido para los expertos del dominio.

Los **objetos de aplicación** representan aspectos computacionales de la aplicación que son visibles para los usuarios. No existen en el espacio del problema, solo tienen sentido en el contexto de la aplicación que vamos a desarrollar para solucionarlo. Sin embargo, no dependen exclusivamente de decisiones de diseño, puesto que son visibles al usuario y no pueden ser

cambiados sin alterar la especificación de la aplicación. No pueden obtenerse analizando el dominio de la aplicación, pero sí pueden ser reutilizados de aplicaciones anteriores, incluso aunque sean de diferente dominio. Los objetos de aplicación incluyen controladores, dispositivos e interfaces.

Los **objetos internos** son componentes de la aplicación que resultan invisibles para el usuario. Su existencia se deriva de decisiones de diseño para implementar el sistema. No deben aparecer durante el análisis. Una parte importante del diseño consiste en añadir objetos internos para hacer factible la implementación del sistema.

Por tanto, en el modelo de objetos figurarán tanto objetos del dominio como objetos de aplicación. Su construcción puede ser realizada en dos fases:

En una primera fase podemos construir un modelo que represente los objetos del dominio del problema y las relaciones que existen entre ellos. Este modelo equivale a lo que se llama *modelo esencial* en la terminología del análisis estructurado.

En una segunda fase podemos construir un *modelo de aplicación*, completando el modelo anterior con objetos de aplicación. Para esto jugarán un papel muy importante los casos de uso desarrollados durante la conceptualización del problema.

Los objetos del modelo pueden ser tanto entidades físicas (como personas, casas y máquinas) como conceptos (como órdenes de compra, reservas de asientos o trayectorias). En cualquier caso, todas las clases deben ser significativas en el dominio de aplicación. Hay que evitar definir clases que se refieren a necesidades de implementación como listas, subrutinas o el reloj del sistema. No todas las clases figuran explícitamente en la especificación preliminar, algunas están implícitas en el dominio de aplicación o son de conocimiento general.

Podemos empezar haciendo una lista de clases candidatas a partir de la especificación preliminar. Normalmente las clases se corresponden con nombres en este documento. No hay que preocuparse aún de establecer relaciones de generalización/especialización entre las clases. Esto se hace para reutilizar código y estas relaciones aparecen más claramente cuando se definan atributos y operaciones.

Una vez que tenemos una lista de clases candidatas hay que proceder a revisarla, eliminando las incorrectas, siguiendo los siguientes criterios:

- **Clases redundantes**. Si dos clases representan la misma información nos quedaremos con la que tenga un nombre más significativa, eliminando la otra. (p. ej. Cliente y Usuario).

- **Clases irrelevantes**. Podemos haber incluido clases que sean importantes en el dominio de aplicación, pero que sean irrelevantes en el sistema que pretendemos implementar. Esto depende mucho del problema concreto.

- **Clases demasiado generales**. Las clases deben ser específicas. Deben tener unos límites claros y unos atributos y un comportamiento comunes para todas las instancias. Debemos eliminar las clases demasiado generales, normalmente creando otras más específicas. Las clases genéricas se incluirán más adelante si es necesario, al observar atributos u operaciones comunes a varias clases.

- **Atributos**. Los nombres que describen propiedades de los objetos son atributos, no clases. Una de las características de un objeto es su identidad (aún cuando los valores de los atributos sean comunes), y otra muy común es su existencia independiente. Los atributos de un objeto no presentan estas dos propiedades. Si la existencia independiente de un atributo es importante, hay que modelarlo como una clase. (P. ej. podemos considerar el Despacho, como un atributo de la clase Empleado, pero esto nos impide hacer una reasignación de despachos, por lo que, si queremos implementar esta operación, deberemos considerar los despachos como objetos en lugar de como atributos).

- **Operaciones**. Una clase no puede ser una operación que se aplique sobre los objetos sin tener independencia por sí misma (p. ej. en nuestro modelo de la línea telefónica la Llamada es una operación, no una clase). Sin embargo, esto depende del dominio de aplicación: en una aplicación de facturación telefónica, la Llamada será una clase que contiene atributos tales como Origen, Destino, Fecha, Hora y Número de Pasos.

- **Roles**. El nombre de una clase debe depender de su naturaleza intrínseca y no del papel que juegue en el sistema. Podemos agrupar varias clases en una, si intrínsecamente son lo mismo, y luego distinguir estos roles en las asociaciones. (P. ej. La clase *Persona* puede jugar varios papeles (*Propietario*, *Conductor*, etc.) en relación con la clase *Vehículo*. En otros casos, una misma entidad física puede modelarse mediante varias clases distintas si los atributos y el comportamiento (y especialmente las instancias) son distintos dependiendo del papel que juegue en el sistema.

- **Objetos internos**. No debemos incluir en el la fase de análisis clases que no se corresponden con el dominio de aplicación sino que tienen relación con la implementación del sistema.

Una vez identificadas las clases debemos empezar a preparar el diccionario de datos del sistema. Cada clase figurará como una entrada en el diccionario donde se define brevemente su

significado y las funciones que realiza. El diccionario de datos también contiene entradas para las asociaciones, las operaciones y los atributos, que deben ser definidos según las vamos incorporando al modelo.

Identificar asociaciones (incluyendo las de composición) entre los objetos.

Cualquier relación entre dos o más clases debe ser modelada como una asociación. No hay que incluir en las clases punteros o atributos que se refieran a objetos de otras clases. Todas estas relaciones deben modelarse como asociaciones para que quede constancia de ellas en el modelo de objetos.

Las relaciones de composición también son asociaciones, simplemente tienen unas características particulares. Hay que distinguir entre unas y otras pero no merece la pena discutir mucho sobre ello.

Una vez identificadas las asociaciones candidatas, procederemos a revisar esta lista, eliminado algunas según los siguientes criterios:

- Asociaciones irrelevantes y de implementación. Hay que eliminar todas las asociaciones que, a pesar de que existan en el muno real, no sean relevantes para el sistema. Lo mismo sucede con las asociaciones cuya existencia se base en la implementación de la solución. Las asociaciones del modelo de objetos deben tener existencia real y ser relevantes para la solución del problema.

- Acciones. Las asociaciones deben describir relaciones estructurales entre las clases. Existen otros tipos de relaciones, (por ejemplo cuando un objeto recibe otro como parámetro en un mensaje que invoca una acción), que no figuran normalmente como relaciones en el modelo de objetos.

- Asociaciones no binarias. La mayor parte de las relaciones entre más de dos clases pueden ser expresadas mediante asociaciones binarias o con atributos (si uno de los términos no tiene existencia propia sino siempre ligada a la asociación), pero en algunos casos no podemos hacer esto sin perder información. Hasta el momento no se a mostrado como necesaria ninguna asociación entre cuatro o más clases.

- Asociaciones derivadas. Hay que eliminar las asociaciones que puedan ser expresadas a partir de otras (p. ej. la asociación Abuelo). La existencia de caminos múltiples entre dos clases suele indicar que existen asociaciones derivadas que pueden ser eliminadas, pero en algunos casos los caminos múltiples son necesarios.

Una vez revisada la lista, procederemos a completar el modelo de objetos, indicando para cada asociación que lo necesite:

- **Roles**. Los roles o papeles son muy útiles cuando hay asociaciones entre la misma clase o hay varias asociaciones entre un par de clases. El rol describe el papel que juega un objeto en la relación desde el punto de vista de la otra clase. (P. ej. Persona (empleado) trabaja para (contratante) Empresa.

- **Calificadores**. Normalmente los nombres sirven para identificar los objetos dentro de un contexto. De esta forma, un atributo puede servir para identificar entre las múltiples instancias conectadas al extremo M de una asociación, reduciendo así la multiplicidad de la relación.

- **Multiplicidad**. Es necesario especificar la multiplicidad, pero no hay que obsesionarse mucho pues cambia con frecuencia a lo largo del análisis. Una vez que el modelo esté completo hay que revisar las conexiones con multiplicidad 1 (que suelen ser opcionales) y las de multiplicidad M, para ver si realmente son necesarias.

- **Ordenación y otras restricciones.**

Identificar atributos y operaciones.

El siguiente paso es identificar los atributos y operaciones de cada clase y de los enlaces que los contengan. Los atributos describen propiedades de los objetos, tales como *peso*, *color* o *edad*, pero no son objetos. Cualquier relación entre objetos debe ser modelada como una asociación, no como un atributo de los objetos relacionados. Tampoco hay que incluir como atributos de los objetos los calificadores de las asociaciones ni los atributos de los enlaces.

Con frecuencia los atributos no figuran en la especificación preliminar. Lo normal es partir de un conjunto básico de atributos para cada objeto e ir añadiendo otros según los vayamos necesitando.

Organizar y simplificar las clases mediante relaciones de generalización y especialización.

El paso siguiente es organizar las clases en jerarquías de forma que se puedan abstraer las características comunes. Esto puede hacerse en dos direcciones:

- Hacia arriba (**generalización**), abstrayendo propiedades (atributos y relaciones) comunes a un grupo de clases en una superclase que las generalice. Muchas relaciones de

generalización se corresponden con clasificaciones (taxonomías) de los objetos en el mundo real.

- Hacia abajo (**especialización**), refinando las clases existentes en subclases especializadas. Estas relaciones de especialización suelen aparecer en la especificación preliminar en forma de nombres adjetivados: lámpara incandescente, lampara fluorescente, etc.

Aunque la herencia facilita la reutilización de código, hay que evitar abusar demasiado de estas relaciones. Aplicaremos la especialización sólo cuando sea necesario, es decir, cuando la distinción entre dos clases especializadas sea significativa para el problema. Esto sucederá cuando ambas clases tengan atributos o operaciones distintos.

Verificar los caminos de acceso.

A continuación hay que comprobar los caminos de acceso en el modelo de objetos. Se trata de comprobar si todas las asociaciones necesarias están presentes en el diagrama, de forma que se pueda hacer un recorrido entre los objetos relacionados. También hay que comprobar la multiplicidad: para relaciones con multiplicidad M, ¿es necesario acceder a las instancias relacionadas de forma individualizada? ¿existe algún mecanismo para hacerlo?, ¿existe un camino para contestar todas las preguntas útiles acerca del problema.? (p. ej. ¿cuáles son las casas de una persona?, ¿a quién le ha comprado esta persona cada casa?, ¿cuándo se realizó la escritura?).

Refinar el modelo iterativamente.

Es muy difícil que el modelo de objetos nos salga bien a la primera. El desarrollo de software sigue normalmente un proceso iterativo: podemos encontrar errores o mejoras en nuestro modelo de objetos, no sólo mientras lo estamos haciendo, sino al hacer el modelo funcional, el dinámico o incluso en las fases de diseño o implementación. Nuestro objetivo es encontrar los errores lo antes posible, especialmente antes de entregar el software al cliente (pérdida de imagen) y antes de implementar los programas (pérdida de tiempo en codificación y pruebas). Cuanto antes encontremos los errores más sencillo y baratos será corregirlos. Por ello hay que tener especial cuidado en la fase de análisis, revisando el modelo cuantas veces sea necesario, evitando especialmente la disparidad con los modelos dinámico y funcional.

5.8.3. Modelo dinámico.

El modelo dinámico muestra el comportamiento de los objetos del modelo de objetos. Podemos empezar estableciendo una lista de los eventos que afectan al sistema, y establecer una o

varias trazas de eventos que muestren secuencias normales de estos eventos. Luego estableceremos las secuencias de eventos permitidas para cada clase de objetos, a partir de las cuales podemos realizar los DEs.

- **Establecer una lista de posibles eventos**. Para ello partimos, como siempre, de la especificación preliminar. Hay que identificar también el objeto que emite el evento (algunos eventos provienen del entorno del sistema), el que lo recibe, y los parámetros que pudiese tener. Podemos agrupar los eventos en clases, especialmente cuando tienen parámetros o para poder utilizarlos con diferentes grados de abstracción.

- **Eliminar de la lista de eventos las operaciones que no afecten al estado de un objeto**. Los objetos reciben mensajes indicando que deben realizar alguna de las operaciones que tienen definidas, pero muchas de estas operaciones no cambiarán el estado del objeto (el objeto se comportará igual después de realizar la operación), por tanto no consideraremos estas órdenes o peticiones como eventos.

- **Realizar varias trazas de eventos**. En cada traza mostraremos una secuencia lógica de eventos junto con los objetos que los emiten y reciben. Podemos empezar haciendo trazas de los casos más habituales de funcionamiento del sistema, luego haremos trazas para situaciones menos corrientes pero igualmente válidas y acabaremos haciendo trazas para determinar el comportamiento del sistema ante situaciones erróneas. El control de errores suele ser la parte más difícil del modelo dinámico de una sistema interactivo. Hay que prever todas las situaciones posibles (incluyendo *timeouts* para las comunicaciones o las peticiones de datos al usuario) y permitir la corrección de cualquier entrada de datos, definiendo claramente mecanismos de vuelta atrás.

- **Construir un DE para cada clase de objetos que presente estados distintos**. Sólo consideraremos los objetos que puedan presentar varios estados, es decir, que se comporten de manera distinta según valores distintos de sus atributos. Para cada una de estas clases iremos tomando las trazas donde aparece, empezando de las más habituales a las más específicas. Cada traza se corresponderá con una secuencia de transiciones en el DE. Cada vez que reciba un evento, un objeto cambiará (posiblemente) de estado, estableciendo un valor nuevo para alguno de sus atributos. Revisaremos cada DE para ver si esta completo (si se han considerado todas las transiciones) y para determinar si los estados finales son efectivamente tales.

- **Verificar la consistencia de los eventos entre los diferentes DEs**. Hay que verificar la completitud y consistencia de los eventos a nivel de todo el sistema. Los eventos que emite un objeto deben ser recibidos por algún otro. Podemos completar el

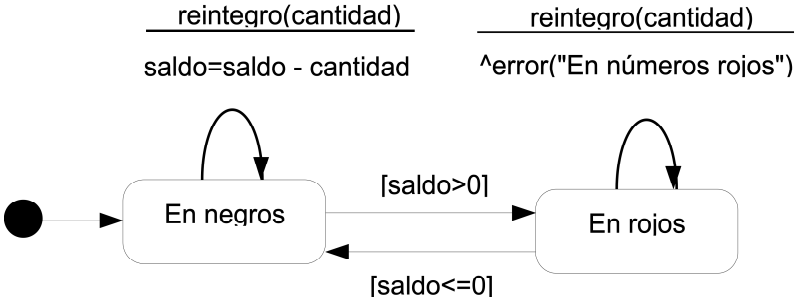
modelo dinámico dibujando uno o varios *diagramas de interacción de objetos,* que nos muestren las relaciones entre las distintas clases del sistema basadas en intercambio de eventos.

5.8.4. Modelo funcional.

El modelo funcional muestra cómo se calculan valores, independientemente de cuándo se realizan esos cálculos o de la estructura de los objetos que almacenan esos valores. Lo que muestra el modelo funcional es las relaciones de dependencia de datos y dependencia funcional.

Normalmente, la aplicación del paradigma orientado a objetos lleva a una mayor fragmentación del código que los métodos estructurados: los métodos tienen una implementación breve, basada normalmente en la llamada a otros métodos de objetos relacionados. Por este motivo, podemos describir la mayoría de los métodos como primitivas de proceso, o mediante *diagramas de interacción*, siendo raro el caso en el sea útil describir una determinada operación mediante un DFD.

El DE de cada clase contiene la mayor parte de la información necesaria para realizar el modelo funcional. Las acciones y actividades asociadas al disparo de una transición se corresponden con la especificación de la operación que figuran como evento en dicha transición. En caso de que un mismo evento figure en varias transiciones, la especificación contendrá las estructuras de control que permitan distinguir entre los diferentes estados.




```
reintegro(cantidad)
if saldo > 0
  then
    saldo = saldo - cantidad
else
    error("En números rojos");
```

Si la especificación de una operación contiene invocaciones de operaciones (mediante envío de eventos) a otras clases, puede especificarse mediante diagramas de interacción. Por el contrario, si se trata de una operación exclusivamente interna, o si no se han realizado DE de la clase, la operación se puede especificar mediante una primitiva de proceso. Esta descripción puede hacerse en lenguaje natural, pseudocódigo, por medio de ecuaciones matemáticas o de tablas. La desventaja de hacerlo en lenguaje natural es que la ambigüedad es mayor, pudiendo presentarse problemas a la hora de implementar y que es más difícil comprobar su consistencia. Aunque utilicemos un algoritmo (pseudocódigo) para describir un proceso, el objetivo es especificar qué hace el proceso

y no cómo lo hace. La elección definitiva del algoritmo depende de consideraciones adicionales que se establecen en las fases de diseño y de implementación.

La especificación de las operaciones debe incluir también cualquier restricción que se deba establecer sobre los datos del sistema, (por ejemplo, precondiciones y postcondiciones de los procesos), así como criterios de optimización que deban tenerse en cuenta en el diseño y la optimización.

5.8.5. Modelo de aplicación.

Una vez realizado el modelo del dominio, donde nos habremos centrado fundamentalmente en los objetos del dominio de aplicación, podemos completarlo, convirtiéndolo en un modelo de aplicación. Para ello nos serán de gran utilidad los casos de uso desarrollados junto con la especificación inicial.

En primer lugar hemos de determinar los límites del sistema, identificando lo que es parte del mismo y lo que son actores externos. Es posible que algunos objetos que aparecen en el modelo del dominio no formen realmente parte de la aplicación. Para determinarlo, debemos observar si es necesario guardar información sobre estos objetos o si vamos a implementar las operaciones que aparecen en ellos.

Por otro lado, el examen de los casos de uso nos permitirá determinar cuál es el acceso necesario para cada clase del dominio. Se pueden construir una o más *vistas* de cada clase del dominio para proporcionar estas distintas formas de acceso. Estas vistas deben añadirse al modelo de objetos, definiendo la traducción entre objetos del dominio y *vistas*.

Debemos identificar cuales son los eventos que se intercambian entre los actores y el sistema. Los periodos de tiempo entre eventos se definirán como estados. Incluiremos objetos controladores para controlar la secuencia de eventos en cada vista. Esto nos permitirá separar la relación estática entre objetos del dominio y vistas de la dinámica de las vistas.

A partir de los casos de uso también podremos determinar cuáles serán los comandos del sistema. Estos comandos serán peticiones que realizan los actores para que el sistema realice una determinada acción. Los comandos del sistema deben añadirse al modelo funcional como operaciones. Si no es posible asignar estas operaciones a ninguna de las clases existentes, la asignaremos a un controlador.

Por último debemos determinar las interfaces externas del sistema, incluyendo las interfaces con dispositivos externos, otros sistemas, etc. Es conveniente rodear los objetos de la aplicación de

objetos interfaz que los aíslen y eviten dependencias de la aplicación respecto a estos sistemas externos.

5.8.6. Refinar el modelo.

Una vez que hemos desarrollado el modelo del sistema, nos quedan aún una serie de cosas por hacer:

Completar la lista de operaciones.

La primera de ellas es **completar la lista de operaciones**, dado que normalmente no es posible hacerlo al desarrollar el modelo de objetos, sino que van apareciendo a lo largo del análisis. Podemos clasificar las operaciones de los objetos en los siguientes grupos:

- **Operaciones implícitas sobre objetos**. Normalmente, cualquier sistema debe ser capaz de crear, destruir y copiar instancias de las clases que define. Algunas de estas operaciones están, directa o indirectamente, descritas en la especificación preliminar, pero la mayor parte de las veces están implícitas. Los LOO suelen incluir primitivas de creación, destrucción y copia de las instancias de una clase.

- **Operaciones implícitas sobre atributos**. Existe otro grupo de operaciones implícitas: las encargadas de consultar o asignar el valor de cada uno de los atributos de un objeto. Así mismo, deben existir operaciones que recorran los enlaces que una instancia tenga con otras. En la descripciones de primitivas de proceso podemos utilizar la notación siguiente, para referirnos a estas operaciones:

`clase.atributo`	operación de consulta que devuelve el valor del atributo.
`clase.atributo := valor`	operación de asignación.
`clase.asociación`	atributo que apunta al objeto asociado.
`clase.asociación[i]`	id. para asociaciones múltiples y ordenadas.
`clase.asociación[calificador]`	id. para asociaciones calificadas.

- **Operaciones invocadas por eventos**. La recepción de un evento trae consigo la ejecución de una serie de operaciones (acciones y actividades) en el objeto que recibe el

mensaje. Es habitual incluir en la clase receptora operaciones con mismo nombre que el evento que las desencadena, en las que se incluyen estas acciones y actividades.

- **Operaciones invocadas por acciones y actividades**. Por el contrario, en algunos casos puede ser conveniente representar las propias acciones y/o actividades como operaciones de las clases. Se tratará en estos casos de operaciones auxiliares que se invocan a la recepción de los eventos correspondientes.

Una vez que hemos completado la lista de operaciones de cada clase, debemos comprobar estas listas con vistas a simplificar operaciones redundantes o a establecer relaciones de generalización entre clases que tengan una serie de operaciones comunes.

Comprobar la consistencia entre los modelos.

El proceso de análisis no es una actividad secuencial, sino que según vamos desarrollando un modelo surgen modificaciones de los anteriores. Una vez que tenemos los tres modelos hay que comprobar la consistencia, para ver si alguna de estas modificaciones se nos ha pasado por alto. Para comprobar la consistencia nos basaremos en las relaciones entre los modelos de las que ya hemos hablado.

Aunque el primer modelo que realicemos sea consistente, rara vez será correcto. Demos revisarlo para ver si se nos ha olvidado algo de lo establecido en la especificación preliminar o si notamos algo extraño.

Comprobar el modelo con el cliente.

Una vez que nosotros hemos dado el visto bueno al análisis, el cliente debe hacer lo mismo. En primer lugar debemos mostrarle los resultados del análisis para ver si son acordes con sus necesidades, o si hemos cometido algún error o se nos ha pasado por alto algún detalle. También debemos discutir con el las modificaciones que se hayan realizado con respecto a lo establecido en la especificación preliminar, para ver si las suposiciones que hemos hecho son correctas.

Deficiencias del análisis estructurado.

- **Descomposición funcional**. Requiere traducir el sistema a una serie de funciones y subfunciones.
- **Flujo de datos**. Es un modelo informático, no nuestra forma habitual de pensar.
- **Modelo de datos**. La relación con el modelo de procesos es débil. Pueden dar visiones muy distintas del mismo problema.

Ventajas del análisis orientado a objetos.

- **Dominio del problema**. Representa el sistema en términos del mundo real, en vez de en términos informáticos.
- **Comunicación**. Usa conceptos más simples. La comunicación con el cliente es más fácil.
- **Consistencia**. Reduce la distancia entre el modelo de procesos y el de datos.
- **Expresión de características comunes**. Evita la duplicación de características comunes a varios elementos.
- **Resistencia al cambio**. Al ser más próximo al sistema real es más estable frente a cambios en los requisitos.
- **Reutilización**. Favorece la reutilización, tanto interna como externa.

Características del enfoque orientado a objetos.

Identidad.

- Los elementos del dominio del problema se organizan en entidades discretas y distinguibles llamadas **objetos**.
- Los objetos encapsulan **atributos** (datos) y **operaciones**.
- Cada objeto tiene **identidad propia**, aunque los valores de sus atributos coincidan con los de otro.

Clasificación.

- Los objetos con propiedades comunes se agrupan en **clases**.
- Las clases son **abstracciones** de características comunes a una serie de objetos.
- Las clases son **arbitrarias**: dependen del dominio del problema.
- Cada uno de los objetos agrupados en una clase se llama **instancia**.
- Las instancias de una clase comparten atributos, operaciones y comportamiento pero tienen valores distintos en los atributos.

Polimorfismo.

- Una misma operación puede realizarse de formas distintas en clases distintas. La **semántica** es común pero la **implementación** varía en cada clase.
- La implementación de una operación en una clase se denomina **método**.

Herencia.

- Es una **relación jerárquica entre clases** con características comunes.
- La **clase padre** define características comunes a todas las hijas.
- Cada **clase hija** hereda las características del padre y las amplia o refina.

Elementos del modelo de objetos

Instancias. Cada uno de los objetos individuales.

Clases. Abstracción de objetos con propiedades comunes.

- **Atributos.** Datos que caracterizan las instancias de una clase**.**
- **Operaciones.** Funciones que pueden realizar las instancias**.**

Relaciones. Se establecen entre clases.

- Asociación. Relación de uso en general.
  - **Multiplicidad.** Número de instancias que intervienen en la relación.
  - **Atributos.** Algunos atributos pueden depender de la asociación.
  - **Calificación.** Limita la multiplicidad de las asociaciones.
  - **Roles.** Indican los papeles de las clases en las relaciones.
  - **Restricciones y ordenación.**
- Composición. Relaciones todo/parte.
- Generalización. Relaciones padre/hijo.
  - **Redefinición**. Modificación de las propiedades heredadas.

Enlaces. Instancias de una relación. Relaciona instancias.

Modelo de objetos: consejos prácticos.

- No lanzarse a dibujar clases y asociaciones sin sentido.
- Intentar que el modelo sea simple, evitando complicaciones innecesarias.
- Los nombres de objetos, asociaciones, atributos y operaciones deben ser significativos.
- No incluir en los objetos punteros a otros objetos.
- Utilizar, si es posible, asociaciones binarias.
- Dejar la definición de la multiplicidad para cuando se tenga un mejor conocimiento del problema.
- No incluir los atributos de las asociaciones en las clases.
- Utilizar preferentemente asociaciones cualificadas en vez de ternarias o con atributos.
- Evitar las jerarquías de composición o generalización de muchos niveles.
- Revisar el modelo hasta que sea satisfactorio.
- Documentar el modelo.
- Utilizar sólo los elementos necesarios.

Modelo dinámico.

- **Estado de un objeto**. Conjunto de valores de los atributos y enlaces que tiene un objeto en un momento determinado.

- **Diagrama de transición de estados**. Diagrama que muestra la secuencia de estados, eventos y acciones de un objeto.

- **Modelo dinámico de un sistema**. Conjunto de DEs de las clase de objetos del sistema.

- **Eventos**. Señales mediante las que se comunican los objetos. Pueden ser señales lógicas o contener atributos.

- **Estado**. Abstracción de los estados de un objeto en el que éste presenta determinado comportamiento. Engloba un conjunto de valores de los atributos y enlaces (un conjunto de estados del objeto).

- **Trazas de eventos**. Diagrama que muestra la secuencia de envíos y recepciones de eventos entre varios objetos.

- **Guarda**. Condición de datos que debe cumplir un objeto para que se dispare una transición al recibir un determinado evento. (*Notación*: [Guarda]).

- **Actividad**. Operación que necesita un tiempo para ejecutarse. Se representa en los estados. (*Notación*: Hacer: Actividad)

- **Acción**. Operación que se realiza (idealmente) de forma instantánea. Se representa en las transiciones. (*Notación*: Evento [Guarda] / Acción).

Modelo dinámico: consejos prácticos.

- No todas las clases necesitan un DE.
- Utilizar trazas de eventos como ayuda para construir el DE.
- Para definir estados, sólo hay que tener en cuenta los atributos que influyen en el comportamiento.
- Decidir la granularidad de estados y eventos según las necesidades del modelo
- Distinguir entre eventos y guardas.
- Distinguir entre actividades y acciones.
- Utilizar diagramas anidados cuando una transición se aplique a varios estados.
- Realizar un DE compuesto si:

o Existen relaciones de composición entre los objetos.

o Existe concurrencia interna en el objeto.

o Podemos dividir el comportamiento en facetas independientes.

o Existen relaciones de herencia entre los objetos.

- En caso de herencia de entre clases, el DE de la subclase **no puede añadir estados o transiciones al DE padre**.
- Comprobar la consistencia de los eventos que se generan en un DE y se utilizan en otros.

Modelo de objetos I

Pasos a seguir.

- Identificar objetos y clases.
- Preparar un diccionario de datos.
- Identificar asociaciones (incluyendo las de composición) entre los objetos.
- Identificar atributos y operaciones.
- Organizar y simplificar las clases mediante relaciones de generalización y especialización.
- Verificar los caminos de acceso.
- Refinar el modelo iterativamente.

Eliminar de la lista de clases candidatas:

- Clases redundantes.
- Clases irrelevantes.
- Clases demasiado generales.
- Atributos.
- Operaciones.
- Roles.
- Construcciones de implementación.

Modelo de objetos II

Eliminar de la lista de asociaciones candidatas:

- Asociaciones irrelevantes y de implementación.
- Acciones.
- Asociaciones no binarias.
- Asociaciones derivadas.

Completar las asociaciones indicando:

- Roles.
- Calificadores.
- Multiplicidad.
- Ordenación.

Modelo dinámico. Pasos a seguir:

- Establecer una lista de posibles eventos.
- Eliminar de la lista de eventos las operaciones que no afecten al estado de un objeto.
- Realizar varias trazas de eventos.
- Construir un DE para cada clase de objetos que presente estados distintos, incorporando una a una las trazas de eventos.
- Verificar la consistencia de los eventos entre los diferentes DEs: construir un diagrama de flujo de eventos.

Modelo funcional. Pasos a seguir.

- Identificar datos de entrada y salida.
- Hacer DFDs para mostrar la dependencia funcional.
- Describir las primitivas de proceso.

Refinar los modelos.

Completar la lista de operaciones.

- Operaciones sobre objetos: creación, destrucción y copia.
- Operaciones sobre atributos: consulta, actualización acceso a los objetos relacionados

`clase.atributo`	devuelve el valor del atributo.
`clase.atributo := valor`	asigna valor al atributo.
`clase.asociación`	devuelve al objeto asociado.
`clase.asociación[i]`	id. para asociaciones múltiples.
`clase.asociación[calificador]`	id. para asociaciones calificadas.

- Operaciones invocadas por eventos.
- Operaciones invocadas por acciones y actividades.

Comprobar la consistencia entre los modelos.

Comprobar el modelo con el cliente.

Ejercicio 1.

Realizar el modelo de objetos de un sistema de ficheros, basándose en las siguientes clases identificadas:

- Sistema de ficheros.
- Fichero.
- Directorio.
- Nombre de fichero.
- Fichero ASCII.
- Fichero ejecutable.
- Fichero de Directorio.
- Disco.
- Drive.
- Pista.
- Sector.

Ejercicio 2.

El sistema de control de la mayoría de los electrodomésticos consta de los siguientes elementos:

- Un botón, que debe mantenerse pulsado para que funcione el electrodoméstico.
- Un motor, con dos contactos: START y RUN.
- Un sensor que detecta cuando está funcionando el motor.
- Un sensor de sobrecalentamiento.

Para arrancar el motor hay que aplicar corriente en los dos contactos (START y RUN) a la vez. Una vez que el motor esta en marcha, basta con mantener la corriente en el contacto RUN.

Si el motor se calienta en exceso (TEMP > TMAX), bien porque esté sobrecargado o bien porque no se consiga arrancar, se desconecta automáticamente, y no puede volver a conectarse hasta que se enfríe (TEMP < TMIN).

- Realizar el DE de un electrodoméstico como el descrito.
- Modificar el DE, generalizando algunos de sus estados de forma que se evite la repetición de transiciones comunes.
- Modificar el modelo del sistema, suponiendo que consta de dos interruptores: `On` y `Off`. Una vez pulsado On el sistema se pone en marcha hasta que se pulse Off. Si ambos están pulsados al mismo

tiempo, el motor debe estar parado. Si estando pulsados ambos, soltamos primero Off, el sistema debe ponerse en marcha.

Ejercicio 3.

Realizar el análisis de un editor de diagramas. Los diagramas pueden constar de varias hojas, cada una de ellas formada por cajas de tamaño variable conectadas mediante enlaces. Cada caja irá etiquetada con un nombre. Los enlaces son secuencias de segmentos rectos que conectan dos cajas. Cada segmento viene especificado por dos puntos. Dos segmentos consecutivos comparten un punto en común.

Las funciones que debe incorporar el editor son las habituales:

- Cargar o salvar el diagrama en un fichero.
- Añadir o borrar páginas del diagrama.
- Seleccionar o deseleccionar cualquiera de los elementos (un elemento activo se representará más brillante en la pantalla).
- Copiar los elementos seleccionados al buffer. Los enlaces entre cajas seleccionadas y no seleccionadas no se copian.
- Borrar los elementos seleccionados. Los enlaces entre cajas seleccionadas y no seleccionadas se borra.
- Vaciar el buffer.
- Copiar el contenido del buffer sobre alguna hoja del diagrama, para lo cual se indicará el desplazamiento que se debe aplicar a las posiciones relativas de los elementos del buffer.
- Crear cajas y enlaces.

Sólo vamos a realizar el análisis del núcleo del editor, no vamos a modelar el ratón, el teclado, ni la representación de los diagramas en la pantalla. Supondremos que los eventos se envían a los objetos del diagrama desde el entorno del sistema, solicitando la ejecución de alguna operación.

Tema 6. Métodos formales de especificación: algebraicos y orientados a modelos

6.1. Introducción.

6.2. Lenguajes de especificación.

6.3. Justificación de los métodos formales.

6.4. Limitaciones de los métodos formales.

6.5. Distintos métodos de especificación formal.

Especificación algebraica

6.6. Propiedades de las especificaciones algebraicas

6.7. El estilo constructivo

6.8 Especificaciones parametrizadas

Especificación orientada a modelos

6.9. La notación Z

6.10. Z orientado a objetos

6.11 Caso de estudio: especificación de una tabla de referencias cruzadas.

Bibliografía

Languages for Specification, Design and Prototyping

V. Berzins, en Modern Software Engineering. P.A. Ng, R.T. Yeh.

Van Nostrand Reinhold, 1990. pp. 83-118.

Formal Methods for Life-Critical Software.

R.W. Butler et al. AIAA Computing in Aerospace 9th Conference.

San Diego, 1993. pp. 319-329.

Estructuras de datos: especificación, diseño e implementación

X. Franch Gutiérrez

Edicions UPC, 1994.

Algebraic Specifications in Software Engineering: An Introduction

I. Horebeek. y J. Lewi.

Springer-Verlag, 1989

Ingeniería del Software: un enfoque práctico

R.S. Pressman. McGraw Hill. Madrid, 1993. 3ª Edición. Cap. 5, 6 y 9.

6.1. Introducción.

Como ya hemos visto, la especificación de requisitos es el documento que describe un sistema software que va a ser construido. Es un documento fundamental, puesto que por un lado representa el contrato acordado con el cliente, y por otra parte sirve de base en todas las etapas posteriores de ingeniería. El principal objetivo de la especificación es definir de forma clara y no ambigua la función y restricciones del software, de forma que se eviten problemas en las etapas de diseño y codificación.

La especificación del software es el resultado del proceso de análisis. El análisis se centra en los ámbitos de información, funcional y de comportamiento del problema en cuestión. Para comprender mejor lo que se requiere se divide el problema en partes y se desarrollan representaciones o modelos que muestren la esencia de los requisitos (modelo esencial). Si éste no refleja los requisitos del cliente, entonces, inevitablemente, el diseñador construirá un sistema incorrecto. Por otra parte, si la especificación es incompleta, ambigua o inconsistente, aunque haya sido aceptada por el cliente, el diseñador no podrá satisfacer los requisitos adecuadamente.

A partir de una especificación confusa, ambigua o contradictoria surgen continuamente dudas a la hora de implementar el software. Cada una de estas dudas plantea una serie de posibilidades y obliga a elegir entre varias alternativas durante el diseño y la implementación. Estas decisiones no se toman con un conocimiento completo del problema (las deben tomar diseñadores y programadores que nunca han hablado con el cliente), por lo que conducen a implementaciones que, con seguridad, no cumplirán las necesidades del cliente.

Por tanto, la forma de especificar un sistema tiene una gran influencia en la calidad de la solución implementada finalmente. Tradicionalmente los ingenieros de software han venido trabajando con especificaciones incompletas, inconsistentes o erróneas, lo que invariablemente lleva a la confusión y a la frustración en todas las etapas del ciclo de vida. Como consecuencia de esto, la calidad, la corrección y la completitud del software disminuyen.

Los métodos de análisis que hemos visto hasta ahora conducen a especificaciones de este tipo, normalmente redactadas en lenguaje natural o, en el mejor de los casos, en pseudocódigo. Para modelar y describir los requisitos del software se utiliza una combinación de diagramas, textos, tablas y notaciones sencillas.

La otra alternativa consiste en describir la el sistema utilizando un lenguaje de especificación formal, basado en la lógica o las matemáticas, con una sintaxis y una semántica formales. De esta forma se elimina la ambigüedad y sería posible demostrar la

completitud de la especificación e incluso la corrección de una implementación de esta especificación. Las continuas revisiones que recomiendan los métodos clásicos se sustituyen aquí por análisis y demostraciones matemáticas, con la ventaja de que, si conseguimos demostrar que un programa es correcto, nunca presentará un error, cosa que no es posible conseguir utilizando métodos informales.

La ingeniería se basa fundamentalmente en el uso de modelos matemáticos y en el cálculo para extraer conclusiones sobre la viabilidad o la utilidad de los diseños que se realizan. Esto contrasta fuertemente con la forma en la que se desarrollan los sistemas software, que se realizan mediante técnicas de andar por casa (ad hoc) y pruebas del producto una vez implementado.

Los métodos formales proporcionan al desarrollo de software las mismas ventajas que utilizan otras ramas de la ingeniería: el análisis matemático basado en modelos. Los métodos formales pueden ser utilizados para especificar el modelo de comportamiento de un sistema y para verificar formalmente que la especificación y la implementación satisfacen propiedades funcionales y de seguridad del sistema. En principio, el uso de técnicas formales puede producir implementaciones libres de errores. Sin embargo, esto requiere una verificación completa a lo largo de todo el proceso de desarrollo, desde la especificación de los requisitos hasta la implementación, lo que, en la práctica, se realiza muy raramente.

Según esto los métodos formales son la matemática aplicada de la ingeniería del software, y juega un papel en el desarrollo de software equivalente al que la dinámica de fluidos juega en el diseño aeronáutico, por ejemplo. Proporcionan un método para calcular, y por tanto para predecir, cuál va a ser el comportamiento de un sistema software con anterioridad a su implementación.

El uso de métodos formales está especialmente indicado en aquellos sistemas en los que la corrección, completitud o fiabilidad del software juegan un papel fundamental (protocolos de comunicación, software de control de una central nuclear, gestión del tráfico aéreo, etc.). Los lenguajes de especificación formal conducen a representaciones de los requisitos que pueden ser analizadas o verificadas formalmente.

Los inconvenientes de las especificaciones formales son que son más difíciles de usar y de entender, por lo que, si bien son más precisas, es discutible que sean más claras. Además se pierde la posibilidad de revisarlas con el cliente, y las demostraciones de corrección y completitud son tan complejas que, hasta ahora, sólo han podido aplicarse a

casos muy sencillos. Por estos motivos, el uso de especificaciones formales se ha venido reservando a aplicaciones muy concretas del software, en las que la seguridad y fiabilidad sean un componente fundamental.

Por último, el uso de métodos formales no debe entenderse como una opción de todo o nada. La aplicación de estos métodos sólo a las partes críticas de un sistema es una estrategia práctica y útil. Aunque una verificación formal completa de un sistema grande y complejo no es posible en la práctica, podemos aumentar la confianza en un sistema mediante el uso de métodos formales en componentes estratégicos del mismo.

6.2. Lenguajes de especificación.

Como hemos visto la especificación consiste en describir un sistema de forma que queden expresadas su funcionalidad, restricciones y rendimiento de la forma más clara y precisa posible. Para ello debemos utilizar un lenguaje, con lo que disponemos de varias alternativas, cada una de ellas con sus ventajas e inconvenientes.

6.2.1. Lenguaje natural.

La solución más intuitiva es utilizar el lenguaje natural, y esta es la opción que ha venido siendo utilizada tradicionalmente. Entre sus ventajas podemos citar que es fácil de usar y de entender: no debemos aprendernos ningún lenguaje nuevo y cualquiera puede leer la especificación y comentarla o criticarla. Entre los inconvenientes están la imprecisión y la ambigüedad. Aunque el análisis de requisitos se haya realizado correctamente, una especificación en lenguaje natural puede dar lugar a que la implementación final no cumpla estos requisitos. Además, debido a su propia facilidad de uso e imprecisión, las especificaciones suelen ocultar lagunas que sólo se pondrán de manifiesto a la hora de programar, es decir, al traducir la especificación a un lenguaje de programación. El uso de subconjuntos del lenguaje, como el llamado “inglés estructurado”, atenúa estas deficiencias pero sigue sin resolver problemas como la corrección, consistencia o completitud de la propia especificación o de los programas desarrollados a partir de ella.

6.2.2. Pseudocódigo o lenguajes procedimentales.

Los lenguajes de programación tienen una sintaxis no ambigua y una semántica más o menos bien definida. Con el fin de llenar el hueco entre la especificación y la implementación, podríamos utilizar estos mismos lenguajes, o un pseudocódigo basado en ellos, para especificar sistemas. De esta forma, al aproximar la especificación a la implementación se reducen los errores de codificación.

Otra ventaja es que, al utilizar un lenguaje de programación, es mucho más fácil definir la interfaz de un proceso (que estaría formada por el conjunto de parámetros de dicha especificación) y podemos comprobar si todos los datos necesarios para realizar un cálculo están indicados en dicha interfaz (esto podría quedar oculto mediante el uso de pronombres en una especificación en lenguaje natural: 'se le coge y se le suma uno').

Los principales inconvenientes son dos: por un lado los lenguajes de programación no son lo suficientemente formales como para poder deducir propiedades de completitud, consistencia o corrección de una especificación. Por otra parte, el uso de lenguajes procedimentales obliga a definir un algoritmo a la hora de especificar un proceso, pero el objetivo de la especificación no es definir el cómo se ha de implementar el sistema sino definir qué sistema se ha de implementar. Con frecuencia los algoritmos descritos por una especificación de este tipo no son los que finalmente se implementan, por lo que se realiza un trabajo de codificación e interpretación de código baldío.

6.2.3. Lenguajes declarativos.

La programación lógica, representada por lenguajes como Prolog, permite utilizar un subconjunto de la lógica de primer orden, concretamente las cláusulas de Horn, para especificar sistemas. Su base formal permite razonar sobre estas especificaciones, que pueden ser interpretadas directamente mediante resolución, con lo que podemos realizar pruebas de la especificación. Además su propio carácter de declarativos permite representar el qué, en lugar del cómo.

Sin embargo, el inconveniente de Prolog es que no está orientado al proceso. Al tratarse de un lenguaje secuencial, no permite especificar sistemas que interactúan con el entorno o sistemas compuestos por muchas partes que funcionan concurrentemente. Existen lenguajes lógicos concurrentes, como Parlog, que subsanan estas deficiencias.

Otro inconveniente que presentan estos lenguajes es que no admiten tipos de datos, ni siquiera definiciones de datos, por lo que son poco adecuados para especificaciones complejas. Podemos decir, por tanto, que, aunque a veces hayan sido usados para la especificación, los lenguajes lógicos no son lenguajes de especificación propiamente dichos.

6.2.4. Lenguajes de especificación no formales.

Existen diversos lenguajes pensados explícitamente para la especificación de sistemas, con un grado diverso de formalismo. Muchos de ellos fueron concebidos dentro del campo de las comunicaciones, para la especificación de protocolos, aunque pueden ser aplicados a la especificación de cualquier tipo de sistema, incluso aunque no sea concurrente ni reactivo.

Un ejemplo de estos lenguajes es SDL. Se trata de un lenguaje que permite la especificación de sistemas concurrentes de forma procedimental e incorpora la definición de datos mediante especificaciones algebraicas. No se trata de un lenguaje formal, estrictamente hablando, lo que impide el análisis de la especificación.

Otros ejemplos dentro de este campo son LOTOS, que deriva de CSP y del que sí se puede decir que es formal, Estelle o Promella, todos ellos desarrollados dentro del campo de las comunicaciones.

6.2.5. Lenguajes de especificación formal.

Los lenguajes formales sí permiten el razonamiento y la demostración de propiedades a partir de la forma de las especificaciones, aunque en la práctica estos razonamientos están limitados por su complejidad y la falta de algoritmos y herramientas para ello. Ejemplos de lenguajes de este tipo son OBJ, la notación Z o las álgebras de procesos, como CSP, CCS, π-cálculo, etc.

El razonamiento es posible porque estos lenguajes disponen de una semántica que permite interpretar de forma exacta la especificación. Esta semántica ha de estar adaptada a las necesidades de la especificación, de manera que permita representar de forma adecuada los requisitos del sistema. Por ejemplo, muchos lenguajes de programación tienen una semántica formal. Sin embargo, un lenguaje de programación no es un buen

lenguaje de especificación puesto que su semántica sólo permite representar funciones computables, es decir, algoritmos que transforman la entrada o la salida. Un lenguaje de especificación debe tener un dominio semántico más amplio. Debe ser capaz de expresar requisitos del tipo de 'Para todo x de un conjunto A existe un y de un conjunto B tal que se cumple la propiedad P(x, y)' ('Todo pedido lleva asociado un proveedor que lo ha realizado'). En otros casos, los lenguajes de especificación tienen un dominio semántico que permite especificar el comportamiento de un sistema: estados y transiciones entre estados, eventos y sus efectos sobre el disparo de transiciones, sincronización, temporización, etc.

Todos los métodos de especificación, ya sean formales o no tienen como objetivo realizar especificaciones que tengan las siguientes propiedades:

- no ambigüedad.
- consistencia.
- completitud.

Con el uso de métodos formales es más probable que logremos estos propósitos. Por un lado, la sintaxis del lenguaje permite interpretar de una sola forma la especificación, eliminando la ambigüedad que surge con el uso de un lenguaje natural o de una notación gráfica que también puede ser interpretada de formas distintas. Además, la potencia expresiva de la teoría de conjuntos y de la lógica permite representar de forma clara los requisitos.

Para lograr la consistencia, no deben existir contradicciones entre lo establecido en puntos distintos de la especificación. La consistencia queda asegurada por la existencia de medios matemáticos que permiten, usando reglas de inferencia, comprobar que los hechos iniciales se corresponden con sentencias posteriores de la especificación.

La completitud es más difícil de conseguir y de demostrar, incluso utilizando lenguajes formales. Al escribir una especificación pueden quedar indefinidos ciertos aspectos del sistema, incluso pueden existir omisiones deliberadas, bien porque estemos desarrollando la especificación de forma incremental o bien porque queramos dejar las manos libres a los diseñadores en ciertos aspectos. Existen algoritmos para demostrar la completitud pero no está asegurado que terminen.

6.3. Justificación de los métodos formales.

Un sistema basado en ordenador puede fallar tanto por avería de sus componentes físicos como por errores de diseño. El desarrollo de un sistema en el que la seguridad sea un factor crítico (un sistema altamente fiable debería ofrecer un margen de confianza de 1 - 10E-9) debe tener en cuenta ambas fuentes de error.

Existen técnicas bien conocidas para evitar que el fallo de componentes físicos afecte a la función del sistema. Estas técnicas utilizan componentes redundantes, al menos tres, de forma que, caso de que uno de los componentes falle, pueda tomarse la salida de los otros dos como válida. Para estudiar el grado de confianza que pueden proporcionar estos sistemas podemos aplicar técnicas estadísticas o cadenas de Markov.

Sin embargo, la presencia de errores de diseño no se puede evitar tan fácilmente. Para evitar el fallo del sistema software debido a errores en sus componentes (en este caso errores latentes en los programas) podemos tomar tres caminos:

- **Realizar montones de pruebas**, tan exhaustivamente como sea posible, aunque nunca llegaremos a tener seguridad de que hayamos detectado todos los errores.

- **Utilizar componentes software redundantes**. Dado que el software no se deteriora, todos los fallos que presenta son fallos de diseño, por lo que única manera de tener componentes redundantes es desarrollarlos mediante equipos de trabajo independientes a partir de la misma especificación. Aunque el realizar varios desarrollos independientes multiplica los costes de producción de software, podemos considerar este método aceptable, teniendo en cuenta que se reduce la necesidad de las pruebas.

- **Desarrollar software que no contenga errores**. Esto sólo puede conseguirse mediante la especificación formal y la verificación, la generación automática de código y la reutilización de software.

Para establecer estadísticamente un nivel de confianza alto p. ej. una probabilidad de error de 10E-9) necesitaríamos hacer pruebas con él durante un tiempo insospechadamente largo (114.000 años para un programa cuya ejecución durase una hora).

Utilizando componentes redundantes podemos obtener fácilmente niveles de confianza muy grandes aunque cada componente individual tenga un nivel de confianza no tan alto, pero para ello debe darse la condición de que los errores se produzcan de forma independiente. Este no suele ser el caso del software, en el que a pesar de desarrollarse de forma independiente, los equipos de desarrollo suelen cometer errores en las mismas partes del código. Intentar demostrar la independencia de las distintas versiones nos obligaría a medir la correlación entre los errores que aparecen en ellas y para esto necesitaríamos unos periodos de pruebas aún mayores que en el caso anterior.

Por tanto, la única posibilidad que tenemos de desarrollar software altamente fiable es la utilización de lenguajes y métodos formales, puesto que son los únicos que ofrecen la posibilidad de obtener implementaciones con esos niveles de confianza.

6.4. Limitaciones de los métodos formales.

Sin embargo los métodos formales tienen una serie de limitaciones. Por varias razones, no proporcionan una garantía absoluta de perfección del software. En primer lugar, los métodos formales no pueden garantizar que la especificación formal de los requisitos refleje estrictamente las necesidades. Podemos demostrar que no es ambigua, que es consistente e incluso que es completa, pero puede ser totalmente errónea si el sistema especificado no es el que necesitamos. El paso de una especificación preliminar, informal, de requisitos a una especificación formal, es una tarea difícil y que no puede ser automatizada.

En segundo lugar, para el caso de especificaciones de sistemas físicos, como una puerta hardware, no podemos asegurar que el modelo matemático usado refleje exactamente el sistema físico. Algunos modelos simplemente representan propiedades lógicas, otros pueden considerar también aspectos como los retrasos que se producen en las señales, pero normalmente los modelos no tienen en cuenta los efectos de la temperatura, o defectos de fabricación. Incluso en el caso del software nunca podemos asegurar que un programa se comporte estrictamente como define la semántica del lenguaje: puede haber errores en el compilador, tales como determinados efectos laterales en el código muy difíciles de detectar y corregir.

En tercer lugar, dada su complejidad, normalmente la verificación formal se aplica a sólo determinadas partes críticas del sistema.

Por último, pueden existir errores en las propias herramientas de verificación formal.

De todas formas, los métodos formales proporcionan una capacidad significativa para descubrir y eliminar errores dentro del proceso de desarrollo de software.

6.5. Distintos métodos de especificación formal.

Dentro de los métodos de formalización podemos distinguir dos enfoques principales: abstracción funcional y abstracción de datos.

Abstracción funcional.

La abstracción funcional estructura las especificaciones en torno a las funciones del sistema que se pretende construir. En primer lugar, se identifican las funciones del sistema y, a continuación se procede a realizar especificaciones del tipo entrada/salida para cada una de esas funciones.

Un ejemplo de especificación mediante abstracción funcional puede ser la especificación de propiedades de las funciones necesarias mediante el uso de prerrequisitos, postrequisitos e invariantes.

Especificación en lenguaje natural.

El vector A, de longitud N, está indexado por el subrango 1..N. El procedimiento debe buscar en este vector un cierto valor X. Si encuentra el elemento, entonces Y se asigna al índice del elemento que es igual a X. Si ningún elemento del array es igual a X, entonces Y se pone a 0.

Especificación formal.

Precondición:

$N > 0$.

Postcondición:

$$\{X = A[Y] \wedge (1 < Y < N) \vee \{ (Y = 0) \wedge (\forall k : (1 < k < N) \supset A[k] \neq X)\}$$

Especificación formal utilizando otra sintaxis.

Precondición:

$N > 0$.

Postcondición:

IF ($\forall k : (1 < k < N) \supset A[k] \neq X$)

THEN ($Y = 0$)

ELSE $X = A[Y] \wedge (1 < Y < N)$

Abstracción de datos.

Por su parte, en la abstracción de datos las especificaciones se estructuran a partir de los datos u objetos que intervienen en el sistema. En primer lugar, se identifican las distintas clases de objetos que intervienen en el sistema, y a continuación se especifica cada clase o grupo de clases junto con sus funciones y operaciones propias.

Todos los métodos de especificación basados en la abstracción de datos definen una cierta disciplina matemática formal, caracterizada por:

- un álgebra, es decir, uno o varios dominios sobre el que se definen una serie de operaciones.

Las especificaciones suelen definir un único dominio pero pueden hacer referencia a dominios definidos en otras especificaciones.

Las operaciones tienen un tratamiento funcional. Vamos a evitar la existencia de operaciones con efectos secundarios. Por ejemplo, operaciones del tipo extraer de una pila, que devuelve la pila modificada y además el elemento de la cima, será considerada incorrecta; para evitar este tipo de situaciones añadiremos tantas operaciones como sea necesario (cima, para consultar el elemento de la cima, y extraer para extraer el elemento de la cima).

- una teoría construida a partir de axiomas relativos a las operaciones.

- un sistema de inferencia, normalmente usando la lógica de primer orden.

Aún dentro de los métodos basados en abstracción de datos, podemos distinguir varias categorías:

1. Métodos basados en la construcción de modelos.

En ellos las especificaciones representan explícitamente un modelo del sistema, construido a partir de las primitivas del lenguaje dentro de una determinada disciplina formal (grafos, conjuntos, etc.), como es el caso de Z, o bien teorías nuevas, construidas para la especificación, como es el caso del método de Viena (VDM).

- Teoría de Grafos Orientados (grafos V).

- **V.D.M.** (método de desarrollo de Viena). No es sólo un método de especificación, sino que abarca todas las etapas de desarrollo.

- **Notación Z**. Basada en la teoría de conjuntos y la lógica de predicados de primer orden.

2. Métodos basados en la definición implícita.

Con ellos no se pretende dar un modelo abstracto, sino definir la abstracción de datos a través de una serie de relaciones entre los valores de los datos y sus propiedades. No hay modelo explícito que represente la abstracción.

Por ejemplo las Máquinas de Estados de Parnas. La idea fundamental consiste en considerar un tipo de datos identificado con una máquina que pueda adoptar una serie de estados. La especificación del tipo se hace viendo cómo cambia de estado en función de las operaciones del tipo.

3. Método axiomático.

La información sobre los valores del dominio y sobre las operaciones se da en forma de axiomas. Hay entera libertad, en principio, para hacerlo. El método no se apoya en ningún marco teórico salvo la lógica de 1er orden.

Cada especificación viene caracterizada por:

- Uno o varios dominios formados por las distintas clases de objetos a los que se refiere la especificación. Intuitivamente, podemos entender por dominio un tipo de datos (concretamente un TAD) o una clase de objetos. Es decir, los dominios son colecciones de objetos con una función y comportamiento comunes.

- Una teoría constituida por los axiomas y teoremas relativos a los objetos y operaciones de los dominios derivables de la especificación mediante razonamiento. Entendida la especificación de esta forma, el sistema software que se construya a partir de ella deberá ser un modelo de dicha teoría.

Cada especificación contiene información de dos tipos:

- información sintáctica, relativa al alfabeto y sintaxis empleados en la especificación. Esta información especificará:
  - la identificación del sistema que estamos especificando.
  - la identificación de los dominios que intervienen en él.
  - la identificación de las operaciones junto con su funcionalidad.

- información semántica, relativa al comportamiento de los objetos definidos en la especificación. Esta información semántica está basada en:
  - la construcción de un **modelo**, en el caso de los métodos orientados a modelos. Este modelo es una representación abstracta de los objetos y operaciones del sistema.
  - la enumeración de **propiedades** y **relaciones** de los objetos y las operaciones, en el caso de los métodos denominados implícitos, entre los que se encuentran las especificaciones algebraicas.

Con respecto a los dominios, en una especificación aparecen por lo general varios dominios: alguno de ellos (posiblemente uno sólo) es el dominio que está definiendo la especificación. El resto son dominios ya conocidos que se utilizan para especificar los nuevos.

Con respecto a las operaciones, pueden ser de tres tipos:

- **generadores**. Son aquellas operaciones que producen resultados de la clase que se está definiendo sin actuar sobre operandos de dicha clase. Es decir, son operaciones que crean objetos de la clase a partir de argumentos de otras clases.

p. ej. la operación de crear un número complejo a partir de dos números reales.

- **transformadores**. Son aquellas operaciones que producen resultados de la clase que se está definiendo, actuando sobre operaciones de dicha clase. Es decir, son operaciones que transforman objetos de la clase que estamos definiendo.

  p. ej. la operación de suma de números complejos.

- **selectores**. Son el resto de las operaciones. Actúan sobre argumentos de la clase que estamos definiendo pero producen resultados de otras clases.

  p. ej. la operación parte real de números complejos.

Ejemplo: Pila de números enteros.

1.- PILA(pila_nula)

2.- PILA(s) & ENTERO(i) => PILA(insertar(i, s))

& [extraer(s) ≠ ERROR_P => PILA(extraer(s))]

& [cima(s) ≠ ERROR_E => ENTERO(cima(s))]

3.- (∀P) P(pila_nula) &

(∀s) (∀i) [PILA(s) & ENTERO(i) & [P(s) => P(insertar(i, s))]

& [s ≠ pila_nula => P(extraer(s))]

=> (∀s) (Pila(s) => P(s))

4.- PILA(s) & ENTERO(i) => cima(insertar(i, s)) = i

5.- cima(pila_nula) = ERROR_E

6.- PILA(s) & ENTERO(i) => extraer(insertar(i, s)) = s

7.- extraer(pila_nula) = ERROR_P

Esta forma de especificación es muy formal. Los métodos axiomáticos son buenos por tener buena minimalidad - se precisa un número mínimo de axiomas- y tener un rango de aplicación muy amplio. Los problemas que presenta son en cuanto a la dificultad de construcción y comprensión, además de la dificultad para decidir si la especificación se adapta a la abstracción que queremos representar.

4. Método Algebraico

Se puede considerar como una variante del método axiomático, surgida del hecho de establecer hipótesis sobre la abstracción que se va a tratar. Partimos de la hipótesis de que las abstracciones de datos tienen dominio numerable y de que sus términos son generables de forma finita (los números reales no podrían representarse). Se enmarca dentro de la teoría de las Álgebras Universales, y en concreto de las Álgebras Multitipos.

Esta teoría establece limitaciones de tipo sintáctico sobre la forma de los axiomas (ecuaciones) y en cuanto a los métodos de razonamiento: ecuacional, asociado al predicado de igualdad, e inducción estructural. Todo esto se incluye implícitamente; además, se

supone que todas las variables están cuantificadas universalmente, con lo que quedan especificaciones más simples y fáciles de entender.

Una especificación algebraica consta de tres partes:

- **Definición de dominios**, donde se establecen los nombres de los dominios que van a intervenir en la especificación.

- **Definición de operaciones**, que es una relación de las operaciones que se definen en la especificación, indicando el nombre y la signatura (tipo de los argumentos y resultado).

- **Axiomas**, que establecen las relaciones entre las operaciones, permitiendo establecer identificaciones entre términos del mismo tipo. En las especificaciones algebraicas, los axiomas tienen la forma de ecuaciones, posiblemente condicionales.

El método algebraico parte de la idea de que todo tipo de datos es un álgebra y que su especificación se puede limitar a la especificación del álgebra que representa; para ello basta enunciar, para cada tipo T,

- una serie de operaciones características, con sus funcionalidades (dominio y recorrido), expresadas con el formato

$$f\colon\ T_1 \times T_2 \times \ldots \times T_k \rightarrow T_l$$

  lo que constituye la signatura del tipo, y

- una relación de ecuaciones para fijar el comportamiento de dichas operaciones en todas las circunstancias posibles, que definen la *semántica* de las operaciones del tipo.

La restricción más típica es considerar únicamente los *modelos iniciales*, es decir, suponer que cada especificación se refiere sólo a las álgebras que se pueden considerar modelos iniciales de dicha especificación (*semántica inicial*). Estos modelos iniciales se caracterizan por lo siguiente:

1. son *álgebras generadas por términos*, es decir, cada valor se puede obtener siempre como resultado de aplicar las operaciones de la signatura (como un término constante del lenguaje asociado a la signatura); y además,

2. son las álgebras *con el mayor número posible de valores distintos*; concretamente, para un álgebra inicial, dos términos denotan siempre dos valores distintos del álgebra salvo que se pueda demostrar mediante las ecuaciones que dichos términos son equivalentes.

Esta restricción permite reducir bastante el número de ecuaciones a la hora de especificar un tipo de datos.

Las especificaciones que se obtienen de este modo se dice que definen **tipos abstractos de datos** (TADs); las implementaciones que de ellos se hagan serán los tipos concretos.

Ejemplo: Pila de números enteros

Dominios: PILA, ENTERO, ERROR_P, ERROR_E

Operaciones:

pila_nula: -> PILA

error_p: -> ERROR_P

error_e: -> ERROR_E

insertar: ENTERO x PILA -> PILA

extraer: PILA -> PILA ∪ ERROR_P

cima: PILA -> ENTERO ∪ ERROR_E

Axiomas: i: ENTERO; p: PILA

1.- cima(insertar(i, p)) == i

2.- cima(pila_nula) == error_e

3.- extraer(insertar(i, p)) == p

4.- extraer(pila_nula) == error_p

Hay varias teorías algebraicas: Álgebras Iniciales y Álgebras Finales o Terminales. Nosotros utilizaremos las Iniciales, caracterizadas porque dos términos son equivalentes siempre que esta igualdad se pueda demostrar a partir de los axiomas. Las Álgebras Terminales por el contrario consideran que dos términos del mismo tipo son iguales a menos que se pueda demostrar lo contrario.

Especificación algebraica

Estas especificaciones tuvieron su origen a mediados de la década de los 70, a partir de la teoría de las álgebras multitipo. Originalmente se desarrollaron para permitir la especificación de tipos abstractos de datos, pero posteriormente se fueron extendiendo de forma que permiten la especificación de sistemas software completos.

Uno de los primeros lenguajes de especificación algebraica es CLEAR, que tuvo considerable influencia sobre el resto. Otros lenguajes de especificación formal derivados de él son ACT ONE y la familia de lenguajes OBJ.

Las especificaciones algebraicas han influido también sobre otros lenguajes de especificación, como LOTOS o SDL, cuya definición de tipos se realiza de esta forma. Conceptos derivados de ellas se utilizan también el lenguajes de programación como ADA o EIFFEL.

6.6. Propiedades de las especificaciones algebraicas

- La **corrección** de una implementación de la especificación se realiza demostrando que dicha implementación define un modelo para la especificación (en otras palabras, que satisface los axiomas de la especificación).

- La **consistencia** (ausencia de contradicciones) de la especificación se realiza demostrando que las ecuaciones, consideradas como reglas de reescritura de izquierda a derecha, presentan la propiedad de Church-Rosser, según la cual, la reducción de un término, aplicando dichas reglas hasta que ya no sea reducible, es independiente del orden de aplicación de las reglas. En general, esto es indecidible.

- La **completitud** (ausencia de ambigüedades) se puede establecer si conseguimos demostrar que cualquier término base (que no contiene variables) se puede reducir a un término en el que sólo aparecen operaciones constructoras. En general también es indecidible.

6.7. El estilo constructivo

A la hora de plantearse la especificación de un tipo de datos conviene seguir algún método que simplifique el trabajo y asegure ciertas condiciones de *consistencia* y *completitud* de la especificación. Igual que para la programación, no sólo hace falta perspicacia, sino también disciplina y estilo.

Para la construcción de especificaciones algebraicas nos basaremos en el hecho de que muchos dominios disponen de un conjunto mínimo de operaciones que permiten generar todos los objetos de ese dominio (éstas son las operaciones constructoras), mientras que el resto de generadores y las operaciones transformadoras no generan valores nuevos ni son capaces de reducir el conjunto de valores generado por las constructoras.

Si existe este conjunto mínimo de generadores, cada objeto del dominio puede ser representado por un término formado exclusivamente por operaciones constructoras. Si, además, este término es único e irreducible, será el representante canónico en la clase de equivalencia correspondiente del álgebra de términos.

6.7.1. Especificaciones constructivas.

El estilo constructivo consiste en:

- Clasificar las operaciones características o necesarias para trabajar con los valores de un tipo T de la siguiente forma:

  operaciones constructoras, que son todas aquellas que producen valores del tipo T; entre ellas cabe distinguir las que no cuentan con ningún argumento del tipo T, que se denominan *constantes relativas*.

  operaciones de observación, consulta o no constructoras, que son todas las que producen valores de tipos distintos al tipo T, aunque actúan sobre algún argumento del tipo T; estas operaciones se suelen utilizar para observar propiedades o consultar componentes de los valores del tipo T.

- Buscar un conjunto G_t de operaciones constructoras, tal que todos los valores del tipo T se puedan expresar como términos finitos utilizando únicamente estas operaciones (aplicadas posiblemente a valores de otros tipos); se tiene así lo que se denomina una *familia de generadores* del tipo T.

Normalmente el conjunto G_t incluirá alguna constante relativa, y cuando incluya además algún otro constructor, los distintos valores del tipo T se podrán generar

mediante la aplicación repetida de este último constructor (o constructores) un número finito de veces tomando como punto de partida las constantes relativas. Los términos generados de esta forma pueden denotar siempre valores distintos, o puede ocurrir que varios términos denoten un mismo valor del tipo T; en el primer caso se dice que G_i es una *familia libre de generadores*, mientras que en el segundo caso no lo será.

Para cada tipo se procurará determinar un G_i lo más reducido posible y libre (aunque no siempre se podrá encontrar) pues esto simplifica el número y la complejidad de las ecuaciones de la especificación, y, además, permite construir un modelo de la especificación, constituido por los distintos términos que se pueden construir con los generadores (el universo de términos), utilizable en los razonamientos sin necesidad de disponer de ninguna implementación.

- Cuando se dispone de una familia de generadores, dichos generadores se toman como funciones primitivas sin ninguna información semántica, salvo cuando dicha familia no sea libre, en cuyo caso habrá que incluir las ecuaciones necesarias para expresar la relación entre los distintos generadores.

  Las demás operaciones *f* deberán especificarse completa y constructivamente en términos de los generadores, posiblemente como funciones parciales, mediante ecuaciones de la forma

  $$f(x_1, ..., x_k) == t(x_1, ..., x_k)$$

  donde $f(x_1, ..., x_k)$ es un término en el que sólo pueden aparecer las variables que aparezcan a la derecha de la ecuación, y procurando que las ecuaciones destinadas a la especificación de una misma función *f* cubran todas las posibles formas de los argumentos de *f* (completitud) y no se solapen ni produzcan identificaciones de valores de otros tipos que deban ser distintos (lo que asegura la consistencia).

  Cuando todas las funciones que aparecen en $t(x_1, ..., x_k)$ están semánticamente definidas, se tiene lo que se llama una *definición directa*, y si aparecen funciones parciales, las condiciones de definición de la parte izquierda de la ecuación coincidirán con las de la parte derecha. También puede aparecer la misma función *f* en $t(x_1, ..., x_k)$, con lo que se tiene una *recursión directa*, o se pueden producir *recursiones mutuas* para un grupo de funciones.

  En la relación de operaciones aparecerán normalmente *operaciones totales*, definidas para todos los valores de su dominio y *operaciones parciales*, distinguidas por el símbolo —/→, definidas únicamente para los valores de sus argumentos que cumplan

unas ciertas condiciones expresadas en el apartado de *precondiciones* mediante predicados de especificación.

En la relación de ecuaciones aparecerán todas aquellas ecuaciones entre términos, construidos con las distintas operaciones, que sean necesarias para expresar de una forma consistente y completa el comportamiento de dichas operaciones respecto a los valores del tipo que se está especificando. El papel de cada ecuación, desde un punto de vista formal, es establecer identificaciones entre términos sintácticamente distintos que deben representar el mismo valor en todos los modelos posibles de la especificación.

Los axiomas se consideran como sistemas de reescritura de términos de izquierda a derecha que reducen los términos que contienen operaciones no constructoras a términos formados por constructores. Se establecen así clases de equivalencia en el conjunto de expresiones bien formadas a partir de las operaciones. Cada una de estas clases de equivalencia del álgebra inicial de una especificación constructiva se puede caracterizar por su forma canónica, el único término constante generado sólo con constructores que contiene.

Como hemos dicho anteriormente, cada operación no constructora se puede ejecutar simbólicamente utilizando los axiomas como reglas de reescritura de izquierda a derecha hasta llegar a una forma canónica (finitud), que será el resultado de dicha ejecución, con lo que se puede hablar de prototipo teórico inmediato del sistema especificado.

Introduciendo restricciones de unicidad y completitud, que pueden ser comprobados mecánicamente, la posibilidad de escribir especificaciones erróneas se reduce considerablemente. De esta forma mejoramos de forma importante la corrección del software.

Ejemplo: los valores lógicos

El tipo correspondiente al álgebra de Boole de los valores lógicos se puede especificar de una manera directa dando las dos constantes true y false, que constituyen un conjunto libre de generadores y denotan los dos valores del tipo, y definiendo las demás operaciones típicas del álgebra de Boole en función de estas constantes. Veáse el ejemplo al final del tema.

Debemos hacer notar que la tercera ecuación de la operación and puede afectar la finitud de la especificación. Si observamos detenidamente los axiomas nos percatamos de que esta ecuación no es precisa, y por lo tanto podemos quitarla sin más -las dos primeras

son suficientes para especificar correctamente dicha operación-. Sin embargo, sabemos que esta es una ecuación importante para nuestra especificación, pues nos dice que la operación and es conmutativa; sería conveniente pues añadir esta ecuación como teorema. Por otra parte vemos que es posible que haya varias formas de especificar lo mismo. Consideremos por ejemplo el siguiente axioma para especificar la operación and:

a and b == si a = true entonces b si no false;

Vemos por una parte que recoge todas las posibles combinaciones de los constructores.

Las operaciones *not* y *and* aparecen como operaciones de enriquecimiento (definidas en función de los generadores), mientras que las operaciones *or*, $\Rightarrow$ y $\Leftrightarrow$ aparecen como operaciones derivadas (definidas como términos construidos con las operaciones anteriores).

Ejemplo: los números naturales

Especificamos el tipo correspondiente al álgebra de los números naturales por recursión, partiendo de los generadores *0* y *suc*, con los que se definen todos los valores distintos del tipo y enriquecemos con las operaciones aritméticas y las relaciones de orden e igualdad. (Ver transparencia)

Ejemplo: el TAD Pila

Los constructores son pila_nueva y insertar, mientras que las operaciones no constructoras son es_pila_vacía, extraer y cima. Con los constructores podemos generar todos los objetos del TAD Pila, y cada objeto del TAD viene dado por un único término formado por pila_nueva y insertar. Mientras, las operaciones no constructoras describen el comportamiento funcional de los objetos del TAD, y son definidos en función de los constructores. (Ver transparencia)

Los axiomas de las operaciones del TAD Pila pueden ser interpretadas como un sistema de reescritura de términos de izquierda a derecha que reduce términos de variables libres que contienen operaciones no constructoras a formas canónicas formadas por constructores. (supongamos que escribimos, por ejemplo, *cinco* en lugar de *suc(suc(suc(suc(suc(cero)))))*, entonces el término

extraer(insertar(extraer(insertar(insertar(pila_nueva, cinco), siete), nueve))

puede ser reducido a la forma canónica

insertar(pila_nueva, cinco)

Otro ejemplo:

es_pila_vacía(extraer(insertar(extraer(insertar(pila_nueva, cinco)), siete)))

se reduce a la forma canónica

true

6.7.2. Especificaciones semiconstructivas

En muchos casos no se consigue la expresión completa del comportamiento de un tipo abstracto mediante especificaciones constructivas debido a que sus constructores presentan propiedades que no son derivables como teoremas, como ocurre por ejemplo con el hecho de que en los números enteros el sucesor del predecesor de un número sea ese mismo número, que la inserción en los conjuntos sea conmutativa o que la inserción repetida de un mismo elemento no altere un conjunto.

En estos casos se hace necesario añadir un apartado de axiomas sobre constructores a la especificación, con lo que la especificación dejará de ser constructiva. En general, un axioma sobre constructores se caracteriza porque sus dos miembros constarán únicamente de variables y nombres de constructores y estos serán de algún tipo definido en el módulo donde figuren.

Estos axiomas, cuando se utilizan como reglas de reescritura de izquierda a derecha, pueden afectar a la finitud de la especificación, por lo que, por lo general, cualquier especificación en la que aparezcan será no constructiva. Sin embargo, se puede conseguir mantener el papel de los axiomas como reglas de reescritura con el objeto de tener un prototipado rápido, imponiendo algunas condiciones que caracterizarán a lo que se denomina especificaciones semiconstructivas.

> ↳ *Si en una especificación semiconstructiva eliminamos los axiomas sobre constructores, la especificación resultante debe ser constructiva.*

Tenemos pues que una especificación semiconstructiva se construye de la misma forma que una constructiva, de hecho, si se borran los axiomas sobre constructores debe quedar una especificación constructiva. Pero además, estos axiomas sobre constructores deben cumplir un par de condiciones para que se asegure la consistencia y la completitud:

↳ *Partiendo de cualquier término formado por variables y constructores, debe ser imposible derivar un número infinito de términos literalmente diferentes aplicando los axiomas sobre constructores.*

Ejemplo: los números enteros

Los números enteros son un ejemplo de especificación semiconstructiva. Además de los generadores *0* y *suc*, utilizados con los números naturales, es necesario incluir como constructora a la operación *pred*, para representar los números negativos. Sin embargo, la familia de generadores resultante no es libre, puesto que cualquier numero entero puede expresarse de infinitas formas utilizando únicamente operaciones constructoras. Así, el número *uno* puede expresarse como:

suc(cero)

suc(pred(suc(cero)))

pred(suc(suc(cero)))

suc(pred(suc(pred(suc(cero)))))

...

Esta dependencia entre los constructores debe quedar reflejada en los axiomas de la especificación, de forma que se identifiquen todos los términos que representan al mismo elemento del dominio. Si no fuese así deberíamos interpretar cada término como un elemento diferente del dominio, puesto que aplicamos el formalismo de las álgebras iniciales. (Ver transparencia).

↳ *Si dos términos constantes construidos únicamente a base de constructores son derivables a partir de un mismo término constante, construido también únicamente a base de constructores por aplicación de los axiomas sobre constructores, entonces a partir de ambos se debe poder llegar a un mismo término constante por aplicación de los axiomas sobre constructores.*

Es posible conseguir una ejecución directa de los axiomas de este tipo de especificaciones parando el proceso de reducción para evitar reducciones infinitas. Una forma sería aplicando los axiomas sobre constructores únicamente cuando no se pueda aplicar otro axioma y además no den lugar a términos ya derivados.

Ejemplo: el TAD Conjunto

En este caso es necesario añadir el axioma sobre el constructor incluir para reflejar la propiedad de que los conjuntos no contienen elementos duplicados y de que el orden de inclusión no es relevante, es decir, que la inclusión cumple la propiedad conmutativa. (Ver transparencia).

La existencia de axiomas conmutativos implica la aparición de derivaciones infinitas de términos (aunque estos términos no sean sintácticamente distintos). Por tanto, para implementar esta especificación deberíamos controlar la aparición de un ciclo al aplicar reiteradamente el axioma de conmutabilidad. Podemos asegurar más fácilmente la terminación escribiendo axiomas condicionales:

i1 = i2 => incluir (i1, incluir (i2, c)) = = incluir (i2, c));

i1 < i2 => incluir (i1, incluir (i2, c)) = = incluir (i2, incluir (i1, c));

(esto último obliga a definir una operación _<_ sobre los naturales)

De esta forma aseguramos la terminación, puesto que ya no es posible una derivación infinita, y es más sencilla la implementación de un prototipo. Además definimos implícitamente una forma canónica para cada clase de equivalencia del álgebra de términos: las formas canónicas se forman al incluir los elementos en orden ascendente.

Ejercicio: Especificar las siguientes operaciones sobre conjuntos: intersección y diferencia de conjuntos, cardinal de un conjunto, y la operación de consulta para comprobar si un conjunto está contenido en otro.

6.7.3. Constructividad y Abstracción

Especificaciones no constructivas son aquellas que no son ni constructivas ni semiconstructivas. La constructividad es una propiedad muy importante, ya que va a permitir un prototipado rápido, de forma que un sistema software pueda ser probado antes de ser implementado. El principal inconveniente de la constructividad es una posible pérdida de abstracción; si se encuentra una especificación no constructiva del mismo sistema, está será probablemente de un mayor nivel de abstracción.

Ejemplo: Un sistema de un robot simple.

El mundo del robot se reduce a una gran cuadrícula regular, atravesada por vías horizontales y verticales, que dan lugar a cruces en los puntos en que se cortan. El robot se

sitúa en un cruce orientado hacia una de las cuatro direcciones. La operación *pos_inicial* coloca al robot en su posición inicial. Cuando el robot ejecuta la operación *avanzar* este se coloca en el siguiente cruce según la orientación que tenía y sigue mirando en la misma dirección. Cuando el robot ejecuta la operación *girar*, este gira noventa grados hacia la izquierda. (Ver transparencias).

Las especificaciones semiconstructivas son más largas y menos abstractas que las no constructivas, pudiéndose considerar las primeras como más cercanas de la implementación que las últimas. La ventaja de las semiconstructivas es que es más fácil y rápido construir un prototipo a partir de ellas. Además, el razonamiento formal es más fácil con las semiconstructivas, y son más adecuadas también a posibles extensiones. En el caso del robot, los axiomas de la especificación no constructiva pueden derivarse como teoremas de los axiomas de la especificación semiconstructiva.

Ejercicio: Demostrar el siguiente teorema.

girar(girar(girar(avanzar(girar(avanzar(pos)))))) ==

avanzar(girar(girar(girar(avanzar(girar(pos))))));

6.8 Especificaciones parametrizadas

Para la especificación de sistemas distintos, pero que presenten comportamientos comunes (tipos genéricos), sería conveniente disponer de algún tipo de construcción en el lenguaje que nos permita escribir especificaciones abiertas, con el comportamiento común, que luego puedan ser completadas con las características particulares.

Esto sucede continuamente con las especificaciones que se refieren a tipos de datos compuestos: una pila de enteros o una pila de caracteres, un punto 2D sobre un espacio entero o sobre un espacio real, etc.

Utilizaremos entonces **esquemas de especificaciones** en las que se definen una serie de parámetros que posteriormente podamos instanciar a valores concretos. Podemos transformar la especificación de la pila de naturales en un esquema de pilas genéricas, independientes de los elementos que contengan.

Ejemplo: Esquema de especificación de una pila.

Un esquema de especificación tiene la misma forma que una especificación, pero añade una definición de requisitos que deben cumplir los parámetros que instancien la especificación. Estos requisitos son de tres tipos: dominios, operaciones y axiomas.

Podemos instanciar un esquema, asignando valores a sus parámetros, pero hay que tener en cuenta que deben cumplirse los requisitos referidos a la signatura de las operaciones y a los axiomas. (Ver transparencia).

Ejemplo: Esquema de especificación de una tabla.

(Ver transparencia). Una posible instanciación del esquema podría ser la siguiente:

```
módulo Vectores_Naturales;
        instancia Vectores cambiando
                Item            por Naturales,
                ITEM  por NATURAL,
                Indice          por Naturales,
                INDICE          por NATURAL;
                _ = _   por _ = _;
fin módulo Vectores_Naturales;
```

6.11 Caso de estudio: construcción de una tabla de referencias cruzadas.

El concepto sobre el que se fundamenta el enfoque orientado a objetos del desarrollo de software es el de Tipo Abstracto de Datos. Veamos cuál es el significado del término TAD y su relación con los programas.

El desarrollo de cualquier sistema software consiste en elegir los TADs y reconocer la relación de jerarquía entre ellos, especificar estos TADs, elegir la modularización del sistema, detectando las relaciones entre los distintos módulos y de estos con los TADs, definir e implementar estos módulos, y finalmente, conectar estos módulos de forma que compongan un programa que sea compilable y ejecutable.

Podríamos suponer que cada objeto tiene un TAD subyacente. Un TAD es una colección de valores, al que llamamos dominio, y una colección de operaciones para construir y consultar estos valores. No hay noción de valores almacenados, o estado, asociados con un TAD. Cuando diseñamos un programa, podemos empezar identificando y definiendo los TADs relevantes al problema.

Ejemplo

Un simple programa de procesamiento de texto que produce un índice de palabras en un fichero de texto. Como entrada tenemos un fichero con líneas de texto, y como salida debe producirse un índice de todas las palabras del texto. El índice será una lista alfabéticamente ordenada de todas las palabras del texto de entrada, y para cada palabra, una lista de todos los números de las líneas en las que aparece cada una de las ocurrencias de esa palabra.

Cada secuencia no interrumpida de letras en el fichero de entrada constituye una palabra. Distinguiremos mayúsculas de minúsculas. Si una palabra aparece varias veces en una misma línea de texto entonces el número de dicha línea ha de aparecer junto a la palabra en el fichero de salida el mismo número de veces. Si una palabra aparece repetida muchas veces en el texto tenemos que considerar la posibilidad de que se necesiten utilizar varias líneas en el fichero de salida. Se tendrán en cuenta aquellas líneas que no contengan palabras para calcular el número de cada línea.

Si el fichero de entrada fuese el siguiente:

```
Esta línea es la línea uno
esta línea es la línea dos
```

entonces el fichero de salida debería tener el siguiente aspecto:

```
dos     2
Esta    1
esta    2
es      1       2
la      1       2
línea   1       1       2       2
uno     1
```

Identificación de los TADs

Tratamos de identificar los TADs que se necesitan para construir la jerarquía con dos objetivos:

•el TAD situado en la cima de la jerarquía recoge aquellos aspectos requeridos del comportamiento del programa que pueden ser contemplados como un TAD

•la organización de la jerarquía recoge un diseño estructural del programa, que puede ser empleado para desarrollar la implementación de este. Los TADs de la jerarquía recogen algunas de las propiedades de los módulos a partir de los cuales se construye el programa.

El comportamiento del programa puede ser especificado como un TAD, aunque este sería muy complicado, por lo que su especificación debe ser organizada.

La visión como TAD del programa vendrá dada por el TAD ÍNDICE, que suministra la operación:

índice: FicheroDeCaracteres -> FicheroDeCaracteres

definida como una función que va desde un fichero de caracteres de entrada a un fichero de salida, que contiene un índice de todas las ocurrencias de todas las palabras del fichero de entrada.

Podemos suponer que el TAD FICHERO_DE_CARACTERES es una secuencia de caracteres, considerando el conjunto de caracteres formado por letras, dígitos y caracteres especiales como el carácter final-de-línea y el separador. Tendremos que ver cómo definimos la comparación de caracteres para poder ordenar alfabéticamente las palabras del índice.

Para conseguir identificar la jerarquía de TADs debemos buscar un conjunto de componentes a partir de los cuales construir la especificación de la operación índice. Inmediatamente podemos considerar el fichero de entrada y el fichero de salida. Un estudio más detallado del problema nos hace pensar que necesitamos alguna estructura de datos intermedia, con la que no sea necesario empezar a escribir en el fichero de salida directamente, pues esto podría complicar en exceso el proceso. Pensemos por ejemplo que casi al final del fichero de entrada nos encontramos con la palabra “al”, que debe ir al principio del fichero de salida. Decidimos por tanto utilizar alguna estructura intermedia que nos permita almacenar palabras y números de línea durante el tiempo que transcurre entre que se lee el fichero de entrada y que se escribe el de salida. Sería conveniente que esta estructura mantuviese los elementos ordenados, por lo que nosotros vamos a llamarla COLECCIÓN_ORDENADA.

Fijémonos primero en el comportamiento que esperamos del fichero de entrada. En principio podríamos pensar en el fichero de entrada formado por líneas que se componen de palabras, e ir leyendo líneas y extrayendo palabras de la línea mientras esta no estuviese vacía, pero esto nos obligaría a mantener un contador para controlar el número de línea en el que estamos. Una posible solución podría consistir en tener una operación que nos devolviese el número de línea en que se encuentra la palabra extraida. Pero esto podría simplificarse considerando el fichero compuesto de palabras, y que cuando se pide una palabra del fichero este nos da la línea en que esta se encuentra. Esto podemos hacerlo definiendo un PAR formado por una palabra y por el número de línea en que esta se encuentra. Así, el TAD FAE (fichero abstracto de entrada):

paresEnFicheroEntradaAbstracto:

FAE -> BOOLEAN;

(devuelve true si hay al menos una palabra en el TAD fichero de entrada abstracto; false en caso contrario)

primerParFicheroEntradaAbstracto:

FAE -> PAR;

(devuelve la primera palabra del fichero de entrada abstracto junto con su número de línea asociado)

ficheroEntradaAbstractoMenosPrimerPar:

FAE -> FAE;

(devuelve un fichero de entrada abstracto con el mismo contenido excepto el primer par en el fichero de entrada abstracto dado)

Con estas operaciones el tratamiento del fichero de entrada queda muy sencillo: mientras haya pares palabra-número de línea en el fichero abstracto de entrada hacemos lo que sea oportuno con el primer par y volvemos a hacer lo mismo con el fichero sin el primer par. Si no hay más pares terminamos.

Fijémonos en que de esta forma el cliente de este TAD no debe preocuparse de detalles innecesarios de la estructura concreta de un fichero de caracteres (por ejemplo, retornos de carro sucesivos, etc.). El TAD PAR puede servirnos también como componente de la colección ordenada, y será a partir de elementos de este tipo a partir de los que construyamos elementos del fichero de salida.

Para representar palabras definimos el TAD STRING, el TAD NATURAL lo utilizamos para representar los números de línea y para definir el TAD CARACTER, en el que necesitamos establecer una ordenación entre los elementos del TAD.

Los TADs que sirven de entrada y de salida al programa se construirán sobre el TAD FICHERO_DE_CARACTERES. Por los requerimientos de la forma en que se trata la entrada, necesitamos dividir la entrada en palabras y en líneas (para poder conocer los números de línea). El programa puede suponer que se hace de esta forma, y de ahí la necesidad de introducir el TAD FAE, que será el encargado de encapsular este comportamiento. Este TAD nos da una visión abstracta del Fichero de Caracteres, que no es otra cosa que una secuencia de caracteres. Por el contrario un fichero de entrada abstracto es una secuencia de palabras, donde cada una de estas palabras tiene asociado un número de línea.

En cuanto a la salida del programa, tenemos que un fichero se construye a partir de palabras y números de línea. El programa puede considerar que las operaciones de escribir una cadena de caracteres y un número no son específicos del programa, de forma que o bien podemos utilizar un TAD que incorpore este comportamiento, o bien definirlo para que sea reutilizado por otros programas. Introducimos el TAD FICHERO_DE_TEXTO_DE_SALIDA como un enriquecimiento de un fichero de caracteres que define operaciones de escritura de cadenas y números naturales (números de línea). Desde el punto de vista del TAD FICHERO_DE_CARACTERES, la escritura de una cadena o un número en el fichero de salida se corresponde con la escritura de cada uno de los caracteres que forman la cadena o el número.

Especificación formal basada en abstracción funcional.

Especificación en lenguaje natural.

El vector A, de longitud N, está indexado por el subrango 1..N. El procedimiento debe buscar en este vector un cierto valor X. Si encuentra el elemento, entonces Y se asigna al índice del elemento que es igual a X. Si ningún elemento del array es igual a X, entonces Y se pone a 0.

Especificación formal.

Precondición:

$N > 0$.

Postcondición:

$$\{X = A[Y] \land (1 < Y < N) \lor (Y = 0) \land (\forall k : (1 < k < N) \supset A[k] \neq X)\}$$

Especificación formal utilizando otra sintaxis.

Precondición:

$N > 0$.

Postcondición:

$$\text{IF } (\forall k : (1 < k < N) \supset A[k] \neq X)$$

$$\text{THEN } (Y = 0)$$

$$\text{ELSE } X = A[Y] \land (1 < Y < N)$$

Especificación axiomática: una pila de números enteros.

1. PILA(pila_nula)
2. PILA(s) & ENTERO(i) =>

   PILA(insertar(i,s))

   & [extraer(s) ≠ ERROR_P => PILA(extraer(s))]

   & [cima(s) ≠ ERROR_E => ENTERO(cima(s))]
3. (∀P) P(pila_nula) &

   (∀s) (∀i) [PILA(s) & ENTERO(i) & [P(s) => P(insertar(i,s))]

   & [s ≠ pila_nula => P(extraer(s))]

   => (∀s) (Pila(s) => P(s))
4. PILA(s) & ENTERO(i) => cima(insertar(i,s)) = i
5. cima(pila_nula) = ERROR_E
6. PILA(s) & ENTERO(i) => extraer(insertar(i,s)) = s
7. extraer(pila_nula) = ERROR_P

Especificación algebraica: una pila de números enteros.

Dominios: PILA, ENTERO, ERROR_P, ERROR_E

Operaciones:

error_p: -> ERROR_P

error_e: -> ERROR_E

pila_nula: -> PILA

insertar: ENTERO x PILA -> PILA

cima: PILA -> ENTERO ∪ ERROR_E

extraer: PILA -> PILA ∪ ERROR_P

Axiomas: i: ENTERO; p: PILA

1.- cima(pila_nula) == error_e

2.- cima(insertar(i, p)) == i

3.- extraer(pila_nula) == error_p

4.- extraer(insertar(i, p)) == p

Valores lógicos

módulo Valores Lógicos

dominio BOOLEAN;

constructores

true, false: → BOOLEAN;

operaciones

not: BOOLEAN → BOOLEAN;

and, _or_: BOOLEAN x BOOLEAN → BOOLEAN;

_⇒_, _⇔_: BOOLEAN x BOOLEAN → BOOLEAN;

axiomas a,b: BOOLEAN;

not true == false;

not false == true;

true and b == b;

false and b == false;

a and b == b and a; // Ojo, no es constructiva

a or b == not ((not a) and (not b));

a ⇒ b == (not a) or b;

a ⇔ b == (a ⇒ b) and (b ⇒ a);

fin módulo Valores Lógicos;

Números naturales

módulo Naturales

importa BOOLEAN;

dominio NATURAL;

constructores

0: → NATURAL;
suc: NATURAL → NATURAL;

operaciones

pred: NATURAL —/→ NATURAL;
_+_ , _*_ : NATURAL x NATURAL → NATURAL;
_-_ : NATURAL x NATURAL —/→ NATURAL;
_≤_ : NATURAL x NATURAL → BOOLEAN;

precondiciones n, m: NATURAL;

pre pred(n): not (n = 0);
pre n-m: m ≤ n;

axiomas n, m: NATURAL;

0 ≤ n == true;
suc(n) ≤ 0 == false ;
suc(n) ≤ suc(m) == n ≤ m;

pred(suc(m)) == m;

n + 0 == n;
n + suc(m) == suc(n + m);

n * 0 == 0;
n * suc(m) == n + (n * m);

n - 0 == n;
n - suc(m) == pred(n - m);

fin módulo Naturales;

Pila de naturales

módulo Pila de Naturales;

importa BOOLEAN, NATURAL;

dominios PILA;

constructores

pila_nueva: -> PILA;

insertar: NATURAL x PILA -> PILA;

operaciones

es_pila_vacía: PILA -> BOOLEAN;

extraer: PILA -> PILA;

cima: PILA -> NATURAL;

precondiciones p: PILA;

pre cima(p): not es_pila_vacía(p);

axiomas p: PILA; e: NATURAL;

es_pila_vacía(pila_nueva) == true;

es_pila_vacía(insertar(e, p)) == false;

extraer(pila_nueva) == pila_nueva;

extraer(insertar(e, p)) == p;

cima(insertar(e, p)) ==e

fin módulo Pila;

Números enteros

módulo Enteros

importa BOOLEAN

dominio ENTERO

constructores

0: → ENTERO;

suc, pred: ENTERO → ENTERO;

operaciones

_+_ , _*_ , _-_ : ENTERO x ENTERO → ENTERO;

axiomas n, m: ENTERO

pred(suc(n)) == n;

suc(pred(n)) == n;

n - 0 == n;

n - suc(m) == pred(n - m);

n - pred(m) == suc(n - m);

n + 0 == n;

n + suc(m) == suc(n + m);

n + pred(m) == pred(n + m);

n * 0 == 0;

n * suc(m) == (n * m) + n;

n * pred(m) == (n * m) - n;

fin módulo Enteros;

TAD Conjunto

módulo Conjunto

importa ELEMENTO, BOOLEAN, NATURAL;

dominio CONJUNTO;

constructores

ConjuntoVacío: → CONJUNTO;

Agregar: ELEMENTO x CONJUNTO → CONJUNTO;

operaciones

Eliminar: ELEMENTO x CONJUNTO → CONJUNTO;

Pertenece: ELEMENTO x CONJUNTO → BOOLEAN;

EsVacío: CONJUNTO → BOOLEAN;

Unión: CONJUNTO x CONJUNTO → CONJUNTO;

axiomas i, j: ELEMENTO; C: CONJUNTO;

Agregar(i, Agregar(j, C)) == si i = j entonces Agregar(i, C)
si no Agregar(j, Agregar(i, C));

EsVacío (ConjuntoVacío) == true;

EsVacío (Agregar(i, C)) == false;

Pertenece(i, ConjuntoVacío) == false;

Pertenece(i, Agregar(j, C)) == si i = j entonces true
si no Pertenece (i, C);

Eliminar(i, ConjuntoVacío) == ConjuntoVacío;

Eliminar(i, Agregar(j, C)) == si i = j entonces C
si no Agregar(j, Eliminar(i, C));

Unión(ConjuntoVacío, C) == C;

Unión(Agregar(i, C_1), C_2) == Agregar(i, Unión(C_1, C_2);

fin módulo Conjuntos;

Robot: Especificación no constructiva.

módulo Robot;

dominio Posición

operaciones

pos_inicial: -> Posición;

avanzar: Posición -> Posición;

girar: Posición -> Posición;

axiomas pos: Posición;

girar(girar(girar(girar(pos)))) == pos;

girar(avanzar(girar(avanzar(girar(avanzar(girar(avanzar(pos))))))) == pos;

girar(girar(avanzar(girar(girar(avanzar(pos)))))) == pos;

fin módulo Robot;

Robot: Especificación semiconstructiva (1/2)

módulo Orientación;

dominio Orientación;

constructores

N, S, E, O: -> Orientación;

operaciones

girardch: Orientación -> Orientación;

girarizq: Orientación -> Orientación;

opuesta: Orientación -> Orientación;

axiomas

girardch(N) == E;

girardch(S) == O;

girardch(E) == S;

girardch(O) == N;

girarizq (N) == O;

girarizq (S) == E;

girarizq (E) == N;

girarizq (O) == S;

opuesta(N) == S;

opuesta(S) == N;

opuesta(E) == O;

opuesta(O) == E;

fin módulo Orientación;

Robot: Especificación semiconstructiva (2/2)

módulo Vía;

dominio Vía;

constructores

vía_inicial: -> Vía;

siguiente: Vía -> Vía;
anterior: Vía -> Vía;

axiomas v: Vía;

anterior(siguiente(v)) == v;
siguiente(anterior(v)) == v;

fin módulo Vía;

módulo Robot;

importa Orientación, Vía;

dominio Posición == Vía x Vía x Orientación;

constructores

pos_inicial: -> Posición;
avanzar: Posición -> Posición;
girar: Posición -> Posición;

axiomas h, v : Vía; o: Orientación; pos: Posición;

pos_inicial == (vía_inicial, vía_inicial, N);

avanzar((h, v, N)) == (siguiente(h), v, N);
avanzar((h, v, S)) == (anterior(h), v, S);
avanzar((h, v, E)) == (h, siguiente(v), E);
avanzar((h, v, O)) == (h, anterior(v), O);
girar((h, v, o)) == (h, v, girarizq(o));

fin módulo Robot;

Esquema genérico del TAD Pila

```
esquema Pilas
requisito Items
        dominio    ITEM;
importa BOOLEAN
dominios PILA;
constructores
        pila_nueva : -> PILA;
        insertar :  ITEM * PILA -> PILA;
operaciones
        es_pila_vacía: PILA -> BOOLEAN;
        extraer :  PILA -> PILA;
        cima :  PILA -> ITEM;
precondiciones p: PILA;
        pre cima(p): not es_pila_vacía(p);
axiomas    i : ITEM; s : PILA;
        . . .  // Los mismos axiomas que en la versión anterior.
fin esquema Pilas;

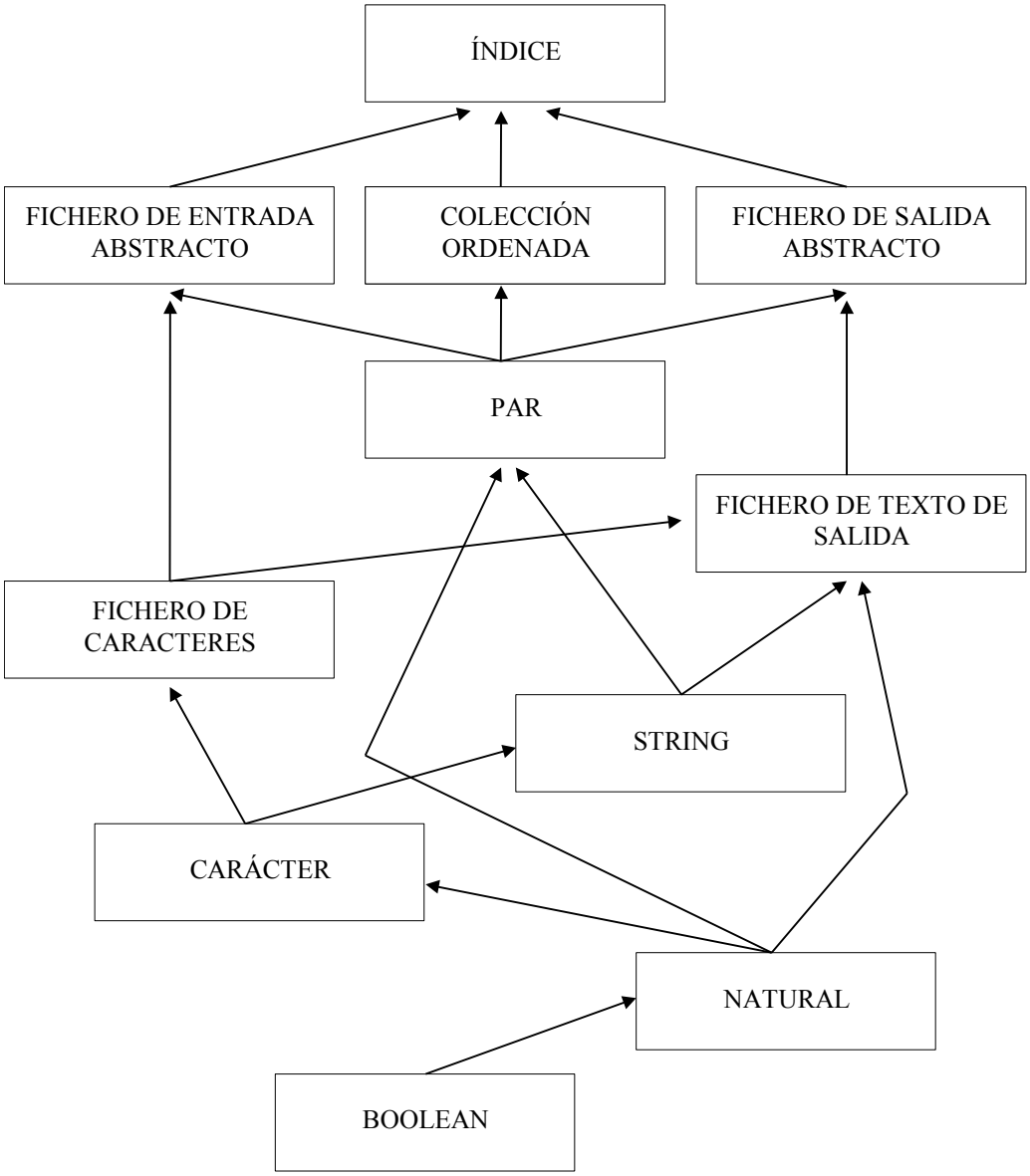
módulo Pila_Enteros;
        instancia Pila cambiando
                Items por Naturales;
                ITEM por NATURAL;
fin módulo Pila_Enteros;
```

Esquema genérico del TAD Vector

```
esquema Vectores;
requisito Item;
```

```
        dominio ITEM;
requisito Indice;
        dominio INDICE;
        operaciones:
                _ = _: INDICE * INDICE  -> BOOLEAN;
importa BOOLEAN;
dominio VECTOR;
operaciones
        vacío : -> VECTOR;
        asignar : VECTOR * INDICE * ITEM -> VECTOR;
        es_vacío: VECTOR -> BOOLEAN;
        valor : VECTOR * INDICE -> ITEM;
precondiciones v: VECTOR; i: INDICE;
        pre valor(v, i): not es_vacío(v);
axiomas    v : VECTOR; i, i1, i2:INDICE; item, item1, item2 : ITEM;
        es_vacío(vacío) == true;
        es_vacío(asignar(v, i, item)) == false;
        valor(asignar(v, i1, item), i2) ==
                si i1 = i2 entonces item
                si no valor(v, i2);
        asignar(asignar(v, i1, item1), i2, item2) ==
                si i1 = i2 entonces asignar(v, i2, item2)
                si no asignar(asignar(v, i2, item2), i1, item1);
fin esquema Vectores;
```

Caso de estudio: Relaciones entre los TADs.



Caso de estudio: TAD Naturales (1/2)

módulo Naturales;

importa BOOLEAN;

dominio NATURAL;

constructores

cero: → NATURAL;

suc: NATURAL → NATURAL;

operaciones

_+_, _*_: NATURAL * NATURAL → NATURAL;

_-_, _div_, _mod_: NATURAL * NATURAL → NATURAL;

_>_, _≥_, _<_, _≤_: NATURAL * NATURAL → BOOLEAN;

precondiciones n, m: NATURAL;

pre n-m: m ≤ n;

pre n div m: not (m == cero);

pre n mod m: not (m == cero);

Caso de estudio: TAD Naturales (2/2)

axiomas n, m: NATURAL;

n + cero == n;

n + suc(m) == suc(n + m);

n * cero == cero;

n * suc(m) == n + (n * m);

n - cero == n;

n - suc(m) == pred(n - m);

n div m == si n > m entonces suc((n - m) div m)

si no cero;

n mod m == si n ≥ m entonces (n - m) mod m

si no n;

n < cero == false;

cero < suc(n) == true ;

suc(n) < suc(m) == n < m;

m ≤ n == (m < n) or m == n;

m > n == not (m ≤ n);

m ≥ n == not (m < n);

fin módulo Naturales;

Caso de estudio: TAD Carácter (1/2)

módulo Caracteres;

importa Naturales, Lógicos;

dominio CARÁCTER;

constructores

A: → CARÁCTER;

a: → CARÁCTER;

. . .

Z : → CARÁCTER;

z : → CARÁCTER;

0 : → CARÁCTER;

. . .

9 : → CARÁCTER;

eol: → CARÁCTER;

sep : → CARÁCTER;

operaciones

valor : CARÁCTER → NATURAL;

_ < _, _ > _ : CARÁCTER * CARÁCTER → BOOLEAN;

es_letra, es_eol : CARÁCTER → BOOLEAN;

carácter : NATURAL –/→ CARÁCTER;

Caso de estudio: TAD Carácter (2/2)

precondiciones n: NATURAL;

pre carácter(n): cero < n < suc^{10}(cero);

axiomas c, c1, c2 : CARÁCTER;

valor(A) == suc(cero);

valor(a) == suc(suc(cero));

. . .

valor(Z) == suc^{51}(cero);

valor(z) == suc^{52}(cero);

valor(0) == suc^{53}(cero);

. . .

valor(9) == suc^{62}(cero);

valor(eof) == suc^{63}(cero);

valor(sep) == suc^{64}(cero);

c1 < c2 == valor(c1) < valor(c2);

c1 > c2 == valor(c1) > valor(c2);

es_letra(c) == (valor(c1) ≥ valor(A)) and (valor(c1) ≤ valor(z));

es_eol(c) == (c == eol);

carácter(cero) == 0;

. . .

carácter(suc^{9}(0)) == 9;

fin módulo Caracteres;

Caso de estudio: TAD Cadena de Caracteres (1/2)

módulo Cadenas;

importa Caracteres, Lógicos;

dominio STRING;

constructores;

cadena_vacía: -> STRING;

añadir: STRING * CARÁCTER -> STRING;

operaciones:

más_caracteres: STRING -> BOOLEAN;

primero: STRING -> CARÁCTER;

consumir: STRING -> STRING;

_ = _, _<_, _>_: STRING * STRING -> BOOLEAN;

precondiciones s: STRING;

pre primero(s): not s == cadena_vacía;

pre consumir(s): not s == cadena_vacía;

Caso de estudio: TAD Cadena de Caracteres (2/2)

axiomas s, s1, s2: STRING; c: CARÁCTER;

más_caracteres(cadena_vacía) == false;

más_caracteres(añadir(s, c)) == true;

primero(añadir(s, c)) ==

si (s == cadena_vacía) entonces c

si no primero(s);

consumir(añadir(s, c)) ==

si (s == cadena_vacía) entonces s

si no añadir(consumir(s), c);

cadena_vacía = cadena_vacía == true;

cadena_vacía = añadir(s, c) == false;

añadir(s, c) = cadena_vacía == false;

añadir(s1, c1) = añadir(s2, c2) ==(s1 = s2) and (c1 == c2);

añadir(s, c) < cadena_vacía == false;

cadena_vacía < añadir(s, c) == true;

añadir(s1, c1) < añadir(s2, c2) ==

(c1 < c2) or ((c1 == c2) and (s1 < s2));

fin módulo Cadenas;

Caso de estudio: TAD Fichero de Caracteres

módulo Fichero de Caracteres

dominio FICHERO;

importa Caracteres, Lógicos;

constructores

fichero_nulo: → FICHERO;

añadir: FICHERO * CARÁCTER → FICHERO;

operaciones

más_caracteres: FICHERO → BOOLEAN;

primero: FICHERO → CARÁCTER;

consumir: FICHERO → FICHERO;

precondiciones

pre primero(f): not (f == fichero_nulo);

pre consumir(f): not (f == fichero_nulo);

axiomas f: FICHERO; c: CARÁCTER;

más_caracteres(fichero_nulo) == false;

más_caracteres(añadir(c,f)) == true;

primero(añadir(c,f)) == si (f == fichero_nulo) entonces c
si no primero(f);

consumir(añadir(c,f)) == si (f == fichero_nulo)
entonces fichero_nulo
si no añadir(c,consumir(f));

fin módulo Fichero de Caracteres;

Caso de estudio: TAD Fichero de Texto de Salida

módulo Fichero de Texto de Salida;

importa Fichero de Caracteres, Cadenas, Naturales;

(* enriquecimiento del Fichero de caracteres *)

operaciones

string: NATURAL → STRING;

string2: NATURAL → STRING;

añadir_string: FICHERO * STRING → FICHERO;

añadir_natural: FICHERO * NATURAL → FICHERO;

axiomas n: NATURAL; f: File; s: STRING; c: carácter;

string(n) ==

si (n == cero) entonces añadir(cadena_vacía, 0)

si no string2(n);

string2(n) ==

si (n == cero) entonces cadena_vacía

sino añadir(string2(n div suc^{10}(0)), carácter(n mod suc^{10}(0)));

añadir_string(f, cadena_vacía) == f;

añadir_string(f, añadir(s, c)) == añadir_string(añadir(f, c), s);

añadir_natural(f, n) == añadir_string(f, string(n));

fin módulo Fichero de Texto de Salida;

Caso de estudio: TAD Par

módulo Pares;

importa Cadenas, Naturales;

dominio PAR;

constructores:

par: STRING * NATURAL -> PAR;

operaciones:

palabra: PAR -> STRING;

línea: PAR -> NATURAL;

_ ≤ _: PAR * PAR -> BOOLEAN;

axiomas p, p1, p2: PAR; s: STRING; l: NATURAL;

palabra(par(s, l)) == s;

línea(par(s, l)) == l;

par(s1, l1) ≤ par(s2, l2) == (s1 < s2) or ((s1 = s2) and (l1 < l2));

fin módulo PAR;

Caso de estudio: TAD Fichero Abstracto de Entrada (1/2)

módulo Fichero Abstracto de Entrada;

importa Fichero de Caracteres, Pares, Lógicos;

dominio FAE;

constructores

fae: FICHERO * NATURAL → FAE;

operaciones

fae: FICHERO → FAE;

más_pares: FAE → BOOLEAN;

primer_par: FAE → PAR;

consumir_par: FAE → FAE;

primera_palabra: FICHERO * STRING → STRING;

consumir_palabra: FICHERO → FICHERO;

precondiciones

primer_par(fae) : más_pares(fae);

consumir_par(fae) : más_pares(fae);

axiomas f:FICHERO;c:CARÁCTER;wd:STRING; ln:NATURAL;

primera_palabra(f, wd) ==

si not es_letra(primero(f)) entonces wd

si no primera_palabra(consumir(f), añadir (wd, primero(f)));

consumir_palabra(f) ==

si not es_letra(primero(f)) entonces f

```
sino consumir_palabra(consumir(f));
```

Caso de estudio: TAD Fichero Abstracto de Entrada (2/2)

```
fae(f) == fae(f, 1);

más_pares(fae(fichero_nulo, ln)) == false;
más_pares(fae(añadir(f, c),ln)) ==
        si es_letra(primero(añadir(f,c)))
                entonces true
                si no más_pares(fae(consumir(añadir(f, c)), ln));

primer_par(fae(f, ln)) ==
        si es_letra(primero(f))
                entonces par(primera_palabra(f,cadena_vacia),ln)
                si no   si es_eol(primero(f))
                                entonces primer_par(fae(consumir(f),suc(ln)))
                                si no primer_par(fae(consumir(f), ln));

consumir_par(fae(f,ln)) ==
        si es_letra(primero(f))
                entonces fae(consumir_palabra(f),ln)
                si no   si es_eol(primero(f))
                                entonces(fae(consumir(f),suc(ln)))
                                si no consumir_par(fae(consumir(f),ln));

fin módulo Fichero Abstracto de Entrada;
```

Caso de estudio: TAD Fichero Abstracto de Salida (1/2)

módulo Fichero Abstracto de Salida;

importa Fichero de Caracteres, Pares, Lógicos, Cadenas;

dominio FAE;

constructores

fas_vacío: -> FAS;

fas: FICHERO * STRING -> FAS;

operaciones

añadir_par: FAS * PAR -> FAS;

fc: FAS -> FICHERO;

axiomas

```
añadir_par(fas_vacío, par(wd, ln)) ==
        fas(    añadir_natural(
                        añadir(
                                añadir_string(fichero_vacío, wd),
                                sep),
                        ln),
                wd);
```

Caso de estudio: TAD Fichero Abstracto de Salida (2/2)

```
        añadir_par(fas(f, wd1), par(wd2, ln)) ==
            si wd1 = wd2 entonces
                fas(    añadir_natural(añadir(f, sep), ln), wd1)
            si no
                fas(    añadir_natural(
                            añadir(
                                añadir_string(
                                    añadir(f, eol),
                                    wd2),
                                sep),
                            ln),
                        wd2);
        fc(fas_vacío) == fichero_nulo;
        fc(fas(f, wd)) == f;
    fin módulo Fichero Abstracto de Salida;
```

Caso de estudio: TAD Colección ordenada

módulo Colección Ordenada;

importa Pares, Lógicos;

dominio COLECCIÓN;

constructores

colección_vacía: -> COLECCIÓN;

añadir_par: COLECCIÓN * PAR -> COLECCIÓN;

operaciones

más_pares: COLECCIÓN -> BOOLEAN;

menor_par: COLECCIÓN -> PAR;

consumir_menor_par: COLECCIÓN -> COLECCIÓN

precondiciones c: COLECCIÓN;

pre menor_par(c): not (c == colección_vacía);

pre consumir_menor_par(c): not (c == colección_vacía);

axiomas c: COLECCIÓN; p: PAR;

más_pares(colección_vacía) == false;

más_pares(añadir_par(c, p)) == true;

menor_par(añadir_par(c, p)) ==

si (c == colección_vacía) entonces p

si no si (p ≤ menor_par(c)) entonces p

si no menor_par(c);

consumir_menor_par(añadir_par(c, p)) ==

si (c == colección_vacía) entonces c

si no si (p ≤ menor_par(c)) entonces c

si no añadir_par(consumir_menor_par(c), p);

fin módulo Colección Ordenada;

Caso de estudio: TAD Indice

módulo Índice;

importa Fichero Abstracto de Entrada, Fichero Abstracto de Salida,

Colección Ordenada;

operaciones

índice: FICHERO -> FICHERO;

ordenar: COLECCIÓN * FAE -> COLECCIÓN;

salida: FAS * COLECCIÓN -> FAS;

axiomas f: FICHERO; fae: FAE; c: COLECCIÓN; fas: FAS;

ordenar(c,fae) == si más_pares(fae)

entonces ordenar(añadir_par(c,primer_par(fae)),consumir_par(fae))

si no c;

salida(fas,c) == si más_pares(c)

entonces salida(añadir_par(fas,menor_par(c)),consumir_menor_par(c))

si no fas;

indice(f) == fc(fas(fas_vacío,ordenar(colección_vacía,fae(f))));

fin módulo Índice;